data-science-ml-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `data_science_ml_ai999x.about()` for the same answer.

Every capability data-science-ml-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import data_science_ml_ai999x

result = data_science_ml_ai999x.solve('ml_binary_confusion_matrix', y_true=[1, 0, 1, 1, 0, 0, 1, 0],
y_pred=[1, 0, 0, 1, 0, 1, 1, 0])
result.show()
# TP = 3, FP = 1, FN = 1, TN = 3, accuracy = 0.75, error_rate = 0.25
```

By topic

topic	capabilities
Classical algorithms	Distances and cosine similarity between two vectors, RBF (Gaussian) kernel matrix, One k-means iteration (assignment and update), K-means clustering from given initial centroids, K-nearest-neighbour classification of a query point, K-nearest-neighbour regression of a query point, Gaussian naive Bayes prediction, Multinomial naive Bayes prediction (word counts), Bernoulli naive Bayes prediction (binary features), Best decision-tree split (classification, Gini or entropy), Agglomerative hierarchical clustering merge heights, Polynomial kernel, Best regression-tree split (squared error)
Classification metrics	Binary confusion matrix from labels, Binary classification metrics from confusion counts, F-beta score, Matthews correlation coefficient, Multi-class confusion matrix, Averaged precision, recall and F1 (multi-class), Per-class precision, recall, F1 and support, Cohen's kappa (unweighted, linear or quadratic), Balanced accuracy (multi-class), Binary log loss (cross-entropy), Multi-class log loss (categorical cross-entropy), Brier score, ROC AUC from scores, ROC curve points, Average precision and precision-recall AUC, Top-k accuracy, Hamming loss and subset accuracy (multi-label), Balanced class weights, Expected calibration error, Confusion counts, precision and recall at a score threshold
Clustering metrics	Normalised mutual information of two clusterings, Adjusted mutual information of two clusterings, Rand index, adjusted Rand index and Fowlkes-Mallows index, Homogeneity, completeness and V-measure, Silhouette score, Davies-Bouldin index, Calinski-Harabasz index
Dimensionality reduction	Principal component analysis (explained variance and first component)
Gradients	Mean squared error loss and gradient of a linear model, Logistic regression loss and gradient (L2-regularised), One gradient-descent step for linear regression
Information theory	KL divergence, Jensen-Shannon divergence and cross-entropy of discrete distributions, Shannon entropy and perplexity of a distribution, Joint, marginal and conditional entropy and mutual information, Mutual information between two label sequences, Information gain and gain ratio of a split, Gini impurity decrease of a split, Node impurity from class counts (entropy, Gini, misclassification)

topic	capabilities
Linear models	Logistic regression fit (L2, stated C), Logistic regression prediction from given weights, Ridge regression coefficients (stated alpha), Lasso regression coefficients (stated alpha), Linear SVM decision values, classes and margin, Perceptron training epoch
Loss functions and activations	Huber, smooth L1 and pseudo-Huber loss, Quantile (pinball) loss, Softmax, Log-softmax, Sigmoid (logistic function) and its derivative, Cross-entropy loss from logits and its gradient, Binary cross-entropy with logits and its gradient, Hinge and squared hinge loss, Multi-class hinge loss (Crammer-Singer and Weston-Watkins), Cross-entropy with label smoothing, Binary focal loss and its gradient, Softmax with temperature, Triplet margin loss
Model selection	Train/test split sizes, K-fold cross-validation fold sizes, K-fold cross-validated MSE of linear regression, Leave-one-out cross-validation of linear regression (PRESS), Bias-variance decomposition of predictions
Neural networks	Forward pass of a one-hidden-layer dense network, Backpropagation gradients of a one-hidden-layer network, Parameter count of a dense (fully connected) network, Convolution output size, Parameter count of a 2-D convolution layer, 2-D convolution of an image with a kernel (deep-learning convention), Max pooling and average pooling, Scaled dot-product attention, Multi-head attention (identity projections), Batch normalisation forward pass (training mode), Parameter count of a transformer encoder, Parameter count of an LSTM, GRU or simple RNN, Receptive field of stacked convolutions
Optimizers	SGD with momentum: one update, Adam optimizer: one update, AdamW optimizer (decoupled weight decay): one update, Step learning-rate schedule, Cosine-annealing learning-rate schedule, Linear warm-up followed by cosine decay
Preprocessing	Standardisation (z-scores of features), [Min-max scaling to [0, 1]](#ml_minmax_scaler), Label encoding and one-hot encoding of categories, Robust scaling (median and interquartile range)
Ranking metrics	Precision@k and recall@k of a ranking, Mean average precision (MAP) of rankings, NDCG@k (normalised discounted cumulative gain), Mean reciprocal rank (MRR)
Regression metrics	Regression metrics (MAE, MSE, RMSE, R^2, explained variance), Mean absolute percentage error, Adjusted R^2, Mean Tweedie deviance (normal, Poisson, gamma, compound Poisson-gamma), Mean squared logarithmic error

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
ml_binary_confusion_matrix	Binary confusion matrix from labels	y_true, y_pred	TP, FP, FN, TN, accuracy, error_rate
ml_binary_metrics_from_counts	Binary classification metrics from confusion counts	TP, FP, FN, TN	accuracy, precision, recall, specificity, f1, npv, balanced_accuracy, fpr, fnr, jaccard
ml_fbeta_score	F-beta score	TP, FP, FN, beta	precision, recall, fbeta
ml_matthews_correlation	Matthews correlation coefficient	TP, FP, FN, TN	mcc
ml_multiclass_confusion_matrix	Multi-class confusion matrix	y_true, y_pred	classes, matrix, n_correct, accuracy
ml_precision_recall_f1_averaged	Averaged precision, recall and F1 (multi-class)	y_true, y_pred, average	precision, recall, f1
ml_classification_report	Per-class precision, recall, F1 and support	y_true, y_pred	classes, precision, recall, f1, support, macro_f1
ml_cohen_kappa	Cohen's kappa (unweighted, linear or quadratic)	y1, y2, weights	kappa

id	operation	inputs	outputs
ml_balanced_accuracy	Balanced accuracy (multi-class)	y_true, y_pred	balanced_accuracy, adjusted
ml_log_loss_binary	Binary log loss (cross-entropy)	y_true, p	log_loss, log_loss_bits
ml_log_loss_multiclass	Multi-class log loss (categorical cross-entropy)	y_true, P	log_loss, perplexity
ml_brier_score	Brier score	y_true, p	brier
ml_roc_auc	ROC AUC from scores	y_true, scores	auc, gini, n_pos, n_neg
ml_roc_curve	ROC curve points	y_true, scores	thresholds, fpr, tpr, auc
ml_average_precision	Average precision and precision-recall AUC	y_true, scores	average_precision, pr_auc_trapezoid
ml_top_k_accuracy	Top-k accuracy	y_true, S, k	top_k_accuracy, n_hits
ml_hamming_loss	Hamming loss and subset accuracy (multi-label)	Y_true, Y_pred	hamming_loss, subset_accuracy, n_mismatches
ml_balanced_class_weights	Balanced class weights	y	classes, weights
ml_expected_calibration_error	Expected calibration error	y_true, p, n_bins	ece, mce, n_nonempty_bins
ml_regression_metrics	Regression metrics (MAE, MSE, RMSE, R^2, explained variance)	y_true, y_pred	mae, mse, rmse, r2, explained_variance, median_ae, max_error
ml_mape	Mean absolute percentage error	y_true, y_pred	mape, mape_percent
ml_adjusted_r2	Adjusted R^2	y_true, y_pred, p	r2, adjusted_r2
ml_huber_loss	Huber, smooth L1 and pseudo-Huber loss	y_true, y_pred, delta	huber, smooth_l1, pseudo_huber
ml_quantile_loss	Quantile (pinball) loss	y_true, y_pred, alpha	pinball
ml_tweedie_deviance	Mean Tweedie deviance (normal, Poisson, gamma, compound Poisson-gamma)	y_true, y_pred, power	deviance
ml_mean_squared_log_error	Mean squared logarithmic error	y_true, y_pred	msle, rmsle
ml_softmax	Softmax	z	probabilities, log_sum_exp
ml_log_softmax	Log-softmax	z	log_probabilities, log_sum_exp
ml_sigmoid	Sigmoid (logistic function) and its derivative	z	sigmoid, derivative, log_sigmoid
ml_cross_entropy_logits	Cross-entropy loss from logits and its gradient	z, y	loss, probability, gradient
ml_bce_with_logits	Binary cross-entropy with logits and its gradient	y, z	loss, gradient
ml_hinge_loss	Hinge and squared hinge loss	y_true, scores	hinge, squared_hinge, n_violations

id	operation	inputs	outputs
ml_multiclass_hinge_loss	Multi-class hinge loss (Crammer-Singer and Weston-Watkins)	y_true, D	crammer_singer, weston_watkins
ml_kl_divergence_discrete	KL divergence, Jensen-Shannon divergence and cross-entropy of discrete distributions	p, q, unit	kl_pq, kl_qp, js_divergence, cross_entropy, entropy_p
ml_label_smoothing_cross_entropy	Cross-entropy with label smoothing	z, y, epsilon	loss, gradient
ml_focal_loss	Binary focal loss and its gradient	y, z, gamma, alpha	loss, gradient
ml_mse_gradient	Mean squared error loss and gradient of a linear model	X, y, w, b	loss, grad_w, grad_b
ml_logistic_loss_gradient	Logistic regression loss and gradient (L2-regularised)	X, y, w, b, lam	loss, grad_w, grad_b
ml_gradient_descent_step_linear	One gradient-descent step for linear regression	X, y, w, b, lr	w_new, b_new, loss_before, loss_after
ml_distance_metrics	Distances and cosine similarity between two vectors	x, z	euclidean, manhattan, chebyshev, cosine_similarity, cosine_distance
ml_rbf_kernel_matrix	RBF (Gaussian) kernel matrix	X, gamma	K, mean_offdiagonal
ml_entropy	Shannon entropy and perplexity of a distribution	p, unit	entropy, perplexity, max_entropy, normalized_entropy
ml_joint_conditional_entropy	Joint, marginal and conditional entropy and mutual information	P, unit	H_XY, H_X, H_Y, H_Y_given_X, H_X_given_Y, mutual_information
ml_mutual_information_labels	Mutual information between two label sequences	x, y, unit	mutual_information, H_x, H_y, H_x_given_y
ml_information_gain	Information gain and gain ratio of a split	y, x	parent_entropy, conditional_entropy, information_gain, split_info, gain_ratio
ml_gini_split	Gini impurity decrease of a split	y, g	gini_parent, gini_children, gini_decrease
ml_normalized_mutual_information	Normalised mutual information of two clusterings	labels_true, labels_pred, method	nmi, mutual_information
ml_adjusted_mutual_information	Adjusted mutual information of two clusterings	labels_true, labels_pred, method	ami, expected_mi, mutual_information
ml_rand_index	Rand index, adjusted Rand index and Fowlkes-Mallows index	labels_true, labels_pred	rand_index, adjusted_rand_index, fowlkes_mallows

id	operation	inputs	outputs
ml_homogeneity_completeness_v_measure	Homogeneity, completeness and V-measure	labels_true, labels_pred, beta	homogeneity, completeness, v_measure
ml_silhouette_score	Silhouette score	X, labels	silhouette, samples
ml_davies_bouldin_index	Davies-Bouldin index	X, labels	davies_bouldin
ml_calinski_harabasz_index	Calinski-Harabasz index	X, labels	calinski_harabasz, between, within
ml_kmeans_step	One k-means iteration (assignment and update)	X, C	labels, new_centroids, inertia
ml_kmeans	k-means clustering from given initial centroids	X, C	centroids, labels, inertia
ml_knn_classify	k-nearest-neighbour classification of a query point	X, y, x, k, metric	prediction, votes, neighbors, neighbor_distances
ml_knn_regress	k-nearest-neighbour regression of a query point	X, y, x, k, metric, weights	prediction, neighbors, neighbor_distances
ml_gaussian_naive_bayes	Gaussian naive Bayes prediction	X, y, x	prediction, classes, posteriors
ml_multinomial_naive_bayes	Multinomial naive Bayes prediction (word counts)	X, y, x, alpha	prediction, classes, posteriors
ml_bernoulli_naive_bayes	Bernoulli naive Bayes prediction (binary features)	X, y, x, alpha	prediction, classes, posteriors
ml_decision_stump_classification	Best decision-tree split (classification, Gini or entropy)	X, y, criterion	feature, threshold, impurity_decrease, n_left, n_right
ml_hierarchical_clustering	Agglomerative hierarchical clustering merge heights	X, method	heights, final_height
ml_pca	Principal component analysis (explained variance and first component)	X	explained_variance, explained_variance_ratio, component_1, scores_1
ml_standard_scaler	Standardisation (z-scores of features)	X	means, stds, scaled
ml_minmax_scaler	Min-max scaling to [0, 1]	X	mins, maxs, scaled
ml_one_hot_encoding	Label encoding and one-hot encoding of categories	categories	categories, counts, codes, one_hot
ml_train_test_split_sizes	Train/test split sizes	n, test_size	n_test, n_train
ml_kfold_sizes	k-fold cross-validation fold sizes	n, k	test_sizes, train_sizes, first_test_fold
ml_kfold_cv_linear_regression	k-fold cross-validated MSE of linear regression	X, y, k	fold_mse, mean_mse

id	operation	inputs	outputs
ml_loocv_linear_regression	Leave-one-out cross-validation of linear regression (PRESS)	X, y	press, loocv_mse
ml_bias_variance_decomposition	Bias-variance decomposition of predictions	P, f	bias_squared, variance, expected_mse
ml_logistic_regression_fit	Logistic regression fit (L2, stated C)	X, y, C	w, b, objective, train_accuracy
ml_logistic_regression_predict	Logistic regression prediction from given weights	w, b, X	probabilities, classes, logits
ml_ridge_regression	Ridge regression coefficients (stated alpha)	X, y, alpha	w, b, train_mse
ml_lasso_regression	Lasso regression coefficients (stated alpha)	X, y, alpha	w, b, n_nonzero
ml_linear_svm_decision	Linear SVM decision values, classes and margin	w, b, X	decision, classes, margin_width, distances
ml_perceptron_epoch	Perceptron training epoch	X, y, w, b, lr	w_new, b_new, n_updates
ml_dense_forward_pass	Forward pass of a one-hidden-layer dense network	x, W1, b1, activation, W2, b2, output	hidden, output, max_output
ml_backpropagation	Backpropagation gradients of a one-hidden-layer network	x, t, W1, b1, activation, W2, b2, loss	loss, dW1, db1, dW2, db2
ml_dense_parameter_count	Parameter count of a dense (fully connected) network	sizes, bias	per_layer, total
ml_conv_output_size	Convolution output size	n, kernel, stride, padding, dilation	output_size, effective_kernel
ml_conv_parameter_count	Parameter count of a 2-D convolution layer	in_channels, out_channels, kernel_h, kernel_w, groups, bias	weights, biases, total
ml_conv2d_forward	2-D convolution of an image with a kernel (deep-learning convention)	image, kernel, stride, padding	output, output_sum
ml_pooling	Max pooling and average pooling	X, size, stride	max_pool, avg_pool
ml_scaled_dot_product_attention	Scaled dot-product attention	Q, K, V, causal	weights, output, output_sum
ml_multihead_attention	Multi-head attention (identity projections)	Q, K, V, heads	output, output_sum
ml_batch_norm_forward	Batch normalisation forward pass (training	X, gamma, beta, eps	output, batch_mean, batch_var

id	operation	inputs	outputs
	mode)		
ml_sgd_momentum_step	SGD with momentum: one update	w, grad, buf, lr, momentum, nesterov, weight_decay	buf_new, w_new
ml_adam_step	Adam optimizer: one update	w, grad, m, v, t, lr, beta1, beta2, eps	m_new, v_new, w_new
ml_adamw_step	AdamW optimizer (decoupled weight decay): one update	w, grad, m, v, t, lr, beta1, beta2, eps, weight_decay	m_new, v_new, w_new
ml_step_lr_schedule	Step learning-rate schedule	lr0, step_size, gamma, epoch	lr, decays
ml_cosine_lr_schedule	Cosine-annealing learning-rate schedule	lr_max, eta_min, T_max, epoch	lr
ml_warmup_cosine_lr_schedule	Linear warm-up followed by cosine decay	lr_max, start_factor, warmup, total, eta_min, step	lr
ml_transformer_parameter_count	Parameter count of a transformer encoder	d_model, n_heads, d_ff, n_layers	attention, feed_forward, layer_norms, per_layer, total
ml_rnn_parameter_count	Parameter count of an LSTM, GRU or simple RNN	cell, input_size, hidden_size, num_layers, bidirectional	per_layer, total
ml_receptive_field	Receptive field of stacked convolutions	kernels, strides, dilations	receptive_field, jump
ml_precision_recall_at_k	Precision@k and recall@k of a ranking	rel, n_relevant, k	precision_at_k, recall_at_k, hits, hit_rate
ml_mean_average_precision	Mean average precision (MAP) of rankings	R	average_precision, map
ml_ndcg_at_k	NDCG@k (normalised discounted cumulative gain)	rel, k, gain	dcg, idcg, ndcg
ml_mean_reciprocal_rank	Mean reciprocal rank (MRR)	R	reciprocal_ranks, mrr
ml_threshold_metrics	Confusion counts, precision and recall at a score threshold	y_true, scores, threshold	TP, FP, FN, TN, precision, recall, accuracy
ml_softmax_temperature	Softmax with temperature	z, T	probabilities, entropy
ml_polynomial_kernel	Polynomial kernel	x, z, gamma, coef0, degree	kernel
ml_triplet_loss	Triplet margin loss	anchor, positive, negative, margin	loss, d_ap, d_an
ml_impurity_from_counts	Node impurity from class counts (entropy, Gini, misclassification)	counts	entropy_bits, gini, misclassification

id	operation	inputs	outputs
ml_robust_scaler	Robust scaling (median and interquartile range)	X	medians, iqr, scaled
ml_decision_stump_regression	Best regression-tree split (squared error)	X, y	feature, threshold, mse_decrease, left_mean, right_mean

Binary confusion matrix from labels, `ml_binary_confusion_matrix`

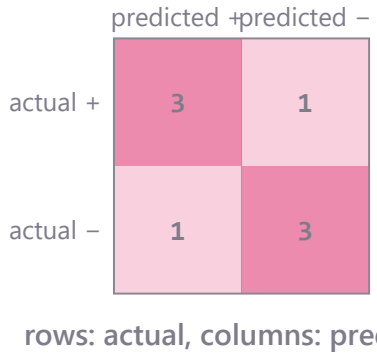
Count true positives, false positives, false negatives and true negatives from true and predicted $0/1$ labels, with the accuracy and error rate.

$$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

input	meaning	condition
`y_true`	true labels (0 or 1, 1 = positive)	
`y_pred`	predicted labels (0 or 1)	

output	meaning
`TP`	true positives
`FP`	false positives
`FN`	false negatives
`TN`	true negatives
`accuracy`	(TP + TN) / n
`error_rate`	(FP + FN) / n

- **Method:** count each (true, predicted) pair; accuracy and error rate exactly as fractions
- **Assumes:** labels are 0 or 1; y_true and y_pred have the same length
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2 × 2 confusion matrix the package returns for y_true = [1, 0, 1, 1, 0, 0, 1, 0] and y_pred = [1, 0, 0, 1, 0, 1, 1, 0]: TP = 3, FN = 1, FP = 1, TN = 3; accuracy 0.75, error rate 0.25.

```
data_science_ml_ai999x.solve('ml_binary_confusion_matrix', y_true=[1, 0, 1, 1, 0, 0, 1, 0], y_pred=
[1, 0, 0, 1, 0, 1, 1, 0])
# -> TP = 3, FP = 1, FN = 1, TN = 3, accuracy = 0.75, error_rate = 0.25   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_binary_confusion_matrix', y_true=[1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
y_pred=[1, 1, 0, 0, 0, 0, 0, 0, 1, 1])
# -> TP = 2, FP = 2, FN = 1, TN = 5, accuracy = 0.7, error_rate = 0.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Binary confusion matrix: y_true = [1, 0, 1, 1, 0, 0, 1, 0], y_pred = [1, 0, 0, 1, 0, 1, 1, 0]	SOLVED	TP = 3, FP = 1, FN = 1, TN = 3, accuracy = 0.75, error_rate = 0.25
Binary confusion matrix: y_true = [1, 1, 1, 0, 0, 0, 0, 0, 0, 0], y_pred = [1, 1, 0, 0, 0, 0, 0, 0, 1, 1]	SOLVED	TP = 2, FP = 2, FN = 1, TN = 5, accuracy = 0.7, error_rate = 0.3
Binary confusion matrix: y_pred = [1, 0, 0, 1, 0, 1, 1, 0]	NEEDS_INPUT	ml_binary_confusion_matrix needs y_true (true labels (0 or 1, 1 = positive)). Example: solve('ml_binary_confusion_matrix
Binary confusion matrix: y_true = [1, 0, 1, 1, 0, 0, 1, 0], y_pred = [1, 0, 0, 1, 0, 1, 1, 0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Binary confusion matrix: y_true = [1, 0, 2], y_pred = [1, 0, 1]	INVALID_INPUT	Binary confusion matrix from labels: every label in y_true must be 0 or 1.
Binary confusion matrix: y_true = [1, 0, 1], y_pred = [1, 0]	INVALID_INPUT	Binary confusion matrix from labels: y_true and y_pred must have the same length.

Binary classification metrics from confusion counts, `ml_binary_metrics_from_counts`

Accuracy, precision, recall (sensitivity), specificity, F1, negative predictive value, balanced accuracy, false positive and false negative rates and the Jaccard index from TP, FP, FN, TN.

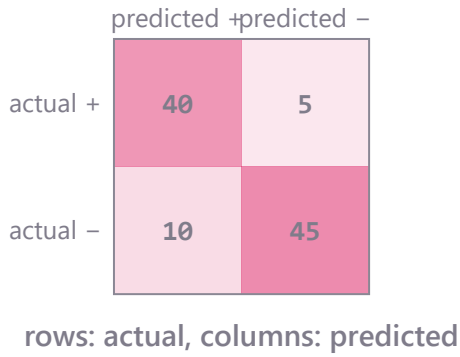
$$P = \frac{TP}{TP + FP},\ R = \frac{TP}{TP + FN},\ F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN}$$

input	meaning	condition
`TP`	true positives	
`FP`	false positives	
`FN`	false negatives	
`TN`	true negatives	

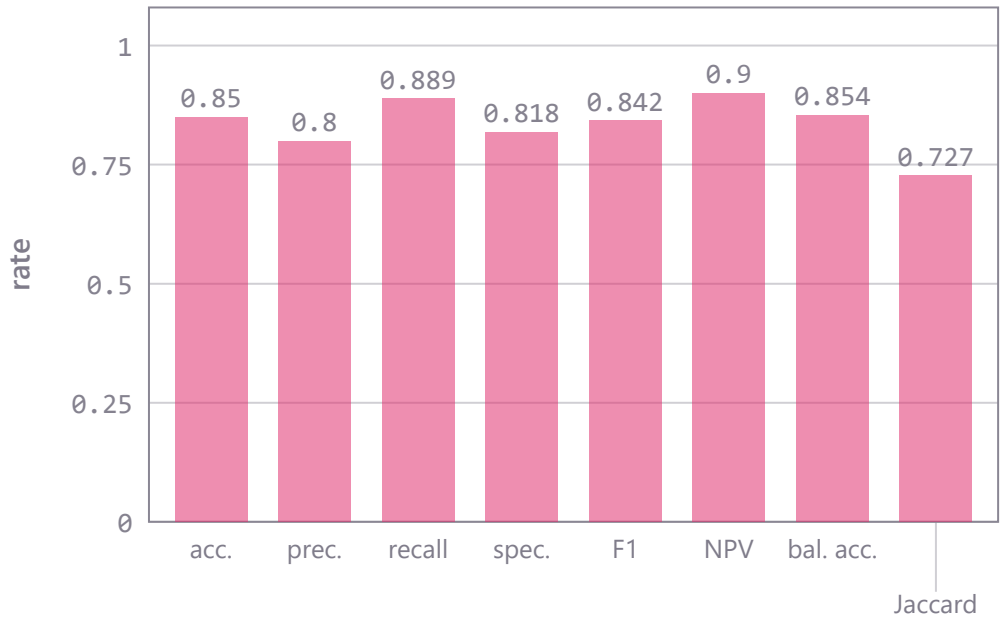
output	meaning
`accuracy`	(TP + TN) / n
`precision`	TP / (TP + FP)
`recall`	TP / (TP + FN), the sensitivity
`specificity`	TN / (TN + FP)
`f1`	2 TP / (2 TP + FP + FN)

output	meaning
`npv`	TN / (TN + FN)
`balanced_accuracy`	(recall + specificity) / 2
`fpr`	FP / (FP + TN)
`fnr`	FN / (FN + TP)
`jaccard`	TP / (TP + FP + FN)

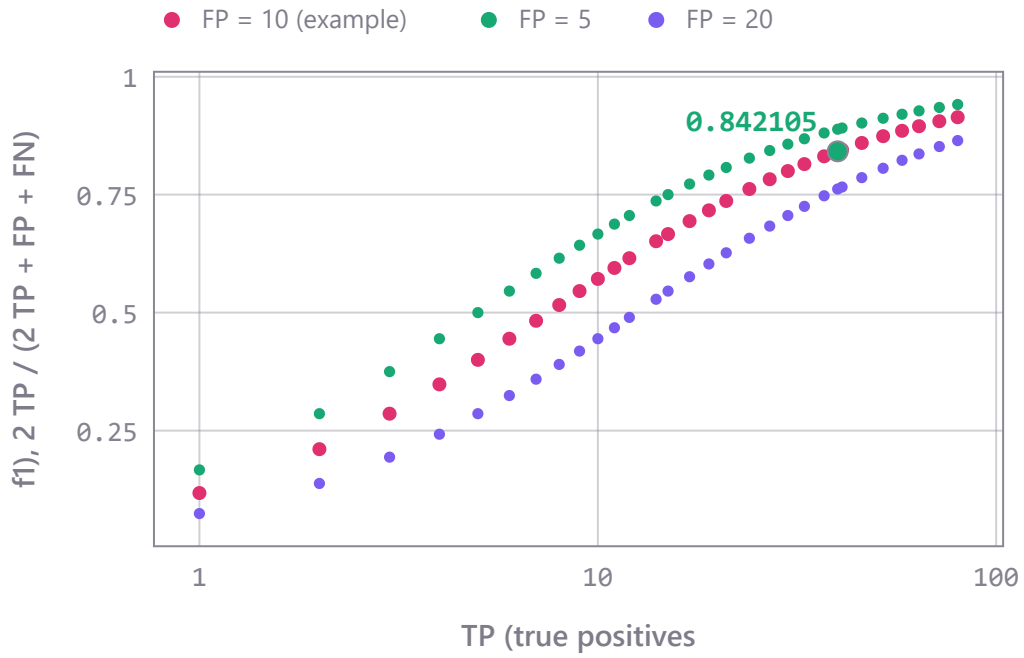
- **Method:** exact rational arithmetic on the four counts
- **Assumes:** counts are whole numbers >= 0; every ratio is defined (TP + FP, TP + FN, TN + FP, TN + FN > 0)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated counts TP = 40, FP = 10, FN = 5, TN = 45 laid out as a confusion matrix.



The rates the package returns for TP = 40, FP = 10, FN = 5, TN = 45: accuracy 0.85, precision 0.8, recall 0.888889, specificity 0.818182, f1 0.842105, npv 0.9, balanced accuracy 0.853535, jaccard 0.727273.



f1 (2 TP / (2 TP + FP + FN)) against TP (true positives) from 1 to 80 on a logarithmic axis. One curve for each value of FP (false positives): 10, 5, 20. Other inputs as in the worked example: FN = 5, TN = 45. 30 points per curve computed by data-science-ml-ai999x; marked: the worked example, TP = 40, f1 = 0.842105.

```
data_science_ml_ai999x.solve('ml_binary_metrics_from_counts', TP=40, FP=10, FN=5, TN=45)
# -> accuracy = 0.85, precision = 0.8, recall = 0.888889, specificity = 0.818182, f1 = 0.842105, npv
= 0.9, balanced_accuracy = 0.853535, fpr = 0.181818, fnr = 0.111111, jaccard = 0.727273   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_binary_metrics_from_counts', TP=90, FP=30, FN=10, TN=870)
# -> accuracy = 0.96, precision = 0.75, recall = 0.9, specificity = 0.966667, f1 = 0.818182, npv =
0.988636, balanced_accuracy = 0.933333, fpr = 0.0333333, fnr = 0.1, jaccard = 0.692308   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Binary metrics from counts: TP = 40, FP = 10, FN = 5, TN = 45	SOLVED	accuracy = 0.85, precision = 0.8, recall = 0.888889, specificity = 0.818182, f1 = 0.842105, npv = 0.9, balanced_accuracy = 0.853535, fpr = 0.181818, fnr = 0.111111, jaccard = 0.727273
Binary metrics from counts: TP = 90, FP = 30, FN = 10, TN = 870	SOLVED	accuracy = 0.96, precision = 0.75, recall = 0.9, specificity = 0.966667, f1 = 0.818182, npv = 0.988636, balanced_accuracy = 0.933333, fpr = 0.0333333, fnr = 0.1, jaccard = 0.692308
Binary metrics from counts: FP = 10, FN = 5, TN = 45	NEEDS_INPUT	ml_binary_metrics_from_counts needs TP (true positives). Example: solve('ml_binary_metrics_from_counts', TP=40, FP=10, F
Binary metrics from counts: TP = 40, FP = 10, FN = 5, TN = 45, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Binary metrics from counts: TP = 0, FP = 0, FN = 5, TN = 45	INVALID_INPUT	Binary classification metrics from confusion counts: precision is undefined: there are no predicted positives (TP + FP =

F-beta score, `ml_fbeta_score`

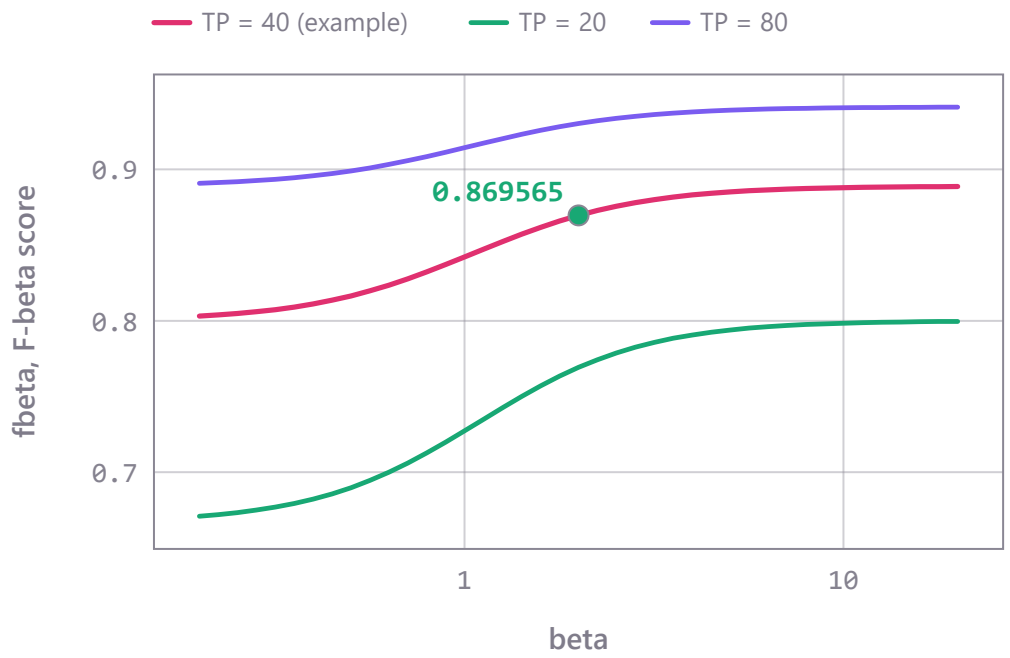
The F-beta score, the weighted harmonic mean of precision and recall in which recall counts beta times as much as precision.

$$F_\beta = \frac{(1+\beta^2)\,PR}{\beta^2 P + R}$$

input	meaning	condition
`TP`	true positives	
`FP`	false positives	
`FN`	false negatives	
`beta`	weight of recall relative to precision (beta > 0; 1 gives F1, 2 favours recall)	

output	meaning
`precision`	TP / (TP + FP)
`recall`	TP / (TP + FN)
`fbeta`	F-beta score

- **Method:** exact rational arithmetic from the counts
- **Assumes:** $\beta > 0$; $TP > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fbeta (F-beta score) against beta (weight of recall relative to precision (beta > 0; 1 gives F1, 2 favours recall)) from 0.2 to 20 on a logarithmic axis. One curve for each value of TP (true positives): 40, 20, 80. Other inputs as in the worked example: FP = 10, FN = 5. 41 points per curve computed by data-science-ml-ai999x; marked: the worked example, beta = 2, fbeta = 0.869565.

```
data_science_ml_ai999x.solve('ml_fbeta_score', TP=40, FP=10, FN=5, beta=2)
# -> precision = 0.8, recall = 0.888889, fbeta = 0.869565   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_fbeta_score', TP=40, FP=10, FN=5, beta=0.5)
# -> precision = 0.8, recall = 0.888889, fbeta = 0.816327   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
F-beta score: TP = 40, FP = 10, FN = 5, beta = 2	SOLVED	precision = 0.8, recall = 0.888889, fbeta = 0.869565
F-beta score: TP = 40, FP = 10, FN = 5, beta = 0.5	SOLVED	precision = 0.8, recall = 0.888889, fbeta = 0.816327
F-beta score: FP = 10, FN = 5, beta = 2	NEEDS_INPUT	ml_fbeta_score needs TP (true positives). Example: solve('ml_fbeta_score', TP=40, FP=10, FN=5, beta=2).
F-beta score: TP = 40, FP = 10, FN = 5, beta = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
F-beta score: TP = 40, FP = 10, FN = 5, beta = -1	INVALID_INPUT	F-beta score: beta must be positive.

Matthews correlation coefficient, `ml_matthews_correlation`

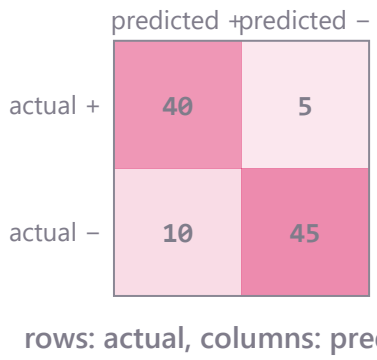
The Matthews correlation coefficient (phi coefficient) of a binary confusion matrix, a balanced measure between -1 and 1.

$$\mathrm{MCC} = \frac{TP\,TN - FP\,FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

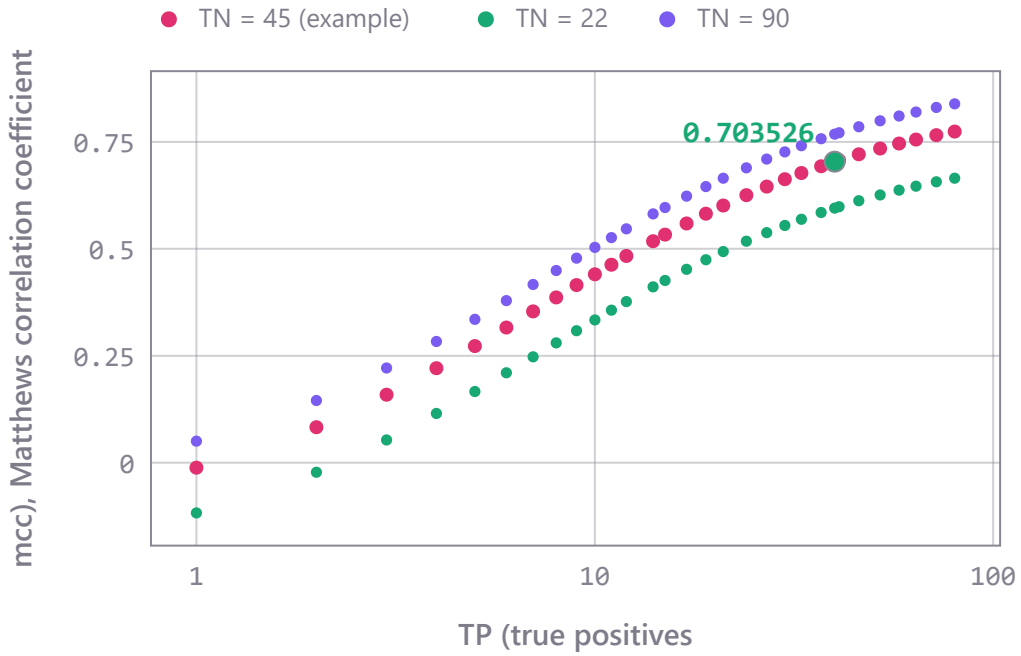
input	meaning	condition
`TP`	true positives	
`FP`	false positives	
`FN`	false negatives	
`TN`	true negatives	

output	meaning
`mcc`	Matthews correlation coefficient

- **Method:** (TP TN - FP FN) / sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN)) with exact integers under the root
- **Assumes:** no empty row or column of the confusion matrix
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated counts TP = 40, FP = 10, FN = 5, TN = 45 as a confusion matrix; returned Matthews correlation coefficient 0.703526.



mcc (Matthews correlation coefficient) against TP (true positives) from 1 to 80 on a logarithmic axis. One curve for each value of TN (true negatives): 45, 22, 90. Other inputs as in the worked example: FP = 10, FN = 5. 30 points per curve computed by data-science-ml-ai999x; marked: the worked example, TP = 40, mcc = 0.703526.

```
data_science_ml_ai999x.solve('ml_matthews_correlation', TP=40, FP=10, FN=5, TN=45)
# -> mcc = 0.703526   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_matthews_correlation', TP=6, FP=1, FN=2, TN=3)
# -> mcc = 0.478091   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Matthews correlation coefficient: TP = 40, FP = 10, FN = 5, TN = 45	SOLVED	mcc = 0.703526
Matthews correlation coefficient: TP = 6, FP = 1, FN = 2, TN = 3	SOLVED	mcc = 0.478091

question	outcome	result
Matthews correlation coefficient: FP = 10, FN = 5, TN = 45	NEEDS_INPUT	ml_matthews_correlation needs TP (true positives). Example: solve('ml_matthews_correlation', TP=40, FP=10, FN=5, TN=45).
Matthews correlation coefficient: TP = 40, FP = 10, FN = 5, TN = 45, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Matthews correlation coefficient: TP = 0, FP = 0, FN = 5, TN = 45	INVALID_INPUT	Matthews correlation coefficient: the MCC is undefined when a row or column of the confusion matrix is empty.

Multi-class confusion matrix, `ml_multiclass_confusion_matrix`

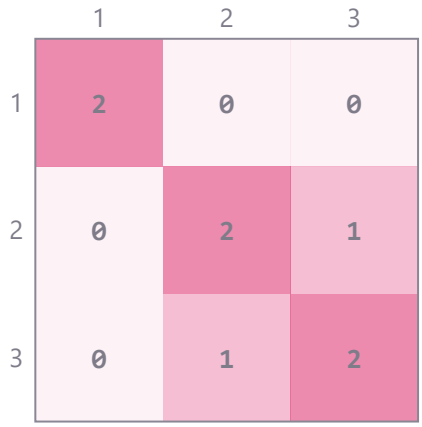
The confusion matrix of a multi-class classifier (true classes in rows, predicted in columns) with the number correct and the accuracy.

$$C_{ij} = \#\{n : y_n = c_i,\ \hat{y}_n = c_j\}$$

input	meaning	condition
`y_true`	true class labels (whole numbers)	
`y_pred`	predicted class labels (whole numbers)	

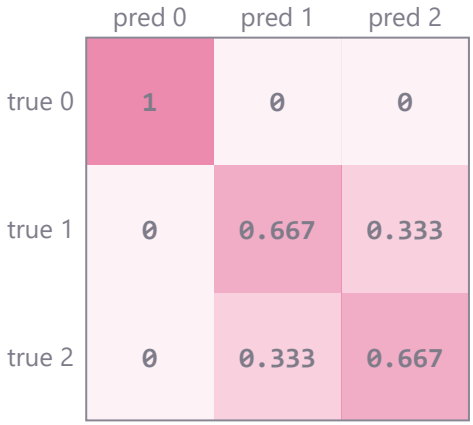
output	meaning
`classes`	classes in sorted order (the union of both label lists)
`matrix`	confusion matrix: row = true class, column = predicted class
`n_correct`	number of correct predictions (the trace)
`accuracy`	fraction correct

- **Method:** count each (true, predicted) pair over the sorted union of classes
- **Assumes:** labels are whole numbers; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: true class, columns: predicted class

The confusion matrix the package returns for true labels [0, 1, 2, 2, 1, 0, 2, 1] and predictions [0, 2, 2, 2, 1, 0, 1, 1]: rows are the true classes 0, 1, 2, columns the predicted ones; 6 of 8 on the diagonal, accuracy 0.75.



rows: true class (fractions), columns: predicted

The confusion matrix the package returns for y_true = [0, 1, 2, 2, 1, 0, 2, 1] and y_pred = [0, 2, 2, 2, 1, 0, 1, 1], each row divided by its total here (so the diagonal is each class's recall); accuracy 0.75, 6 correct.

```
data_science_ml_ai999x.solve('ml_multiclass_confusion_matrix', y_true=[0, 1, 2, 2, 1, 0, 2, 1], y_pred=[0, 2, 2, 2, 1, 0, 1, 1])
# -> classes = [0, 1, 2], matrix = [[2, 0, 0], [0, 2, 1], [0, 1, 2]], n_correct = 6, accuracy = 0.75  (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_multiclass_confusion_matrix', y_true=[3, 1, 2, 3, 3, 1], y_pred=[3, 3, 2, 3, 1, 1])
# -> classes = [1, 2, 3], matrix = [[1, 0, 1], [0, 1, 0], [1, 0, 2]], n_correct = 4, accuracy = 0.666667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiclass confusion matrix: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1]	SOLVED	classes = [0, 1, 2], matrix = [[2, 0, 0], [0, 2, 1], [0, 1, 2]], n_correct = 6, accuracy = 0.75

question	outcome	result
Multiclass confusion matrix: y_true = [3, 1, 2, 3, 3, 1], y_pred = [3, 3, 2, 3, 1, 1]	SOLVED	classes = [1, 2, 3], matrix = [[1, 0, 1], [0, 1, 0], [1, 0, 2]], n_correct = 4, accuracy = 0.666667
Multiclass confusion matrix: y_pred = [0, 2, 2, 2, 1, 0, 1, 1]	NEEDS_INPUT	ml_multiclass_confusion_matrix needs y_true (true class labels (whole numbers)). Example: solve('ml_multiclass_confusion
Multiclass confusion matrix: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Multiclass confusion matrix: y_true = [0, 1.5, 2], y_pred = [0, 1, 2]	INVALID_INPUT	Multi-class confusion matrix: every label in y_true must be a whole number.

Averaged precision, recall and F1 (multi-class), `ml_precision_recall_f1_averaged`

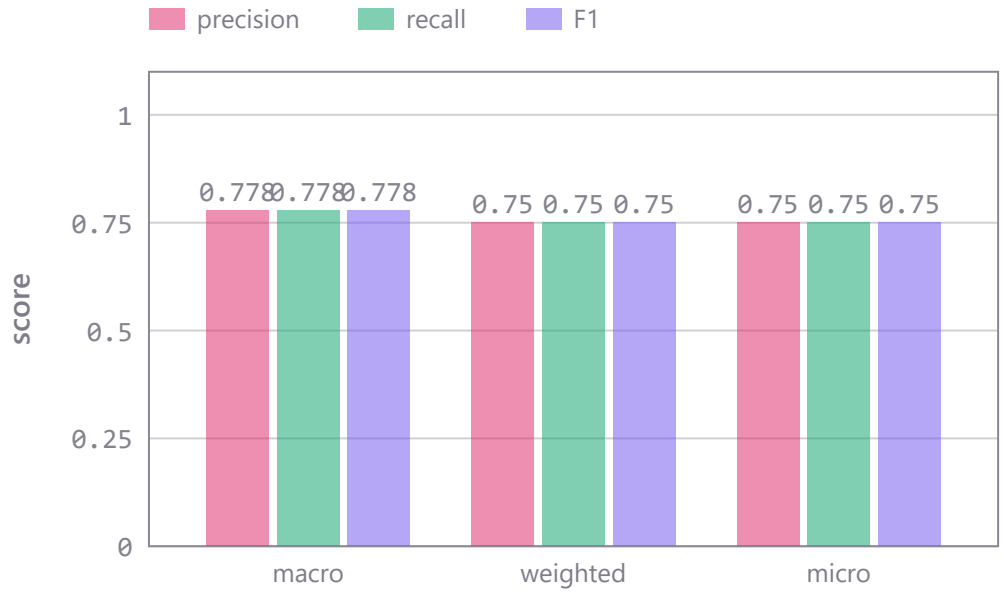
Multi-class precision, recall and F1 averaged over classes: macro (unweighted), weighted (by support) or micro (pooled counts).

$$F_1^{\text{macro}} = \frac{1}{K} \sum_k \frac{2P_k R_k}{P_k + R_k}$$

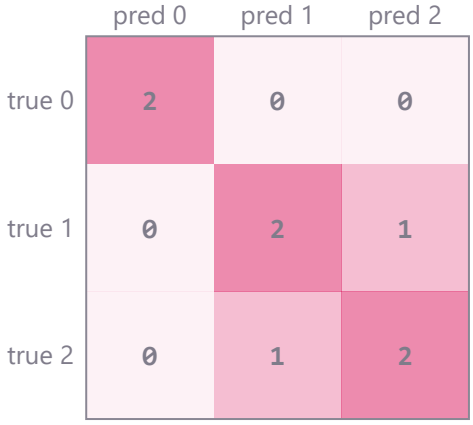
input	meaning	condition
`y_true`	true class labels (whole numbers)	
`y_pred`	predicted class labels (whole numbers)	
`average`	averaging: macro, weighted or micro	

output	meaning
`precision`	averaged precision
`recall`	averaged recall
`f1`	averaged F1 (macro and weighted: the average of the per-class F1)

- **Method:** per-class counts over the sorted union of classes, exact fractions; an undefined per-class ratio counts as 0
- **Assumes:** labels are whole numbers; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Precision, recall and F1 of y_true = [0, 1, 2, 2, 1, 0, 2, 1] against y_pred = [0, 2, 2, 2, 1, 0, 1, 1] under each averaging, one run each: macro: P 0.777778, R 0.777778, F1 0.777778; weighted: P 0.75, R 0.75, F1 0.75; micro: P 0.75, R 0.75, F1 0.75.



rows: true class, columns: predicted class

The confusion matrix of y_true = [0, 1, 2, 2, 1, 0, 2, 1] and y_pred = [0, 2, 2, 2, 1, 0, 1, 1] (from the package): precision is read down each column, recall along each row.

```
data_science_ml_ai999x.solve('ml_precision_recall_f1_averaged', y_true=[0, 1, 2, 2, 1, 0, 2, 1],
y_pred=[0, 2, 2, 2, 1, 0, 1, 1], average='macro')
# -> precision = 0.777778, recall = 0.777778, f1 = 0.777778   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_precision_recall_f1_averaged', y_true=[0, 1, 2, 2, 1, 0, 2, 1],
y_pred=[0, 2, 2, 2, 1, 0, 1, 1], average='weighted')
# -> precision = 0.75, recall = 0.75, f1 = 0.75   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_precision_recall_f1_averaged', y_true=[0, 1, 2, 2, 1, 0, 2, 1],
y_pred=[0, 2, 2, 2, 1, 0, 1, 1], average='micro')
# -> precision = 0.75, recall = 0.75, f1 = 0.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Averaged precision recall F1: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1], average = macro	SOLVED	precision = 0.777778, recall = 0.777778, f1 = 0.777778
Averaged precision recall F1: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1], average = weighted	SOLVED	precision = 0.75, recall = 0.75, f1 = 0.75
Averaged precision recall F1: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1], average = micro	SOLVED	precision = 0.75, recall = 0.75, f1 = 0.75
Averaged precision recall F1: y_pred = [0, 2, 2, 2, 1, 0, 1, 1], average = macro	NEEDS_INPUT	ml_precision_recall_f1_averaged needs y_true (true class labels (whole numbers)). Example: solve('ml_precision_recall_f1
Averaged precision recall F1: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1], average = macro, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Averaged precision recall F1: y_true = [0, 1, 2], y_pred = [0, 1, 2], average = samples	INVALID_INPUT	average: one of macro, weighted, micro; got 'samples'.

Per-class precision, recall, F1 and support, `ml_classification_report`

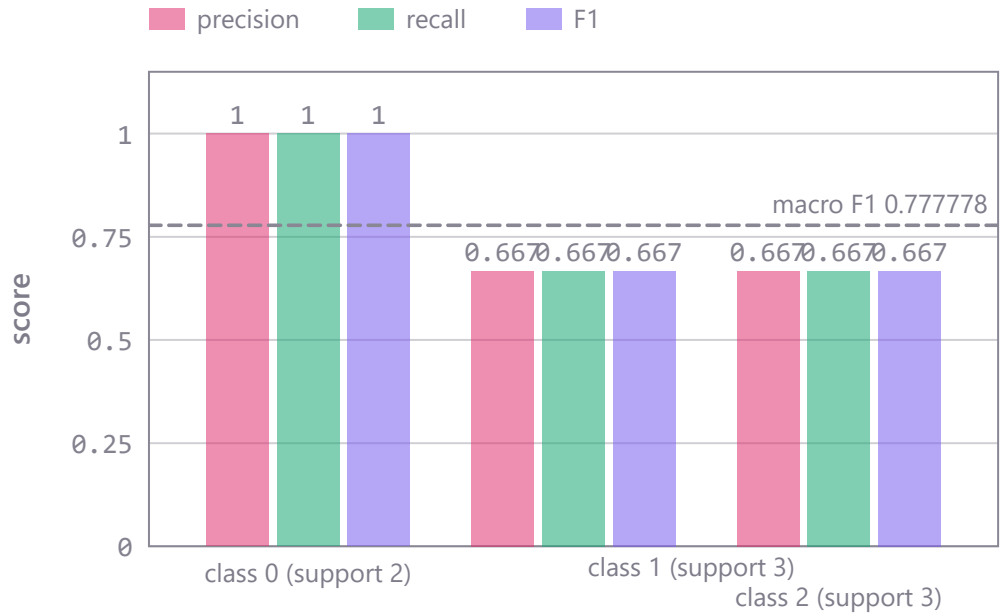
A classification report: precision, recall, F1 and support for every class, with the macro-averaged F1.

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}$$

input	meaning	condition
`y_true`	true class labels (whole numbers)	
`y_pred`	predicted class labels (whole numbers)	

output	meaning
`classes`	classes in sorted order
`precision`	precision of each class
`recall`	recall of each class
`f1`	F1 of each class
`support`	number of true members of each class
`macro_f1`	mean of the per-class F1

- **Method:** per-class counts, exact fractions; an undefined ratio counts as 0
- **Assumes:** labels are whole numbers; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Per-class precision, recall and F1 the package returns for y_true = [0, 1, 2, 2, 1, 0, 2, 1] and y_pred = [0, 2, 2, 2, 1, 0, 1, 1] (support 2, 3, 3); macro F1 0.777778.

```
data_science_ml_ai999x.solve('ml_classification_report', y_true=[0, 1, 2, 2, 1, 0, 2, 1], y_pred=[0, 2, 2, 2, 1, 0, 1, 1])
# -> classes = [0, 1, 2], precision = [1.0, 0.666666666667, 0.666666666667], recall = [1.0, 0.666666666667, 0.666666666667], f1 = [1.0, 0.666666666667, 0.666666666667], support = [2, 3, 3], macro_f1 = 0.777778   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_classification_report', y_true=[1, 1, 2, 2, 3, 3, 3], y_pred=[1, 2, 2, 2, 3, 1, 3])
# -> classes = [1, 2, 3], precision = [0.5, 0.666666666667, 1.0], recall = [0.5, 1.0, 0.666666666667], f1 = [0.5, 0.8, 0.8], support = [2, 2, 3], macro_f1 = 0.7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Classification report: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1]	SOLVED	classes = [0, 1, 2], precision = [1.0, 0.666666666667, 0.666666666667], recall = [1.0, 0.666666666667, 0.666666666667], f1 = [1.0, 0.666666666667, 0.666666666667], support = [2, 3, 3], macro_f1 = 0.777778
Classification report: y_true = [1, 1, 2, 2, 3, 3, 3], y_pred = [1, 2, 2, 2, 3, 1, 3]	SOLVED	classes = [1, 2, 3], precision = [0.5, 0.666666666667, 1.0], recall = [0.5, 1.0, 0.666666666667], f1 = [0.5, 0.8, 0.8], support = [2, 2, 3], macro_f1 = 0.7
Classification report: y_pred = [0, 2, 2, 2, 1, 0, 1, 1]	NEEDS_INPUT	ml_classification_report needs y_true (true class labels (whole numbers)). Example: solve('ml_classification_report', y_
Classification report: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2, 2, 1, 0, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Cohen's kappa (unweighted, linear or quadratic), `ml_cohen_kappa`

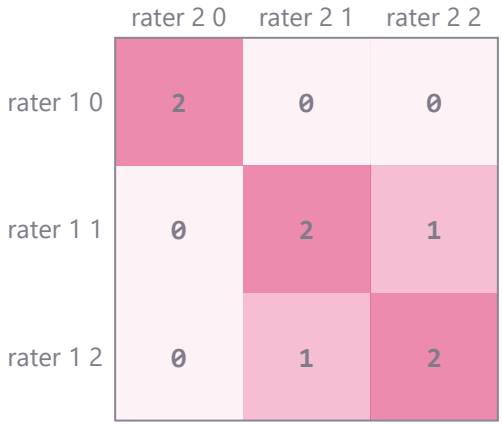
Cohen's kappa, the agreement between two raters (or a classifier and the truth) corrected for chance, optionally weighted by the distance between classes.

$$\kappa = 1 - \frac{\sum_{ij} w_{ij} O_{ij}}{\sum_{ij} w_{ij} E_{ij}}, \quad E_{ij} = \frac{r_i c_j}{n}$$

input	meaning	condition
`y1`	labels of the first rater (whole numbers)	
`y2`	labels of the second rater (whole numbers)	
`weights`	disagreement weights: none, linear or quadratic (distance between class positions in sorted order)	

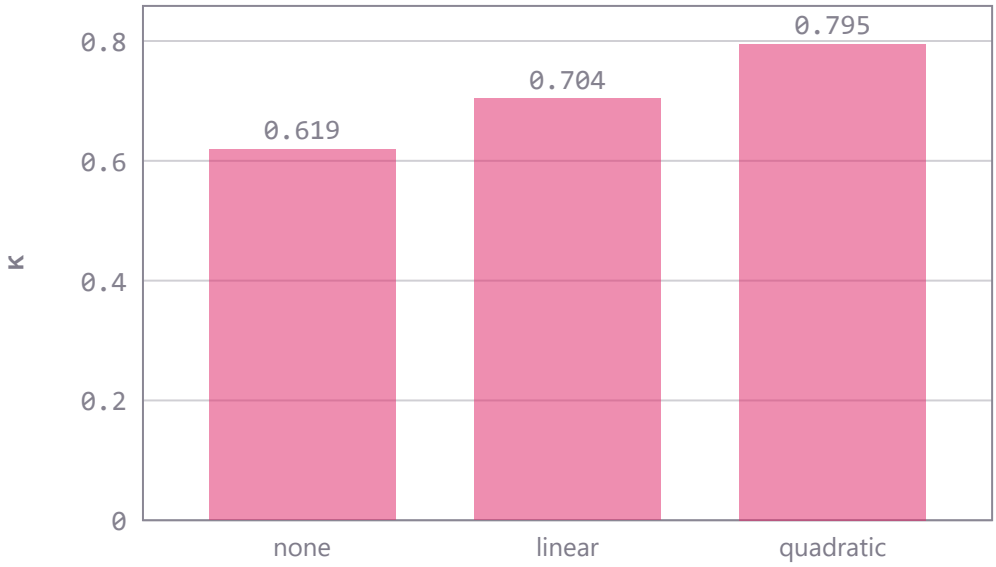
output	meaning
`kappa`	Cohen's kappa

- **Method:** kappa = 1 - (weighted observed disagreement) / (weighted chance disagreement), exact fractions
- **Assumes:** labels are whole numbers; at least two classes; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: rater 1, columns: rater 2

Counts of each pair (rater 1, rater 2) tallied from the stated labels rater 1 = [0, 1, 2, 2, 1, 0, 2, 1] and rater 2 = [0, 2, 2, 2, 1, 0, 1, 1]; 6 of 8 items on the diagonal; returned Cohen's κ = 0.619048 (weights: none).



Cohen's κ of y1 = [0, 1, 2, 2, 1, 0, 2, 1] and y2 = [0, 2, 2, 2, 1, 0, 1, 1] with each disagreement weighting, one run each: none 0.619048, linear 0.703704, quadratic 0.794872.

```
data_science_ml_ai999x.solve('ml_cohen_kappa', y1=[0, 1, 2, 2, 1, 0, 2, 1], y2=[0, 2, 2, 2, 1, 0, 1, 1], weights='none')
# -> kappa = 0.619048   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_cohen_kappa', y1=[1, 2, 3, 3, 2, 1, 3, 2, 1, 1], y2=[1, 2, 3, 2, 2, 1, 3, 3, 2, 1], weights='quadratic')
# -> kappa = 0.769231   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cohen's kappa: y1 = [0, 1, 2, 2, 1, 0, 2, 1], y2 = [0, 2, 2, 2, 1, 0, 1, 1], weights = none	SOLVED	kappa = 0.619048
Cohen's kappa: y1 = [1, 2, 3, 3, 2, 1, 3, 2, 1, 1], y2 = [1, 2, 3, 2, 2, 1, 3, 3, 2, 1], weights = quadratic	SOLVED	kappa = 0.769231
Cohen's kappa: y2 = [0, 2, 2, 2, 1, 0, 1, 1], weights = none	NEEDS_INPUT	ml_cohen_kappa needs y1 (labels of the first rater (whole numbers)). Example: solve('ml_cohen_kappa', y1=[0, 1, 2, 2, 1,
Cohen's kappa: y1 = [0, 1, 2, 2, 1, 0, 2, 1], y2 = [0, 2, 2, 2, 1, 0, 1, 1], weights = none, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Balanced accuracy (multi-class), `ml_balanced_accuracy`

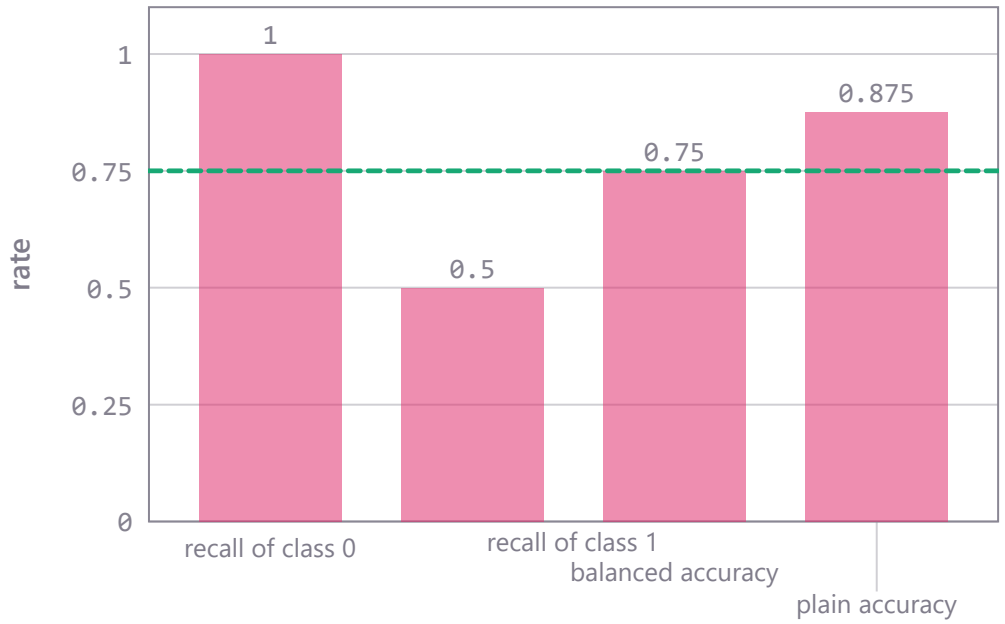
Balanced accuracy, the mean recall over the classes present in the true labels, and its chance-adjusted form.

$$\mathrm{BA} = \frac{1}{K} \sum_{k} \frac{TP_k}{TP_k + FN_k}$$

input	meaning	condition
`y_true`	true class labels (whole numbers)	
`y_pred`	predicted class labels (whole numbers)	

output	meaning
`balanced_accuracy`	mean per-class recall over the true classes
`adjusted`	(balanced accuracy - 1/K) / (1 - 1/K), K = number of true classes

- **Method:** per-class recall as exact fractions, averaged
- **Assumes:** at least two true classes; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



For y_true = [0, 0, 0, 0, 0, 0, 1, 1] and y_pred = [0, 0, 0, 0, 0, 0, 0, 1]: the recall of each class (from the package's classification report: class 0 1, class 1 0.5); their mean is the returned balanced accuracy 0.75 (adjusted 0.5), against the plain accuracy 0.875 that the majority class inflates.

```
data_science_ml_ai999x.solve('ml_balanced_accuracy', y_true=[0, 0, 0, 0, 0, 0, 1, 1], y_pred=[0, 0, 0, 0, 0, 0, 0, 1])
# -> balanced_accuracy = 0.75, adjusted = 0.5   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_balanced_accuracy', y_true=[0, 1, 2, 2, 1, 0, 2, 1], y_pred=[0, 2, 2, 2, 1, 0, 1, 1])
# -> balanced_accuracy = 0.777778, adjusted = 0.666667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Balanced accuracy: y_true = [0, 0, 0, 0, 0, 0, 1, 1], y_pred = [0, 0, 0, 0, 0, 0, 0, 1]	SOLVED	balanced_accuracy = 0.75, adjusted = 0.5
Balanced accuracy: y_true = [0, 1, 2, 2, 1, 0, 2, 1], y_pred = [0, 2, 2,	SOLVED	balanced_accuracy = 0.777778, adjusted = 0.666667

question	outcome	result
2, 1, 0, 1, 1]		
Balanced accuracy: y_pred = [0, 0, 0, 0, 0, 0, 0, 1]	NEEDS_INPUT	ml_balanced_accuracy needs y_true (true class labels (whole numbers)). Example: solve('ml_balanced_accuracy', y_true=[0,
Balanced accuracy: y_true = [0, 0, 0, 0, 0, 0, 1, 1], y_pred = [0, 0, 0, 0, 0, 0, 0, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Balanced accuracy: y_true = [1, 1, 1], y_pred = [1, 0, 1]	INVALID_INPUT	Balanced accuracy (multi-class): the adjusted balanced accuracy needs at least two true classes.

Binary log loss (cross-entropy), `ml_log_loss_binary`

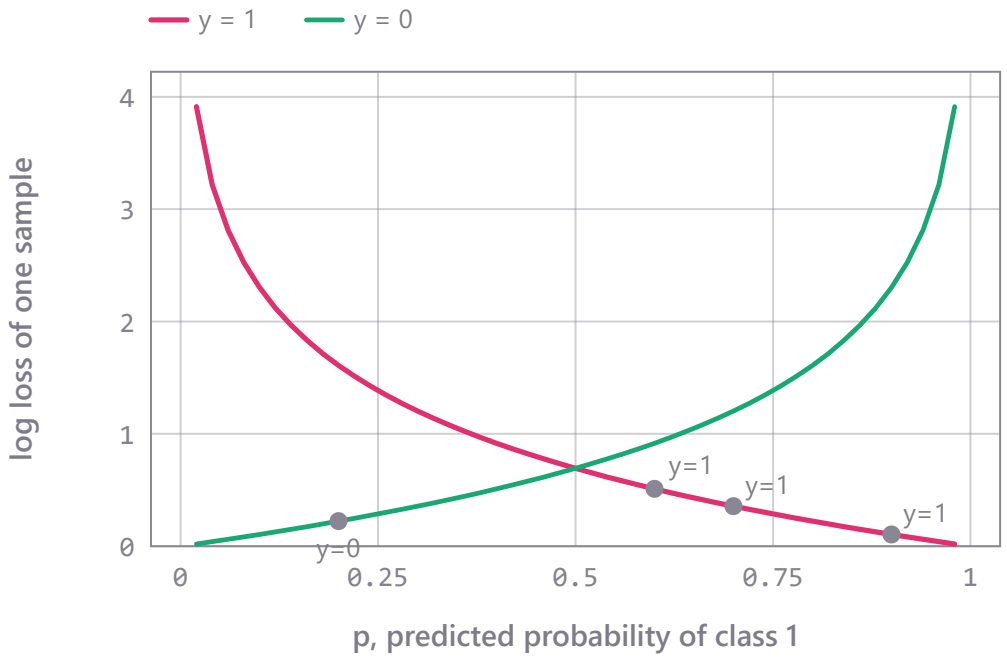
The mean binary cross-entropy (log loss) of predicted probabilities of the positive class, in nats and in bits.

$$\mathcal{L} = -\frac{1}{n}\sum_i [y_i \ln p_i + (1 - y_i)\ln(1 - p_i)]$$

input	meaning	condition
`y_true`	true labels (0 or 1)	
`p`	predicted probability of class 1 for each sample $(0 < p < 1)$	

output	meaning
`log_loss`	mean log loss in nats
`log_loss_bits`	mean log loss in bits

- **Method:** -mean(y log $p + (1 - y)$ log(1 - p)) with log1p for the negative class
- **Assumes:** labels 0 or 1; $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Log loss of one sample as its predicted probability runs from 0.02 to 0.98, for y = 1 and y = 0 (49 runs each). Marked: the worked example's samples (y_true [1, 0, 1, 1], p [0.9, 0.2, 0.7, 0.6]), one run each; returned mean 0.299001 nats (0.431367 bits).

```
data_science_ml_ai999x.solve('ml_log_loss_binary', y_true=[1, 0, 1, 1], p=[0.9, 0.2, 0.7, 0.6])
# -> log_loss = 0.299001, log_loss_bits = 0.431367   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_log_loss_binary', y_true=[0, 0, 1], p=[0.1, 0.4, 0.8])
# -> log_loss = 0.279777, log_loss_bits = 0.403632   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Binary log loss: y_true = [1, 0, 1, 1], p = [0.9, 0.2, 0.7, 0.6]	SOLVED	log_loss = 0.299001, log_loss_bits = 0.431367
Binary log loss: y_true = [0, 0, 1], p = [0.1, 0.4, 0.8]	SOLVED	log_loss = 0.279777, log_loss_bits = 0.403632
Binary log loss: p = [0.9, 0.2, 0.7, 0.6]	NEEDS_INPUT	ml_log_loss_binary needs y_true (true labels (0 or 1)). Example: solve('ml_log_loss_binary', y_true=[1, 0, 1, 1], p=[0.9
Binary log loss: y_true = [1, 0, 1, 1], p = [0.9, 0.2, 0.7, 0.6], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Binary log loss: y_true = [1, 0], p = [1, 0.2]	INVALID_INPUT	Binary log loss (cross-entropy): every probability must lie strictly between 0 and 1.

Multi-class log loss (categorical cross-entropy), `ml_log_loss_multiclass`

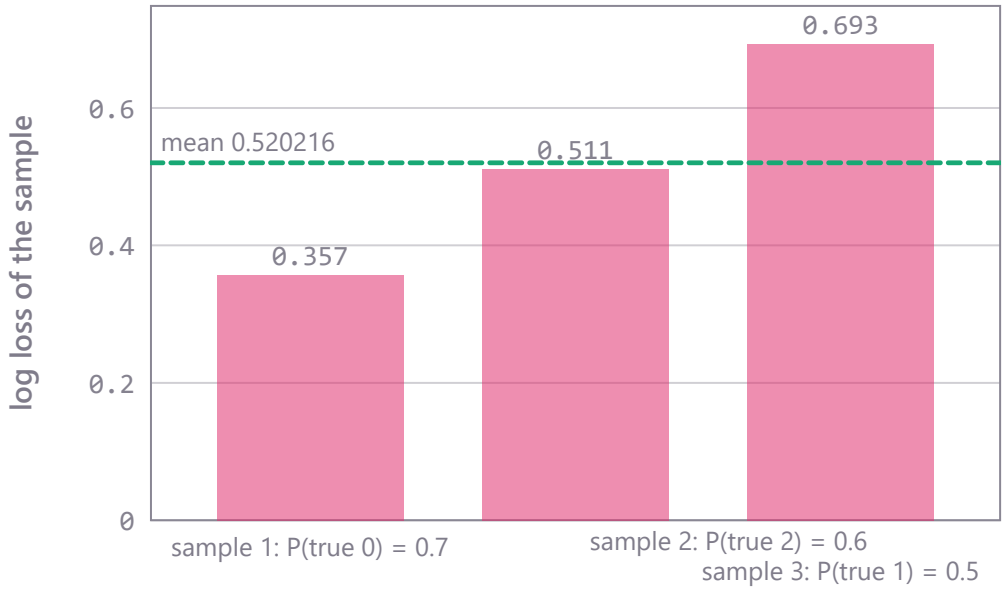
The mean categorical cross-entropy of predicted class probabilities (rows of P), with the perplexity exp(loss).

$$\mathcal{L} = -\frac{1}{n}\sum_{i} \ln P_{i,y_i}$$

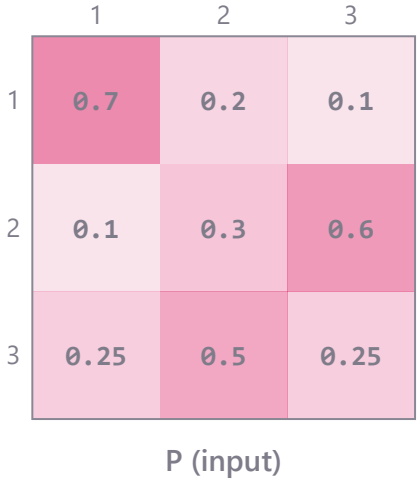
input	meaning	condition
`y_true`	true class index of each sample (0 to K-1)	
`P`	predicted probabilities: one row per sample, one column per class, rows sum to 1	

output	meaning
`log_loss`	mean log loss in nats
`perplexity`	exp(log loss)

- **Method:** -mean of ln P[i, y_i]
- **Assumes:** labels are column indices of P; rows of P sum to 1; P[i, y_i] > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Log loss of each sample of the stated probabilities P = [[0.7, 0.2, 0.1], [0.1, 0.3, 0.6], [0.25, 0.5, 0.25]] with true classes [0, 2, 1], one run per sample (−ln of the probability given to the true class); mean = returned log loss 0.520216, perplexity 1.68239.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): P (input). Returned with them by data-science-ml-ai999x: log_loss = 0.520216, perplexity = 1.68239. Other inputs: y_true = [0, 2, 1].

```
data_science_ml_ai999x.solve('ml_log_loss_multiclass', y_true=[0, 2, 1], P=[[0.7, 0.2, 0.1], [0.1, 0.3, 0.6], [0.25, 0.5, 0.25]])
# -> log_loss = 0.520216, perplexity = 1.68239   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_log_loss_multiclass', y_true=[1, 1], P=[[0.2, 0.8], [0.6, 0.4]])
# -> log_loss = 0.569717, perplexity = 1.76777   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiclass log loss: y_true = [0, 2, 1], P = [[0.7, 0.2, 0.1], [0.1, 0.3, 0.6],	SOLVED	log_loss = 0.520216, perplexity = 1.68239

question	outcome	result
[0.25, 0.5, 0.25]]		
Multiclass log loss: y_true = [1, 1], P = [[0.2, 0.8], [0.6, 0.4]]	SOLVED	log_loss = 0.569717, perplexity = 1.76777
Multiclass log loss: P = [[0.7, 0.2, 0.1], [0.1, 0.3, 0.6], [0.25, 0.5, 0.25]]	NEEDS_INPUT	ml_log_loss_multiclass needs y_true (true class index of each sample (0 to K-1)). Example: solve('ml_log_loss_multiclass
Multiclass log loss: y_true = [0, 2, 1], P = [[0.7, 0.2, 0.1], [0.1, 0.3, 0.6], [0.25, 0.5, 0.25]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Multiclass log loss: y_true = [0, 3], P = [[0.7, 0.2, 0.1], [0.1, 0.3, 0.6]]	INVALID_INPUT	Multi-class log loss (categorical cross-entropy): every label must be a column index of P (0 to K-1).

Brier score, `ml_brier_score`

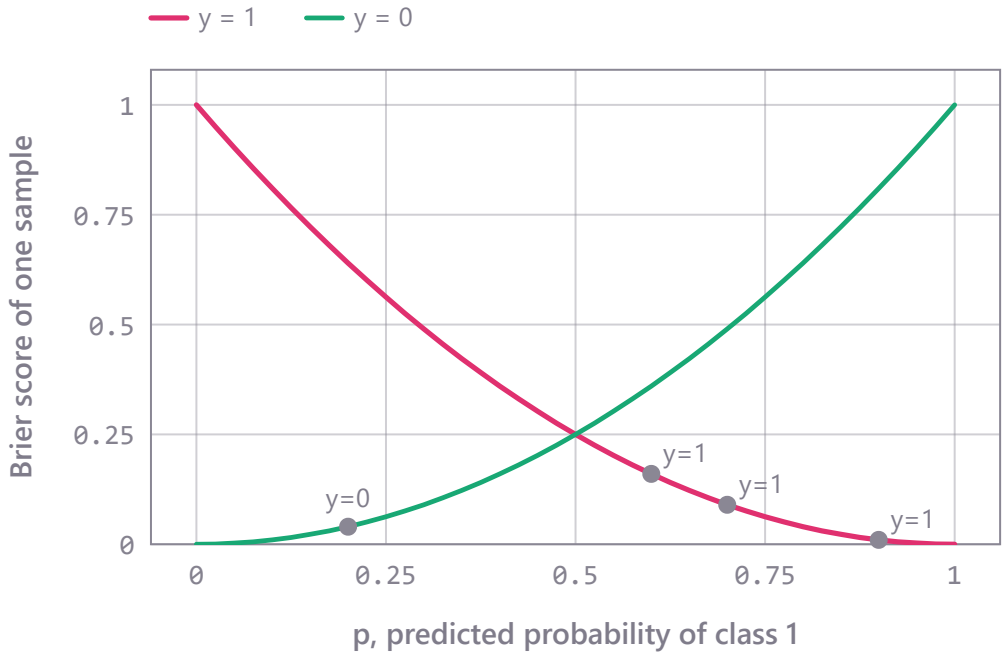
The Brier score, the mean squared difference between predicted probabilities of the positive class and the $0/1$ outcomes.

$$\mathrm{BS} = \frac{1}{n}\sum_i (p_i - y_i)^2$$

input	meaning	condition
`y_true`	true labels (0 or 1)	
`p`	predicted probability of class 1 (0 to 1)	

output	meaning
`brier`	mean squared error of the probabilities

- **Method:** exact mean of $(p - y)^2$
- **Assumes:** labels 0 or 1; $0 \le p \le 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Brier score of one sample as its predicted probability runs from 0 to 1, for y = 1 and y = 0 (41 runs each). Marked: the worked example's samples (y_true [1, 0, 1, 1], p [0.9, 0.2, 0.7, 0.6]), one run each; their mean is the returned Brier score 0.075.

```
data_science_ml_ai999x.solve('ml_brier_score', y_true=[1, 0, 1, 1], p=[0.9, 0.2, 0.7, 0.6])
# -> brier = 0.075   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_brier_score', y_true=[0, 1, 0], p=[0.3, 0.6, 0.1])
# -> brier = 0.0866667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Brier score: y_true = [1, 0, 1, 1], p = [0.9, 0.2, 0.7, 0.6]	SOLVED	brier = 0.075
Brier score: y_true = [0, 1, 0], p = [0.3, 0.6, 0.1]	SOLVED	brier = 0.0866667
Brier score: p = [0.9, 0.2, 0.7, 0.6]	NEEDS_INPUT	ml_brier_score needs y_true (true labels (0 or 1)). Example: solve('ml_brier_score', y_true=[1, 0, 1, 1], p=[0.9, 0.2, 0
Brier score: y_true = [1, 0, 1, 1], p = [0.9, 0.2, 0.7, 0.6], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Brier score: y_true = [1, 0], p = [1.2, 0.2]	INVALID_INPUT	Brier score: every probability must lie between 0 and 1.

ROC AUC from scores, `ml_roc_auc`

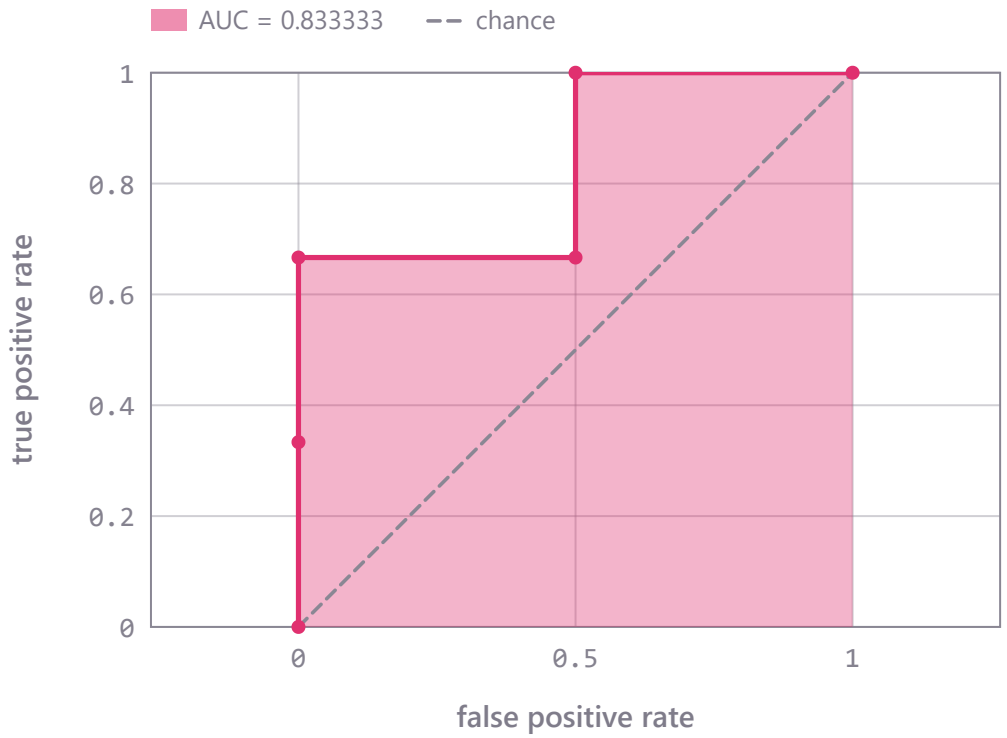
The area under the ROC curve from $0/1$ labels and classifier scores: the probability that a random positive scores above a random negative (ties count one half), with the Gini coefficient 2 AUC - 1.

$$\mathrm{AUC} = \frac{1}{n_+ n_-} \sum_{i \in +} \sum_{j \in -} \left[\mathbb{1}(s_i > s_j) + \tfrac{1}{2} \mathbb{1}(s_i = s_j) \right]$$

input	meaning	condition
`y_true`	true labels (0 or 1)	
`scores`	classifier scores (higher = more likely positive)	

output	meaning
`auc`	area under the ROC curve
`gini`	2 AUC - 1
`n_pos`	number of positives
`n_neg`	number of negatives

- **Method:** exact count over all positive-negative pairs (Mann-Whitney), ties counted $1/2$
- **Assumes:** labels 0 or 1; at least one positive and one negative
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The ROC curve of labels [0, 1, 1, 0, 1] with scores [0.1, 0.9, 0.8, 0.3, 0.2] (points from the package's ROC capability); the shaded area is the returned AUC 0.833333 (Gini 0.666667; 3 positives, 2 negatives).

```
data_science_ml_ai999x.solve('ml_roc_auc', y_true=[0, 1, 1, 0, 1], scores=[0.1, 0.9, 0.8, 0.3, 0.2])
# -> auc = 0.833333, gini = 0.666667, n_pos = 3, n_neg = 2   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_roc_auc', y_true=[0, 0, 1, 1], scores=[0.1, 0.4, 0.35, 0.8])
# -> auc = 0.75, gini = 0.5, n_pos = 2, n_neg = 2   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_roc_auc', y_true=[1, 0, 1, 0], scores=[0.5, 0.5, 0.7, 0.2])
# -> auc = 0.875, gini = 0.75, n_pos = 2, n_neg = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ROC AUC: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2]	SOLVED	auc = 0.833333, gini = 0.666667, n_pos = 3, n_neg = 2
ROC AUC: y_true = [0, 0, 1, 1], scores = [0.1, 0.4, 0.35, 0.8]	SOLVED	auc = 0.75, gini = 0.5, n_pos = 2, n_neg = 2
ROC AUC: y_true = [1, 0, 1, 0], scores = [0.5, 0.5, 0.7, 0.2]	SOLVED	auc = 0.875, gini = 0.75, n_pos = 2, n_neg = 2
ROC AUC: scores = [0.1, 0.9, 0.8, 0.3, 0.2]	NEEDS_INPUT	ml_roc_auc needs y_true (true labels (0 or 1)). Example: solve('ml_roc_auc', y_true=[0, 1, 1, 0, 1], scores=[0.1, 0.9, 0
ROC AUC: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
ROC AUC: y_true = [1, 1, 1], scores = [0.2, 0.5, 0.7]	INVALID_INPUT	ROC AUC from scores: ROC AUC needs at least one positive and one negative sample.

ROC curve points, `ml_roc_curve`

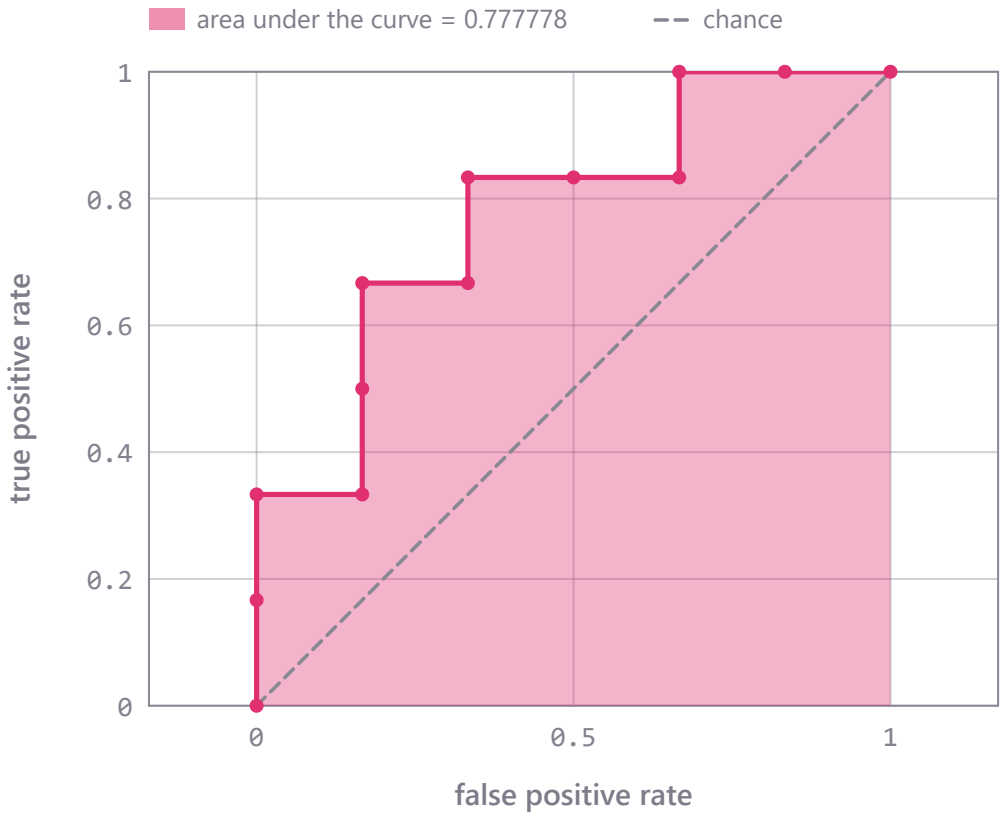
The ROC curve of a scored classifier: the false and true positive rates at every distinct score threshold (a score at or above the threshold is positive), and the trapezoidal area under it.

$$\mathrm{TPR}(t) = \frac{TP(t)}{P},\ \mathrm{FPR}(t) = \frac{FP(t)}{N}$$

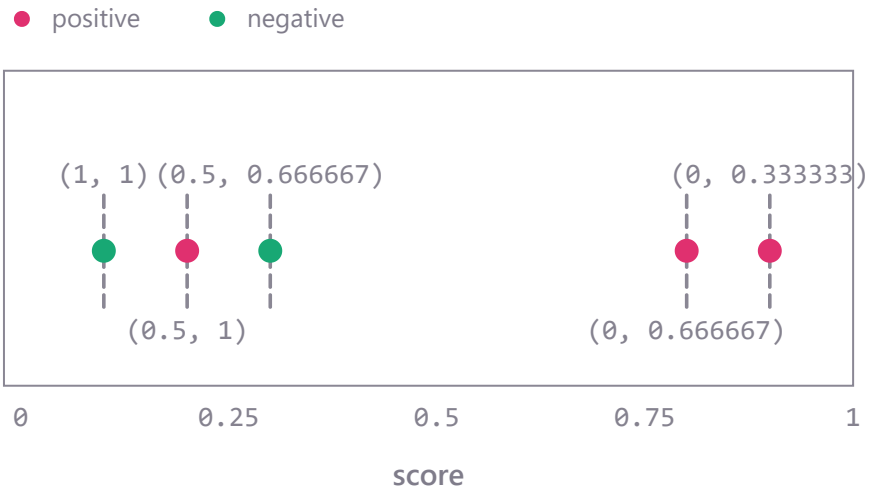
input	meaning	condition
`y_true`	true labels (0 or 1)	
`scores`	classifier scores	

output	meaning
`thresholds`	distinct scores, descending (one per curve point after the origin)
`fpr`	false positive rates, starting at 0
`tpr`	true positive rates, starting at 0
`auc`	trapezoidal area under the curve

- **Method:** sort by score, accumulate TP and FP per distinct score, exact fractions
- **Assumes:** labels 0 or 1; at least one positive and one negative
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The ROC curve the package returns for 12 scored examples (labels 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0; scores 0.95, 0.9, 0.85, 0.8, 0.7, 0.65, 0.6, 0.5, 0.4, 0.35, 0.3, 0.1): true against false positive rate at each of its thresholds, with area under the curve 0.777778.



The stated scores [0.1, 0.9, 0.8, 0.3, 0.2] on one axis, coloured by label [0, 1, 1, 0, 1]; dashed: the returned thresholds 0.9, 0.8, 0.3, 0.2, 0.1, each labelled with the (false, true) positive rate the package returns when everything at or above it is called positive; AUC 0.833333.

```
data_science_ml_ai999x.solve('ml_roc_curve', y_true=[0, 1, 1, 0, 1], scores=[0.1, 0.9, 0.8, 0.3, 0.2])
# -> thresholds = [0.9, 0.8, 0.3, 0.2, 0.1], fpr = [0.0, 0.0, 0.0, 0.5, 0.5, 1.0], tpr = [0.0, 0.333333333333, 0.666666666667, 0.666666666667, 1.0, 1.0], auc = 0.833333   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_roc_curve', y_true=[1, 0, 1, 0, 0, 1], scores=[3, 1, 2, 2, 0, 4])
# -> thresholds = [4.0, 3.0, 2.0, 1.0, 0.0], fpr = [0.0, 0.0, 0.0, 0.333333333333, 0.666666666667, 1.0], tpr = [0.0, 0.333333333333, 0.666666666667, 1.0, 1.0, 1.0], auc = 0.944444   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ROC curve: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2]	SOLVED	thresholds = [0.9, 0.8, 0.3, 0.2, 0.1], fpr = [0.0, 0.0, 0.0, 0.5, 0.5, 1.0], tpr = [0.0, 0.333333333333, 0.666666666667, 0.666666666667, 1.0, 1.0], auc = 0.833333
ROC curve: y_true = [1, 0, 1, 0, 0, 1], scores = [3, 1, 2, 2, 0, 4]	SOLVED	thresholds = [4.0, 3.0, 2.0, 1.0, 0.0], fpr = [0.0, 0.0, 0.0, 0.333333333333, 0.666666666667, 1.0], tpr = [0.0, 0.333333333333, 0.666666666667, 1.0, 1.0, 1.0], auc = 0.944444
ROC curve: scores = [0.1, 0.9, 0.8, 0.3, 0.2]	NEEDS_INPUT	ml_roc_curve needs y_true (true labels (0 or 1)). Example: solve('ml_roc_curve', y_true=[0, 1, 1, 0, 1], scores=[0.1, 0.
ROC curve: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Average precision and precision-recall AUC, `ml_average_precision`

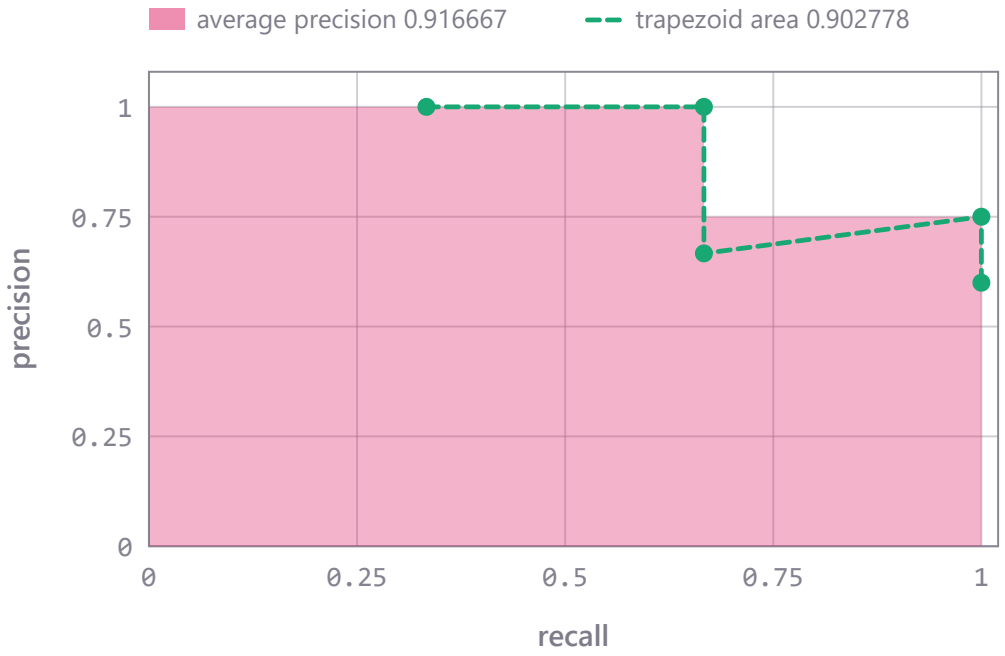
Average precision (the step-wise area under the precision-recall curve, sum of (R_k - R_{$k-1$}) P_k over distinct score thresholds) and the trapezoidal area under the precision-recall curve.

$$\mathrm{AP} = \sum_{k} (R_k - R_{k-1})\, P_k$$

input	meaning	condition
`y_true`	true labels (0 or 1)	
`scores`	classifier scores	

output	meaning
`average_precision`	average precision
`pr_auc_trapezoid`	trapezoidal area under the precision-recall curve, from (recall 0, precision 1)

- **Method:** precision and recall at every distinct threshold, exact fractions
- **Assumes:** labels 0 or 1; at least one positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Precision–recall points of labels [0, 1, 1, 0, 1] with scores [0.1, 0.9, 0.8, 0.3, 0.2], one per score used as the threshold (from the package's threshold metrics); the shaded steps are the returned average precision 0.916667, the dashed path its trapezoid area 0.902778.

```
data_science_ml_ai999x.solve('ml_average_precision', y_true=[0, 1, 1, 0, 1], scores=[0.1, 0.9, 0.8, 0.3, 0.2])
# -> average_precision = 0.916667, pr_auc_trapezoid = 0.902778   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_average_precision', y_true=[1, 0, 1, 0, 0, 1], scores=[3, 1, 2, 2, 0, 4])
# -> average_precision = 0.916667, pr_auc_trapezoid = 0.958333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Average precision: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2]	SOLVED	average_precision = 0.916667, pr_auc_trapezoid = 0.902778
Average precision: y_true = [1, 0, 1, 0, 0, 1], scores = [3, 1, 2, 2, 0, 4]	SOLVED	average_precision = 0.916667, pr_auc_trapezoid = 0.958333
Average precision: scores = [0.1, 0.9, 0.8, 0.3, 0.2]	NEEDS_INPUT	ml_average_precision needs y_true (true labels (0 or 1)). Example: solve('ml_average_precision', y_true=[0, 1, 1, 0, 1],
Average precision: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Top-k accuracy, `ml_top_k_accuracy`

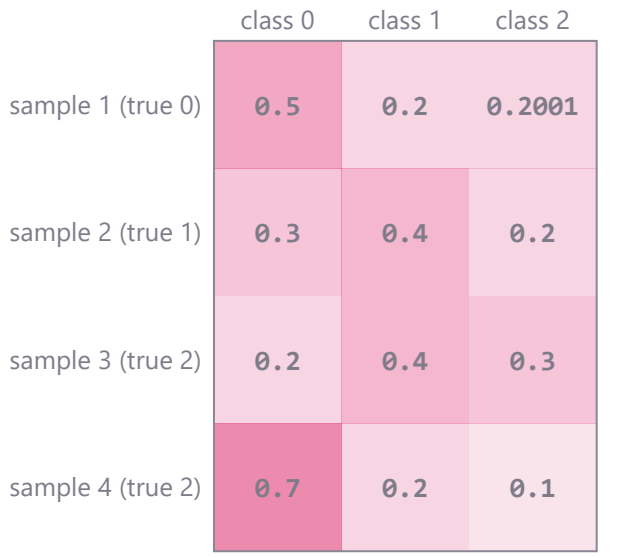
The fraction of samples whose true class is among the k highest-scored classes.

$$\text{top-}k = \frac{1}{n}\sum_{i} \mathbb{1}\left[\text{rank}_i(y_i) \le k\right]$$

input	meaning	condition
`y_true`	true class index of each sample (0 to K-1)	
`S`	class scores: one row per sample, one column per class $(K \geq 3)$	
`k`	number of top classes that count as a hit	

output	meaning
`top_k_accuracy`	fraction of samples with the true class in the top k
`n_hits`	number of such samples

- **Method:** rank of the true class = number of classes scored strictly higher; a hit when rank < k
- **Assumes:** $K \geq 3$ classes; $1 \leq k < K$; distinct scores within each row
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: samples, columns: classes

The stated scores S, one row per sample, labelled with its true class y = [0, 1, 2, 2]; returned top-2 accuracy 0.75 (3 hits).

```
data_science_ml_ai999x.solve('ml_top_k_accuracy', y_true=[0, 1, 2, 2], S=[[0.5, 0.2, 0.2001], [0.3, 0.4, 0.2], [0.2, 0.4, 0.3], [0.7, 0.2, 0.1]], k=2)
# -> top_k_accuracy = 0.75, n_hits = 3   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_top_k_accuracy', y_true=[3, 0, 1], S=[[0.1, 0.2, 0.3, 0.4], [0.2, 0.5, 0.1, 0.25], [0.6, 0.1, 0.2, 0.15]], k=2)
# -> top_k_accuracy = 0.333333, n_hits = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Top-k accuracy: y_true = [0, 1, 2, 2], S = [[0.5, 0.2, 0.2001], [0.3, 0.4, 0.2], [0.2, 0.4, 0.3], [0.7, 0.2, 0.1]], k = 2	SOLVED	top_k_accuracy = 0.75, n_hits = 3
Top-k accuracy: y_true = [3, 0, 1], S = [[0.1, 0.2, 0.3, 0.4], [0.2, 0.5, 0.1, 0.25],	SOLVED	top_k_accuracy = 0.333333, n_hits = 1

question	outcome	result
[0.6, 0.1, 0.2, 0.15]], k = 2		
Top-k accuracy: S = [[0.5, 0.2, 0.2001], [0.3, 0.4, 0.2], [0.2, 0.4, 0.3], [0.7, 0.2, 0.1]], k = 2	NEEDS_INPUT	ml_top_k_accuracy needs y_true (true class index of each sample (0 to K-1)). Example: solve('ml_top_k_accuracy', y_true=
Top-k accuracy: y_true = [0, 1, 2, 2], S = [[0.5, 0.2, 0.2001], [0.3, 0.4, 0.2], [0.2, 0.4, 0.3], [0.7, 0.2, 0.1]], k = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Top-k accuracy: y_true = [0, 1], S = [[0.5, 0.2, 0.3], [0.3, 0.4, 0.2]], k = 3	INVALID_INPUT	Top-k accuracy: k must be at least 1 and less than the number of classes.

Hamming loss and subset accuracy (multi-label), `ml_hamming_loss`

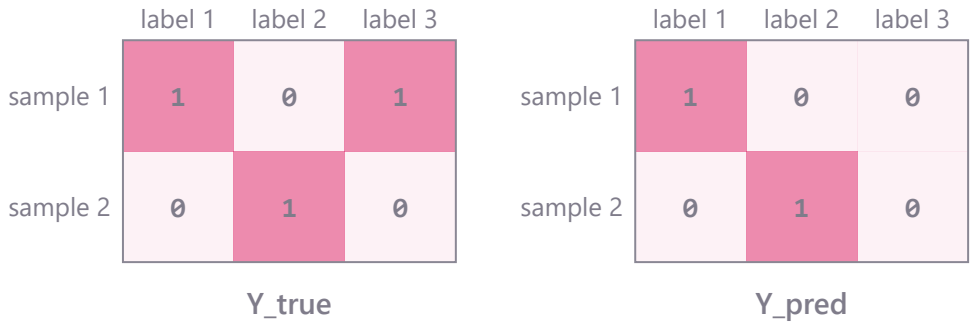
For multi-label predictions (0/1 indicator matrices): the Hamming loss (fraction of wrong labels) and the subset accuracy (fraction of samples with every label right).

$$\mathrm{HL} = \frac{1}{nL}\sum_{i,j}\mathbb{1}(Y_{ij} \neq \hat{Y}_{ij})$$

input	meaning	condition
`Y_true`	true label indicators: one row per sample, one column per label (0 or 1)	
`Y_pred`	predicted label indicators, same shape	

output	meaning
`hamming_loss`	wrong label entries / (samples x labels)
`subset_accuracy`	fraction of rows predicted exactly
`n_mismatches`	number of wrong label entries

- **Method:** count mismatching entries and exactly matching rows
- **Assumes:** entries 0 or 1; same shape
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Stated true and predicted label sets of each sample: Y_true = [[1, 0, 1], [0, 1, 0]], Y_pred = [[1, 0, 0], [0, 1, 0]]; returned Hamming loss 0.166667 (1 mismatched entry of 6), subset accuracy 0.5.

```
data_science_ml_ai999x.solve('ml_hamming_loss', Y_true=[[1, 0, 1], [0, 1, 0]], Y_pred=[[1, 0, 0], [0, 1, 0]])
# -> hamming_loss = 0.166667, subset_accuracy = 0.5, n_mismatches = 1   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_hamming_loss', Y_true=[[1, 1], [0, 1], [1, 0]], Y_pred=[[1, 0], [0, 1], [0, 1]])
# -> hamming_loss = 0.5, subset_accuracy = 0.333333, n_mismatches = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hamming loss: Y_true = [[1, 0, 1], [0, 1, 0]], Y_pred = [[1, 0, 0], [0, 1, 0]]	SOLVED	hamming_loss = 0.166667, subset_accuracy = 0.5, n_mismatches = 1
Hamming loss: Y_true = [[1, 1], [0, 1], [1, 0]], Y_pred = [[1, 0], [0, 1], [0, 1]]	SOLVED	hamming_loss = 0.5, subset_accuracy = 0.333333, n_mismatches = 3
Hamming loss: Y_pred = [[1, 0, 0], [0, 1, 0]]	NEEDS_INPUT	ml_hamming_loss needs Y_true (true label indicators: one row per sample, one column per label (0 or 1)). Example: solve(
Hamming loss: Y_true = [[1, 0, 1], [0, 1, 0]], Y_pred = [[1, 0, 0], [0, 1, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Hamming loss: Y_true = [[1, 2], [0, 1]], Y_pred = [[1, 0], [0, 1]]	INVALID_INPUT	Hamming loss and subset accuracy (multi-label): entries must be 0 or 1.

Balanced class weights, `ml_balanced_class_weights`

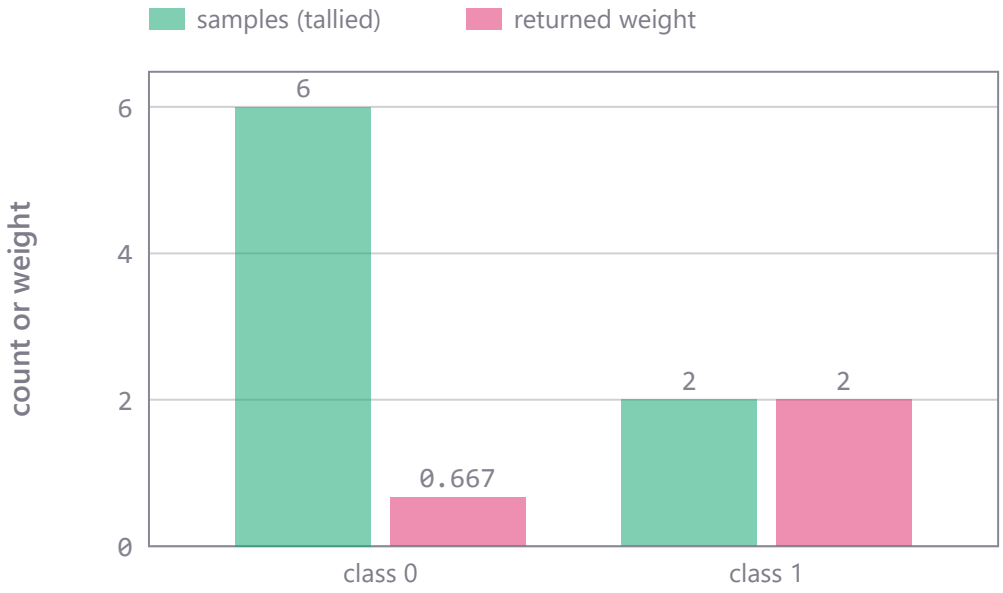
Class weights that balance an imbalanced training set: n / (K n_c) for each class c.

$$w_c = \frac{n}{K\, n_c}$$

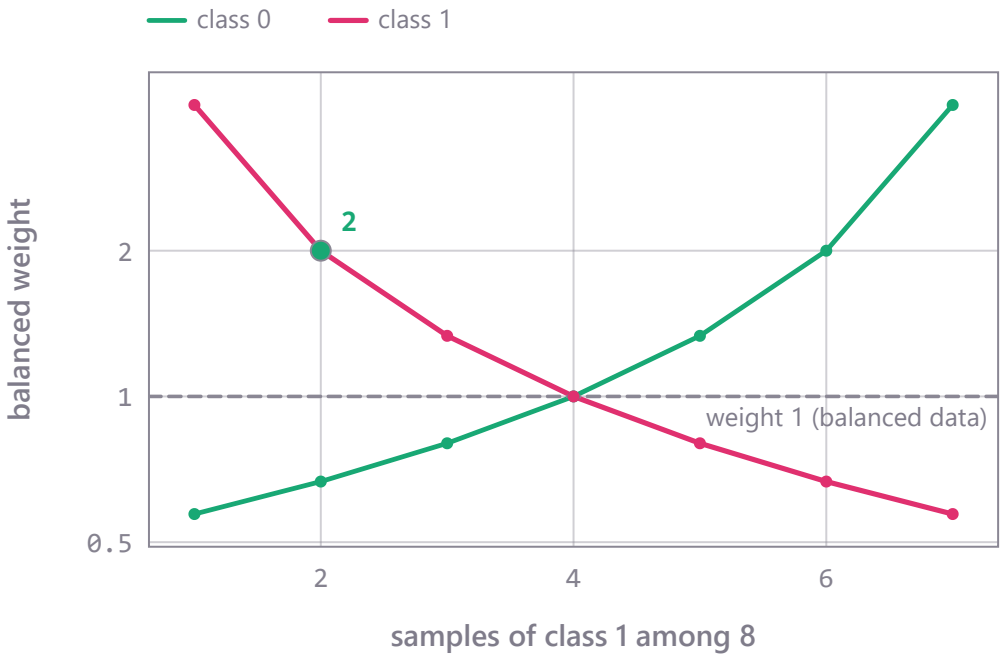
input	meaning	condition
`y`	training labels (whole numbers)	

output	meaning
`classes`	classes in sorted order
`weights`	weight of each class, n / (K n_c)

- **Method:** exact fractions n / (K n_c)
- **Assumes:** labels are whole numbers
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Class counts tallied from y = [0, 0, 0, 0, 0, 0, 1, 1] (class 0: 6, class 1: 2) and the returned balanced weights 0.666667, 2: weight × count is the same for every class.



Balanced weights of a two-class label list of 8 samples (the worked example's size) with k samples of class 1, k = 1 … 7 (one run each; logarithmic axis): the rare class's weight grows as n / (2 k); at k = n/2 both are 1. Marked: the worked example, k = 2: weights 0.6667 and 2.

```
data_science_ml_ai999x.solve('ml_balanced_class_weights', y=[0, 0, 0, 0, 0, 0, 1, 1])
# -> classes = [0, 1], weights = [0.666666666667, 2.0]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_balanced_class_weights', y=[1, 2, 2, 3, 3, 3])
# -> classes = [1, 2, 3], weights = [2.0, 1.0, 0.666666666667]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Balanced class weights: y = [0, 0, 0, 0, 0, 0, 1, 1]	SOLVED	classes = [0, 1], weights = [0.666666666667, 2.0]
Balanced class weights: y = [1, 2, 2, 3, 3, 3]	SOLVED	classes = [1, 2, 3], weights = [2.0, 1.0, 0.666666666667]
Balanced class weights:	NEEDS_INPUT	ml_balanced_class_weights needs y (training labels (whole numbers)). Example: solve('ml_balanced_class_weights', y=[0, 0
Balanced class weights: y = [0, 0, 0, 0, 0, 0, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Expected calibration error, `ml_expected_calibration_error`

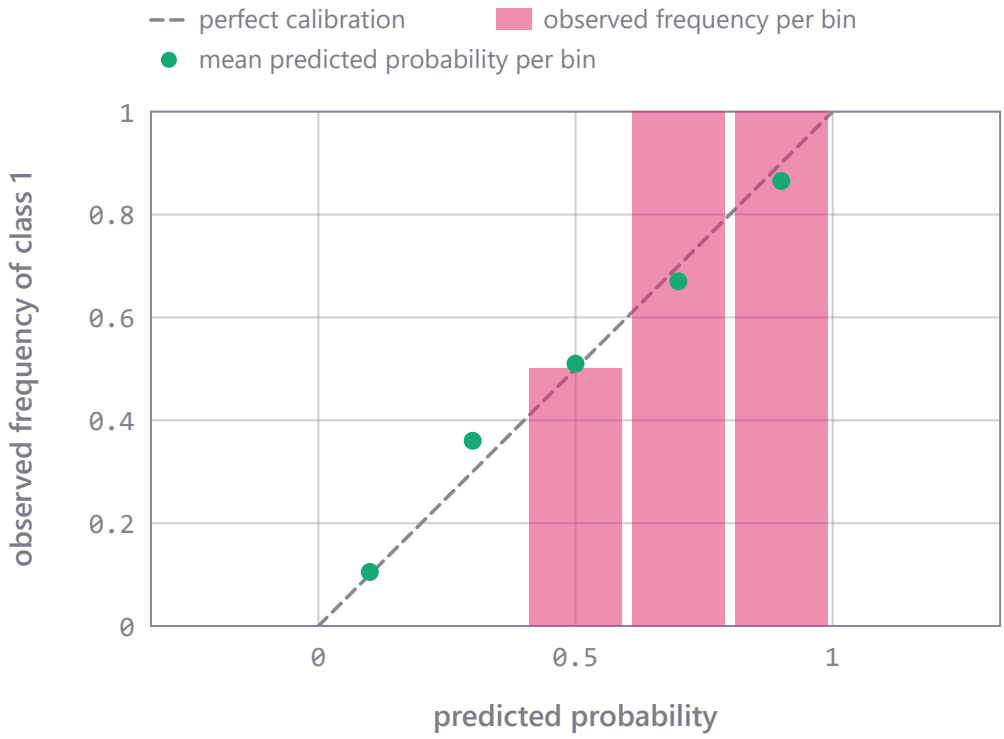
The expected calibration error (support-weighted mean gap between accuracy and mean confidence over equal-width probability bins) and the maximum calibration error.

$$\mathrm{ECE} = \sum_{b} \frac{n_b}{n} |\bar{y}_b - \bar{p}_b|$$

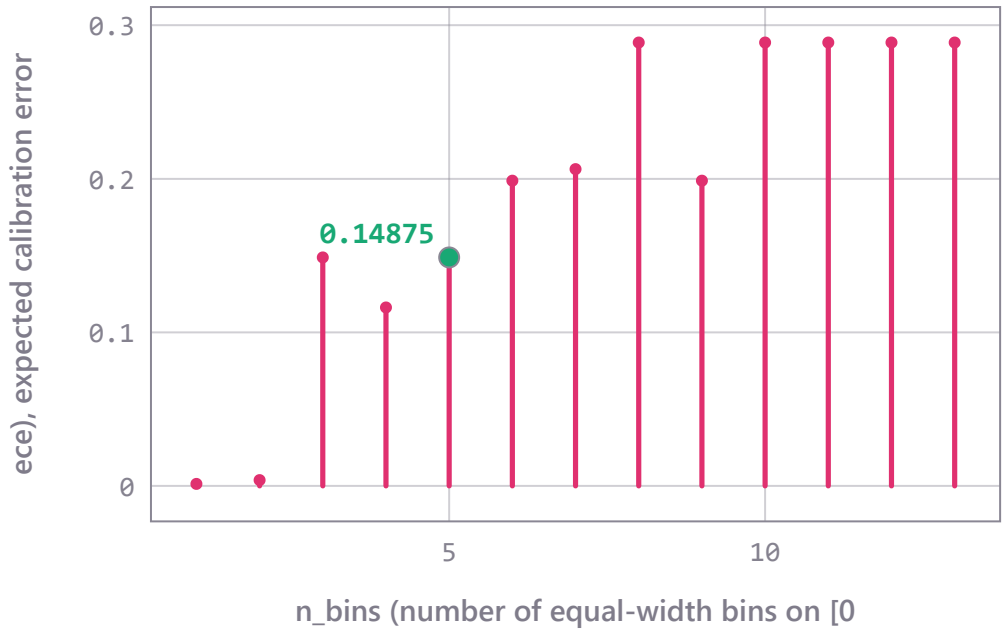
input	meaning	condition
`y_true`	true labels (0 or 1)	
`p`	predicted probability of class 1	
`n_bins`	number of equal-width bins on [0, 1]	

output	meaning
`ece`	expected calibration error
`mce`	maximum calibration error over non-empty bins
`n_nonempty_bins`	number of non-empty bins

- **Method:** bin b = floor(p * n_bins) (p = 1 in the last bin); per bin |mean(y) - mean(p)|, weighted by the bin's share
- **Assumes:** labels 0 or 1; $0 \le p \le 1$; bins are [b/n, (b+1)/n); a probability on a bin edge goes to the upper bin
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Reliability diagram of the stated predictions p = [0.92, 0.13, 0.67, 0.81, 0.36, 0.08, 0.44, 0.58] for labels [1, 0, 1, 1, 0, 0, 1, 0] in 5 equal-width bins: the observed frequency of class 1 and the mean prediction of each bin, tallied here from the stated lists. Returned: expected calibration error 0.14875, maximum 0.36, 5 non-empty bins.



ece (expected calibration error) against n_bins (number of equal-width bins on [0, 1]) from 1 to 13. Other inputs as in the worked example: y_true = [1, 0, 1, 1, 0, 0, 1, 0], p = [0.92, 0.13, 0.67, 0.81, 0.36, 0.08, 0.44, 0.58]. 13 points computed by data-science-ml-ai999x; marked: the worked example, n_bins = 5, ece = 0.14875.

```
data_science_ml_ai999x.solve('ml_expected_calibration_error', y_true=[1, 0, 1, 1, 0, 0, 1, 0], p=
[0.92, 0.13, 0.67, 0.81, 0.36, 0.08, 0.44, 0.58], n_bins=5)
# -> ece = 0.14875, mce = 0.36, n_nonempty_bins = 5   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_expected_calibration_error', y_true=[0, 1, 1, 0, 1], p=[0.15, 0.85, 0.65, 0.45, 0.95], n_bins=2)
# -> ece = 0.23, mce = 0.3, n_nonempty_bins = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Expected calibration error: y_true = [1, 0, 1, 1, 0, 0, 1, 0], p = [0.92, 0.13, 0.67, 0.81, 0.36, 0.08, 0.44, 0.58], n_bins = 5	SOLVED	ece = 0.14875, mce = 0.36, n_nonempty_bins = 5
Expected calibration error: y_true = [0, 1, 1, 0, 1], p = [0.15, 0.85, 0.65, 0.45, 0.95], n_bins = 2	SOLVED	ece = 0.23, mce = 0.3, n_nonempty_bins = 2
Expected calibration error: p = [0.92, 0.13, 0.67, 0.81, 0.36, 0.08, 0.44, 0.58], n_bins = 5	NEEDS_INPUT	ml_expected_calibration_error needs y_true (true labels (0 or 1)). Example: solve('ml_expected_calibration_error', y_tru
Expected calibration error: y_true = [1, 0, 1, 1, 0, 0, 1, 0], p = [0.92, 0.13, 0.67, 0.81, 0.36, 0.08, 0.44, 0.58], n_bins = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Regression metrics (MAE, MSE, RMSE, R^2, explained variance), `ml_regression_metrics`

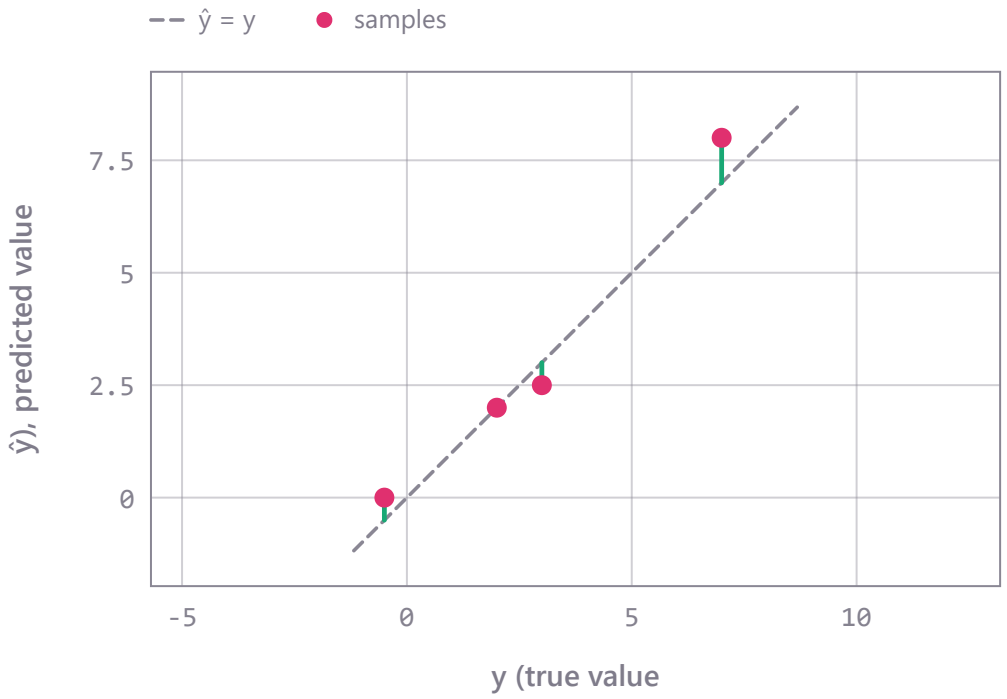
The standard regression error metrics of predictions against true values: mean absolute error, mean squared error, root mean squared error, R^2, explained variance, median absolute error and maximum error.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}, \quad \mathrm{RMSE} = \sqrt{\tfrac{1}{n}\sum (y_i - \hat{y}_i)^2}$$

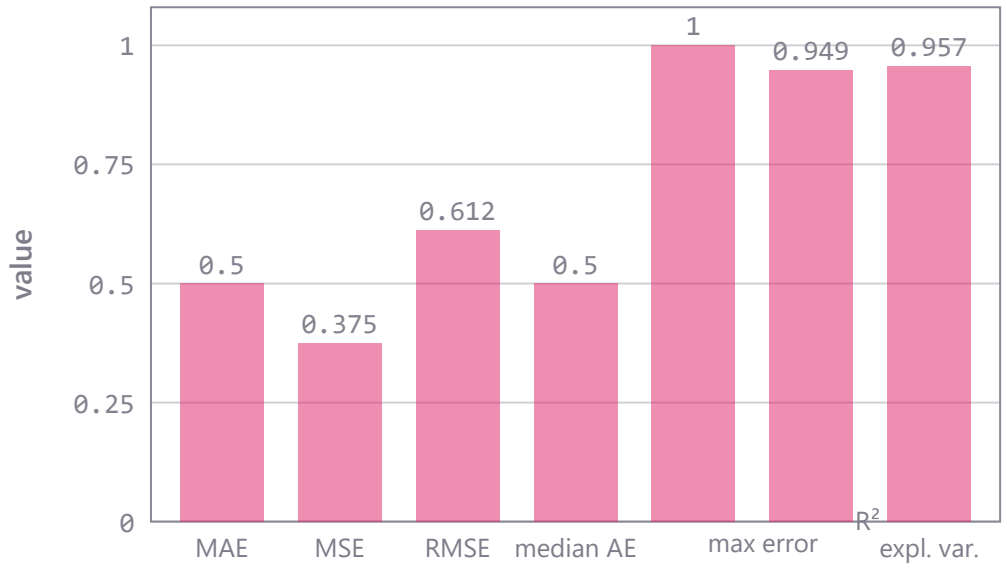
input	meaning	condition
`y_true`	true values	
`y_pred`	predicted values	

output	meaning
`mae`	mean absolute error
`mse`	mean squared error
`rmse`	root mean squared error
`r2`	coefficient of determination 1 - SS_res / SS_tot
`explained_variance`	1 - Var(y - y_hat) / Var(y)
`median_ae`	median absolute error
`max_error`	largest absolute error

- **Method:** exact rational sums of the errors; the square root rounded once
- **Assumes:** at least two values; the true values are not all equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Predicted against true values (y_true [3, -0.5, 2, 7], y_pred [2.5, 0, 2, 8]); each vertical segment is a residual. Returned: MAE 0.5, MSE 0.375, RMSE 0.612372, R^2 0.948608, explained variance 0.957173, median absolute error 0.5, max error 1.



The seven metrics the package returns for y_true [3, -0.5, 2, 7] and y_pred [2.5, 0, 2, 8]: MAE 0.5, MSE 0.375, RMSE 0.612372, median AE 0.5, max error 1, R^2 0.948608, expl. var. 0.957173.

```
data_science_ml_ai999x.solve('ml_regression_metrics', y_true=[3, -0.5, 2, 7], y_pred=[2.5, 0, 2, 8])
# -> mae = 0.5, mse = 0.375, rmse = 0.612372, r2 = 0.948608, explained_variance = 0.957173,
median_ae = 0.5, max_error = 1   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_regression_metrics', y_true=[10, 20, 30, 40, 50], y_pred=[12, 18,
33, 37, 52])
# -> mae = 2.4, mse = 6, rmse = 2.44949, r2 = 0.97, explained_variance = 0.9708, median_ae = 2,
max_error = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Regression metrics: y_true = [3, -0.5, 2, 7], y_pred = [2.5, 0, 2, 8]	SOLVED	mae = 0.5, mse = 0.375, rmse = 0.612372, r2 = 0.948608, explained_variance = 0.957173, median_ae = 0.5, max_error = 1
Regression metrics: y_true = [10, 20, 30, 40, 50], y_pred = [12, 18, 33, 37, 52]	SOLVED	mae = 2.4, mse = 6, rmse = 2.44949, r2 = 0.97, explained_variance = 0.9708, median_ae = 2, max_error = 3
Regression metrics: y_pred = [2.5, 0, 2, 8]	NEEDS_INPUT	ml_regression_metrics needs y_true (true values). Example: solve('ml_regression_metrics', y_true=[3, -0.5, 2, 7], y_pred
Regression metrics: y_true = [3, -0.5, 2, 7], y_pred = [2.5, 0, 2, 8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Regression metrics: y_true = [5, 5, 5], y_pred = [4, 5, 6]	INVALID_INPUT	Regression metrics (MAE, MSE, RMSE, R^2, explained variance): R^2 is undefined: every true value is the same.

Mean absolute percentage error, `ml_mape`

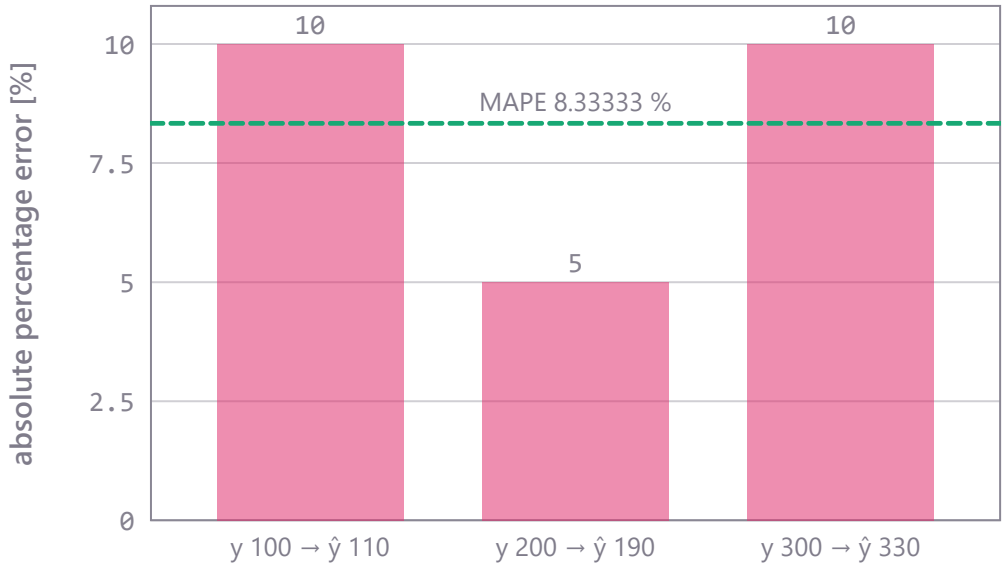
The mean absolute percentage error of predictions, as a fraction and in per cent.

$$\mathrm{MAPE} = \frac{1}{n} \sum_{i} \frac{|y_i - \hat{y}_i|}{|y_i|}$$

input	meaning	condition
`y_true`	true values (none equal to 0)	
`y_pred`	predicted values	

output	meaning
`mape`	mean of |y - y_hat| / |y| (a fraction)
`mape_percent`	the same in per cent

- **Method:** exact mean of the relative absolute errors
- **Assumes:** no true value equal to 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Absolute percentage error of each sample (y_true [100, 200, 300], y_pred [110, 190, 330]), each one run of the package on that sample alone; their mean is the returned MAPE 8.33333 %.

```
data_science_ml_ai999x.solve('ml_mape', y_true=[100, 200, 300], y_pred=[110, 190, 330])
# -> mape = 0.0833333, mape_percent = 8.33333   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_mape', y_true=[3, -0.5, 2, 7], y_pred=[2.5, 0, 2, 8])
# -> mape = 0.327381, mape_percent = 32.7381   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mean absolute percentage error: y_true = [100, 200, 300], y_pred = [110, 190, 330]	SOLVED	mape = 0.0833333, mape_percent = 8.33333
Mean absolute percentage error: y_true = [3, -0.5, 2, 7], y_pred = [2.5, 0, 2, 8]	SOLVED	mape = 0.327381, mape_percent = 32.7381
Mean absolute percentage error: y_pred = [110, 190, 330]	NEEDS_INPUT	ml_mape needs y_true (true values (none equal to 0)). Example: solve('ml_mape', y_true=[100, 200, 300], y_pred= [110, 190
Mean absolute percentage error: y_true = [100, 200, 300], y_pred = [110, 190, 330], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Mean absolute percentage error: y_true = [0, 2], y_pred = [1, 2]	INVALID_INPUT	Mean absolute percentage error: MAPE is undefined when a true value is 0.

Adjusted R^2, `ml_adjusted_r2`

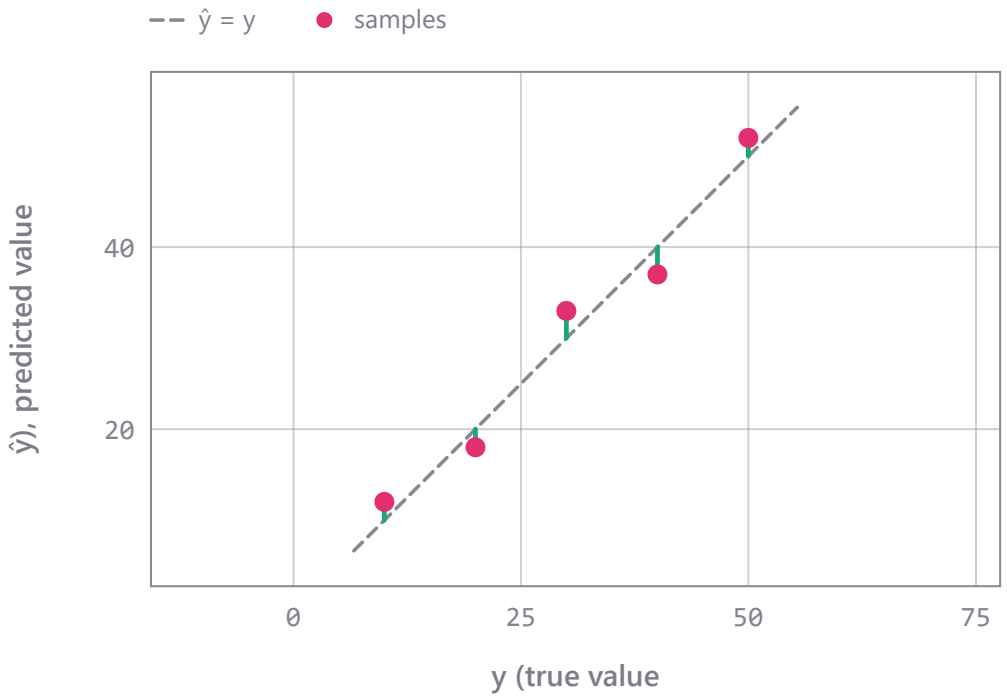
R^2 and the adjusted R^2 that penalises the number of predictors p of the model.

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

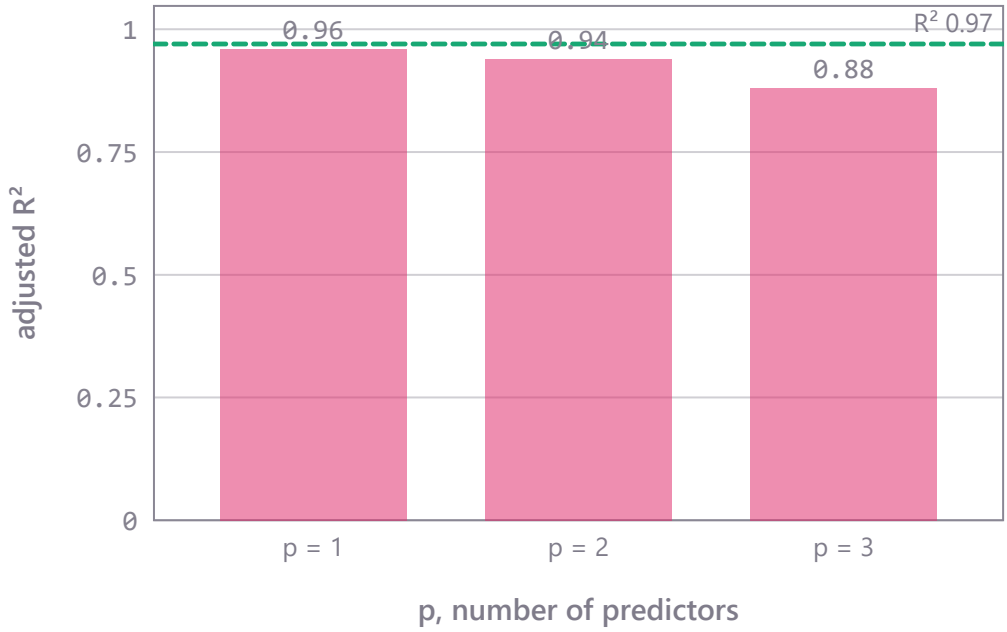
input	meaning	condition
`y_true`	true values	
`y_pred`	fitted values	
`p`	number of predictors (excluding the intercept)	

output	meaning
`r2`	R^2
`adjusted_r2`	$1 - (1 - R^2)(n - 1)/(n - p - 1)$

- **Method:** exact rational arithmetic
- **Assumes:** $n - p - 1 > 0$; true values not all equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Predicted against true values (y_true [10, 20, 30, 40, 50], y_pred [12, 18, 33, 37, 52]); returned R^2 0.97 and adjusted R^2 0.96 for p = 1 predictor.



Adjusted R^2 of the same predictions (n = 5 samples, R^2 = 0.97) charged for p = 1, 2, 3 predictors, one run each: 0.96, 0.94, 0.88.

```
data_science_ml_ai999x.solve('ml_adjusted_r2', y_true=[10, 20, 30, 40, 50], y_pred=[12, 18, 33, 37, 52], p=1)
# -> r2 = 0.97, adjusted_r2 = 0.96   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_adjusted_r2', y_true=[3, 5, 7, 6, 9, 11, 10, 14], y_pred=[3.5, 4.5, 6.5, 7, 8.5, 10.5, 11, 13], p=2)
# -> r2 = 0.95218, adjusted_r2 = 0.933052   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Adjusted R squared: y_true = [10, 20, 30, 40, 50], y_pred = [12, 18, 33, 37, 52], p = 1	SOLVED	r2 = 0.97, adjusted_r2 = 0.96
Adjusted R squared: y_true = [3, 5, 7, 6, 9, 11, 10, 14], y_pred = [3.5, 4.5, 6.5, 7, 8.5, 10.5, 11, 13], p = 2	SOLVED	r2 = 0.95218, adjusted_r2 = 0.933052
Adjusted R squared: y_pred = [12, 18, 33, 37, 52], p = 1	NEEDS_INPUT	ml_adjusted_r2 needs y_true (true values). Example: solve('ml_adjusted_r2', y_true=[10, 20, 30, 40, 50], y_pred=[12, 18,
Adjusted R squared: y_true = [10, 20, 30, 40, 50], y_pred = [12, 18, 33, 37, 52], p = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Adjusted R squared: y_true = [1, 2, 4], y_pred = [1, 2, 3], p = 2	INVALID_INPUT	Adjusted R^2: adjusted R^2 needs more samples than predictors plus one (n - p - 1 > 0).

Huber, smooth L1 and pseudo-Huber loss, `ml_huber_loss`

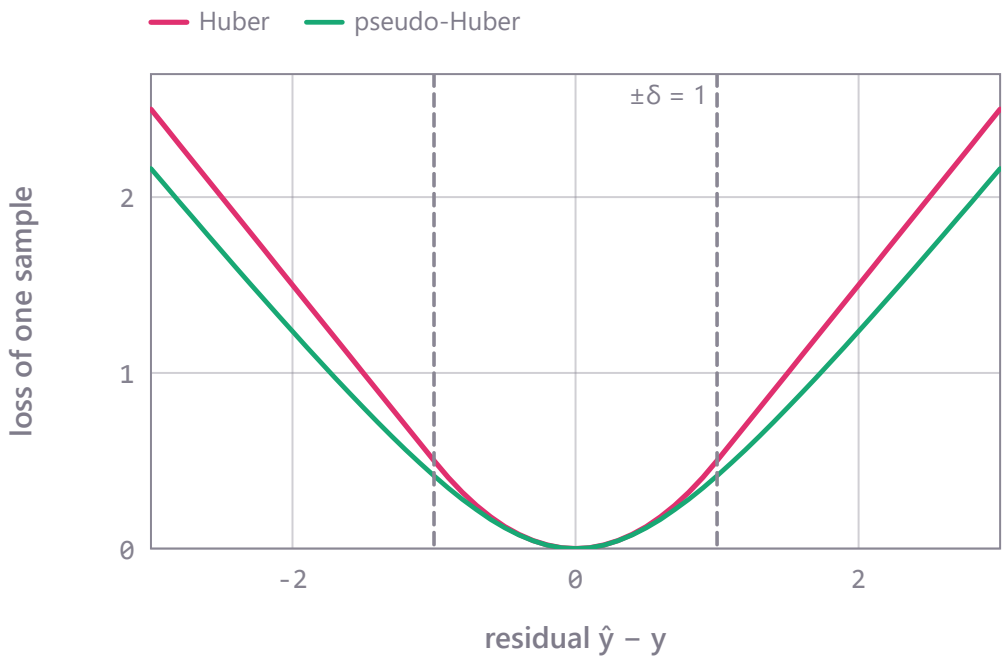
The mean Huber loss with threshold delta (quadratic for small errors, linear for large), the smooth L1 loss (Huber / delta) and the pseudo-Huber loss.

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \le \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right) & |r| > \delta \end{cases}$$

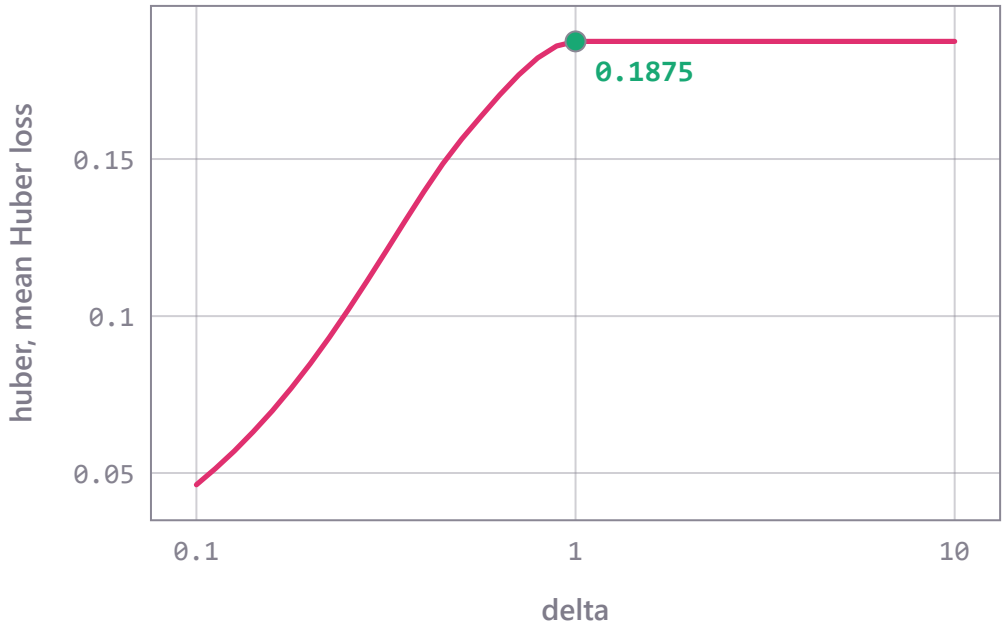
input	meaning	condition
`y_true`	true values	
`y_pred`	predicted values	
`delta`	threshold between the quadratic and linear parts $(\delta > 0)$	

output	meaning
`huber`	mean Huber loss
`smooth_l1`	mean smooth L1 loss with $\beta = \delta$
`pseudo_huber`	mean pseudo-Huber loss

- **Method:** exact piecewise Huber terms; pseudo-Huber in the cancellation-free form δ^2 x^2 / (sqrt(1 + x^2) + 1)
- **Assumes:** $\delta > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Huber and pseudo-Huber loss of one sample as the residual runs from −3 to 3 (δ = 1, 61 runs each): quadratic inside ±δ, linear outside. Worked example y_true = [3, -0.5, 2, 7], y_pred = [2.5, 0, 2, 8]: returned Huber 0.1875, smooth L1 0.1875, pseudo-Huber 0.16257.



huber (mean Huber loss) against delta (threshold between the quadratic and linear parts (delta > 0)) from 0.1 to 10 on a logarithmic axis. Other inputs as in the worked example: y_true = [3, -0.5, 2, 7], y_pred = [2.5, 0, 2, 8]. 41 points computed by data-science-ml-ai999x; marked: the worked example, delta = 1, huber = 0.1875.

```
data_science_ml_ai999x.solve('ml_huber_loss', y_true=[3, -0.5, 2, 7], y_pred=[2.5, 0, 2, 8],
delta=1)
# -> huber = 0.1875, smooth_l1 = 0.1875, pseudo_huber = 0.16257   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_huber_loss', y_true=[1, 2, 3], y_pred=[4, 2, 2.5], delta=1.5)
# -> huber = 1.16667, smooth_l1 = 0.777778, pseudo_huber = 0.96762   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Huber loss: y_true = [3, -0.5, 2, 7], y_pred = [2.5, 0, 2, 8], delta = 1	SOLVED	huber = 0.1875, smooth_l1 = 0.1875, pseudo_huber = 0.16257
Huber loss: y_true = [1, 2, 3], y_pred = [4, 2, 2.5], delta = 1.5	SOLVED	huber = 1.16667, smooth_l1 = 0.777778, pseudo_huber = 0.96762
Huber loss: y_pred = [2.5, 0, 2, 8], delta = 1	NEEDS_INPUT	ml_huber_loss needs y_true (true values). Example: solve('ml_huber_loss', y_true=[3, -0.5, 2, 7], y_pred=[2.5, 0, 2, 8],
Huber loss: y_true = [3, -0.5, 2, 7], y_pred = [2.5, 0, 2, 8], delta = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Huber loss: y_true = [1, 2], y_pred = [1, 3], delta = 0	INVALID_INPUT	Huber, smooth L1 and pseudo-Huber loss: delta must be positive.

Quantile (pinball) loss, `ml_quantile_loss`

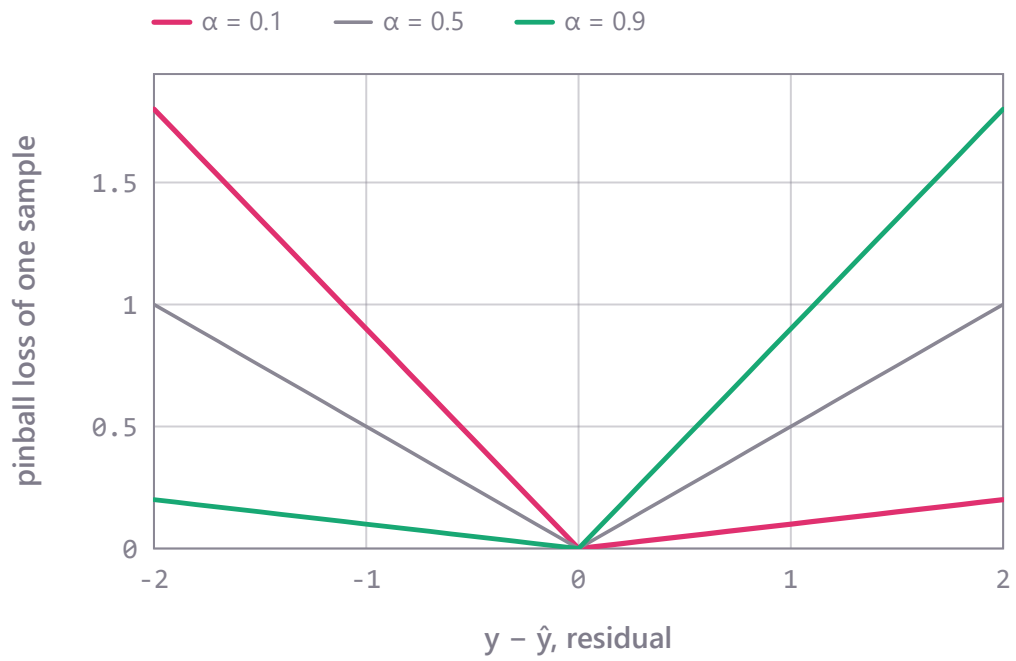
The mean pinball loss of predictions of the alpha-quantile: under-predictions cost alpha per unit, over-predictions $\mathbf{1-\alpha}$.

$$L_\alpha(y, q) = \max\{\alpha(y-q),\ (\alpha-1)(y-q)\}$$

input	meaning	condition
`y_true`	true values	
`y_pred`	predicted quantiles	
`alpha`	quantile level $(0 < \alpha < 1)$	

output	meaning
`pinball`	mean pinball loss

- **Method:** exact mean of max(alpha (y - q), $(\alpha - 1)$(y - q))
- **Assumes:** $0 < \alpha < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Pinball (quantile) loss of one sample against the residual y − ŷ from −2 to 2, for α = 0.1, 0.5, 0.9 (41 runs each): under-prediction costs α per unit, over-prediction 1 − α. Worked example (α = 0.1): returned 0.0333333.

```
data_science_ml_ai999x.solve('ml_quantile_loss', y_true=[1, 2, 3], y_pred=[0, 2, 3], alpha=0.1)
# -> pinball = 0.0333333   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_quantile_loss', y_true=[10, 20, 30], y_pred=[12, 15, 33],
alpha=0.9)
# -> pinball = 1.66667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pinball loss: y_true = [1, 2, 3], y_pred = [0, 2, 3], alpha = 0.1	SOLVED	pinball = 0.0333333
Pinball loss: y_true = [10, 20, 30], y_pred = [12, 15, 33], alpha = 0.9	SOLVED	pinball = 1.66667

question	outcome	result
Pinball loss: y_pred = [0, 2, 3], alpha = 0.1	NEEDS_INPUT	ml_quantile_loss needs y_true (true values). Example: solve('ml_quantile_loss', y_true=[1, 2, 3], y_pred=[0, 2, 3], alph
Pinball loss: y_true = [1, 2, 3], y_pred = [0, 2, 3], alpha = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Pinball loss: y_true = [1, 2], y_pred = [1, 3], alpha = 1.5	INVALID_INPUT	Quantile (pinball) loss: alpha (the quantile level) must lie strictly between 0 and 1.

Mean Tweedie deviance (normal, Poisson, gamma, compound Poisson-gamma), `ml_tweedie_deviance`

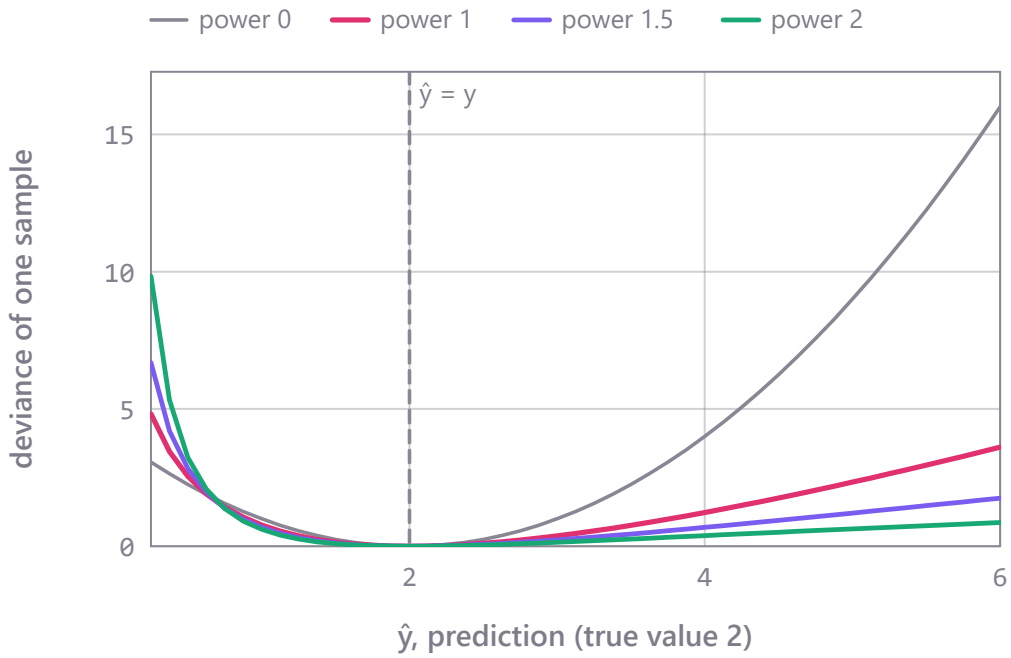
The mean Tweedie deviance of predictions for a stated power: 0 squared error, 1 Poisson, 2 gamma, 3 inverse Gaussian, between 1 and 2 compound Poisson-gamma.

$$d_p(y,\mu) = 2\left(\frac{y^{2-p}}{(1-p)(2-p)} - \frac{y\mu^{1-p}}{1-p} + \frac{\mu^{2-p}}{2-p}\right)$$

input	meaning	condition
`y_true`	true values	
`y_pred`	predicted means (positive for power > 0)	
`power`	Tweedie power p (0, or p >= 1)	

output	meaning
`deviance`	mean Tweedie deviance

- **Method:** unit deviance of the Tweedie family for each sample, averaged
- **Assumes:** power 0 $or \geq 1$; positive predictions (power > 0); $y \geq 0\ (1 \leq p < 2)$ or $y > 0\ (p \geq 2)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tweedie deviance of one sample with true value 2 as the prediction runs from 0.25 to 6, for powers 0 (squared error), 1 (Poisson), 1.5 and 2 (gamma), 47 runs each: all vanish at ŷ = y, higher powers forgive over-prediction more. Worked example (power 1): returned 1.42602.

```
data_science_ml_ai999x.solve('ml_tweedie_deviance', y_true=[2, 0, 1, 4], y_pred=[0.5, 0.5, 2, 2], power=1)
# -> deviance = 1.42602   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_tweedie_deviance', y_true=[2, 0.5, 1, 4], y_pred=[0.5, 0.5, 2, 2], power=2)
# -> deviance = 1.05685   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_tweedie_deviance', y_true=[2, 0, 1, 4], y_pred=[0.5, 0.5, 2, 2], power=1.5)
# -> deviance = 1.77817   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tweedie deviance: y_true = [2, 0, 1, 4], y_pred = [0.5, 0.5, 2, 2], power = 1	SOLVED	deviance = 1.42602
Tweedie deviance: y_true = [2, 0.5, 1, 4], y_pred = [0.5, 0.5, 2, 2], power = 2	SOLVED	deviance = 1.05685
Tweedie deviance: y_true = [2, 0, 1, 4], y_pred = [0.5, 0.5, 2, 2], power = 1.5	SOLVED	deviance = 1.77817
Tweedie deviance: y_pred = [0.5, 0.5, 2, 2], power = 1	NEEDS_INPUT	ml_tweedie_deviance needs y_true (true values). Example: solve('ml_tweedie_deviance', y_true=[2, 0, 1, 4], y_pred=[0.5,
Tweedie deviance: y_true = [2, 0, 1, 4], y_pred = [0.5, 0.5, 2, 2], power = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

question	outcome	result
Tweedie deviance: y_true = [2, 1], y_pred = [0.5, 0.5], power = 0.5	INVALID_INPUT	Mean Tweedie deviance (normal, Poisson, gamma, compound Poisson-gamma): power must be 0 (normal) or at least 1 (1 Poisso

Mean squared logarithmic error, `ml_mean_squared_log_error`

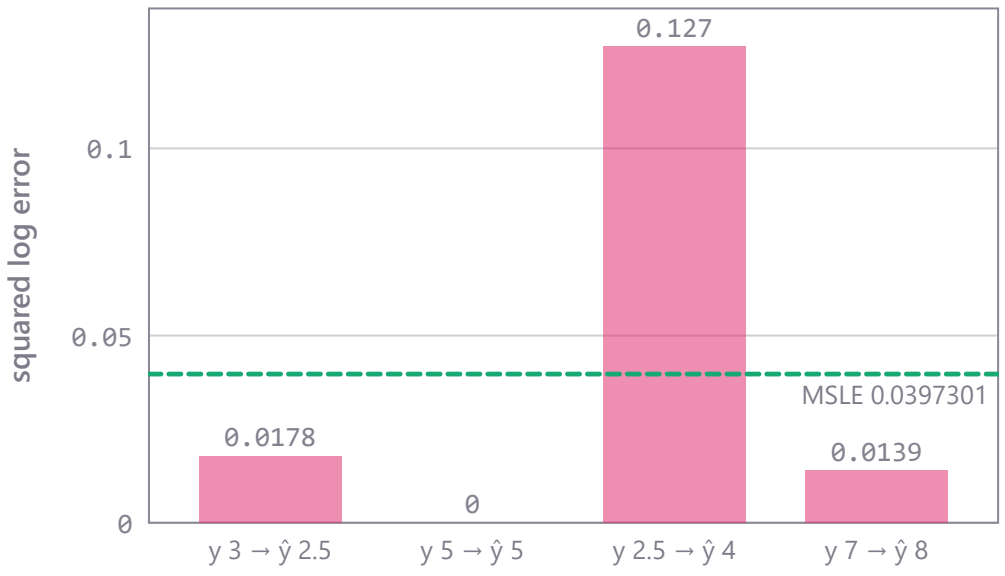
The mean squared logarithmic error ln(1 + y) - ln(1 + y_hat) and its square root (RMSLE), for non-negative targets such as counts and prices.

$$\mathrm{MSLE} = \frac{1}{n}\sum_{i}\left(\ln(1+y_i) - \ln(1+\hat{y}_i)\right)^2$$

input	meaning	condition
`y_true`	true values (>= 0)	
`y_pred`	predicted values (>= 0)	

output	meaning
`msle`	mean squared log error
`rmsle`	root mean squared log error

- **Method:** log1p differences squared and averaged
- **Assumes:** values >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Squared log error of each sample (y_true [3, 5, 2.5, 7], y_pred [2.5, 5, 4, 8]), one run per sample; their mean is the returned MSLE 0.0397301 (RMSLE 0.199324). Under-predicting 2.5 as 4 costs more than over-predicting 7 as 8.

```
data_science_ml_ai999x.solve('ml_mean_squared_log_error', y_true=[3, 5, 2.5, 7], y_pred=[2.5, 5, 4, 8])
# -> msle = 0.0397301, rmsle = 0.199324   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_mean_squared_log_error', y_true=[100, 200], y_pred=[110, 150])
# -> msle = 0.0453618, rmsle = 0.212983   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mean squared log error: y_true = [3, 5, 2.5, 7], y_pred = [2.5, 5, 4, 8]	SOLVED	msle = 0.0397301, rmsle = 0.199324
Mean squared log error: y_true = [100, 200], y_pred = [110, 150]	SOLVED	msle = 0.0453618, rmsle = 0.212983
Mean squared log error: y_pred = [2.5, 5, 4, 8]	NEEDS_INPUT	ml_mean_squared_log_error needs y_true (true values (>= 0)). Example: solve('ml_mean_squared_log_error', y_true=[3, 5, 2
Mean squared log error: y_true = [3, 5, 2.5, 7], y_pred = [2.5, 5, 4, 8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Mean squared log error: y_true = [-3, 5], y_pred = [2.5, 5]	INVALID_INPUT	Mean squared logarithmic error: the squared log error needs non-negative values.

Softmax, `ml_softmax`

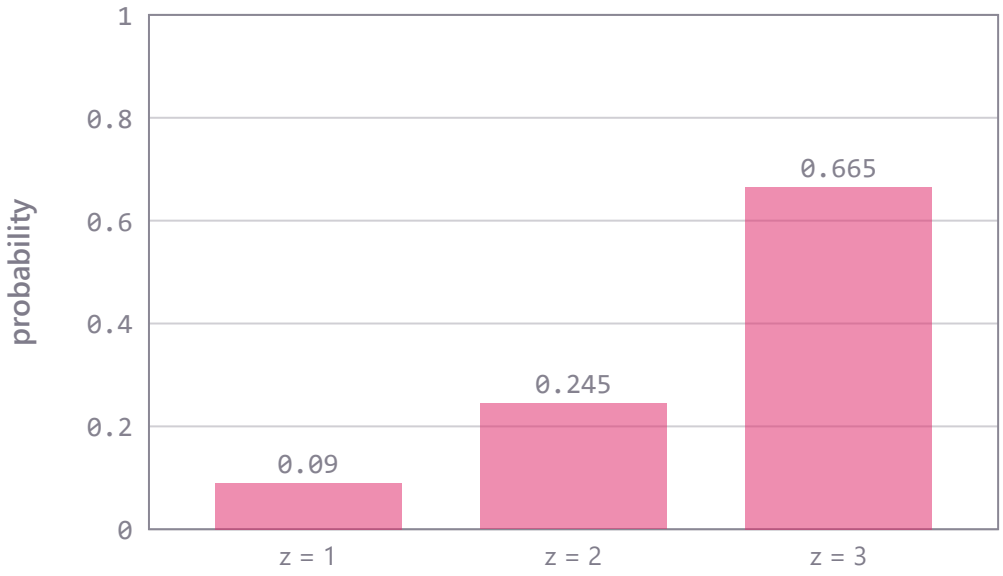
The softmax of a vector of logits (numerically stable: the largest logit is subtracted first), with log-sum-exp.

$$\sigma(z)_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}},\ m = \max_j z_j$$

input	meaning	condition
`z`	logits (at least two)	

output	meaning
`probabilities`	softmax probabilities exp(z_i) / sum exp(z_j)
`log_sum_exp`	log sum exp(z_j)

- **Method:** subtract max(z), exponentiate, normalise (fsum)
- **Assumes:** at least two logits
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Softmax of the logits z = [1, 2, 3]: returned probabilities 0.0900306, 0.244728, 0.665241 (log-sum-exp 3.40761); each unit of z multiplies the probability by e.

```
data_science_ml_ai999x.solve('ml_softmax', z=[1, 2, 3])
# -> probabilities = [0.0900305731704, 0.244728471055, 0.665240955775], log_sum_exp = 3.40761
(SOLVED)
```

```
data_science_ml_ai999x.solve('ml_softmax', z=[1000, 1001])
# -> probabilities = [0.26894142137, 0.73105857863], log_sum_exp = 1001.31   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Softmax: z = [1, 2, 3]	SOLVED	probabilities = [0.0900305731704, 0.244728471055, 0.665240955775], log_sum_exp = 3.40761
Softmax: z = [1000, 1001]	SOLVED	probabilities = [0.26894142137, 0.73105857863], log_sum_exp = 1001.31
Softmax:	NEEDS_INPUT	ml_softmax needs z (logits (at least two)). Example: solve('ml_softmax', z=[1, 2, 3]).
Softmax: z = [1, 2, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Log-softmax, `ml_log_softmax`

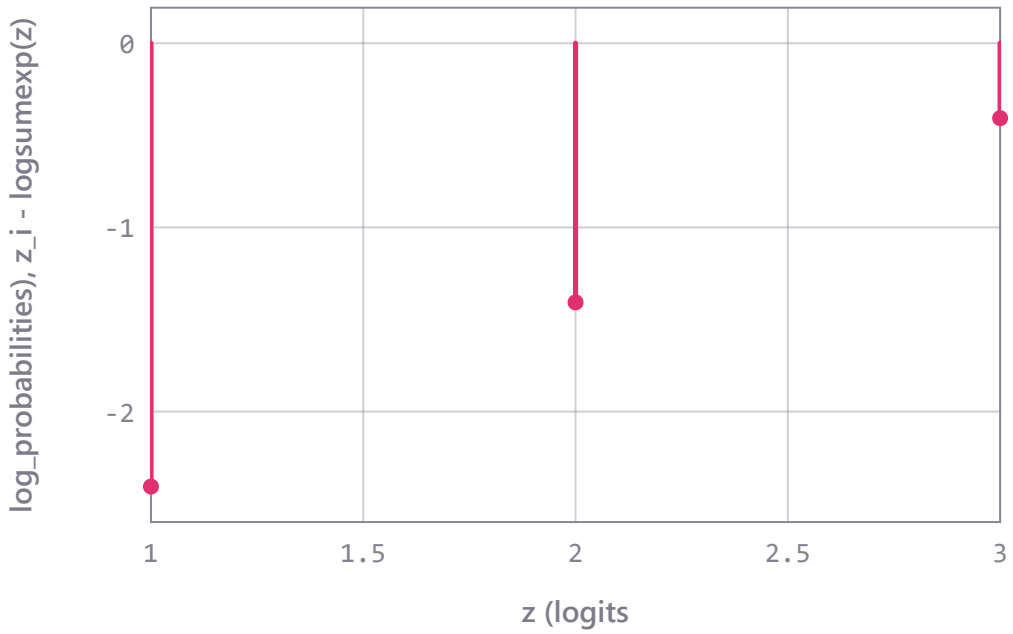
The log-softmax of logits, z_i - log sum exp(z), computed stably even for very large logits.

$$\log \sigma(z)_i = z_i - m - \log \sum_j e^{z_j - m}$$

input	meaning	condition
`z`	logits	

output	meaning
`log_probabilities`	z_i - logsumexp(z)
`log_sum_exp`	logsumexp(z)

- **Method:** log-sum-exp with the maximum subtracted
- **Assumes:** at least two logits
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



z_i - logsumexp(z) (log_probabilities) returned by data-science-ml-ai999x for the worked example, one value for each z (logits) (3 values, first -2.40761, last -0.407606, largest -0.407606). Inputs: z = [1, 2, 3].

```
data_science_ml_ai999x.solve('ml_log_softmax', z=[1, 2, 3])
# -> log_probabilities = [-2.40760596444, -1.40760596444, -0.407605964444], log_sum_exp = 3.40761
(SOLVED)
```

```
data_science_ml_ai999x.solve('ml_log_softmax', z=[1000, 0, -1000])
# -> log_probabilities = [0.0, -1000.0, -2000.0], log_sum_exp = 1000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Log-softmax: z = [1, 2, 3]	SOLVED	log_probabilities = [-2.40760596444, -1.40760596444, -0.407605964444], log_sum_exp = 3.40761
Log-softmax: z = [1000, 0, -1000]	SOLVED	log_probabilities = [0.0, -1000.0, -2000.0], log_sum_exp = 1000
Log-softmax:	NEEDS_INPUT	ml_log_softmax needs z (logits). Example: solve('ml_log_softmax', z=[1, 2, 3]).
Log-softmax: z = [1, 2, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Sigmoid (logistic function) and its derivative, `ml_sigmoid`

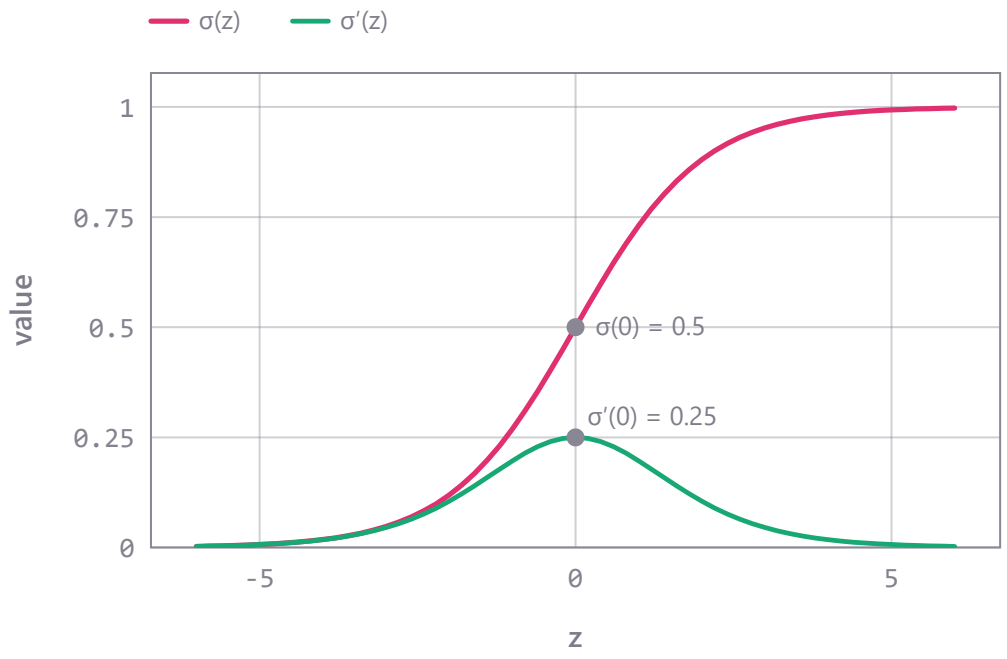
The logistic sigmoid $1/(1+e^{-z})$, its derivative sigma(1 - sigma) and log sigma(z), computed stably for large |z|.

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)\,(1-\sigma(z))$$

input	meaning	condition
`z`	input (logit)	

output	meaning
`sigmoid`	1 / (1 + exp(-z))
`derivative`	sigma(z)(1 - sigma(z))
`log_sigmoid`	log sigma(z)

- **Method:** branch on the sign of z so no exponential overflows; derivative as e^-|z| / (1 + e^-|z|)^2
- **Assumes:** finite z
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The logistic sigmoid and its derivative for z from -6 to 6 (61 runs). Marked: the worked example z = 0, σ = 0.5, σ′ = 0.25 (the derivative peaks at z = 0).

```
data_science_ml_ai999x.solve('ml_sigmoid', z=0)
# -> sigmoid = 0.5, derivative = 0.25, log_sigmoid = -0.693147   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_sigmoid', z=2)
# -> sigmoid = 0.880797, derivative = 0.104994, log_sigmoid = -0.126928   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_sigmoid', z=-35)
# -> sigmoid = 6.30512e-16, derivative = 6.30512e-16, log_sigmoid = -35   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sigmoid: z = 0	SOLVED	sigmoid = 0.5, derivative = 0.25, log_sigmoid = -0.693147
Sigmoid: z = 2	SOLVED	sigmoid = 0.880797, derivative = 0.104994, log_sigmoid = -0.126928
Sigmoid: z = -35	SOLVED	sigmoid = 6.30512e-16, derivative = 6.30512e-16, log_sigmoid = -35
Sigmoid:	NEEDS_INPUT	ml_sigmoid needs z (input (logit)). Example: solve('ml_sigmoid', z=0).
Sigmoid: z = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Cross-entropy loss from logits and its gradient, `ml_cross_entropy_logits`

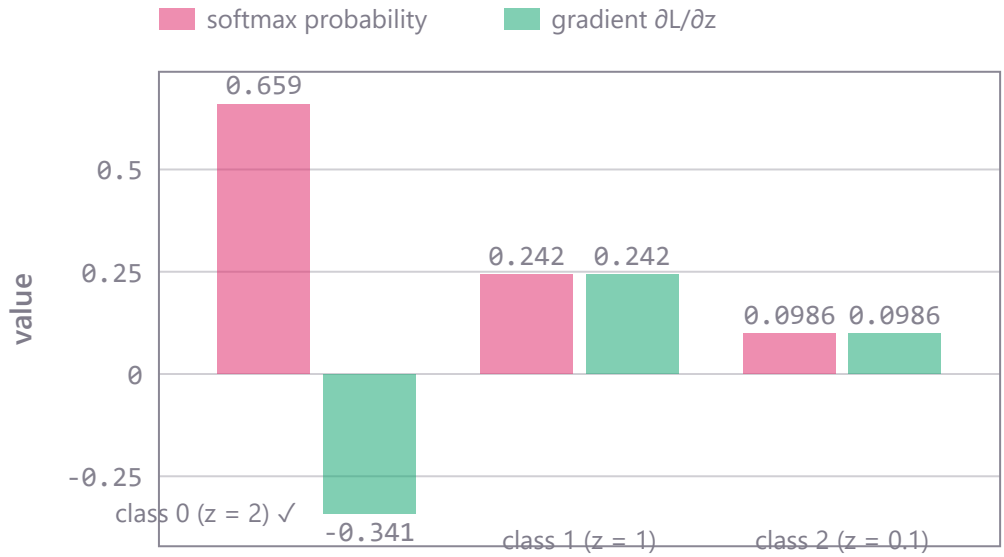
The cross-entropy loss of one sample from its logits and true class, the predicted probability of that class, and the gradient of the loss with respect to the logits (softmax minus one-hot).

$$L = -\log \frac{e^{z_y}}{\sum_j e^{z_j}}, \quad \frac{\partial L}{\partial z_j} = p_j - \mathbb{1}(j = y)$$

input	meaning	condition
`z`	logits of the classes	
`y`	index of the true class (0 to K-1)	

output	meaning
`loss`	logsumexp(z) - z_y
`probability`	softmax probability of the true class
`gradient`	dL/dz = softmax(z) - onehot(y)

- **Method:** log-sum-exp form; the true-class gradient as minus the sum of the other probabilities (no cancellation)
- **Assumes:** y is a class index
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Logits z = [2, 1, 0.1] with true class 0 (✓): the softmax probabilities (from the package's softmax: 0.659001, 0.242433, 0.0985659) and the returned gradient of the loss, probability minus one-hot (-0.340999, 0.242433, 0.0985659); returned loss 0.41703 = −ln 0.659001.

```
data_science_ml_ai999x.solve('ml_cross_entropy_logits', z=[2, 1, 0.1], y=0)
# -> loss = 0.41703, probability = 0.659001, gradient = [-0.340998861114, 0.242432970705,
0.0985658904093]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_cross_entropy_logits', z=[0.5, -1, 3, 0], y=2)
# -> loss = 0.139925, probability = 0.869423, gradient = [0.0713666110359, 0.0159240433497,
-0.130576692059, 0.0432860376731]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cross-entropy from logits: z = [2, 1, 0.1], y = 0	SOLVED	loss = 0.41703, probability = 0.659001, gradient = [-0.340998861114, 0.242432970705, 0.0985658904093]
Cross-entropy from logits: z = [0.5, -1, 3, 0], y = 2	SOLVED	loss = 0.139925, probability = 0.869423, gradient = [0.0713666110359, 0.0159240433497, -0.130576692059, 0.0432860376731]
Cross-entropy from logits: y = 0	NEEDS_INPUT	ml_cross_entropy_logits needs z (logits of the classes). Example: solve('ml_cross_entropy_logits', z=[2, 1, 0.1], y=0).
Cross-entropy from logits: z = [2, 1, 0.1], y = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Cross-entropy from logits: z = [2, 1, 0.1], y = 3	INVALID_INPUT	Cross-entropy loss from logits and its gradient: y must be the index of the true class (0 to K-1).

Binary cross-entropy with logits and its gradient, `ml_bce_with_logits`

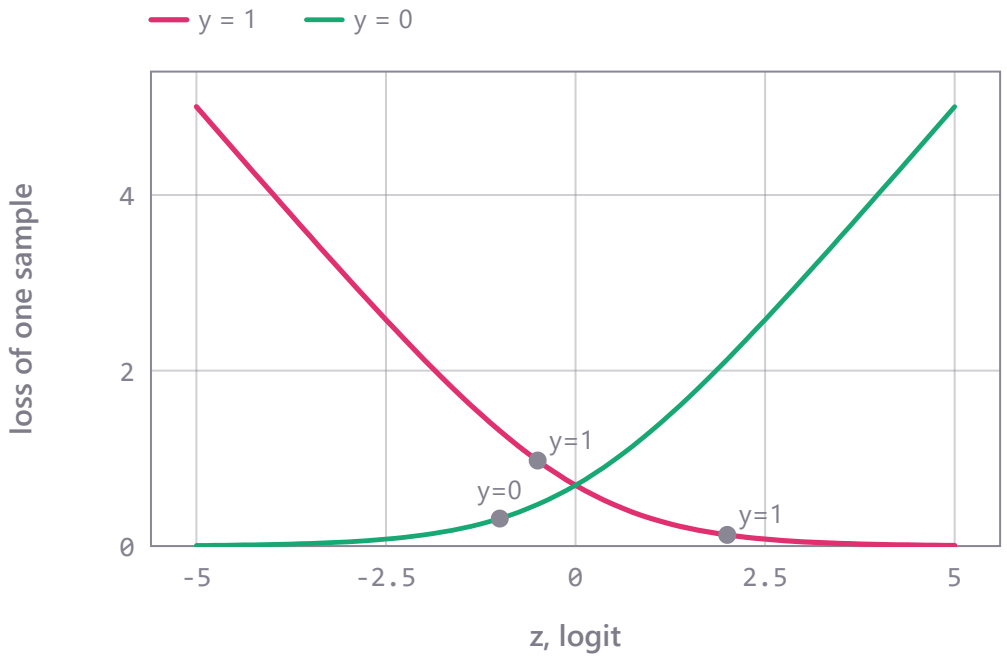
The mean binary cross-entropy computed directly from logits (stable for large |z|; soft targets allowed) and its gradient with respect to each logit.

$$\ell(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}), \quad \frac{\partial \ell}{\partial z} = \sigma(z) - y$$

input	meaning	condition
`y`	targets between 0 and 1	
`z`	logits	

output	meaning
`loss`	mean of max(z, 0) - z y + log(1 + e^-|z|)
`gradient`	dL/dz_i = (sigma(z_i) - y_i) / n

- **Method:** the log-sum-exp form max(z, 0) - z y + $log1p$(e^-|z|)
- **Assumes:** $0 \le y \le 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Binary cross-entropy of one sample as its logit z runs from −5 to 5, for y = 1 and y = 0 (81 runs each). Marked: the worked example's samples (y, z) = [(1, 2), (0, -1), (1, -0.5)], each one run; their mean is the returned loss 0.471422.

```
data_science_ml_ai999x.solve('ml_bce_with_logits', y=[1, 0, 1], z=[2, -1, -0.5])
# -> loss = 0.471422, gradient = [-0.0397343073407, 0.0896471404567, -0.207486443734]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_bce_with_logits', y=[0.8, 0.1], z=[1.5, 0.3])
# -> loss = 0.662884, gradient = [0.00878723809682, 0.237221258406]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Binary cross-entropy with logits: y = [1, 0, 1], z = [2, -1, -0.5]	SOLVED	loss = 0.471422, gradient = [-0.0397343073407, 0.0896471404567, -0.207486443734]

question	outcome	result
Binary cross-entropy with logits: y = [0.8, 0.1], z = [1.5, 0.3]	SOLVED	loss = 0.662884, gradient = [0.00878723809682, 0.237221258406]
Binary cross-entropy with logits: z = [2, -1, -0.5]	NEEDS_INPUT	ml_bce_with_logits needs y (targets between 0 and 1). Example: solve('ml_bce_with_logits', y=[1, 0, 1], z=[2, -1, -0.5])
Binary cross-entropy with logits: y = [1, 0, 1], z = [2, -1, -0.5], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Binary cross-entropy with logits: y = [2, 0], z = [1, 1]	INVALID_INPUT	Binary cross-entropy with logits and its gradient: targets y must lie between 0 and 1.

Hinge and squared hinge loss, `ml_hinge_loss`

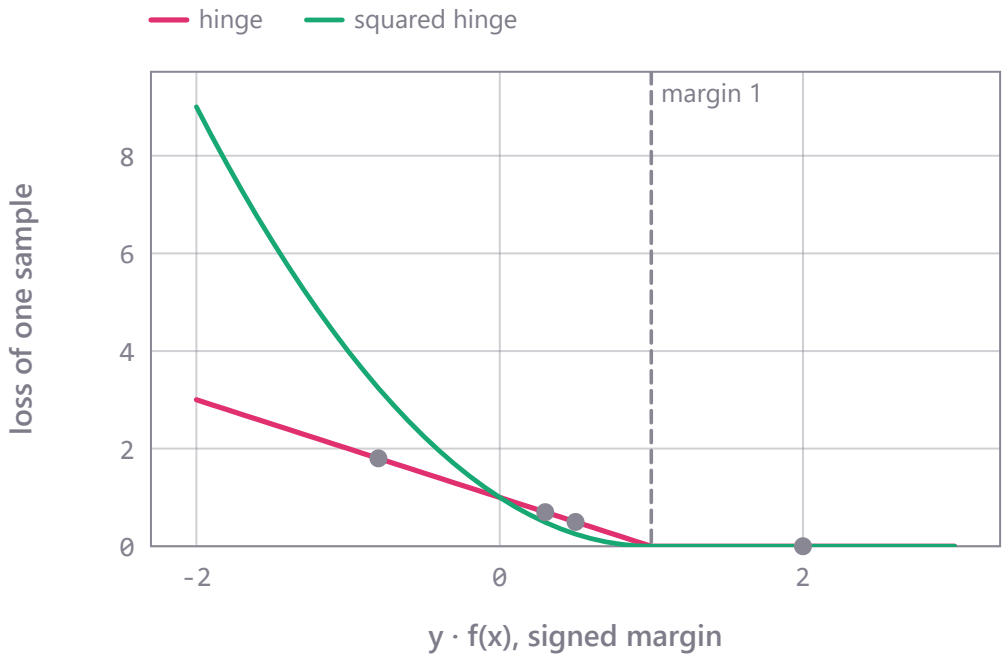
The mean hinge loss max(0, $1 - yf$) of a margin classifier with labels -1/+1, the mean squared hinge loss and the number of margin violations.

$$\ell(y, f) = \max(0,\ 1 - yf)$$

input	meaning	condition
`y_true`	true labels (-1 or +1)	
`scores`	decision values f(x)	

output	meaning
`hinge`	mean max(0, $1 - yf$)
`squared_hinge`	mean max(0, $1 - yf$)^2
`n_violations`	samples with y $yf < 1$

- **Method:** exact fractions
- **Assumes:** labels -1 $or + 1$, both present
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hinge and squared hinge loss of one sample against its signed margin y·f(x) from −2 to 3 (51 runs each; each run holds a +1 and a −1 sample at the same margin, since both classes must occur, so the mean is the loss of one sample). Marked: the worked example's samples (labels [1, -1, 1, -1], scores [2, -0.5, 0.3, 0.8]) at their margins, each one run; returned means: hinge 0.75, squared hinge 0.995, 3 samples inside the margin.

```
data_science_ml_ai999x.solve('ml_hinge_loss', y_true=[1, -1, 1, -1], scores=[2, -0.5, 0.3, 0.8])
# -> hinge = 0.75, squared_hinge = 0.995, n_violations = 3   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_hinge_loss', y_true=[1, 1, -1], scores=[0.9, 1.5, -2])
# -> hinge = 0.0333333, squared_hinge = 0.00333333, n_violations = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hinge loss: y_true = [1, -1, 1, -1], scores = [2, -0.5, 0.3, 0.8]	SOLVED	hinge = 0.75, squared_hinge = 0.995, n_violations = 3
Hinge loss: y_true = [1, 1, -1], scores = [0.9, 1.5, -2]	SOLVED	hinge = 0.0333333, squared_hinge = 0.00333333, n_violations = 1
Hinge loss: scores = [2, -0.5, 0.3, 0.8]	NEEDS_INPUT	ml_hinge_loss needs y_true (true labels (-1 or +1)). Example: solve('ml_hinge_loss', y_true=[1, -1, 1, -1], scores=[2, -
Hinge loss: y_true = [1, -1, 1, -1], scores = [2, -0.5, 0.3, 0.8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Hinge loss: y_true = [1, 0, 1], scores = [2, -0.5, 0.3]	INVALID_INPUT	Hinge and squared hinge loss: labels must be -1 or +1.

Multi-class hinge loss (Crammer-Singer and Weston-Watkins), `ml_multiclass_hinge_loss`

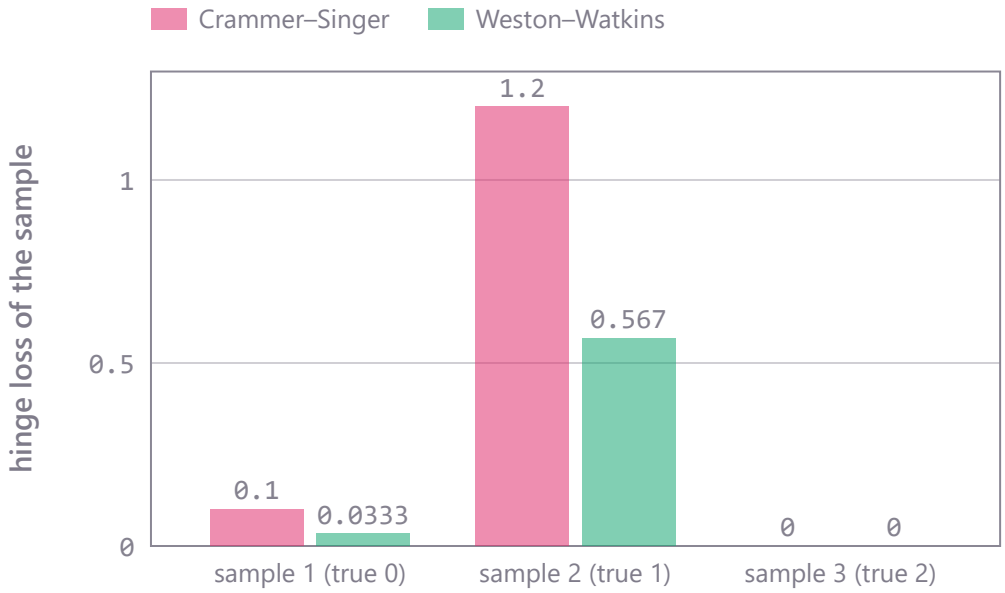
The multi-class hinge losses of a decision matrix: Crammer-Singer max(0, 1 - f_y + max_{$j \neq y$} f_j) and Weston-Watkins sum_{$j \neq y$} max(0, 1 - f_y + f_j) / K, averaged over samples.

$$\ell_{CS} = \max\left(0,\, 1 - f_y + \max_{j \neq y} f_j\right), \quad \ell_{WW} = \frac{1}{K}\sum_{j \neq y} \max(0,\, 1 - f_y + f_j)$$

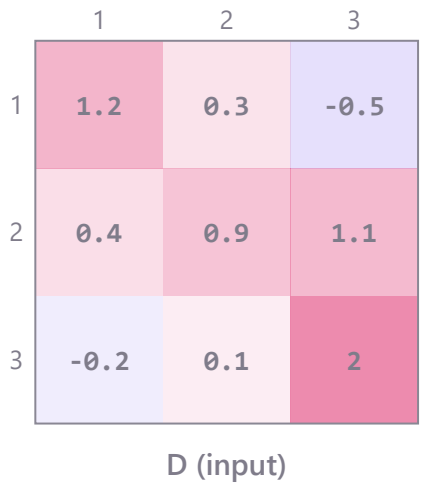
input	meaning	condition
`y_true`	true class index of each sample (0 to K-1)	
`D`	decision values: one row per sample, one column per class $(K \geq 3)$	

output	meaning
`crammer_singer`	mean Crammer-Singer hinge loss
`weston_watkins`	mean Weston-Watkins hinge loss (sum over wrong classes divided by K)

- **Method:** exact fractions
- **Assumes:** $K \geq 3$ classes; labels are column indices
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Multiclass hinge loss of each sample of the decision values D = [[1.2, 0.3, -0.5], [0.4, 0.9, 1.1], [-0.2, 0.1, 2.0]] with true classes [0, 1, 2], one run per sample; returned means: Crammer–Singer 0.433333, Weston–Watkins 0.2. Sample 2's true score barely beats one rival and loses to another.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): D (input). Returned with them by data-science-ml-ai999x: crammer_singer = 0.433333, weston_watkins = 0.2. Other inputs: y_true = [0, 1, 2].

```
data_science_ml_ai999x.solve('ml_multiclass_hinge_loss', y_true=[0, 1, 2], D=[[1.2, 0.3, -0.5],
[0.4, 0.9, 1.1], [-0.2, 0.1, 2.0]])
# -> crammer_singer = 0.433333, weston_watkins = 0.2   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_multiclass_hinge_loss', y_true=[2, 0], D=[[0.5, 0.2, 0.4], [1.0,
1.5, -1.0]])
# -> crammer_singer = 1.3, weston_watkins = 0.566667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiclass hinge loss: y_true = [0, 1, 2], D = [[1.2, 0.3, -0.5], [0.4, 0.9, 1.1], [-0.2, 0.1, 2]]	SOLVED	crammer_singer = 0.433333, weston_watkins = 0.2
Multiclass hinge loss: y_true = [2, 0], D = [[0.5, 0.2, 0.4], [1, 1.5, -1]]	SOLVED	crammer_singer = 1.3, weston_watkins = 0.566667
Multiclass hinge loss: D = [[1.2, 0.3, -0.5], [0.4, 0.9, 1.1], [-0.2, 0.1, 2]]	NEEDS_INPUT	ml_multiclass_hinge_loss needs y_true (true class index of each sample (0 to K-1)). Example: solve('ml_multiclass_hinge_
Multiclass hinge loss: y_true = [0, 1, 2], D = [[1.2, 0.3, -0.5], [0.4, 0.9, 1.1], [-0.2, 0.1, 2]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

KL divergence, Jensen-Shannon divergence and cross-entropy of discrete distributions, `ml_kl_divergence_discrete`

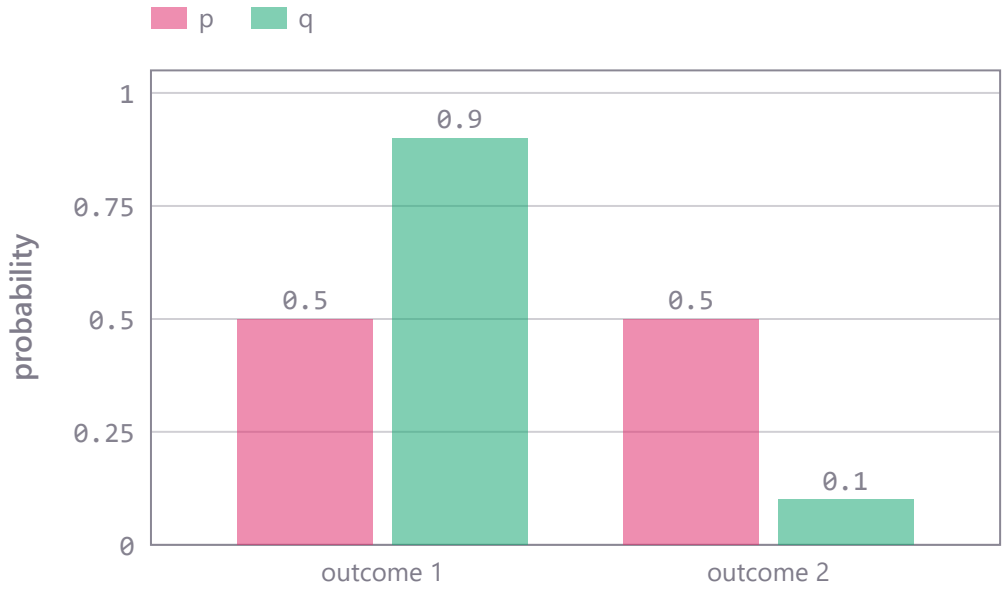
The Kullback-Leibler divergence of two discrete distributions in both directions, the Jensen-Shannon divergence, the cross-entropy H(p, q) and the entropy H(p).

$$D_{KL}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i}, \quad H(p,q) = H(p) + D_{KL}(p\|q)$$

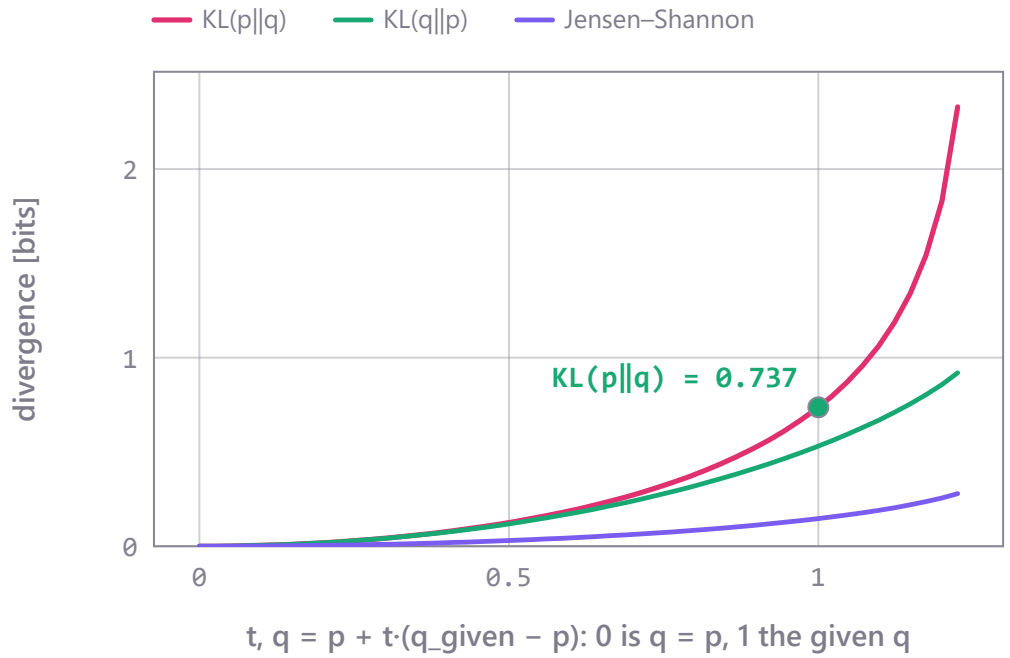
input	meaning	condition
`p`	first distribution (positive, sums to 1)	
`q`	second distribution (positive, sums to 1)	
`unit`	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	

output	meaning
`kl_pq`	KL(p || q)
`kl_qp`	KL(q || p)
`js_divergence`	Jensen-Shannon divergence
`cross_entropy`	H(p, q) = -sum p log q
`entropy_p`	H(p)

- **Method:** direct sums with fsum
- **Assumes:** positive probabilities summing to 1; same length
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated distributions p = [0.5, 0.5] and q = [0.9, 0.1]: returned KL(p||q) = 0.736966, KL(q||p) = 0.531004 (not symmetric), Jensen–Shannon 0.146793, cross-entropy 1.73697 = H(p) 1 + KL(p||q), in bits.



Divergences between the worked example's p = ['0.5', '0.5'] and a q moved from p towards and past its q = ['0.9', '0.1'] (49 runs each): both KL directions vanish at q = p and grow, at different rates, as q moves away; Jensen–Shannon stays below 1 bit. Marked: the worked example, KL(p||q) = 0.736966, KL(q||p) = 0.531004, JS = 0.146793 bits.

```
data_science_ml_ai999x.solve('ml_kl_divergence_discrete', p=[0.5, 0.5], q=[0.9, 0.1], unit='bits')
# -> kl_pq = 0.736966, kl_qp = 0.531004, js_divergence = 0.146793, cross_entropy = 1.73697,
entropy_p = 1   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_kl_divergence_discrete', p=[0.36, 0.48, 0.16], q=[0.333, 0.333,
0.334], unit='nats')
# -> kl_pq = 0.0858203, kl_qp = 0.0980925, js_divergence = 0.0226001, cross_entropy = 1.09913,
entropy_p = 1.01331   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
KL divergence: p = [0.5, 0.5], q = [0.9, 0.1], unit = bits	SOLVED	kl_pq = 0.736966, kl_qp = 0.531004, js_divergence = 0.146793, cross_entropy = 1.73697, entropy_p = 1
KL divergence: p = [0.36, 0.48, 0.16], q = [0.333, 0.333, 0.334], unit = nats	SOLVED	kl_pq = 0.0858203, kl_qp = 0.0980925, js_divergence = 0.0226001, cross_entropy = 1.09913, entropy_p = 1.01331
KL divergence: q = [0.9, 0.1], unit = bits	NEEDS_INPUT	ml_kl_divergence_discrete needs p (first distribution (positive, sums to 1)). Example: solve('ml_kl_divergence_discrete'
KL divergence: p = [0.5, 0.5], q = [0.9, 0.1], unit = bits, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
KL divergence: p = [0.5, 0.6], q = [0.9, 0.1], unit = bits	INVALID_INPUT	KL divergence, Jensen-Shannon divergence and cross-entropy of discrete distributions: p must sum to 1.

Cross-entropy with label smoothing, `ml_label_smoothing_cross_entropy`

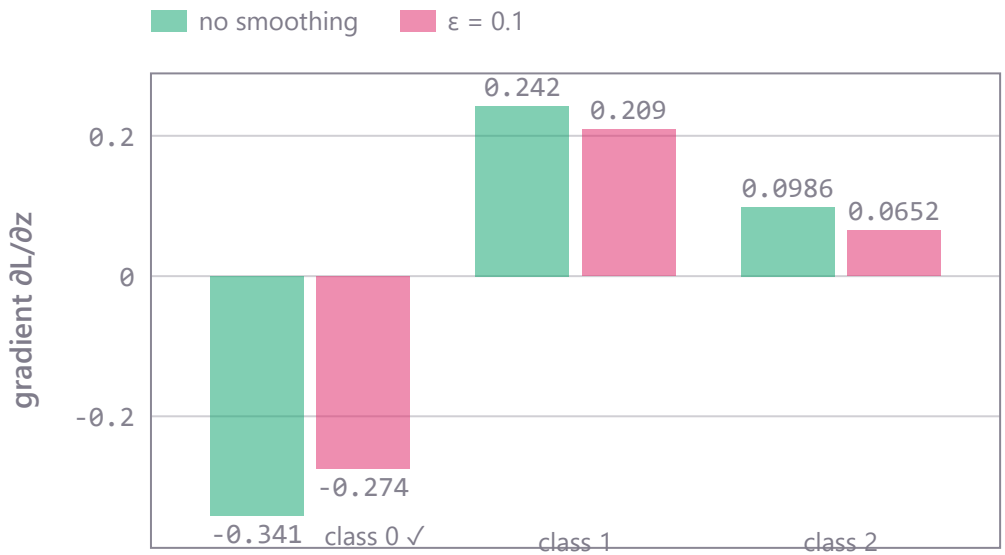
The cross-entropy of logits against a smoothed target (1 - epsilon on the true class plus epsilon spread evenly over all K classes) and its gradient with respect to the logits.

$$q_j = \frac{\varepsilon}{K} + (1-\varepsilon)\mathbb{1}(j=y), \quad L = -\sum_j q_j \log \sigma(z)_j$$

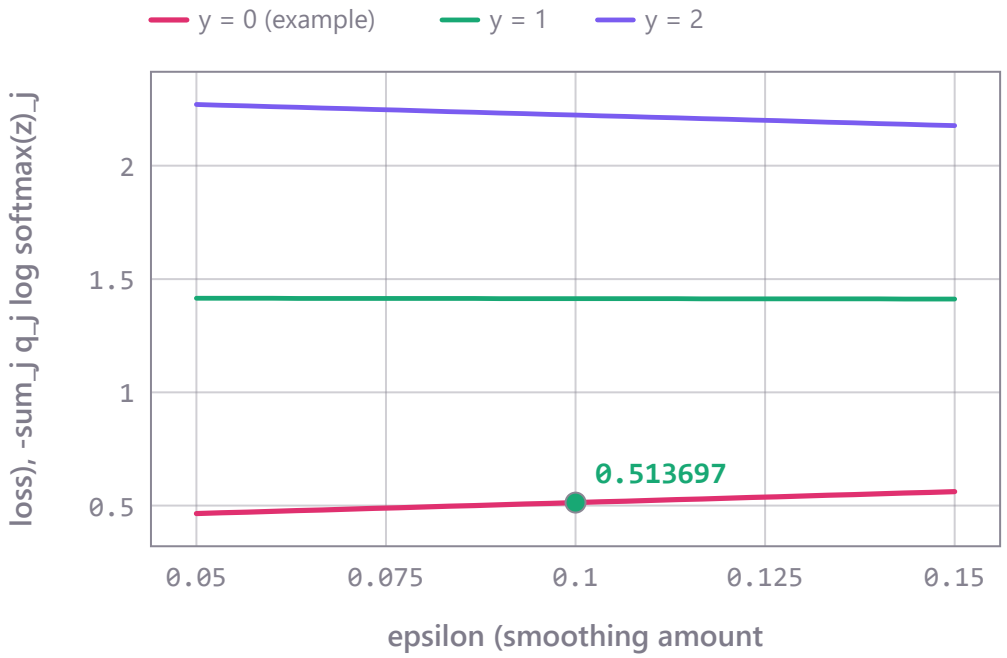
input	meaning	condition
`z`	logits	
`y`	index of the true class (0 to K-1)	
`epsilon`	smoothing amount (0 <= epsilon < 1)	

output	meaning
`loss`	-sum_j q_j log softmax(z)_j
`gradient`	softmax(z) - q

- **Method:** smoothed target q_j = epsilon/K + (1 - epsilon) 1(j = y); stable log-softmax
- **Assumes:** 0 <= epsilon < 1; y is a class index
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Gradient of the loss for logits z = [2, 1, 0.1], true class 0 (✓): plain cross-entropy (loss 0.41703) against label smoothing with ε = 0.1 (returned loss 0.513697); smoothing shrinks the push on the true class and on the others.



loss (-sum_j q_j log softmax(z)_j) against epsilon (smoothing amount (0 <= epsilon < 1)) from 0.05 to 0.15. One curve for each value of y (index of the true class (0 to K-1)): 0, 1, 2. Other inputs as in the worked example: z = [2, 1, 0.1]. 31 points per curve computed by data-science-ml-ai999x; marked: the worked example, epsilon = 0.1, loss = 0.513697.

```
data_science_ml_ai999x.solve('ml_label_smoothing_cross_entropy', z=[2, 1, 0.1], y=0, epsilon=0.1)
# -> loss = 0.513697, gradient = [-0.274332194447, 0.209099637371, 0.065232557076]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_label_smoothing_cross_entropy', z=[0.5, -1, 3, 0], y=1,
epsilon=0.2)
# -> loss = 3.81493, gradient = [0.0213666110359, -0.83407595665, 0.819423307941, -0.00671396232691]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Label smoothing cross-entropy: z = [2, 1, 0.1], y = 0, epsilon = 0.1	SOLVED	loss = 0.513697, gradient = [-0.274332194447, 0.209099637371, 0.065232557076]
Label smoothing cross-entropy: z = [0.5, -1, 3, 0], y = 1, epsilon = 0.2	SOLVED	loss = 3.81493, gradient = [0.0213666110359, -0.83407595665, 0.819423307941, -0.00671396232691]
Label smoothing cross-entropy: y = 0, epsilon = 0.1	NEEDS_INPUT	ml_label_smoothing_cross_entropy needs z (logits). Example: solve('ml_label_smoothing_cross_entropy', z=[2, 1, 0.1], y=0
Label smoothing cross-entropy: z = [2, 1, 0.1], y = 0, epsilon = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Label smoothing cross-entropy: z = [2, 1, 0.1], y = 0, epsilon = 1.2	INVALID_INPUT	Cross-entropy with label smoothing: epsilon must lie in [0, 1).

Binary focal loss and its gradient, `ml_focal_loss`

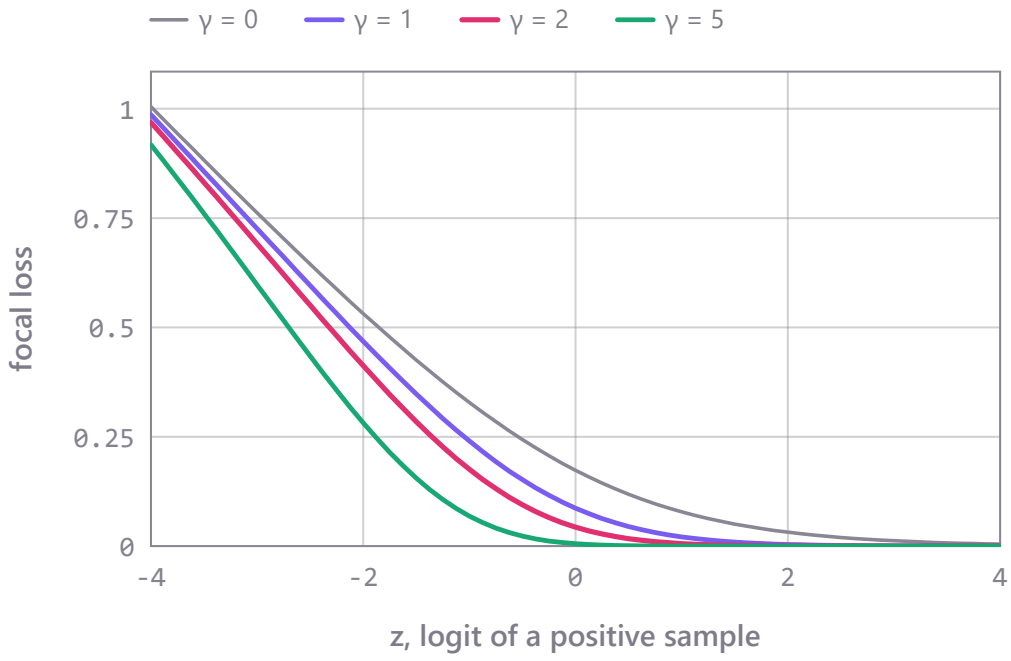
The mean binary focal loss -alpha_t $(1 - p_t)^\gamma$ log p_t from logits (it down-weights easy examples) and its gradient with respect to each logit.

$$FL = -\alpha_t (1 - p_t)^\gamma \log p_t$$

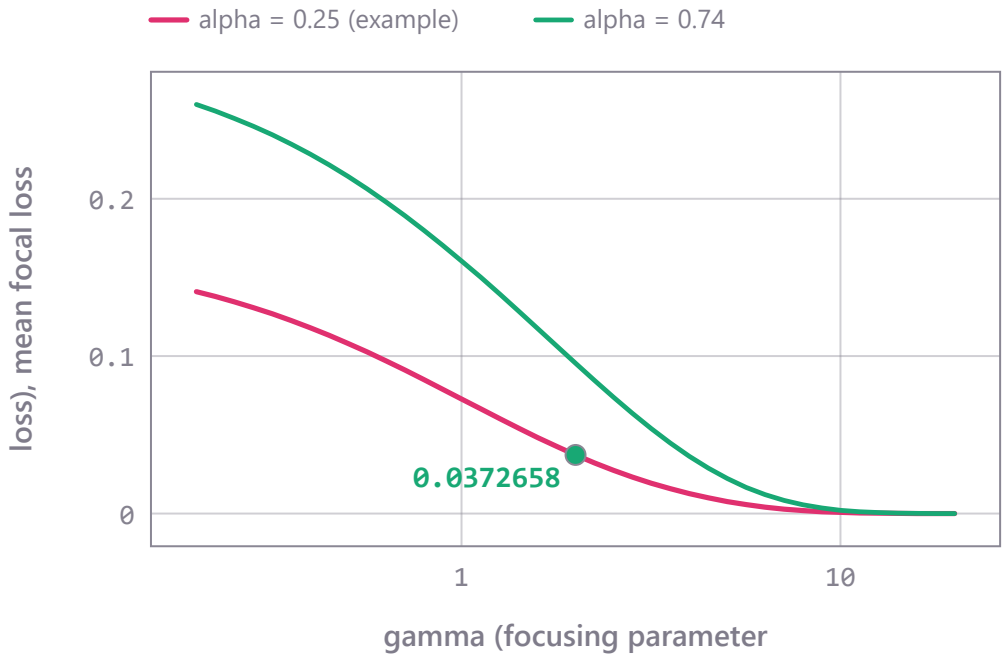
input	meaning	condition
`y`	labels (0 or 1)	
`z`	logits	
`gamma`	focusing parameter (>= 0; 0 gives weighted cross-entropy)	
`alpha`	weight of the positive class (0 to 1; negatives get 1 - alpha)	

output	meaning
`loss`	mean focal loss
`gradient`	dL/dz_i

- **Method:** $p_t = \sigma$(+-z), $1 - p_t = \sigma$(-+z) (no cancellation); analytic gradient alpha_t $(1 - p_t)^\gamma$ (gamma p_t log $p_t - (1 - p_t)$) with the sign of the label
- **Assumes:** labels 0 or 1; $\gamma \geq 0$; $0 \leq \alpha \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Focal loss of one positive sample (y = 1) as its logit z runs from −4 to 4, for focusing parameters γ = 0, 1, 2, 5 with α = 0.25 (65 runs each): larger γ silences the well-classified samples (large z). γ = 0 is the α-weighted cross-entropy.



loss (mean focal loss) against gamma (focusing parameter (>= 0; 0 gives weighted cross-entropy)) from 0.2 to 20 on a logarithmic axis. One curve for each value of alpha (weight of the positive class (0 to 1; negatives get 1 - alpha)): 0.25, 0.74. Other inputs as in the worked example: y = [1, 0, 1], z = [2, -1, -0.5]. 41 points per curve computed by data-science-ml-ai999x; marked: the worked example, gamma = 2, loss = 0.0372658.

```
data_science_ml_ai999x.solve('ml_focal_loss', y=[1, 0, 1], z=[2, -1, -0.5], gamma=2, alpha=0.25)
# -> loss = 0.0372658, gradient = [-0.000405911682949, 0.0131452824944, -0.0438459851562]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_focal_loss', y=[0, 1], z=[1.5, 0.3], gamma=0, alpha=0.5)
# -> loss = 0.563942, gradient = [0.204393619048, -0.106389370797]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Focal loss: y = [1, 0, 1], z = [2, -1, -0.5], gamma = 2, alpha = 0.25	SOLVED	loss = 0.0372658, gradient = [-0.000405911682949, 0.0131452824944, -0.0438459851562]
Focal loss: y = [0, 1], z = [1.5, 0.3], gamma = 0, alpha = 0.5	SOLVED	loss = 0.563942, gradient = [0.204393619048, -0.106389370797]
Focal loss: z = [2, -1, -0.5], gamma = 2, alpha = 0.25	NEEDS_INPUT	ml_focal_loss needs y (labels (0 or 1)). Example: solve('ml_focal_loss', y=[1, 0, 1], z=[2, -1, -0.5], gamma=2, alpha=0.
Focal loss: y = [1, 0, 1], z = [2, -1, -0.5], gamma = 2, alpha = 0.25, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Focal loss: y = [1, 0], z = [2, -1], gamma = -1, alpha = 0.25	INVALID_INPUT	Binary focal loss and its gradient: gamma must be >= 0.

Mean squared error loss and gradient of a linear model, `ml_mse_gradient`

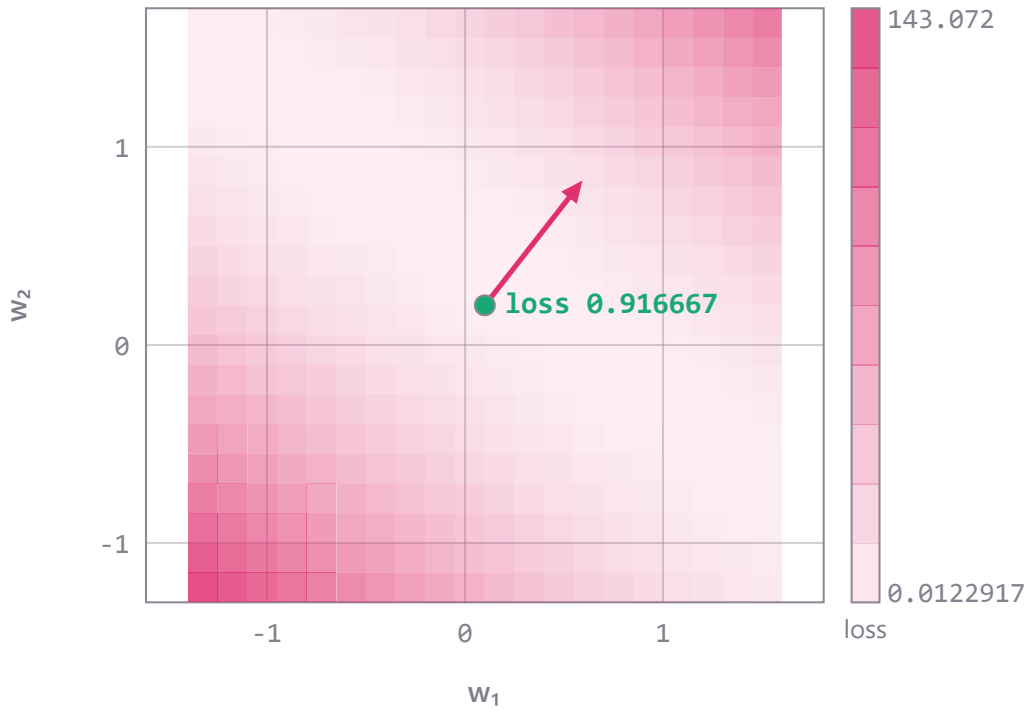
The mean squared error of a linear model y_hat = X w + b and its gradient with respect to the weights and the bias.

$$L = \frac{1}{n}\|Xw + b - y\|^2, \quad \nabla_w L = \frac{2}{n} X^\top (Xw + b - y)$$

input	meaning	condition
`X`	feature matrix: one row per sample	
`y`	targets, one per row	
`w`	weights, one per column of X	
`b`	bias (intercept)	

output	meaning
`loss`	mean (X w + b - y)^2
`grad_w`	$dL/dw = (2/n)\ X^T$ (X w + b - y)
`grad_b`	$dL/db = (2/n)$ sum of residuals

- **Method:** exact rational arithmetic (the loss and gradient are polynomials in the inputs)
- **Assumes:** dimensions agree
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The mean squared error over weights (w_1, w_2) within ±1.5 of the worked example's w = [0.1, 0.2], other inputs as in the worked example (400 runs, darker = higher). Marked: the worked example, returned loss 0.916667; the arrow points along the returned negative gradient −(-6.46667, -8.26667), drawn 0.8 long.

```
data_science_ml_ai999x.solve('ml_mse_gradient', X=[[1, 2], [3, 4], [5, 6]], y=[1, 2, 3], w=[0.1,
0.2], b=0)
# -> loss = 0.916667, grad_w = [-6.46666666667, -8.26666666667], grad_b = -1.8   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_mse_gradient', X=[[1], [2], [3], [4]], y=[2, 4, 6, 8], w=[1], b=1)
# -> loss = 3.5, grad_w = [-10.0], grad_b = -3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MSE gradient of a linear model: X = [[1, 2], [3, 4], [5, 6]], y = [1, 2, 3], w = [0.1, 0.2], b = 0	SOLVED	loss = 0.916667, grad_w = [-6.46666666667, -8.26666666667], grad_b = -1.8
MSE gradient of a linear model: X = [[1], [2], [3], [4]], y = [2, 4, 6, 8], w = [1], b = 1	SOLVED	loss = 3.5, grad_w = [-10.0], grad_b = -3
MSE gradient of a linear model: y = [1, 2, 3], w = [0.1, 0.2], b = 0	NEEDS_INPUT	ml_mse_gradient needs X (feature matrix: one row per sample). Example: solve('ml_mse_gradient', X=[[1, 2], [3, 4], [5, 6
MSE gradient of a linear model: X = [[1, 2], [3, 4], [5, 6]], y = [1, 2, 3], w = [0.1, 0.2], b = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
MSE gradient of a linear model: X = [[1, 2], [3, 4]], y = [1, 2], w = [0.1], b = 0	INVALID_INPUT	Mean squared error loss and gradient of a linear model: w needs one weight per column of X.

Logistic regression loss and gradient (L2-regularised), `ml_logistic_loss_gradient`

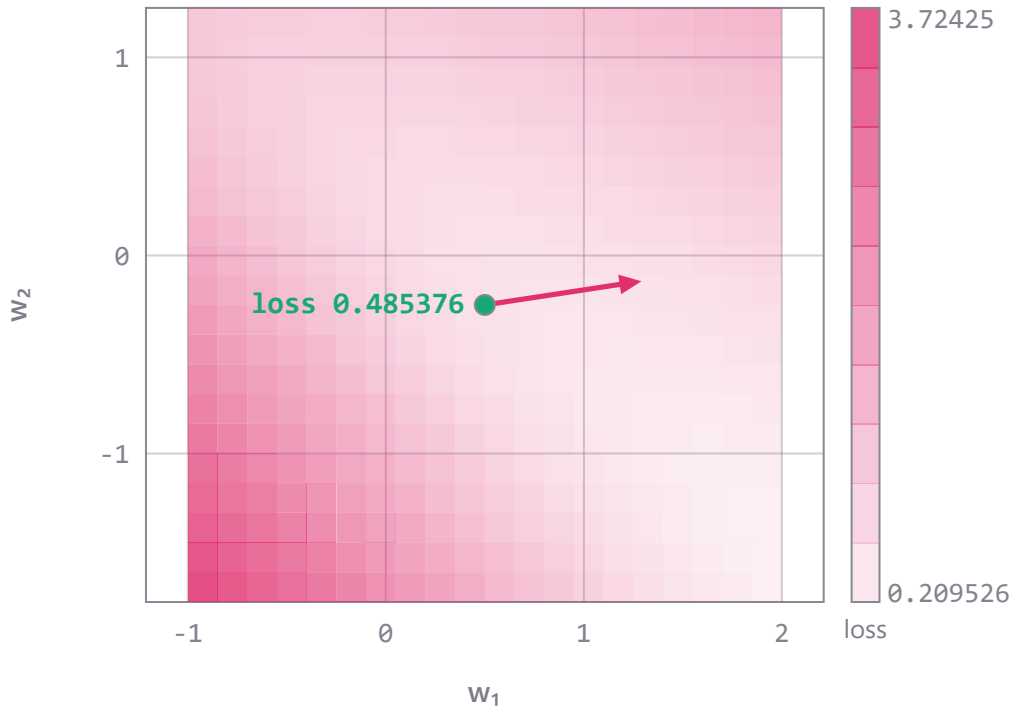
The mean logistic (binary cross-entropy) loss of a linear model with an L2 penalty (lam / 2) ||w||^2 and its gradient with respect to the weights and the bias.

$$L = \frac{1}{n}\sum_i \left[\log(1+e^{z_i}) - y_i z_i\right] + \frac{\lambda}{2}\|w\|^2, \quad \nabla_w L = \frac{1}{n} X^\top(\sigma(z) - y) + \lambda w$$

input	meaning	condition
`X`	feature matrix	
`y`	labels (0 or 1)	
`w`	weights	
`b`	bias	
`lam`	L2 regularisation strength (>= 0; the bias is not penalised)	

output	meaning
`loss`	mean log loss + (lam/2)||w||^2
`grad_w`	X^T (sigma(Xw + b) - y) / n + lam w
`grad_b`	mean of sigma(Xw + b) - y

- **Method:** stable log-sum-exp form of the log loss; analytic gradient
- **Assumes:** labels 0 or 1; lam >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The mean logistic loss over weights (w_1, w_2) within ±1.5 of the worked example's w = [0.5, -0.25], other inputs as in the worked example (400 runs, darker = higher). Marked: the worked example, returned loss 0.485376; the arrow points along the returned negative gradient −(-0.324062, -0.0492577), drawn 0.8 long.

```
data_science_ml_ai999x.solve('ml_logistic_loss_gradient', X=[[1, 2], [2, 1], [3, 3]], y=[0, 1, 1], w=[0.5, -0.25], b=0.1, lam=0)
# -> loss = 0.485376, grad_w = [-0.32406170005, -0.0492576850487], grad_b = -0.0246288   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_logistic_loss_gradient', X=[[0.5], [1.5], [-1]], y=[1, 1, 0], w=[1], b=0, lam=0.1)
# -> loss = 0.379584, grad_w = [-0.14378334716], grad_b = -0.0970083   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic regression gradient: X = [[1, 2], [2, 1], [3, 3]], y = [0, 1, 1], w = [0.5, -0.25], b = 0.1, lam = 0	SOLVED	loss = 0.485376, grad_w = [-0.32406170005, -0.0492576850487], grad_b = -0.0246288
Logistic regression gradient: X = [[0.5], [1.5], [-1]], y = [1, 1, 0], w = [1], b = 0, lam = 0.1	SOLVED	loss = 0.379584, grad_w = [-0.14378334716], grad_b = -0.0970083
Logistic regression gradient: y = [0, 1, 1], w = [0.5, -0.25], b = 0.1, lam = 0	NEEDS_INPUT	ml_logistic_loss_gradient needs X (feature matrix). Example: solve('ml_logistic_loss_gradient', X=[[1, 2], [2, 1], [3, 3
Logistic regression gradient: X = [[1, 2], [2, 1], [3, 3]], y = [0, 1, 1], w = [0.5, -0.25], b = 0.1, lam = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Logistic regression gradient: X = [[1, 2], [2, 1]], y = [0, 2], w = [0.5, -0.25], b = 0.1, lam = 0	INVALID_INPUT	Logistic regression loss and gradient (L2-regularised): labels y must be 0 or 1.

One gradient-descent step for linear regression, `ml_gradient_descent_step_linear`

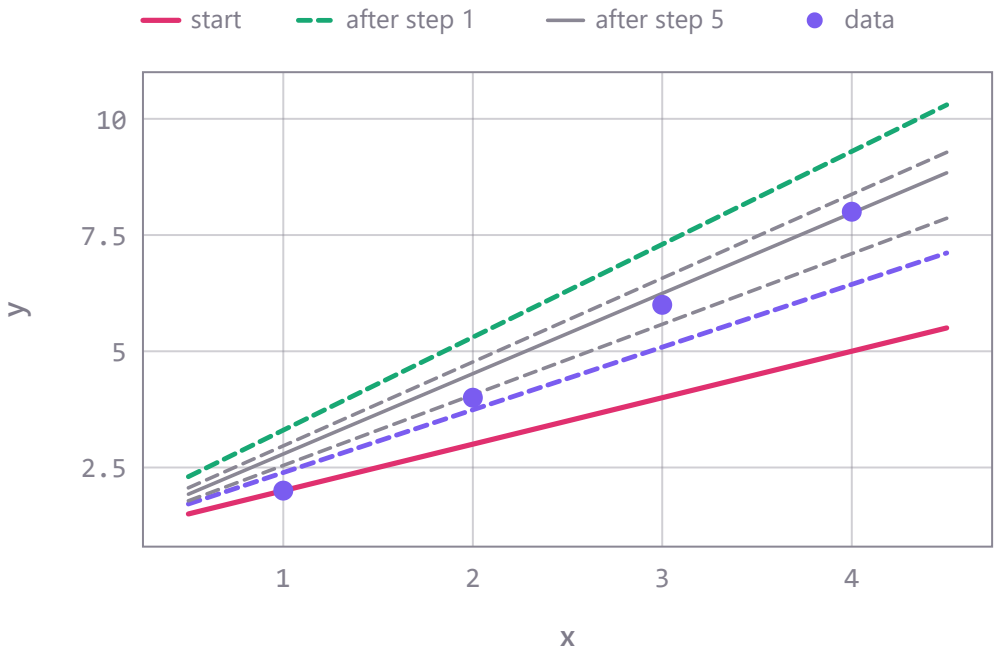
One full-batch gradient-descent step on the mean squared error of a linear model: the new weights and bias and the loss before and after the step.

$$w \leftarrow w - \eta \nabla_w L, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

input	meaning	condition
`X`	feature matrix: one row per sample	
`y`	targets, one per row	
`w`	weights, one per column of X	
`b`	bias (intercept)	
`lr`	learning rate (> 0)	

output	meaning
`w_new`	w - lr dL/dw
`b_new`	b - lr dL/db
`loss_before`	MSE before the step
`loss_after`	MSE after the step

- **Method:** exact rational arithmetic
- **Assumes:** $lr > 0$; dimensions agree
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Five gradient-descent steps for a line through x = [1, 2, 3, 4], y = [2, 4, 6, 8] from w = [1], b = 1 (lr = 0.1), each a run of the package from the w, b the previous run returned; MSE after each step: 1.69, 0.87035, 0.495762, 0.321386, 0.237267.

```
data_science_ml_ai999x.solve('ml_gradient_descent_step_linear', X=[[1], [2], [3], [4]], y=[2, 4, 6, 8], w=[1], b=1, lr=0.1)
# -> w_new = [2.0], b_new = 1.3, loss_before = 3.5, loss_after = 1.69   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_gradient_descent_step_linear', X=[[1, 2], [3, 4], [5, 6]], y=[1, 2, 3], w=[0.1, 0.2], b=0, lr=0.01)
# -> w_new = [0.164666666667, 0.282666666667], b_new = 0.018, loss_before = 0.916667, loss_after = 0.135084   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gradient descent step for linear regression: X = [[1], [2], [3], [4]], y = [2, 4, 6, 8], w = [1], b = 1, lr = 0.1	SOLVED	w_new = [2.0], b_new = 1.3, loss_before = 3.5, loss_after = 1.69
Gradient descent step for linear regression: X = [[1, 2], [3, 4], [5, 6]], y = [1, 2, 3], w = [0.1, 0.2], b = 0, lr = 0.01	SOLVED	w_new = [0.164666666667, 0.282666666667], b_new = 0.018, loss_before = 0.916667, loss_after = 0.135084
Gradient descent step for linear regression: y = [2, 4, 6, 8], w = [1], b = 1, lr = 0.1	NEEDS_INPUT	ml_gradient_descent_step_linear needs X (feature matrix: one row per sample). Example: solve('ml_gradient_descent_step_l
Gradient descent step for linear regression: X = [[1], [2], [3], [4]], y = [2, 4, 6, 8], w = [1], b = 1, lr = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Distances and cosine similarity between two vectors, `ml_distance_metrics`

The Euclidean, Manhattan and Chebyshev distances between two feature vectors, and their cosine similarity and cosine distance.

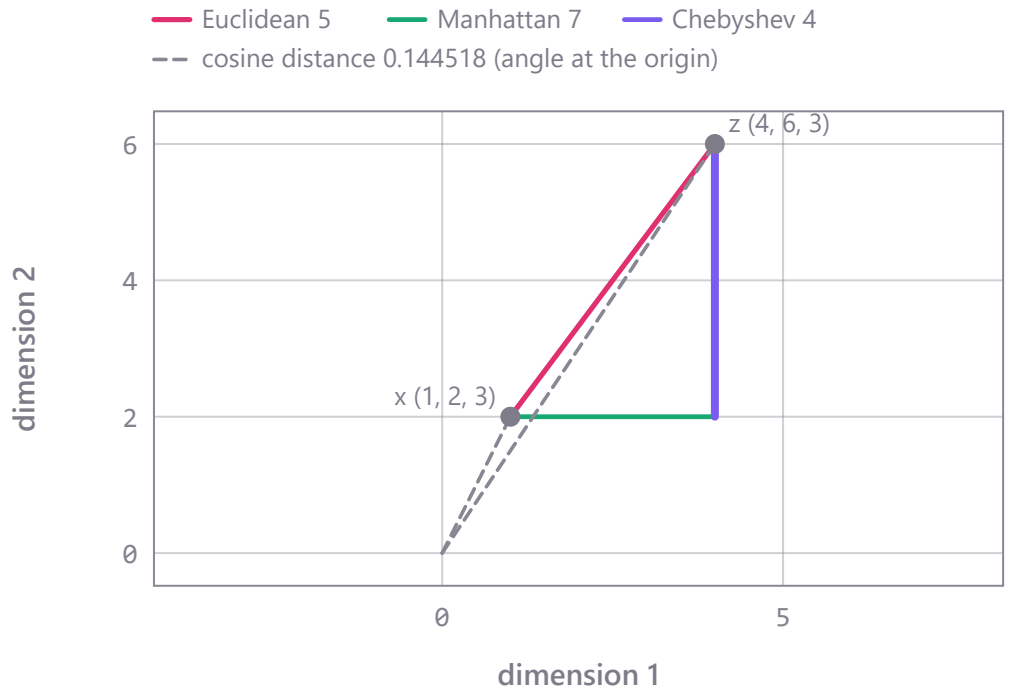
$$d_2 = \sqrt{\sum_i (x_i - z_i)^2}, \quad \cos\theta = \frac{x \cdot z}{\|x\|\|z\|}$$

input	meaning	condition
`x`	first vector	
`z`	second vector	

output	meaning
`euclidean`	sqrt(sum (x - z)^2)
`manhattan`	sum |x - z|
`chebyshev`	max |x - z|
`cosine_similarity`	x.z / (|x| |z|)
`cosine_distance`	1 - cosine similarity

- **Method:** direct sums with fsum
- **Assumes:** equal lengths; non-zero vectors

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The distances the package returns between the worked example's points x and z (3 dimensions; drawn in the first two, the others shown in the labels): straight line 5, city-block path 7, largest coordinate gap 4, cosine similarity 0.855482 (distance 0.144518). The legs drawn are those of the first two coordinates.

```
data_science_ml_ai999x.solve('ml_distance_metrics', x=[1, 2, 3], z=[4, 6, 3])
# -> euclidean = 5, manhattan = 7, chebyshev = 4, cosine_similarity = 0.855482, cosine_distance =
0.144518   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_distance_metrics', x=[1, 0], z=[1, 1])
# -> euclidean = 1, manhattan = 1, chebyshev = 1, cosine_similarity = 0.707107, cosine_distance =
0.292893   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Distance metrics: x = [1, 2, 3], z = [4, 6, 3]	SOLVED	euclidean = 5, manhattan = 7, chebyshev = 4, cosine_similarity = 0.855482, cosine_distance = 0.144518
Distance metrics: x = [1, 0], z = [1, 1]	SOLVED	euclidean = 1, manhattan = 1, chebyshev = 1, cosine_similarity = 0.707107, cosine_distance = 0.292893
Distance metrics: z = [4, 6, 3]	NEEDS_INPUT	ml_distance_metrics needs x (first vector). Example: solve('ml_distance_metrics', x=[1, 2, 3], z=[4, 6, 3]).
Distance metrics: x = [1, 2, 3], z = [4, 6, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Distance metrics: x = [0, 0], z = [1, 1]	INVALID_INPUT	Distances and cosine similarity between two vectors: cosine similarity needs two non-zero vectors.

RBF (Gaussian) kernel matrix, `ml_rbf_kernel_matrix`

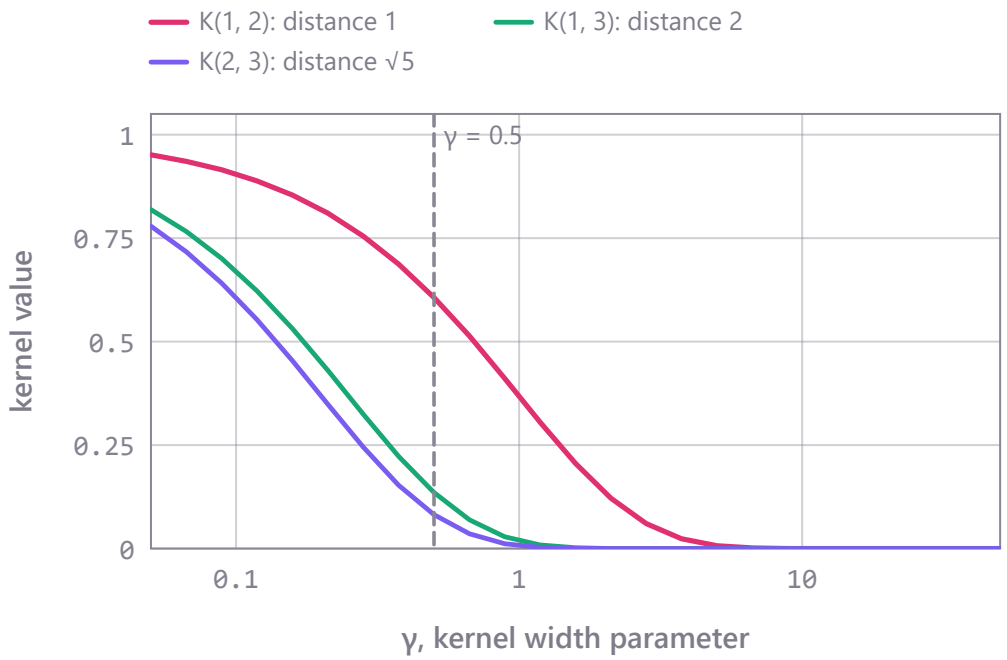
The Gram matrix of the radial basis function kernel exp(-gamma ||x_i - x_j||^2) for a set of points, and the mean off-diagonal similarity.

$$K_{ij} = \exp\left(-\gamma \left\| x_i - x_j \right\|^2\right)$$

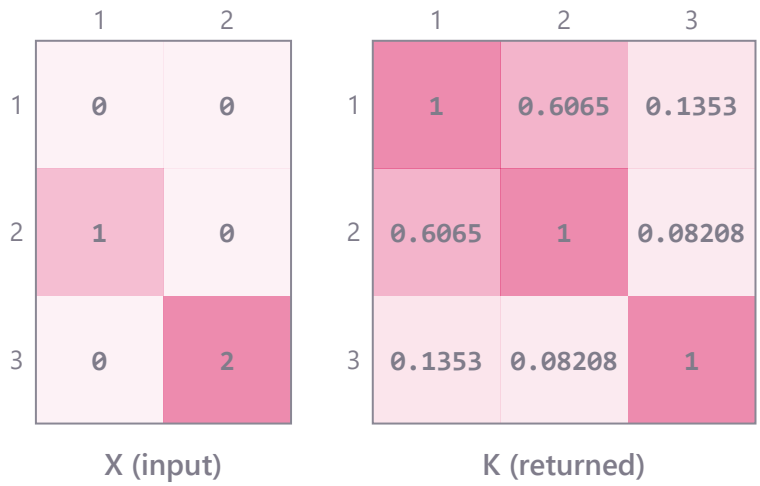
input	meaning	condition
`X`	points: one row per point (at least two)	
`gamma`	kernel width parameter (> 0)	

output	meaning
`K`	kernel matrix $K_ij = exp$(-gamma ||x_i - x_j||^2)
`mean_offdiagonal`	mean of the off-diagonal entries

- **Method:** squared distances with fsum, then exp
- **Assumes:** $\gamma > 0$; at least two points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Off-diagonal entries of the RBF kernel matrix of the stated points X = [[0, 0], [1, 0], [0, 2]] as γ runs from 0.05 to 50 (25 runs): exp(−γ d^2) falls fastest for the farthest pair; the worked example uses γ = 0.5.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); K (returned). Returned with them by data-science-ml-ai999x: mean_offdiagonal = 0.27465. Other inputs: gamma = 0.5.

```
data_science_ml_ai999x.solve('ml_rbf_kernel_matrix', X=[[0, 0], [1, 0], [0, 2]], gamma=0.5)
# -> K = [[1.0, 0.606530659713, 0.135335283237], [0.606530659713, 1.0, 0.0820849986239],
[0.135335283237, 0.0820849986239, 1.0]], mean_offdiagonal = 0.27465   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_rbf_kernel_matrix', X=[[1], [2], [4]], gamma=1)
# -> K = [[1.0, 0.367879441171, 0.000123409804087], [0.367879441171, 1.0, 0.0183156388887],
[0.000123409804087, 0.0183156388887, 1.0]], mean_offdiagonal = 0.128773   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
RBF kernel matrix: X = [[0, 0], [1, 0], [0, 2]], gamma = 0.5	SOLVED	K = [[1.0, 0.606530659713, 0.135335283237], [0.606530659713, 1.0, 0.0820849986239], [0.135335283237, 0.0820849986239, 1.0]], mean_offdiagonal = 0.27465
RBF kernel matrix: X = [[1], [2], [4]], gamma = 1	SOLVED	K = [[1.0, 0.367879441171, 0.000123409804087], [0.367879441171, 1.0, 0.0183156388887], [0.000123409804087, 0.0183156388887, 1.0]], mean_offdiagonal = 0.128773
RBF kernel matrix: gamma = 0.5	NEEDS_INPUT	ml_rbf_kernel_matrix needs X (points: one row per point (at least two)). Example: solve('ml_rbf_kernel_matrix', X=[[0, 0
RBF kernel matrix: X = [[0, 0], [1, 0], [0, 2]], gamma = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
RBF kernel matrix: X = [[0, 0], [1, 0]], gamma = -1	INVALID_INPUT	RBF (Gaussian) kernel matrix: gamma must be positive.

Shannon entropy and perplexity of a distribution, `ml_entropy`

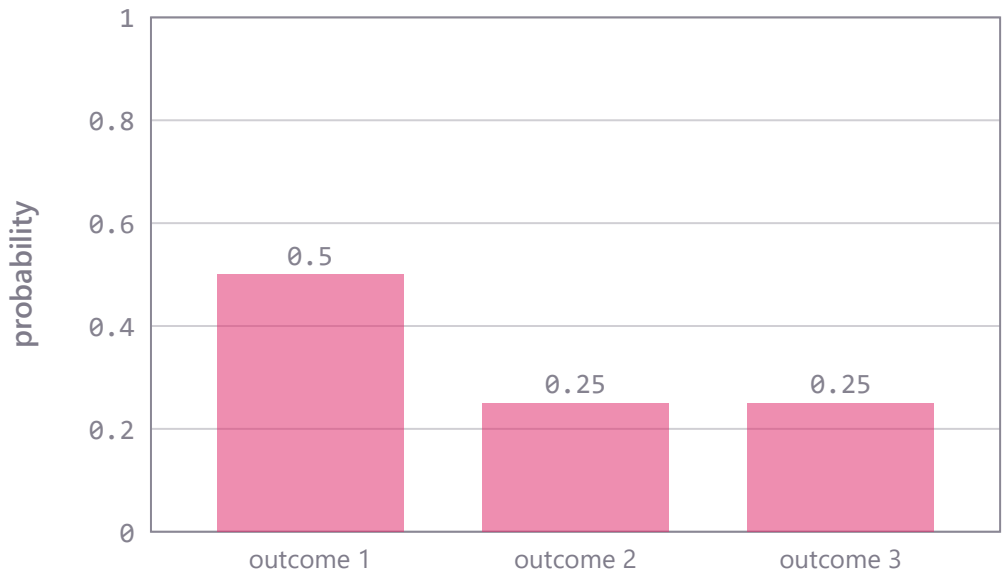
The Shannon entropy of a discrete probability distribution in the stated unit, its perplexity (e^H in nats = 2^H in bits), the maximum entropy log K and the normalised entropy H / log K.

$$H(p) = -\sum_i p_i \log p_i$$

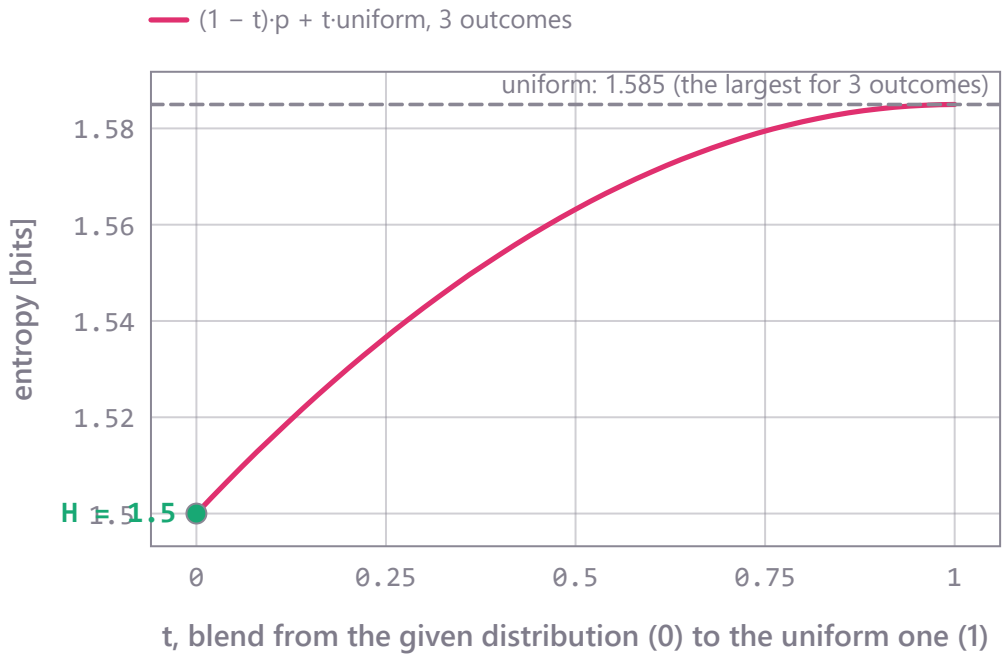
input	meaning	condition
`p`	probabilities (>= 0, sum to 1)	
`unit`	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	

output	meaning
`entropy`	H(p) in the stated unit
`perplexity`	exp(H in nats)
`max_entropy`	log K in the stated unit
`normalized_entropy`	H / log K (0 for K = 1)

- **Method:** -sum p log p with 0 log $0 = 0$ (fsum)
- **Assumes:** probabilities >= 0 summing to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated distribution p = [0.5, 0.25, 0.25]: returned entropy 1.5 bits (maximum 1.58496 for three outcomes, normalised 0.946395), perplexity 2.82843, as uncertain as 2.82843 equally likely outcomes.



Shannon entropy (bits) of the worked example's distribution p = ['0.5', '0.25', '0.25'] blended step by step with the uniform distribution over its 3 outcomes (51 runs): entropy rises to its largest value, 1.58496, when every outcome is equally likely. Marked: the worked example itself, H = 1.5.

```
data_science_ml_ai999x.solve('ml_entropy', p=[0.5, 0.25, 0.25], unit='bits')
# -> entropy = 1.5, perplexity = 2.82843, max_entropy = 1.58496, normalized_entropy = 0.946395
(SOLVED)
```

```
data_science_ml_ai999x.solve('ml_entropy', p=[0.9, 0.1], unit='nats')
# -> entropy = 0.325083, perplexity = 1.38415, max_entropy = 0.693147, normalized_entropy = 0.468996
(SOLVED)
```

```
data_science_ml_ai999x.solve('ml_entropy', p=[0.25, 0.25, 0.25, 0.25], unit='bits')
# -> entropy = 2, perplexity = 4, max_entropy = 2, normalized_entropy = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Entropy of a distribution: p = [0.5, 0.25, 0.25], unit = bits	SOLVED	entropy = 1.5, perplexity = 2.82843, max_entropy = 1.58496, normalized_entropy = 0.946395
Entropy of a distribution: p = [0.9, 0.1], unit = nats	SOLVED	entropy = 0.325083, perplexity = 1.38415, max_entropy = 0.693147, normalized_entropy = 0.468996
Entropy of a distribution: p = [0.25, 0.25, 0.25, 0.25], unit = bits	SOLVED	entropy = 2, perplexity = 4, max_entropy = 2, normalized_entropy = 1
Entropy of a distribution: unit = bits	NEEDS_INPUT	ml_entropy needs p (probabilities (>= 0, sum to 1)). Example: solve('ml_entropy', p=[0.5, 0.25, 0.25], unit='bits').
Entropy of a distribution: p = [0.5, 0.25, 0.25], unit = bits, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Entropy of a distribution: p = [0.5, 0.6], unit = bits	INVALID_INPUT	Shannon entropy and perplexity of a distribution: the probabilities must sum to 1.

Joint, marginal and conditional entropy and mutual information, `ml_joint_conditional_entropy`

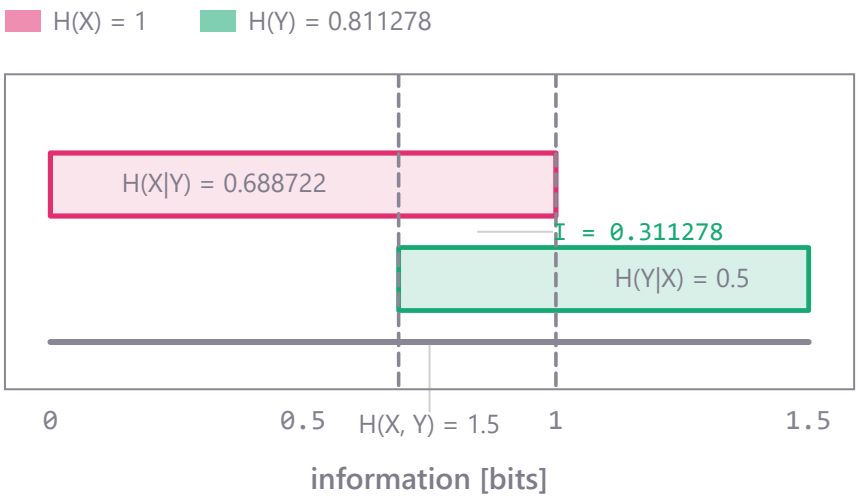
From a joint probability table of X (rows) and Y (columns): the joint entropy, both marginal entropies, both conditional entropies and the mutual information.

$$H(Y \mid X) = H(X,Y) - H(X), \quad I(X;Y) = \sum_{ij} P_{ij} \log \frac{P_{ij}}{P_i P_j}$$

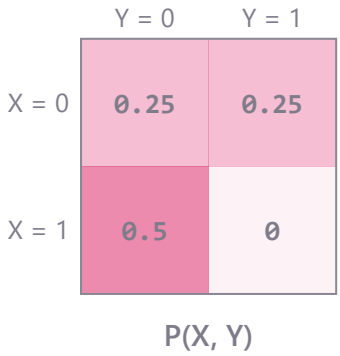
input	meaning	condition
`P`	joint probabilities P($X = i$, Y = j): rows are values of X, columns values of Y; sums to 1	
`unit`	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	

output	meaning
`H_XY`	joint entropy
`H_X`	entropy of X
`H_Y`	entropy of Y
`H_Y_given_X`	H(Y | X) = H(X, Y) - H(X)
`H_X_given_Y`	H(X | Y)
`mutual_information`	I(X; Y) = sum P log P / (P_X P_Y)

- **Method:** entropies of the table and its marginals; mutual information summed directly
- **Assumes:** probabilities >= 0 summing to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Information diagram of the joint distribution P = [[0.25, 0.25], [0.5, 0]] (rows X, columns Y): H(X) = 1 and H(Y) = 0.811278 drawn as overlapping bars; the overlap is the mutual information 0.311278, the rest the conditional entropies H(X|Y) = 0.688722 and H(Y|X) = 0.5; the line spans H(X, Y) = 1.5 bits, all returned values.



The stated joint distribution P(X, Y) = [[0.25, 0.25], [0.5, 0]], rows the values of X and columns the values of Y.

```
data_science_ml_ai999x.solve('ml_joint_conditional_entropy', P=[[0.25, 0.25], [0.5, 0]],
unit='bits')
# -> H_XY = 1.5, H_X = 1, H_Y = 0.811278, H_Y_given_X = 0.5, H_X_given_Y = 0.688722,
mutual_information = 0.311278   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_joint_conditional_entropy', P=[[0.1, 0.2, 0.1], [0.3, 0.1, 0.2]],
unit='nats')
# -> H_XY = 1.69574, H_X = 0.673012, H_Y = 1.0889, H_Y_given_X = 1.02273, H_X_given_Y = 0.606843,
mutual_information = 0.0661691   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Joint and conditional entropy: P = [[0.25, 0.25], [0.5, 0]], unit = bits	SOLVED	H_XY = 1.5, H_X = 1, H_Y = 0.811278, H_Y_given_X = 0.5, H_X_given_Y = 0.688722, mutual_information = 0.311278
Joint and conditional entropy: P = [[0.1, 0.2, 0.1], [0.3, 0.1, 0.2]], unit = nats	SOLVED	H_XY = 1.69574, H_X = 0.673012, H_Y = 1.0889, H_Y_given_X = 1.02273, H_X_given_Y = 0.606843, mutual_information = 0.0661691
Joint and conditional entropy: unit = bits	NEEDS_INPUT	ml_joint_conditional_entropy needs P (joint probabilities P(X = i, Y = j): rows are values of X, columns values of Y; su
Joint and conditional entropy: P = [[0.25, 0.25], [0.5, 0]], unit = bits, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Joint and conditional entropy: P = [[0.5, 0.5], [0.5, 0]], unit = bits	INVALID_INPUT	Joint, marginal and conditional entropy and mutual information: the joint probabilities must sum to 1.

Mutual information between two label sequences, `ml_mutual_information_labels`

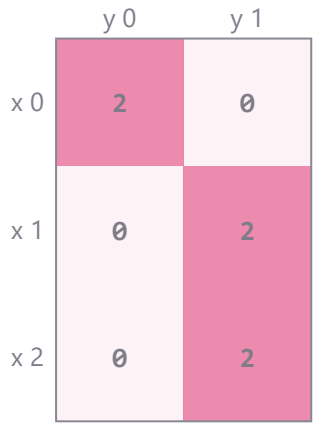
The mutual information between two discrete variables observed together (two label lists), with both entropies and the conditional entropy H(x | y), from the empirical joint distribution.

$$I(X;Y) = \sum_{ij} \frac{n_{ij}}{n} \log \frac{n\, n_{ij}}{a_i b_j}$$

input	meaning	condition
`x`	values of the first variable (whole numbers)	
`y`	values of the second variable (whole numbers)	
`unit`	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	

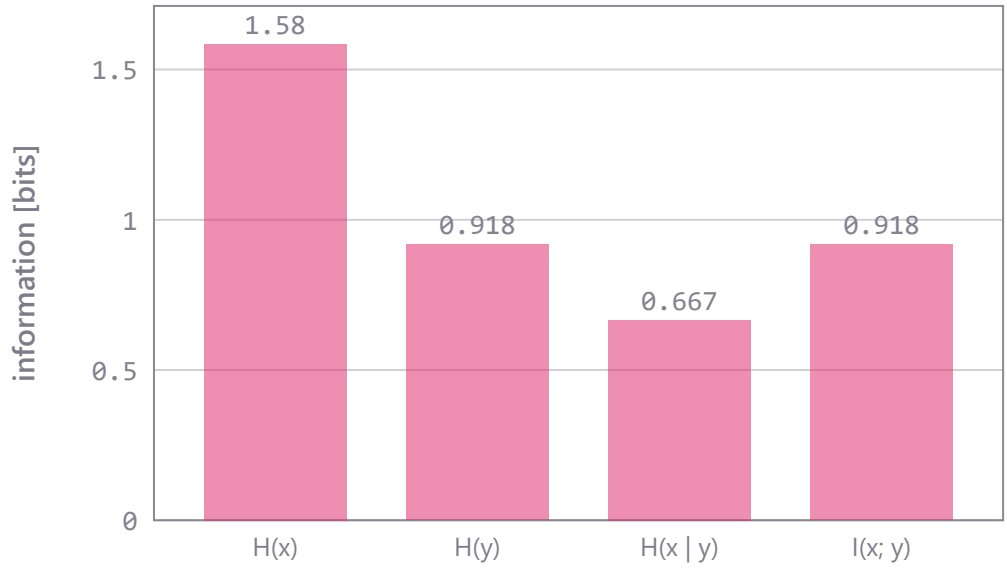
output	meaning
`mutual_information`	I(x; y)
`H_x`	entropy of x
`H_y`	entropy of y
`H_x_given_y`	H(x | y) = H(x) - I

- **Method:** contingency table of counts; I = sum n_ij/n log(n n_ij / (a_i b_j))
- **Assumes:** labels are whole numbers; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: x, columns: y

Counts of each pair (x, y) tallied from the stated labels x = [0, 0, 1, 1, 2, 2] and y = [0, 0, 1, 1, 1, 1]; returned (bits): H(x) = 1.58496, H(y) = 0.918296, H(x | y) = 0.666667, mutual information 0.918296.



Entropies the package returns for x = [0, 0, 1, 1, 2, 2] and y = [0, 0, 1, 1, 1, 1] (in bits): H(x) = 1.58496, H(y) = 0.918296, H(x | y) = 0.666667, I(x; y) = 0.918296; I(x; y) = H(x) − H(x | y).

```
data_science_ml_ai999x.solve('ml_mutual_information_labels', x=[0, 0, 1, 1, 2, 2], y=[0, 0, 1, 1, 1, 1], unit='bits')
# -> mutual_information = 0.918296, H_x = 1.58496, H_y = 0.918296, H_x_given_y = 0.666667   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_mutual_information_labels', x=[1, 1, 2, 2, 3, 3, 3, 1], y=[1, 2, 2, 2, 3, 3, 1, 1], unit='nats')
# -> mutual_information = 0.60481, H_x = 1.0822, H_y = 1.0822, H_x_given_y = 0.477386   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mutual information of labels: x = [0, 0, 1, 1, 2, 2], y = [0, 0, 1, 1, 1, 1], unit = bits	SOLVED	mutual_information = 0.918296, H_x = 1.58496, H_y = 0.918296, H_x_given_y = 0.666667
Mutual information of labels: x = [1, 1, 2, 2, 3, 3, 3, 1], y = [1, 2, 2, 2, 3, 3, 1, 1], unit = nats	SOLVED	mutual_information = 0.60481, H_x = 1.0822, H_y = 1.0822, H_x_given_y = 0.477386
Mutual information of labels: y = [0, 0, 1, 1, 1, 1], unit = bits	NEEDS_INPUT	ml_mutual_information_labels needs x (values of the first variable (whole numbers)). Example: solve('ml_mutual_informati
Mutual information of labels: x = [0, 0, 1, 1, 2, 2], y = [0, 0, 1, 1, 1, 1], unit = bits, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Information gain and gain ratio of a split, `ml_information_gain`

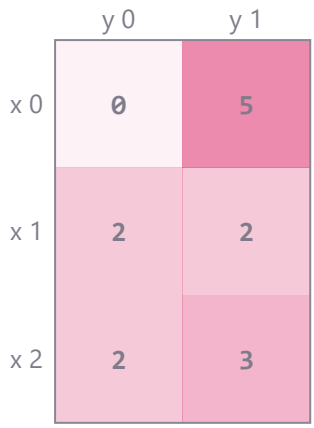
The information gain (in bits) of splitting class labels y by a discrete feature x, with the parent and conditional entropies, the split information and the gain ratio (C4.5).

$$IG = H(Y) - \sum_{v} \frac{|S_v|}{|S|} H(Y \mid X = v), \quad GR = \frac{IG}{H(X)}$$

input	meaning	condition
`y`	class labels (whole numbers)	
`x`	feature value of each sample (whole numbers; each value is one branch)	

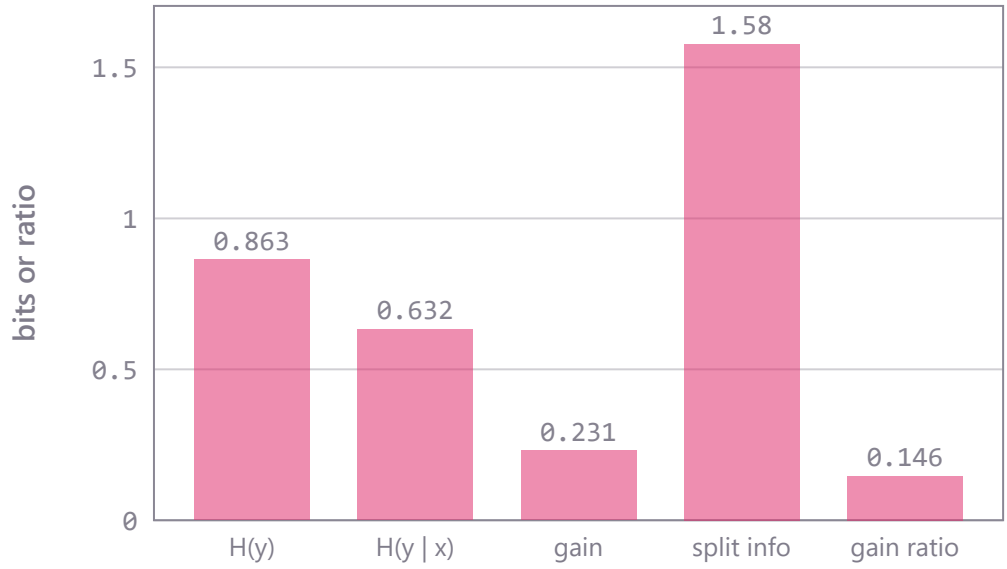
output	meaning
`parent_entropy`	H(y) in bits
`conditional_entropy`	H(y | x) in bits
`information_gain`	H(y) - H(y | x) in bits
`split_info`	H(x) in bits
`gain_ratio`	information gain / split information

- **Method:** entropies of the branches weighted by their sizes
- **Assumes:** the feature takes at least two values
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: x, columns: y

Counts of each pair (x, y) tallied from the stated labels x = [0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1, 1, 2] and y = [1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0]; returned: parent entropy 0.863121, conditional entropy 0.632482, information gain 0.230638 bits, split information 1.57741, gain ratio 0.146214.



The values the package returns for splitting y = [1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0] on x = [0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1, 1, 2]: H(y) 0.863121, H(y | x) 0.632482, gain 0.230638, split info 1.57741, gain ratio 0.146214; gain = H(y) − H(y | x), ratio = gain / split info.

```
data_science_ml_ai999x.solve('ml_information_gain', y=[1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0], x=
[0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1, 1, 2])
# -> parent_entropy = 0.863121, conditional_entropy = 0.632482, information_gain = 0.230638,
split_info = 1.57741, gain_ratio = 0.146214   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_information_gain', y=[0, 0, 1, 1], x=[0, 0, 1, 1])
# -> parent_entropy = 1, conditional_entropy = 0, information_gain = 1, split_info = 1, gain_ratio =
1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Information gain: y = [1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0], x = [0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1, 1, 2]	SOLVED	parent_entropy = 0.863121, conditional_entropy = 0.632482, information_gain = 0.230638, split_info = 1.57741, gain_ratio = 0.146214
Information gain: y = [0, 0, 1, 1], x = [0, 0, 1, 1]	SOLVED	parent_entropy = 1, conditional_entropy = 0, information_gain = 1, split_info = 1, gain_ratio = 1
Information gain: x = [0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1, 1, 2]	NEEDS_INPUT	ml_information_gain needs y (class labels (whole numbers)). Example: solve('ml_information_gain', y=[1, 1, 0, 0, 1, 1, 0
Information gain: y = [1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0], x = [0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1, 1, 2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Information gain: y = [0, 1, 1], x = [2, 2, 2]	INVALID_INPUT	Information gain and gain ratio of a split: the gain ratio is undefined when the feature takes a single value.

Gini impurity decrease of a split, `ml_gini_split`

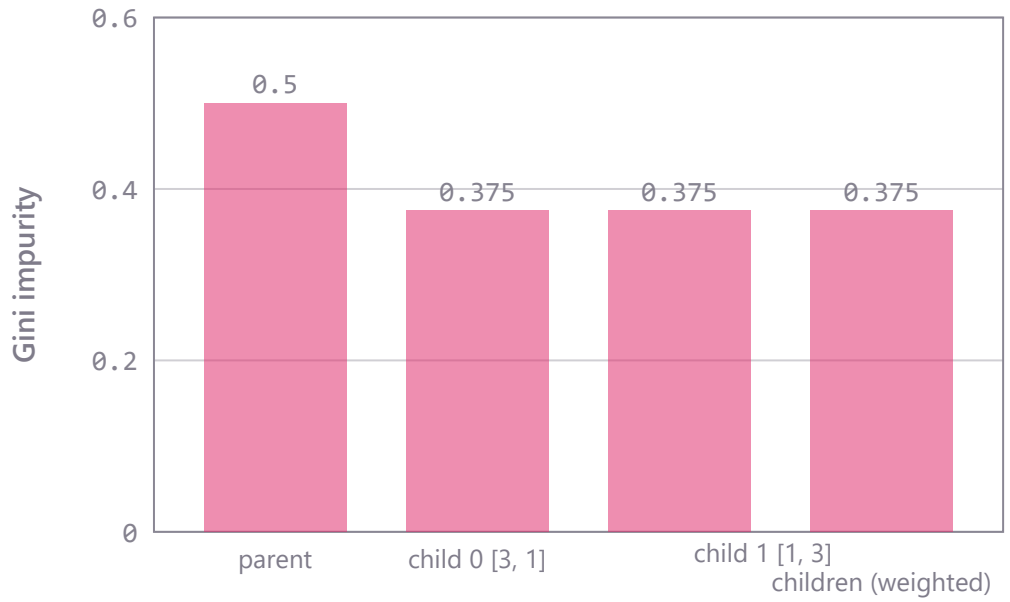
The Gini impurity of a node, the size-weighted Gini impurity of its children after a split, and the decrease (the CART split criterion).

$$\Delta G = G(S) - \sum_v \frac{|S_v|}{|S|} G(S_v)$$

input	meaning	condition
`y`	class labels (whole numbers)	
`g`	child (group) each sample goes to (whole numbers)	

output	meaning
`gini_parent`	Gini impurity before the split
`gini_children`	weighted Gini impurity of the children
`gini_decrease`	parent minus children

- **Method:** exact fractions
- **Assumes:** at least two groups
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Gini impurity of the parent y = [0, 0, 0, 1, 1, 1, 1, 0] and of the children given by g = [0, 0, 0, 0, 1, 1, 1, 1]: class counts per child tallied from the stated lists (child 0: [3, 1]; child 1: [1, 3]), each child's Gini from the package's impurity capability; returned: parent 0.5, weighted children 0.375, decrease 0.125.

```
data_science_ml_ai999x.solve('ml_gini_split', y=[0, 0, 0, 1, 1, 1, 1, 0], g=[0, 0, 0, 0, 1, 1, 1,
1])
# -> gini_parent = 0.5, gini_children = 0.375, gini_decrease = 0.125   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_gini_split', y=[1, 2, 2, 3, 3, 3], g=[0, 0, 1, 1, 2, 2])
# -> gini_parent = 0.611111, gini_children = 0.333333, gini_decrease = 0.277778   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gini split: y = [0, 0, 0, 1, 1, 1, 1, 0], g = [0, 0, 0, 0, 1, 1, 1, 1]	SOLVED	gini_parent = 0.5, gini_children = 0.375, gini_decrease = 0.125
Gini split: y = [1, 2, 2, 3, 3, 3], g = [0, 0, 1, 1, 2, 2]	SOLVED	gini_parent = 0.611111, gini_children = 0.333333, gini_decrease = 0.277778
Gini split: g = [0, 0, 0, 0, 1, 1, 1, 1]	NEEDS_INPUT	ml_gini_split needs y (class labels (whole numbers)). Example: solve('ml_gini_split', y=[0, 0, 0, 1, 1, 1, 1, 0], g=[0,
Gini split: y = [0, 0, 0, 1, 1, 1, 1, 0], g = [0, 0, 0, 0, 1, 1, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Normalised mutual information of two clusterings, `ml_normalized_mutual_information`

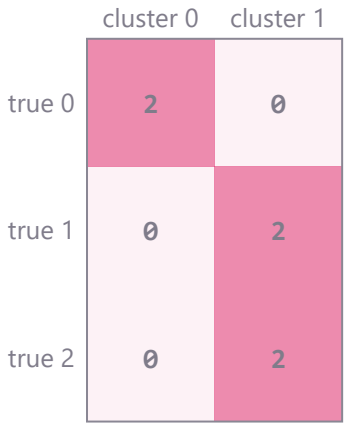
The normalised mutual information between two clusterings (or a clustering and the true classes), with the stated normaliser of the two entropies, and the mutual information in nats.

$$\mathrm{NMI} = \frac{I(U, V)}{\mathrm{mean}(H(U), H(V))}$$

input	meaning	condition
`labels_true`	reference labels (whole numbers)	
`labels_pred`	cluster labels (whole numbers)	
`method`	normaliser: arithmetic, geometric, min or max of the two entropies	

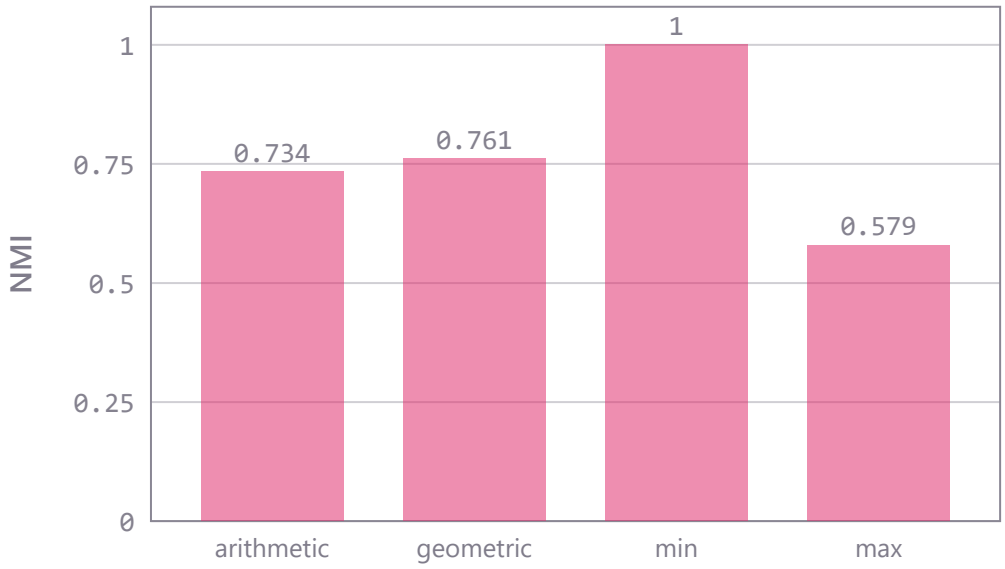
output	meaning
`nmi`	I / normaliser(H_true, H_pred)
`mutual_information`	I in nats

- **Method:** contingency counts; mutual information and entropies in nats
- **Assumes:** each labelling has at least two clusters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: true, columns: cluster

Counts of each pair (true, cluster) tallied from the stated labels true = [0, 0, 1, 1, 2, 2] and cluster = [0, 0, 1, 1, 1, 1]; returned: mutual information 0.636514, normalised mutual information 0.73368 (arithmetic normaliser).



Normalised mutual information of labels_true = [0, 0, 1, 1, 2, 2] and labels_pred = [0, 0, 1, 1, 1, 1] with each normaliser, one run each: arithmetic 0.73368, geometric 0.76117, min 1, max 0.57938; the min normaliser gives 1 here because the clustering is a coarsening of the reference.

```
data_science_ml_ai999x.solve('ml_normalized_mutual_information', labels_true=[0, 0, 1, 1, 2, 2],
labels_pred=[0, 0, 1, 1, 1, 1], method='arithmetic')
# -> nmi = 0.73368, mutual_information = 0.636514   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_normalized_mutual_information', labels_true=[0, 0, 0, 1, 1, 1],
labels_pred=[1, 1, 0, 0, 0, 0], method='geometric')
# -> nmi = 0.479139, mutual_information = 0.318257   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normalized mutual information: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1], method = arithmetic	SOLVED	nmi = 0.73368, mutual_information = 0.636514
Normalized mutual information: labels_true = [0, 0, 0, 1, 1, 1], labels_pred = [1, 1, 0, 0, 0, 0], method = geometric	SOLVED	nmi = 0.479139, mutual_information = 0.318257
Normalized mutual information: labels_pred = [0, 0, 1, 1, 1, 1], method = arithmetic	NEEDS_INPUT	ml_normalized_mutual_information needs labels_true (reference labels (whole numbers)). Example: solve('ml_normalized_mut
Normalized mutual information: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1], method = arithmetic, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Normalized mutual information: labels_true = [0, 0, 0], labels_pred = [0, 1, 1], method = arithmetic	INVALID_INPUT	Normalised mutual information of two clusterings: each labelling must have at least two clusters.

Adjusted mutual information of two clusterings, `ml_adjusted_mutual_information`

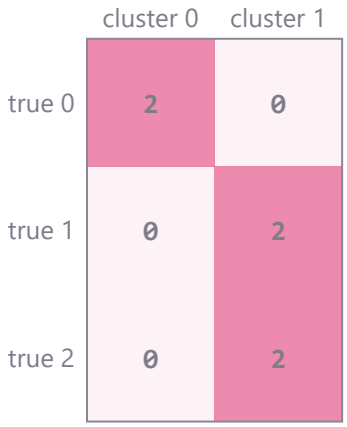
The adjusted mutual information between two clusterings: the mutual information corrected for chance by its expected value under random labellings with the same cluster sizes (hypergeometric model).

$$\mathrm{AMI} = \frac{I - E[I]}{\mathrm{mean}(H(U), H(V)) - E[I]}$$

input	meaning	condition
`labels_true`	reference labels (whole numbers)	
`labels_pred`	cluster labels (whole numbers)	
`method`	normaliser: arithmetic, geometric, min or max of the two entropies	

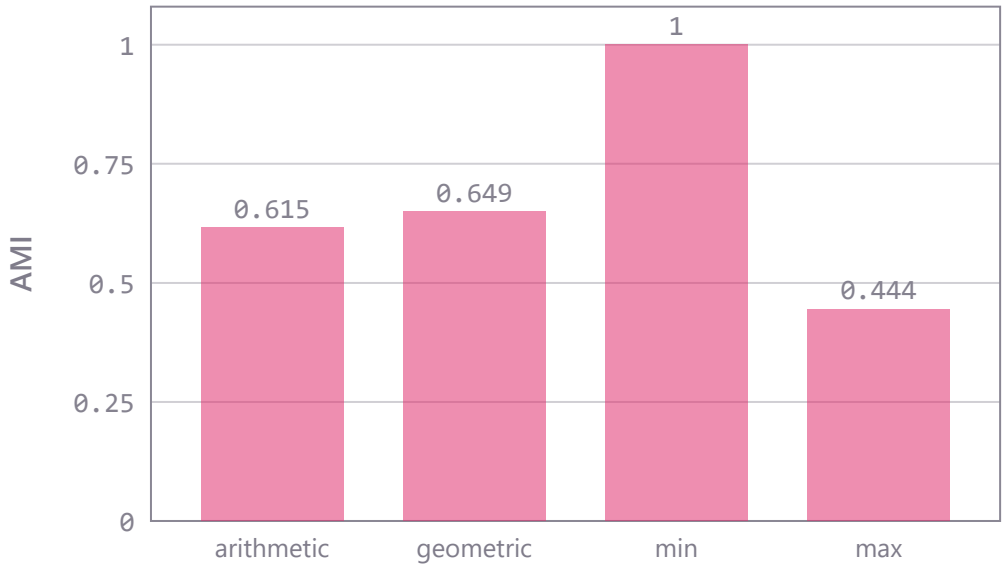
output	meaning
`ami`	(I - E[I]) / (normaliser - E[I])
`expected_mi`	E[I] in nats
`mutual_information`	I in nats

- **Method:** expected mutual information summed over the hypergeometric distribution of each cell (log-gamma form)
- **Assumes:** each labelling has at least two clusters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: true, columns: cluster

Counts of each pair (true, cluster) tallied from the stated labels true = [0, 0, 1, 1, 2, 2] and cluster = [0, 0, 1, 1, 1, 1]; returned: mutual information 0.636514, its expected value under chance 0.266836, adjusted mutual information 0.615385 (arithmetic normaliser).



Adjusted mutual information of labels_true = [0, 0, 1, 1, 2, 2] and labels_pred = [0, 0, 1, 1, 1, 1] with each normaliser, one run each: arithmetic 0.615385, geometric 0.649248, min 1, max 0.444444.

```
data_science_ml_ai999x.solve('ml_adjusted_mutual_information', labels_true=[0, 0, 1, 1, 2, 2],
labels_pred=[0, 0, 1, 1, 1, 1], method='arithmetic')
# -> ami = 0.615385, expected_mi = 0.266836, mutual_information = 0.636514   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_adjusted_mutual_information', labels_true=[0, 0, 0, 1, 1, 1, 2, 2,
2], labels_pred=[0, 0, 1, 1, 2, 2, 2, 0, 1], method='max')
# -> ami = -0.037037, expected_mi = 0.336299, mutual_information = 0.308065   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Adjusted mutual information: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1], method = arithmetic	SOLVED	ami = 0.615385, expected_mi = 0.266836, mutual_information = 0.636514
Adjusted mutual information: labels_true = [0, 0, 0, 1, 1, 1, 2, 2, 2], labels_pred = [0, 0, 1, 1, 2, 2, 2, 0, 1], method = max	SOLVED	ami = -0.037037, expected_mi = 0.336299, mutual_information = 0.308065
Adjusted mutual information: labels_pred = [0, 0, 1, 1, 1, 1], method = arithmetic	NEEDS_INPUT	ml_adjusted_mutual_information needs labels_true (reference labels (whole numbers)). Example: solve('ml_adjusted_mutual_
Adjusted mutual information: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1], method = arithmetic, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Rand index, adjusted Rand index and Fowlkes-Mallows index, `ml_rand_index`

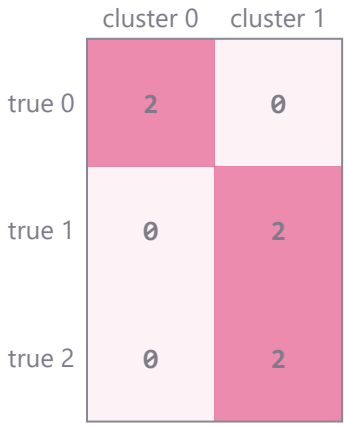
Pair-counting agreement between two clusterings: the Rand index, the adjusted Rand index (corrected for chance) and the Fowlkes-Mallows index.

$$\mathrm{ARI} = \frac{\sum_{ij}\binom{n_{ij}}{2} - \frac{\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}}{\binom{n}{2}}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \frac{\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}}{\binom{n}{2}}}$$

input	meaning	condition
`labels_true`	reference labels (whole numbers)	
`labels_pred`	cluster labels (whole numbers)	

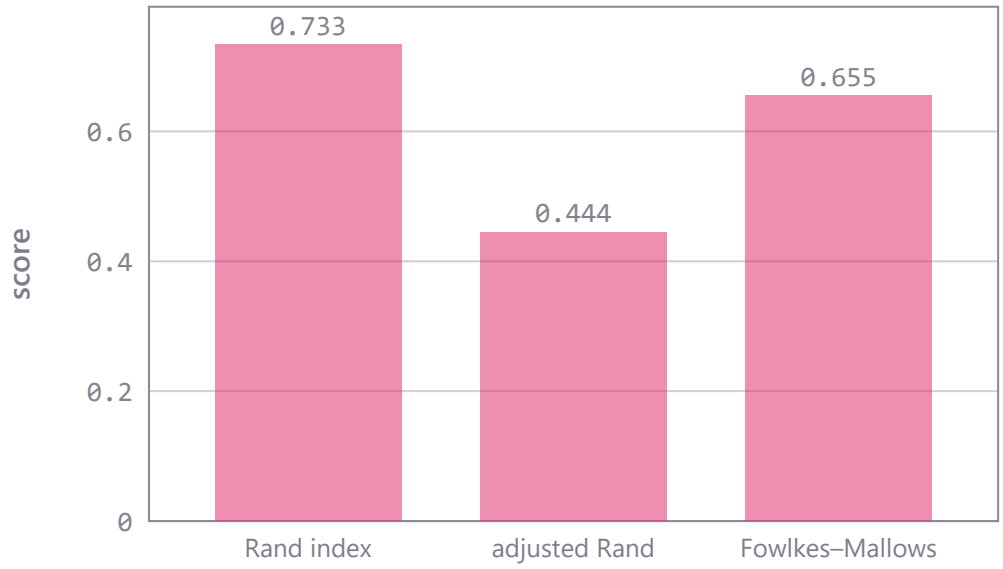
output	meaning
`rand_index`	fraction of sample pairs on which the two labellings agree
`adjusted_rand_index`	ARI
`fowlkes_mallows`	TP / sqrt((TP + FP)(TP + FN)) over pairs

- **Method:** pair counts from the contingency table, exact integers
- **Assumes:** at least one within-cluster pair in each labelling
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: true, columns: cluster

Counts of each pair (true, cluster) tallied from the stated labels true = [0, 0, 1, 1, 2, 2] and cluster = [0, 0, 1, 1, 1, 1]; returned: Rand index 0.733333, adjusted Rand index 0.444444, Fowlkes–Mallows 0.654654.



The three pair-counting scores the package returns for labels_true = [0, 0, 1, 1, 2, 2] and labels_pred = [0, 0, 1, 1, 1, 1]: Rand index 0.733333, adjusted Rand index 0.444444 (corrected for chance), Fowlkes–Mallows 0.654654.

```
data_science_ml_ai999x.solve('ml_rand_index', labels_true=[0, 0, 1, 1, 2, 2], labels_pred=[0, 0, 1, 1, 1, 1])
# -> rand_index = 0.733333, adjusted_rand_index = 0.444444, fowlkes_mallows = 0.654654   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_rand_index', labels_true=[0, 0, 0, 1, 1, 1], labels_pred=[0, 0, 1, 1, 2, 2])
# -> rand_index = 0.666667, adjusted_rand_index = 0.242424, fowlkes_mallows = 0.471405   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Adjusted Rand index: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0,	SOLVED	rand_index = 0.733333, adjusted_rand_index = 0.444444, fowlkes_mallows = 0.654654

question	outcome	result
0, 1, 1, 1, 1]		
Adjusted Rand index: labels_true = [0, 0, 0, 1, 1, 1], labels_pred = [0, 0, 1, 1, 2, 2]	SOLVED	rand_index = 0.666667, adjusted_rand_index = 0.242424, fowlkes_mallows = 0.471405
Adjusted Rand index: labels_pred = [0, 0, 1, 1, 1, 1]	NEEDS_INPUT	ml_rand_index needs labels_true (reference labels (whole numbers)). Example: solve('ml_rand_index', labels_true=[0, 0, 1
Adjusted Rand index: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Homogeneity, completeness and V-measure, `ml_homogeneity_completeness_v_measure`

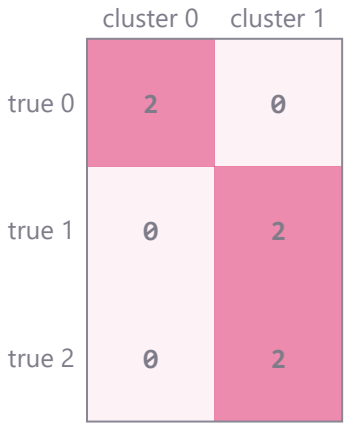
Entropy-based clustering scores against true classes: homogeneity (each cluster holds one class), completeness (each class sits in one cluster) and their weighted harmonic mean, the V-measure.

$$h = 1 - \frac{H(C \mid K)}{H(C)}, \quad c = 1 - \frac{H(K \mid C)}{H(K)}, \quad V_\beta = \frac{(1+\beta)hc}{\beta h + c}$$

input	meaning	condition
`labels_true`	true class labels (whole numbers)	
`labels_pred`	cluster labels (whole numbers)	
`beta`	weight of completeness relative to homogeneity in the V-measure (1 = equal)	

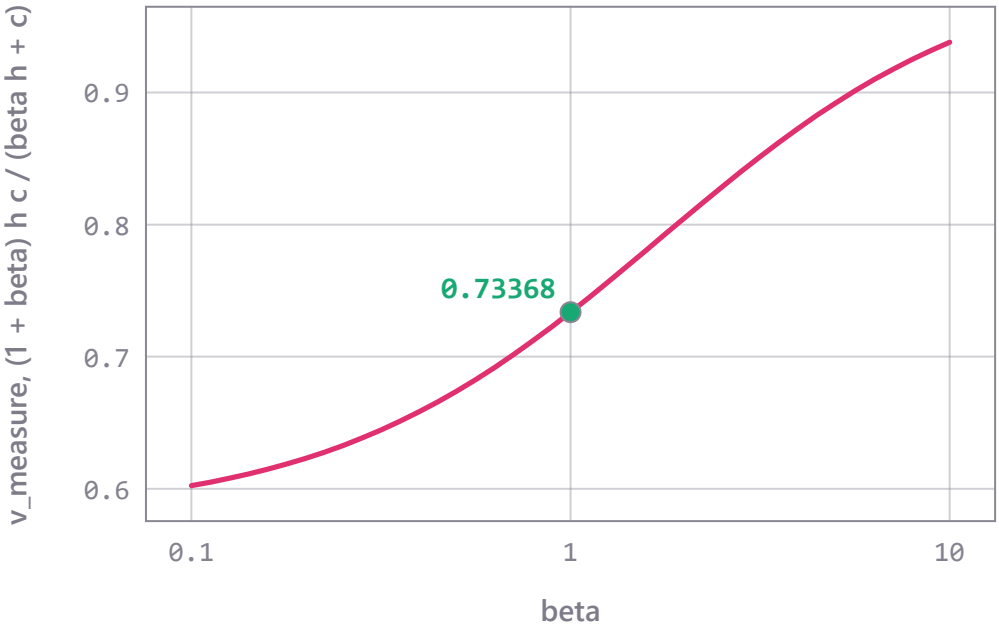
output	meaning
`homogeneity`	$1 - H$(C | K) / H(C) = I / H(C)
`completeness`	I / H(K)
`v_measure`	$(1+\beta)$ h c / (beta h + c)

- **Method:** mutual information and entropies of the contingency table
- **Assumes:** each labelling has at least two clusters; $\beta > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rows: true, columns: cluster

Counts of each pair (true, cluster) tallied from the stated labels true = [0, 0, 1, 1, 2, 2] and cluster = [0, 0, 1, 1, 1, 1]; returned: homogeneity 0.57938 (cluster 1 mixes two classes), completeness 1 (each class sits in one cluster), V-measure 0.73368 (β = 1).



v_measure ((1 + beta) h c / (beta h + c)) against beta (weight of completeness relative to homogeneity in the V-measure (1 = equal)) from 0.1 to 10 on a logarithmic axis. Other inputs as in the worked example: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1]. 41 points computed by data-science-ml-ai999x; marked: the worked example, beta = 1, v_measure = 0.73368.

```
data_science_ml_ai999x.solve('ml_homogeneity_completeness_v_measure', labels_true=[0, 0, 1, 1, 2, 2], labels_pred=[0, 0, 1, 1, 1, 1], beta=1)
# -> homogeneity = 0.57938, completeness = 1, v_measure = 0.73368   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_homogeneity_completeness_v_measure', labels_true=[0, 0, 0, 1, 1, 1], labels_pred=[0, 0, 1, 1, 2, 2], beta=1)
# -> homogeneity = 0.666667, completeness = 0.42062, v_measure = 0.515804   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
V-measure: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1], beta = 1	SOLVED	homogeneity = 0.57938, completeness = 1, v_measure = 0.73368
V-measure: labels_true = [0, 0, 0, 1, 1, 1], labels_pred = [0, 0, 1, 1, 2, 2], beta = 1	SOLVED	homogeneity = 0.666667, completeness = 0.42062, v_measure = 0.515804
V-measure: labels_pred = [0, 0, 1, 1, 1, 1], beta = 1	NEEDS_INPUT	ml_homogeneity_completeness_v_measure needs labels_true (true class labels (whole numbers)). Example: solve('ml_homogene
V-measure: labels_true = [0, 0, 1, 1, 2, 2], labels_pred = [0, 0, 1, 1, 1, 1], beta = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Silhouette score, `ml_silhouette_score`

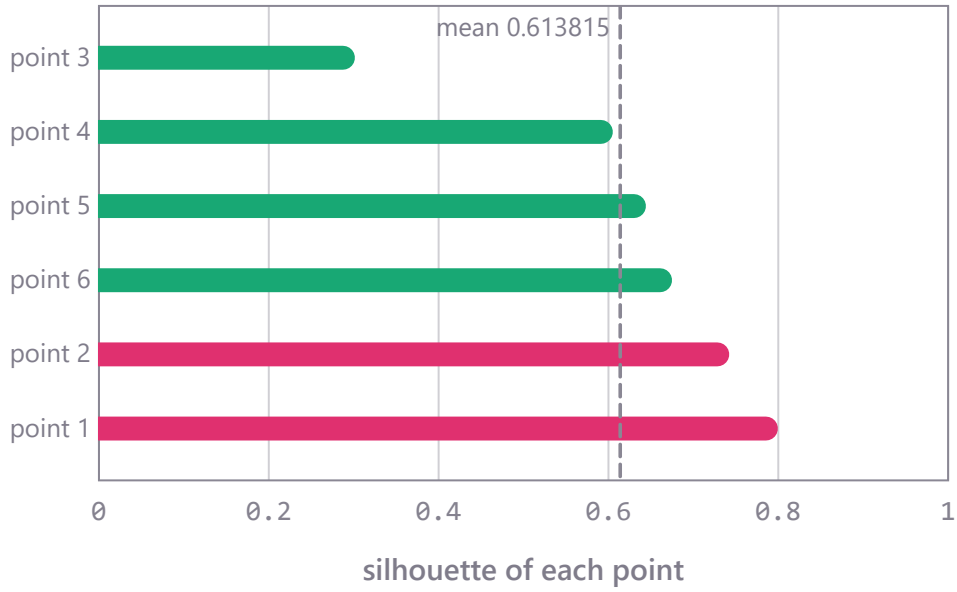
The mean silhouette coefficient of a clustering (Euclidean distance) and the silhouette of each sample: (b - a) / max(a, b), a = mean distance within its cluster, b = mean distance to the nearest other cluster.

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

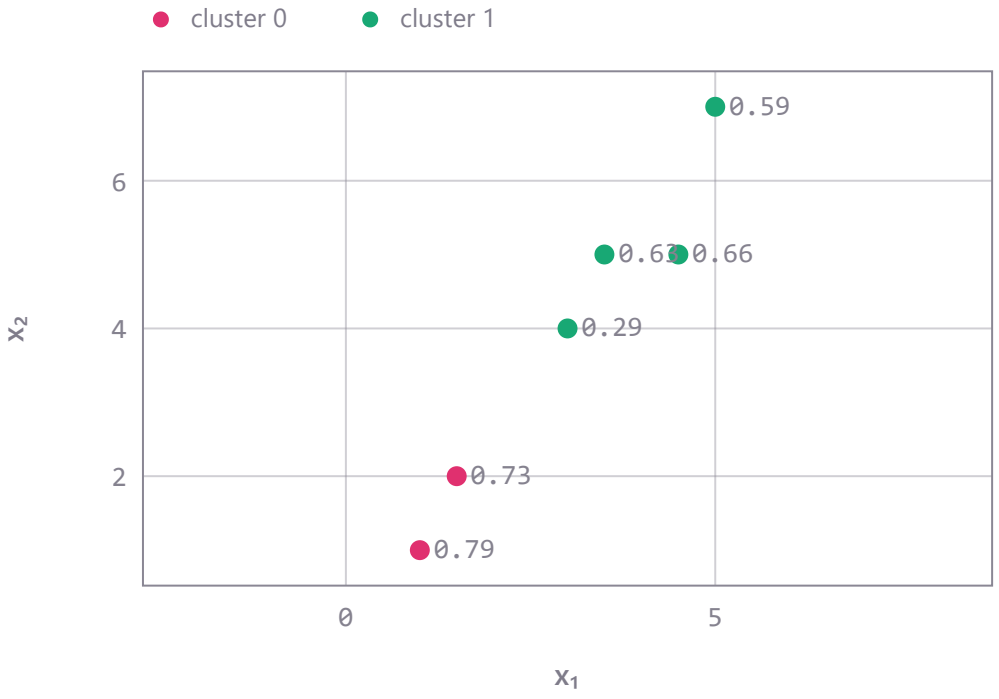
input	meaning	condition
`X`	points: one row per sample	
`labels`	cluster label of each point (whole numbers)	

output	meaning
`silhouette`	mean silhouette coefficient
`samples`	silhouette of each sample (0 for a point alone in its cluster)

- **Method:** all pairwise Euclidean distances, per-sample a and b
- **Assumes:** 2 <= number of clusters <= n - 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The silhouette of each of the 6 stated points (returned: 0.785496, 0.728162, 0.287385, 0.591064, 0.630078, 0.660705), grouped by cluster label [0, 0, 1, 1, 1, 1] and sorted; the dashed line is the returned mean silhouette 0.613815. Point 3 sits closest to the other cluster.



The 6 stated points coloured by label [0, 0, 1, 1, 1, 1], each labelled with its returned silhouette; mean 0.613815.

```
data_science_ml_ai999x.solve('ml_silhouette_score', X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels=[0, 0, 1, 1, 1, 1])
# -> silhouette = 0.613815, samples = [0.785495870518, 0.728161920035, 0.287385025256, 0.591063503361, 0.630078239981, 0.660704874448]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_silhouette_score', X=[[0], [1], [5], [6], [11]], labels=[0, 0, 1, 1, 2])
# -> silhouette = 0.634747, samples = [0.818181818182, 0.777777777778, 0.777777777778, 0.8, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Silhouette score: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels = [0, 0, 1, 1, 1, 1]	SOLVED	silhouette = 0.613815, samples = [0.785495870518, 0.728161920035, 0.287385025256, 0.591063503361, 0.630078239981, 0.660704874448]
Silhouette score: X = [[0], [1], [5], [6], [11]], labels = [0, 0, 1, 1, 2]	SOLVED	silhouette = 0.634747, samples = [0.818181818182, 0.777777777778, 0.777777777778, 0.8, 0.0]
Silhouette score: labels = [0, 0, 1, 1, 1, 1]	NEEDS_INPUT	ml_silhouette_score needs X (points: one row per sample). Example: solve('ml_silhouette_score', X=[[1, 1], [1.5, 2], [3,
Silhouette score: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels = [0, 0, 1, 1, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Silhouette score: X = [[0], [1], [5]], labels = [0, 1, 2]	INVALID_INPUT	Silhouette score: the silhouette needs between 2 and n - 1 clusters.

Davies-Bouldin index, `ml_davies_bouldin_index`

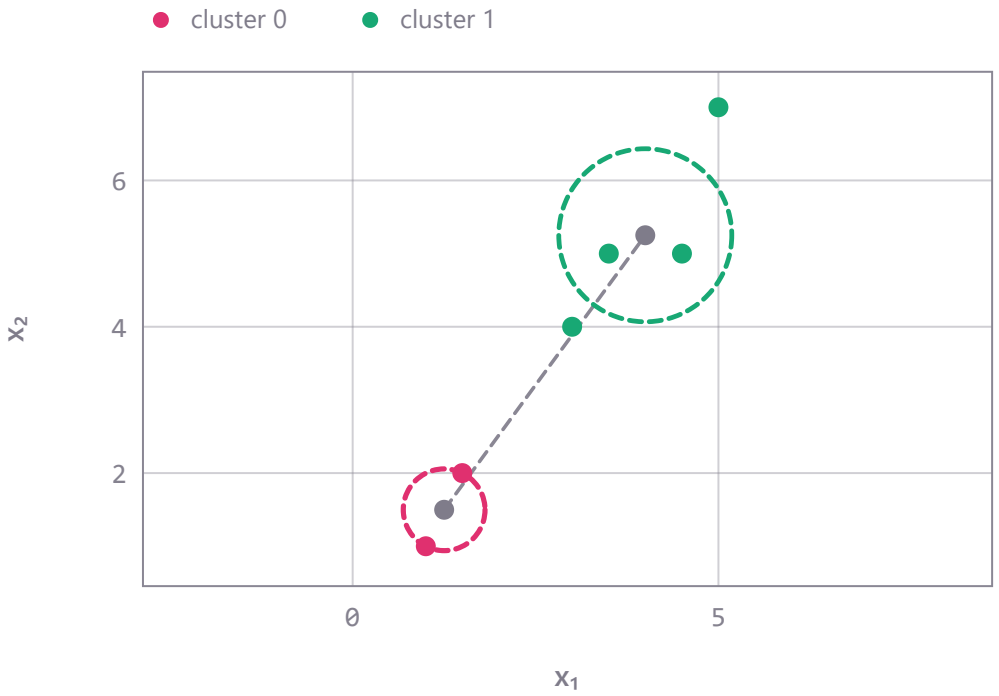
The Davies-Bouldin index of a clustering: the mean over clusters of the worst ratio of within-cluster scatter to between-centroid distance (lower is better).

$$DB = \frac{1}{K} \sum_{k} \max_{m \neq k} \frac{S_k + S_m}{\|c_k - c_m\|}$$

input	meaning	condition
`X`	points: one row per sample	
`labels`	cluster label of each point (whole numbers)	

output	meaning
`davies_bouldin`	Davies-Bouldin index

- **Method:** centroids, mean distance to the centroid as scatter, Euclidean centroid distances
- **Assumes:** 2 <= number of clusters <= n - 1; distinct centroids
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



6 stated points with labels [0, 0, 1, 1, 1, 1]; black dots: cluster means, dashed circles: each cluster's mean distance to its mean, dashed line: the distance between the means (all computed here from the stated points, the ratio the index is built from). Returned Davies–Bouldin index 0.374734 (lower is better).

```
data_science_ml_ai999x.solve('ml_davies_bouldin_index', X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels=[0, 0, 1, 1, 1, 1])
# -> davies_bouldin = 0.374734   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_davies_bouldin_index', X=[[0], [1], [5], [6], [11]], labels=[0, 0, 1, 1, 2])
# -> davies_bouldin = 0.163636   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Davies-Bouldin index: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels = [0, 0, 1, 1, 1, 1]	SOLVED	davies_bouldin = 0.374734
Davies-Bouldin index: X = [[0], [1], [5], [6], [11]], labels = [0, 0, 1, 1, 2]	SOLVED	davies_bouldin = 0.163636
Davies-Bouldin index: labels = [0, 0, 1, 1, 1, 1]	NEEDS_INPUT	ml_davies_bouldin_index needs X (points: one row per sample). Example: solve('ml_davies_bouldin_index', X=[[1, 1], [1.5,
Davies-Bouldin index: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels = [0, 0, 1, 1, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Calinski-Harabasz index, `ml_calinski_harabasz_index`

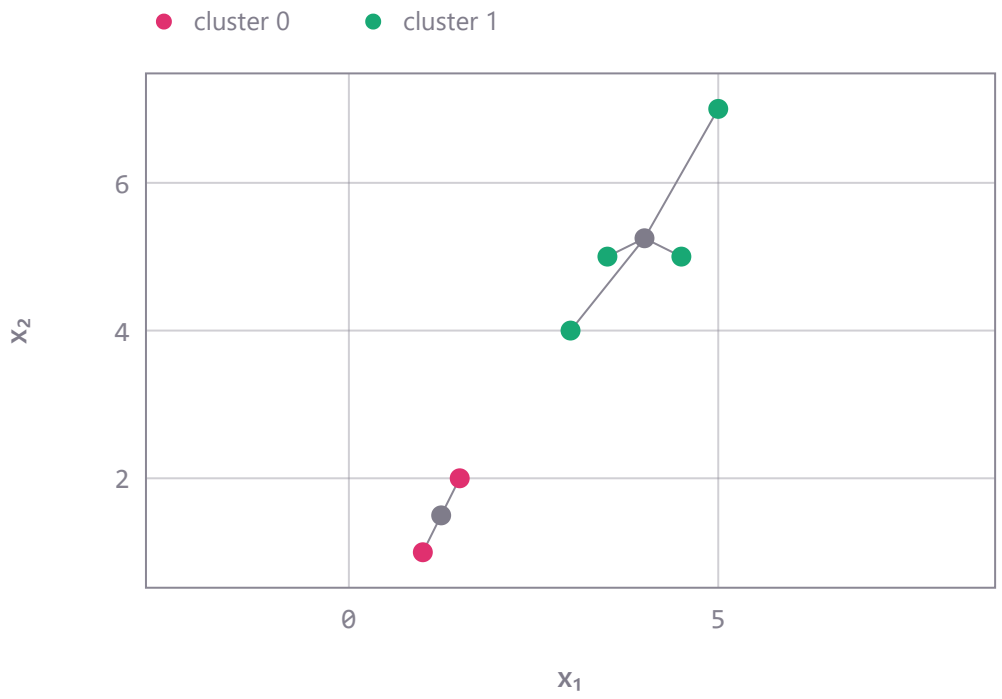
The Calinski-Harabasz (variance ratio) index of a clustering: between-cluster over within-cluster dispersion, scaled by $(n - K)/(K - 1)$ (higher is better).

$$CH = \frac{\mathrm{tr}(B)}{\mathrm{tr}(W)} \cdot \frac{n - K}{K - 1}$$

input	meaning	condition
`X`	points: one row per sample	
`labels`	cluster label of each point (whole numbers)	

output	meaning
`calinski_harabasz`	CH index
`between`	between-cluster dispersion B
`within`	within-cluster dispersion W

- **Method:** exact rational sums of squares
- **Assumes:** 2 <= number of clusters <= n - 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



6 stated points with their cluster labels [0, 0, 1, 1, 1, 1]; black dots: each cluster's mean (computed here from the stated points), grey: the within-cluster spread. Returned: between-cluster dispersion 28.8333, within 7.875, Calinski–Harabasz index 14.6455.

```
data_science_ml_ai999x.solve('ml_calinski_harabasz_index', X=[[1, 1], [1.5, 2], [3, 4], [5, 7],
[3.5, 5], [4.5, 5]], labels=[0, 0, 1, 1, 1, 1])
# -> calinski_harabasz = 14.6455, between = 28.8333, within = 7.875   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_calinski_harabasz_index', X=[[0], [1], [5], [6], [11]], labels=[0,
0, 1, 1, 2])
# -> calinski_harabasz = 76.2, between = 76.2, within = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Calinski-Harabasz index: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels = [0, 0, 1, 1, 1, 1]	SOLVED	calinski_harabasz = 14.6455, between = 28.8333, within = 7.875
Calinski-Harabasz index: X = [[0], [1], [5], [6], [11]], labels = [0, 0, 1, 1, 2]	SOLVED	calinski_harabasz = 76.2, between = 76.2, within = 1
Calinski-Harabasz index: labels = [0, 0, 1, 1, 1, 1]	NEEDS_INPUT	ml_calinski_harabasz_index needs X (points: one row per sample). Example: solve('ml_calinski_harabasz_index', X=[[1, 1],
Calinski-Harabasz index: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]], labels = [0, 0, 1, 1, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

One k-means iteration (assignment and update), `ml_kmeans_step`

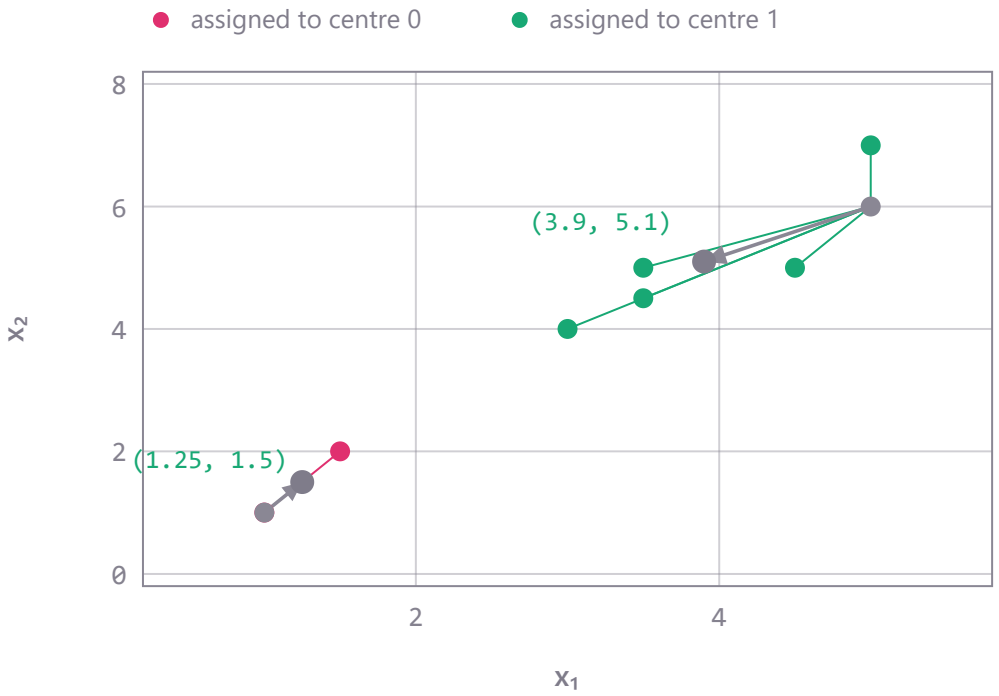
One Lloyd iteration of k-means from given centroids: assign every point to its nearest centroid, give the inertia of that assignment and the updated centroids (the means of the assigned points).

$$c(i) = \arg\min_k \|x_i - \mu_k\|^2, \quad \mu'_k = \frac{1}{|S_k|} \sum_{i \in S_k} x_i$$

input	meaning	condition
`X`	points: one row per sample	
`C`	current (or initial) centroids: one row per cluster	

output	meaning
`labels`	index (0-based row of C) of the nearest centroid of each point
`new_centroids`	mean of the points assigned to each centroid
`inertia`	sum of squared distances of the points to their assigned (current) centroids

- **Method:** exact squared Euclidean distances; exact means
- **Assumes:** no exact distance tie; no empty cluster; $2 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



One k-means step on the 7 stated points: each point joined to its nearest current centre [[1, 1], [5, 6]] (returned labels [0, 0, 1, 1, 1, 1, 1]); arrows move each centre to the mean of its points, the returned new centroids (black). Inertia before the move 19.25.

```
data_science_ml_ai999x.solve('ml_kmeans_step', X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]], C=[[1, 1], [5, 6]])
# -> labels = [0, 0, 1, 1, 1, 1, 1], new_centroids = [[1.25, 1.5], [3.9, 5.1]], inertia = 19.25  (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_kmeans_step', X=[[0], [1], [5], [6], [11]], C=[[0], [6]])
# -> labels = [0, 0, 1, 1, 1], new_centroids = [[0.5], [7.33333333333]], inertia = 27   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
K-means step: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]], C = [[1, 1], [5, 6]]	SOLVED	labels = [0, 0, 1, 1, 1, 1, 1], new_centroids = [[1.25, 1.5], [3.9, 5.1]], inertia = 19.25
K-means step: X = [[0], [1], [5], [6], [11]], C = [[0], [6]]	SOLVED	labels = [0, 0, 1, 1, 1], new_centroids = [[0.5], [7.33333333333]], inertia = 27
K-means step: C = [[1, 1], [5, 6]]	NEEDS_INPUT	ml_kmeans_step needs X (points: one row per sample). Example: solve('ml_kmeans_step', X=[[1, 1], [1.5, 2], [3, 4], [5, 7
K-means step: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]], C = [[1, 1], [5, 6]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
K-means step: X = [[0], [2], [4]], C = [[1], [3]]	INVALID_INPUT	One k-means iteration (assignment and update): a point is exactly equidistant from two centroids (the assignment is ambi

k-means clustering from given initial centroids, `ml_kmeans`

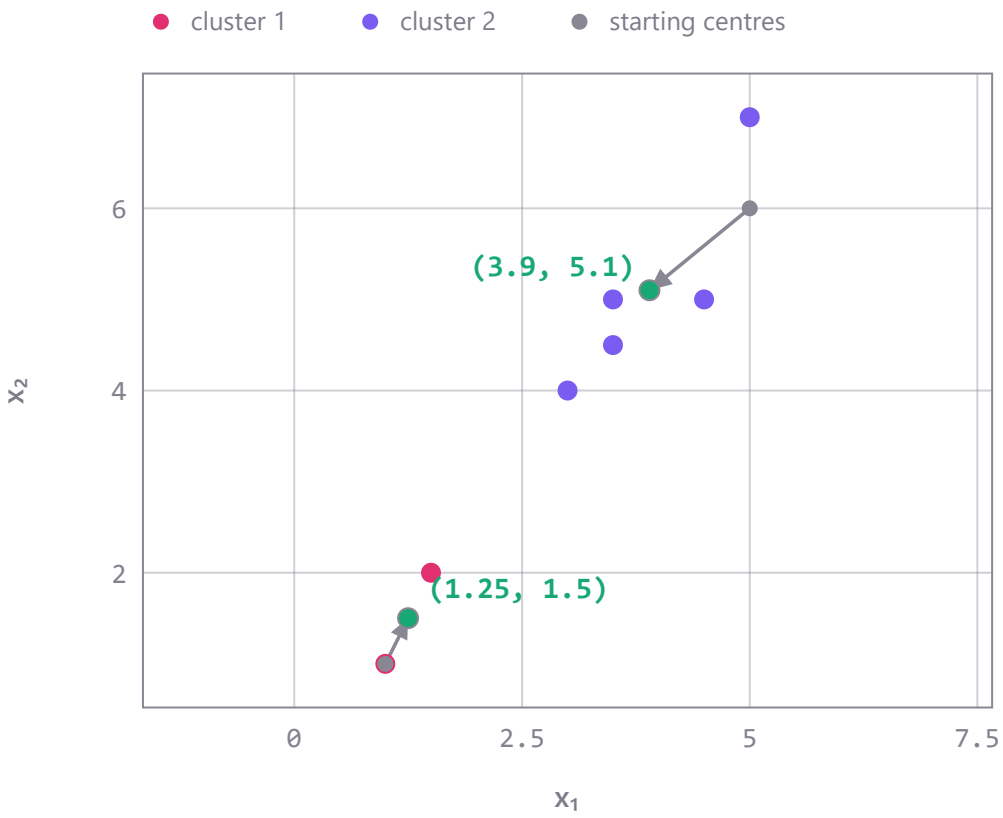
Lloyd's k-means algorithm run to convergence from stated initial centroids: the final centroids, the cluster of each point and the inertia (within-cluster sum of squares).

$$\min \sum_{k} \sum_{i \in S_k} \|x_i - \mu_k\|^2$$

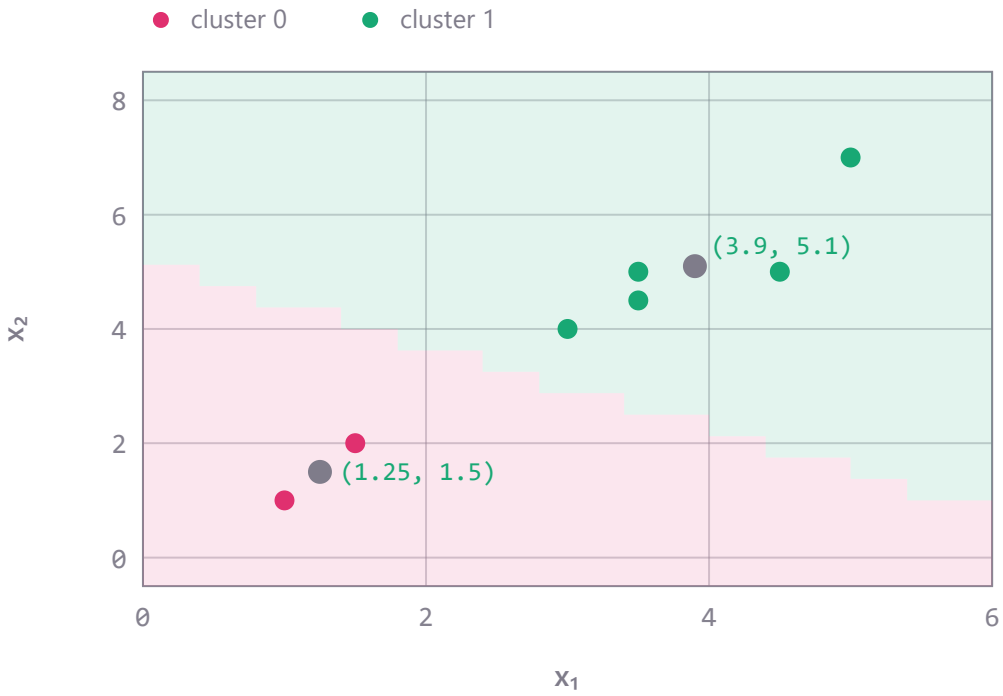
input	meaning	condition
`X`	points: one row per sample	
`C`	initial centroids: one row per cluster	

output	meaning
`centroids`	final centroids
`labels`	cluster (0-based row of C) of each point
`inertia`	sum of squared distances to the final centroids

- **Method:** alternate exact assignment and mean update until no assignment changes (at most 100 iterations)
- **Assumes:** no exact distance tie; no empty cluster during the iterations; convergence within 100 iterations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k-means on 7 points from the starting centres [[1, 1], [5, 6]]: the package's final centroids (1.25, 1.5), (3.9, 5.1) (marked; arrows from the starting centres), cluster labels 1, 1, 2, 2, 2, 2, 2 and inertia 8.525.



k-means on the 7 stated points from the starting centres [[1, 1], [5, 6]]: the returned centroids (black, labelled) and labels; the shaded regions are the plane split by nearest centroid, each of 720 cells assigned by the package's assignment step. Inertia 8.525.

```
data_science_ml_ai999x.solve('ml_kmeans', X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]], C=[[1, 1], [5, 6]])
# -> centroids = [[1.25, 1.5], [3.9, 5.1]], labels = [0, 0, 1, 1, 1, 1, 1], inertia = 8.525  (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_kmeans', X=[[0], [1], [5], [6], [11], [12]], C=[[0], [1], [12]])
# -> centroids = [[0.5], [5.5], [11.5]], labels = [0, 0, 1, 1, 2, 2], inertia = 1.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
K-means clustering: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]], C = [[1, 1], [5, 6]]	SOLVED	centroids = [[1.25, 1.5], [3.9, 5.1]], labels = [0, 0, 1, 1, 1, 1, 1], inertia = 8.525
K-means clustering: X = [[0], [1], [5], [6], [11], [12]], C = [[0], [1], [12]]	SOLVED	centroids = [[0.5], [5.5], [11.5]], labels = [0, 0, 1, 1, 2, 2], inertia = 1.5
K-means clustering: C = [[1, 1], [5, 6]]	NEEDS_INPUT	ml_kmeans needs X (points: one row per sample). Example: solve('ml_kmeans', X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5
K-means clustering: X = [[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]], C = [[1, 1], [5, 6]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

k-nearest-neighbour classification of a query point, `ml_knn_classify`

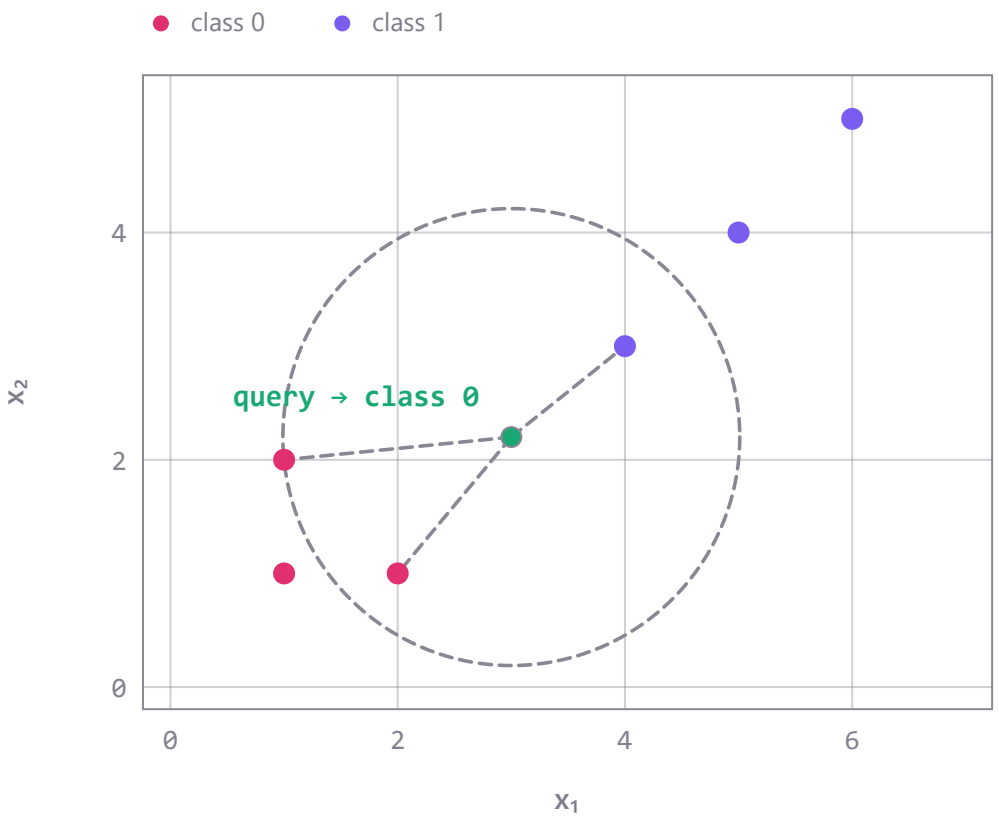
Classify a query point by majority vote among its k nearest training points (stated metric), with the neighbours and their distances.

$$\hat{y} = \arg\max_{c} \sum_{i \in N_k(x)} \mathbb{1}(y_i = c)$$

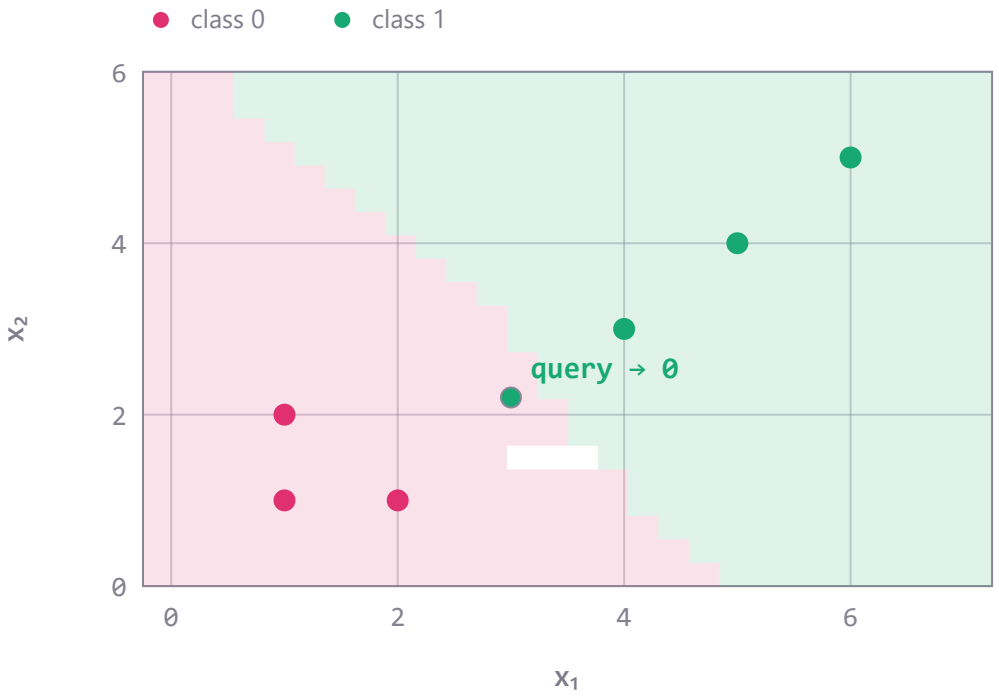
input	meaning	condition
`X`	training points: one row per sample	
`y`	training labels (whole numbers)	
`x`	query point	
`k`	number of neighbours	
`metric`	distance: euclidean or manhattan	

output	meaning
`prediction`	majority class among the k neighbours
`votes`	number of neighbours of that class
`neighbors`	indices (0-based rows of X) of the k nearest points, ascending
`neighbor_distances`	their distances, ascending

- **Method:** exact distances (squared for Euclidean), stable sort, majority vote
- **Assumes:** no distance tie at the k-th neighbour; no tied vote; $1 \leq k \leq n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k-nearest-neighbour classification of the query point (3, 2.2) among 6 labelled points (k = 3, euclidean distance): the package's 3 nearest neighbours are points 2, 3, 5 at distances 1.28062, 1.56205, 2.00998 (dashed lines; the circle reaches the farthest of them); 2 of 3 votes give class 0.



The 3-nearest-neighbour vote (euclidean distance) over the plane: each of 616 cells is coloured by the class the package predicts at its centre, from the 6 stated labelled points. 3 cells where the k-th and (k+1)-th neighbours tie are refused and left blank. Marked: the worked example's query [3, 2.2] → class 0.

```
data_science_ml_ai999x.solve('ml_knn_classify', X=[[1, 1], [2, 1], [4, 3], [5, 4], [1, 2], [6, 5]], y=[0, 0, 1, 1, 0, 1], x=[3, 2.2], k=3, metric='euclidean')
# -> prediction = 0, votes = 2, neighbors = [1, 2, 4], neighbor_distances = [1.28062484749, 1.56204993518, 2.00997512422]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_knn_classify', X=[[0], [1], [2], [6], [7], [8]], y=[1, 1, 1, 2, 2, 2], x=[5.5], k=5, metric='manhattan')
# -> prediction = 2, votes = 3, neighbors = [1, 2, 3, 4, 5], neighbor_distances = [0.5, 1.5, 2.5, 3.5, 4.5]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
kNN classification: X = [[1, 1], [2, 1], [4, 3], [5, 4], [1, 2], [6, 5]], y = [0, 0, 1, 1, 0, 1], x = [3, 2.2], k = 3, metric = euclidean	SOLVED	prediction = 0, votes = 2, neighbors = [1, 2, 4], neighbor_distances = [1.28062484749, 1.56204993518, 2.00997512422]
kNN classification: X = [[0], [1], [2], [6], [7], [8]], y = [1, 1, 1, 2, 2, 2], x = [5.5], k = 5, metric = manhattan	SOLVED	prediction = 2, votes = 3, neighbors = [1, 2, 3, 4, 5], neighbor_distances = [0.5, 1.5, 2.5, 3.5, 4.5]
kNN classification: y = [0, 0, 1, 1, 0, 1], x = [3, 2.2], k = 3, metric = euclidean	NEEDS_INPUT	ml_knn_classify needs X (training points: one row per sample). Example: solve('ml_knn_classify', X=[[1, 1], [2, 1], [4,
kNN classification: X = [[1, 1], [2, 1], [4, 3], [5, 4], [1, 2], [6, 5]], y = [0, 0, 1, 1, 0, 1], x = [3, 2.2], k = 3, metric = euclidean, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
kNN classification: X = [[0], [2]], y = [0, 1], x = [1], k = 1, metric = euclidean	INVALID_INPUT	k-nearest-neighbour classification of a query point: the k-th and (k+1)-th nearest neighbours are at the same distance (

k-nearest-neighbour regression of a query point, `ml_knn_regress`

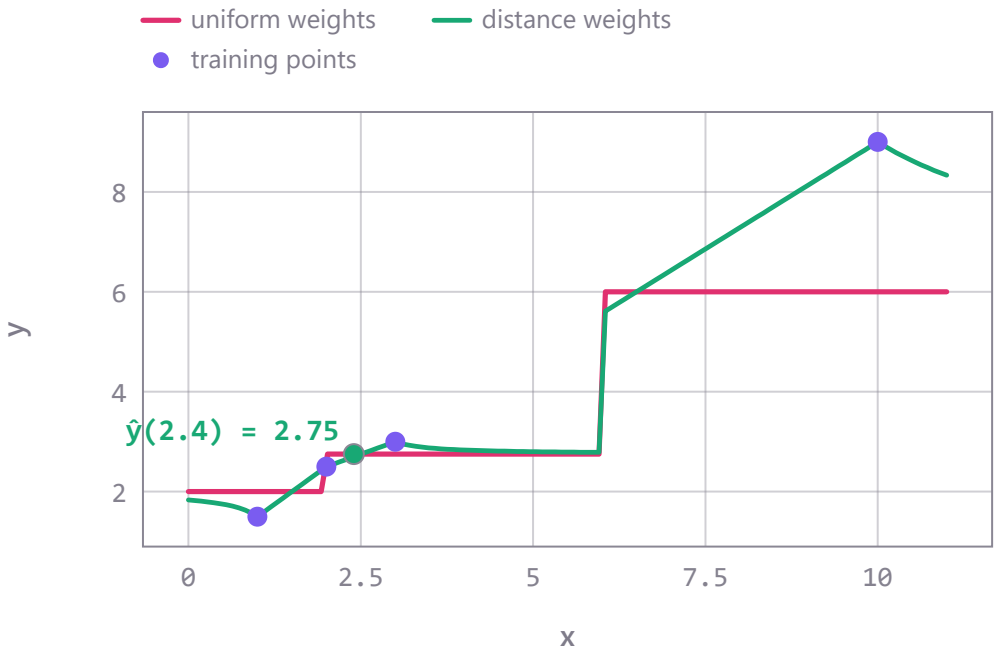
Predict a value at a query point as the mean (uniform) or inverse-distance-weighted mean of the targets of its k nearest training points.

$$\hat{y} = \frac{\sum_{i \in N_k} w_i y_i}{\sum_{i \in N_k} w_i}, \quad w_i = 1 \text{ or } 1/d_i$$

input	meaning	condition
`X`	training points	
`y`	training targets	
`x`	query point	
`k`	number of neighbours	
`metric`	distance: euclidean or manhattan	
`weights`	uniform mean or inverse-distance weights	

output	meaning
`prediction`	predicted value
`neighbors`	indices of the k nearest points, ascending
`neighbor_distances`	their distances, ascending

- **Method:** exact distances, stable sort, (weighted) mean
- **Assumes:** no distance tie at the k-th neighbour; distance weights: the query is not a training point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



k-nearest-neighbour regression (k = 2) through the stated points x = [1, 2, 3, 10], y = [1.5, 2.5, 3.0, 9.0]: the prediction at 121 query points for uniform and distance weights (one run each). Marked: the worked example x = 2.4, ŷ = 2.75 (neighbours [1, 2]).

```
data_science_ml_ai999x.solve('ml_knn_regress', X=[[1], [2], [3], [10]], y=[1.5, 2.5, 3.0, 9.0], x=
[2.4], k=2, metric='euclidean', weights='uniform')
# -> prediction = 2.75, neighbors = [1, 2], neighbor_distances = [0.4, 0.6]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_knn_regress', X=[[1, 1], [2, 1], [4, 3], [5, 4]], y=[10, 12, 20,
24], x=[3, 2], k=3, metric='manhattan', weights='distance')
# -> prediction = 14.5, neighbors = [0, 1, 2], neighbor_distances = [2.0, 2.0, 3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
kNN regression: X = [[1], [2], [3], [10]], y = [1.5, 2.5, 3, 9], x = [2.4], k = 2, metric = euclidean, weights = uniform	SOLVED	prediction = 2.75, neighbors = [1, 2], neighbor_distances = [0.4, 0.6]
kNN regression: X = [[1, 1], [2, 1], [4, 3], [5, 4]], y = [10, 12, 20, 24], x = [3, 2], k = 3, metric = manhattan, weights = distance	SOLVED	prediction = 14.5, neighbors = [0, 1, 2], neighbor_distances = [2.0, 2.0, 3.0]
kNN regression: y = [1.5, 2.5, 3, 9], x = [2.4], k = 2, metric = euclidean, weights = uniform	NEEDS_INPUT	ml_knn_regress needs X (training points). Example: solve('ml_knn_regress', X=[[1], [2], [3], [10]], y=[1.5, 2.5, 3.0, 9.
kNN regression: X = [[1], [2], [3], [10]], y = [1.5, 2.5, 3, 9], x = [2.4], k = 2, metric = euclidean, weights = uniform, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Gaussian naive Bayes prediction, `ml_gaussian_naive_bayes`

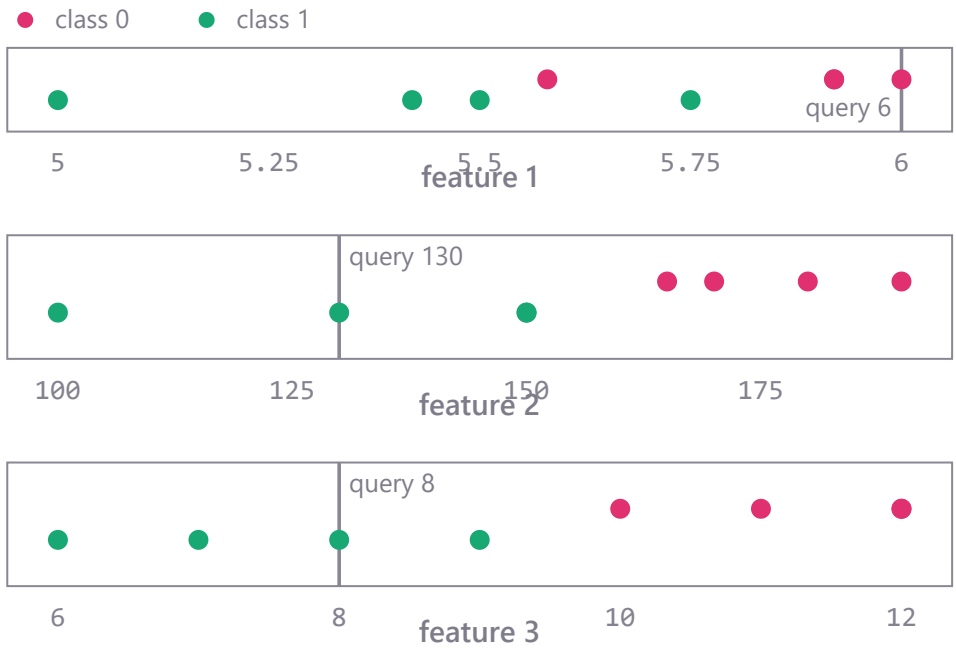
Fit a Gaussian naive Bayes classifier (class frequencies as priors, per-class feature means and population variances) and give the predicted class and posterior probabilities of a query point.

$$P(c \mid x) \propto P(c) \prod_j \mathcal{N}(x_j;\ \mu_{cj}, \sigma^2_{cj})$$

input	meaning	condition
`X`	training features: one row per sample	
`y`	training labels (whole numbers)	
`x`	query point	

output	meaning
`prediction`	class with the largest posterior
`classes`	classes in sorted order
`posteriors`	posterior probability of each class

- **Method:** log prior + sum of Gaussian log densities (variance divides by n_c), normalised by log-sum-exp
- **Assumes:** at least two classes; positive within-class variance of every feature
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated training rows split by class (y = [0, 0, 0, 0, 1, 1, 1, 1]), one axis per feature, with the query x = [6, 130, 8] as the vertical line: the second and third features place it among class 1. Returned posteriors: class 0 1.5443e-07, class 1 1; prediction 1.

```
data_science_ml_ai999x.solve('ml_gaussian_naive_bayes', X=[[6, 180, 12], [5.92, 190, 11], [5.58, 170, 12], [5.92, 165, 10], [5, 100, 6], [5.5, 150, 8], [5.42, 130, 7], [5.75, 150, 9]], y=[0, 0, 0, 0, 1, 1, 1, 1], x=[6, 130, 8])
# -> prediction = 1, classes = [0, 1], posteriors = [1.54428667821e-07, 0.999999845571]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_gaussian_naive_bayes', X=[[1.0], [1.2], [0.8], [3.0], [3.3], [2.9]], y=[0, 0, 0, 1, 1, 1], x=[2.0])
# -> prediction = 0, classes = [0, 1], posteriors = [0.727570105482, 0.272429894518]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gaussian naive Bayes: X = [[6, 180, 12], [5.92, 190, 11], [5.58, 170, 12], [5.92, 165, 10], [5, 100, 6], [5.5, 150, 8], [5.42, 130, 7], [5.75, 150, 9]], y = [0, 0, 0, 0, 1, 1, 1, 1], x = [6, 130, 8]	SOLVED	prediction = 1, classes = [0, 1], posteriors = [1.54428667821e-07, 0.999999845571]
Gaussian naive Bayes: X = [[1], [1.2], [0.8], [3], [3.3], [2.9]], y = [0, 0, 0, 1, 1, 1], x = [2]	SOLVED	prediction = 0, classes = [0, 1], posteriors = [0.727570105482, 0.272429894518]
Gaussian naive Bayes: y = [0, 0, 0, 0, 1, 1, 1, 1], x = [6, 130, 8]	NEEDS_INPUT	ml_gaussian_naive_bayes needs X (training features: one row per sample). Example: solve('ml_gaussian_naive_bayes', X=[[6
Gaussian naive Bayes: X = [[6, 180, 12], [5.92, 190, 11], [5.58, 170, 12], [5.92, 165, 10], [5, 100, 6], [5.5, 150, 8], [5.42, 130, 7], [5.75, 150, 9]], y = [0, 0, 0, 0, 1, 1, 1, 1], x = [6, 130, 8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Gaussian naive Bayes: X = [[1], [1], [3], [4]], y = [0, 0, 1, 1], x = [2]	INVALID_INPUT	Gaussian naive Bayes prediction: a feature has zero variance within a class (the Gaussian likelihood is undefined).

Multinomial naive Bayes prediction (word counts), `ml_multinomial_naive_bayes`

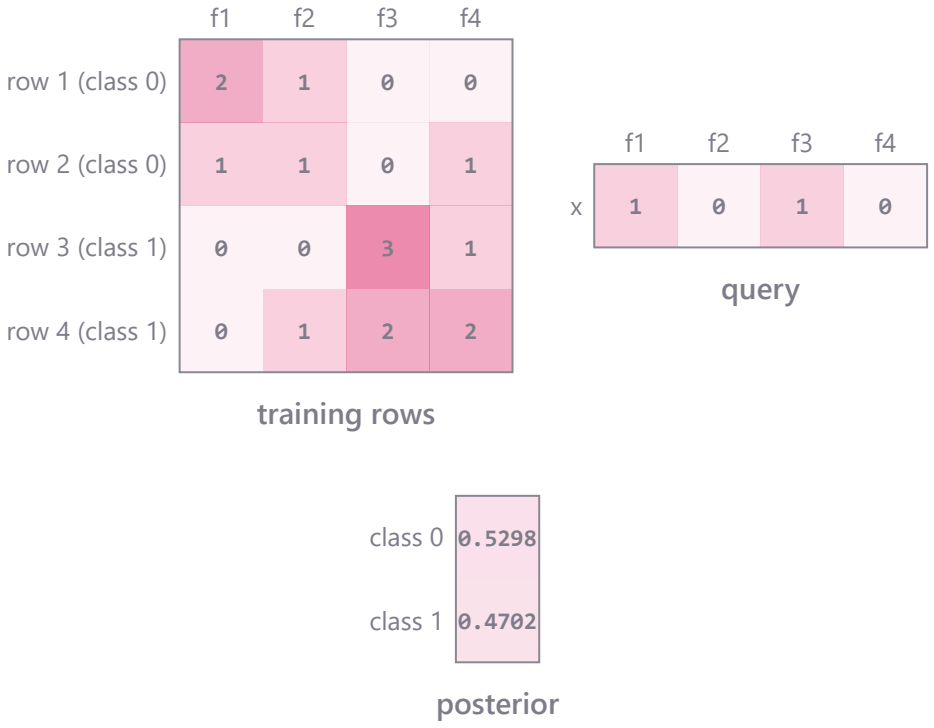
Fit a multinomial naive Bayes classifier on count features (such as word counts) with additive (Laplace/Lidstone) smoothing alpha and give the predicted class and posteriors of a query count vector.

$$\hat{\theta}_{cj} = \frac{N_{cj} + \alpha}{N_c + \alpha d}, \quad \log P(c \mid x) = \log P(c) + \sum_j x_j \log \hat{\theta}_{cj} + \text{const}$$

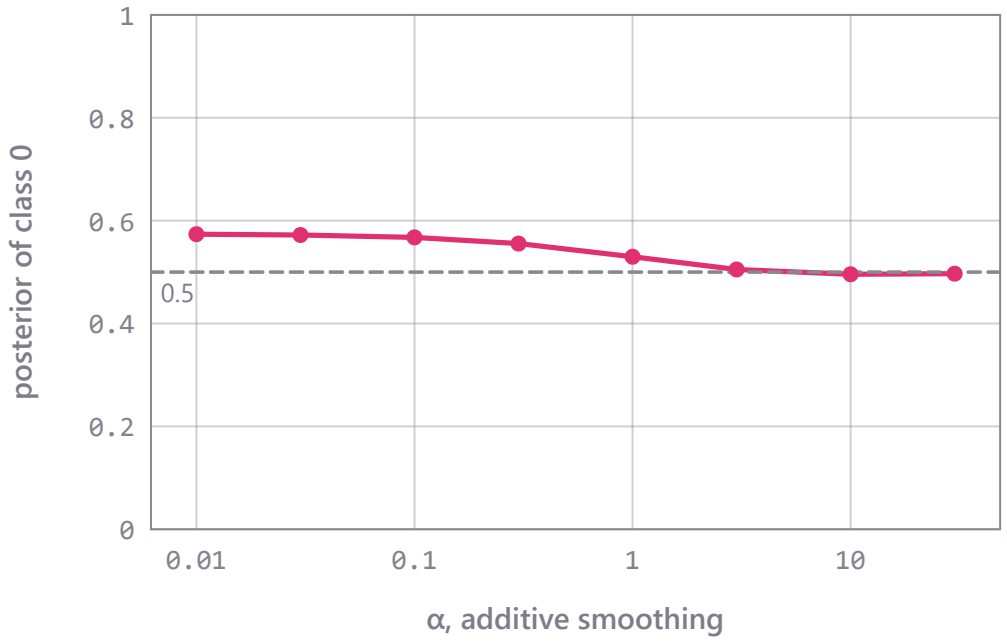
input	meaning	condition
`X`	training counts: one row per document, one column per term	
`y`	training labels (whole numbers)	
`x`	query counts	
`alpha`	additive smoothing (> 0; 1 = Laplace)	

output	meaning
`prediction`	class with the largest posterior
`classes`	classes in sorted order
`posteriors`	posterior probability of each class

- **Method:** theta_cj = (N_cj + alpha) / (N_c + alpha d); log prior + sum x_j log theta_cj; log-sum-exp normalisation
- **Assumes:** $\alpha > 0$; counts >= 0; at least two classes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated training rows (labelled with their class y = [0, 0, 1, 1]) and the query x = [1, 0, 1, 0]; returned posteriors class 0: 0.529781, class 1: 0.470219, prediction class 0 (smoothing α = 1).



Posterior probability of class 0 for the query x = [1, 0, 1, 0] as the smoothing α runs over 0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30 (one run each, same training rows): the query's third word never occurs in class 0, so little smoothing hands it to class 1; more smoothing pulls both posteriors toward 0.5.

```
data_science_ml_ai999x.solve('ml_multinomial_naive_bayes', X=[[2, 1, 0, 0], [1, 1, 0, 1], [0, 0, 3, 1], [0, 1, 2, 2]], y=[0, 0, 1, 1], x=[1, 0, 1, 0], alpha=1)
# -> prediction = 0, classes = [0, 1], posteriors = [0.529780564263, 0.470219435737]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_multinomial_naive_bayes', X=[[3, 0, 1], [2, 1, 0], [0, 2, 3], [1, 3, 2], [0, 1, 4]], y=[1, 1, 2, 2, 2], x=[2, 1, 1], alpha=0.5)
# -> prediction = 1, classes = [1, 2], posteriors = [0.854392490787, 0.145607509213]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multinomial naive Bayes: X = [[2, 1, 0, 0], [1, 1, 0, 1], [0, 0, 3, 1], [0, 1, 2, 2]], y = [0, 0, 1, 1], x = [1, 0, 1, 0], alpha = 1	SOLVED	prediction = 0, classes = [0, 1], posteriors = [0.529780564263, 0.470219435737]
Multinomial naive Bayes: X = [[3, 0, 1], [2, 1, 0], [0, 2, 3], [1, 3, 2], [0, 1, 4]], y = [1, 1, 2, 2, 2], x = [2, 1, 1], alpha = 0.5	SOLVED	prediction = 1, classes = [1, 2], posteriors = [0.854392490787, 0.145607509213]
Multinomial naive Bayes: y = [0, 0, 1, 1], x = [1, 0, 1, 0], alpha = 1	NEEDS_INPUT	ml_multinomial_naive_bayes needs X (training counts: one row per document, one column per term). Example: solve('ml_mult
Multinomial naive Bayes: X = [[2, 1, 0, 0], [1, 1, 0, 1], [0, 0, 3, 1], [0, 1, 2, 2]], y = [0, 0, 1, 1], x = [1, 0, 1, 0], alpha = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Multinomial naive Bayes: X = [[2, 1], [0, 3]], y = [0, 1], x = [1, 1], alpha = 0	INVALID_INPUT	Multinomial naive Bayes prediction (word counts): alpha (additive smoothing) must be positive.

Bernoulli naive Bayes prediction (binary features), `ml_bernoulli_naive_bayes`

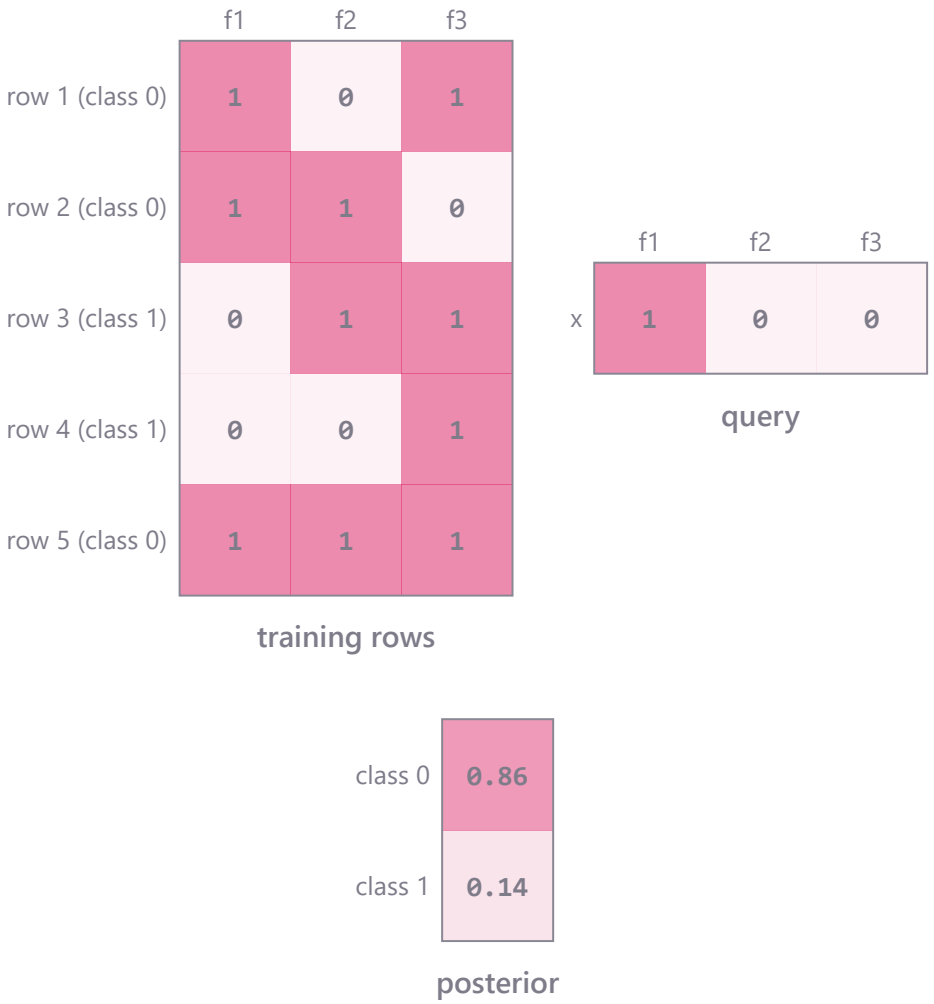
Fit a Bernoulli naive Bayes classifier on $0/1$ features with additive smoothing alpha (absent features count too) and give the predicted class and posteriors of a query vector.

$$P(x \mid c) = \prod_j p_{cj}^{x_j}(1 - p_{cj})^{1 - x_j}$$

input	meaning	condition
`X`	training features (0 or 1)	
`y`	training labels (whole numbers)	
`x`	query features (0 or 1)	
`alpha`	additive smoothing (> 0)	

output	meaning
`prediction`	class with the largest posterior
`classes`	classes in sorted order
`posteriors`	posterior probability of each class

- **Method:** p_cj = (N_cj + alpha)/(N_c + 2 alpha); log prior + sum of log p or log(1 - p)
- **Assumes:** features 0 or 1; $\alpha > 0$; at least two classes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated training rows (labelled with their class y = [0, 0, 1, 1, 0]) and the query x = [1, 0, 0]; returned posteriors class 0: 0.860022, class 1: 0.139978, prediction class 0 (smoothing α = 1).

```
data_science_ml_ai999x.solve('ml_bernoulli_naive_bayes', X=[[1, 0, 1], [1, 1, 0], [0, 1, 1], [0, 0, 1], [1, 1, 1]], y=[0, 0, 1, 1, 0], x=[1, 0, 0], alpha=1)
# -> prediction = 0, classes = [0, 1], posteriors = [0.860022396417, 0.139977603583]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_bernoulli_naive_bayes', X=[[1, 0], [0, 1], [1, 1], [0, 0]], y=[1, 2, 1, 2], x=[0, 1], alpha=0.5)
# -> prediction = 2, classes = [1, 2], posteriors = [0.166666666667, 0.833333333333]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bernoulli naive Bayes: X = [[1, 0, 1], [1, 1, 0], [0, 1, 1], [0, 0, 1], [1, 1, 1]], y = [0, 0, 1, 1, 0], x = [1, 0, 0], alpha = 1	SOLVED	prediction = 0, classes = [0, 1], posteriors = [0.860022396417, 0.139977603583]
Bernoulli naive Bayes: X = [[1, 0], [0, 1], [1, 1], [0, 0]], y = [1, 2, 1, 2], x = [0, 1], alpha = 0.5	SOLVED	prediction = 2, classes = [1, 2], posteriors = [0.166666666667, 0.833333333333]
Bernoulli naive Bayes: y = [0, 0, 1, 1, 0], x = [1, 0, 0], alpha = 1	NEEDS_INPUT	ml_bernoulli_naive_bayes needs X (training features (0 or 1)). Example: solve('ml_bernoulli_naive_bayes', X=[[1, 0, 1],

question	outcome	result
Bernoulli naive Bayes: X = [[1, 0, 1], [1, 1, 0], [0, 1, 1], [0, 0, 1], [1, 1, 1]], y = [0, 0, 1, 1, 0], x = [1, 0, 0], alpha = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Best decision-tree split (classification, Gini or entropy), `ml_decision_stump_classification`

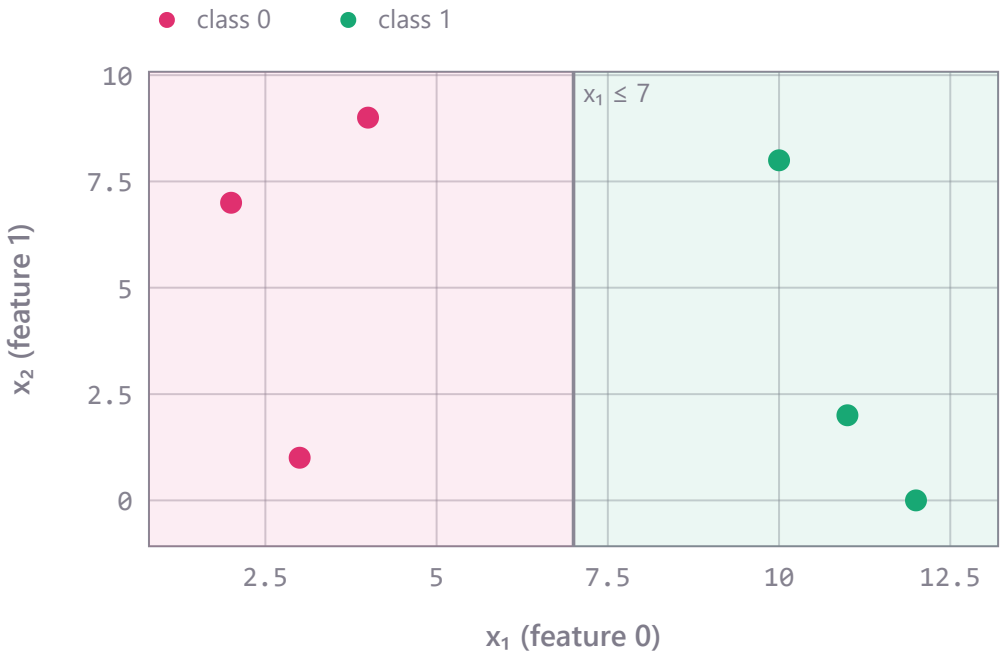
The best single split $x_j \le t$ of a classification tree's root by Gini impurity or entropy (bits): the feature, the threshold (midway between consecutive values), the impurity decrease and the sizes of both sides.

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

input	meaning	condition
`X`	features: one row per sample (whole numbers)	
`y`	class labels (whole numbers)	
`criterion`	split criterion: gini or entropy	

output	meaning
`feature`	index (0-based column of X) of the best feature
`threshold`	threshold t (go left when x_j <= t)
`impurity_decrease`	parent impurity minus the size-weighted child impurities
`n_left`	samples going left
`n_right`	samples going right

- **Method:** scan every feature and every midpoint between consecutive distinct values; exact fractions for Gini
- **Assumes:** at least two classes; whole-number features; a unique best split
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The best single split of the 6 stated points (labels [0, 0, 1, 1, 0, 1], gini criterion): the package returns feature 0, threshold 7, 3 points left and 3 right, impurity decrease 0.5, the split separates the classes perfectly.

```
data_science_ml_ai999x.solve('ml_decision_stump_classification', X=[[2, 7], [3, 1], [10, 8], [11, 2], [4, 9], [12, 0]], y=[0, 0, 1, 1, 0, 1], criterion='gini')
# -> feature = 0, threshold = 7, impurity_decrease = 0.5, n_left = 3, n_right = 3   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_decision_stump_classification', X=[[1], [2], [3], [4], [5], [6], [7]], y=[0, 0, 1, 0, 1, 1, 1], criterion='entropy')
# -> feature = 0, threshold = 4.5, impurity_decrease = 0.521641, n_left = 4, n_right = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decision tree best split: X = [[2, 7], [3, 1], [10, 8], [11, 2], [4, 9], [12, 0]], y = [0, 0, 1, 1, 0, 1], criterion = gini	SOLVED	feature = 0, threshold = 7, impurity_decrease = 0.5, n_left = 3, n_right = 3
Decision tree best split: X = [[1], [2], [3], [4], [5], [6], [7]], y = [0, 0, 1, 0, 1, 1, 1], criterion = entropy	SOLVED	feature = 0, threshold = 4.5, impurity_decrease = 0.521641, n_left = 4, n_right = 3
Decision tree best split: y = [0, 0, 1, 1, 0, 1], criterion = gini	NEEDS_INPUT	ml_decision_stump_classification needs X (features: one row per sample (whole numbers)). Example: solve('ml_decision_stu
Decision tree best split: X = [[2, 7], [3, 1], [10, 8], [11, 2], [4, 9], [12, 0]], y = [0, 0, 1, 1, 0, 1], criterion = gini, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Decision tree best split: X = [[1, 5], [2, 6]], y = [1, 1], criterion = gini	INVALID_INPUT	Best decision-tree split (classification, Gini or entropy): the node is pure: there is nothing to split.

Agglomerative hierarchical clustering merge heights, `ml_hierarchical_clustering`

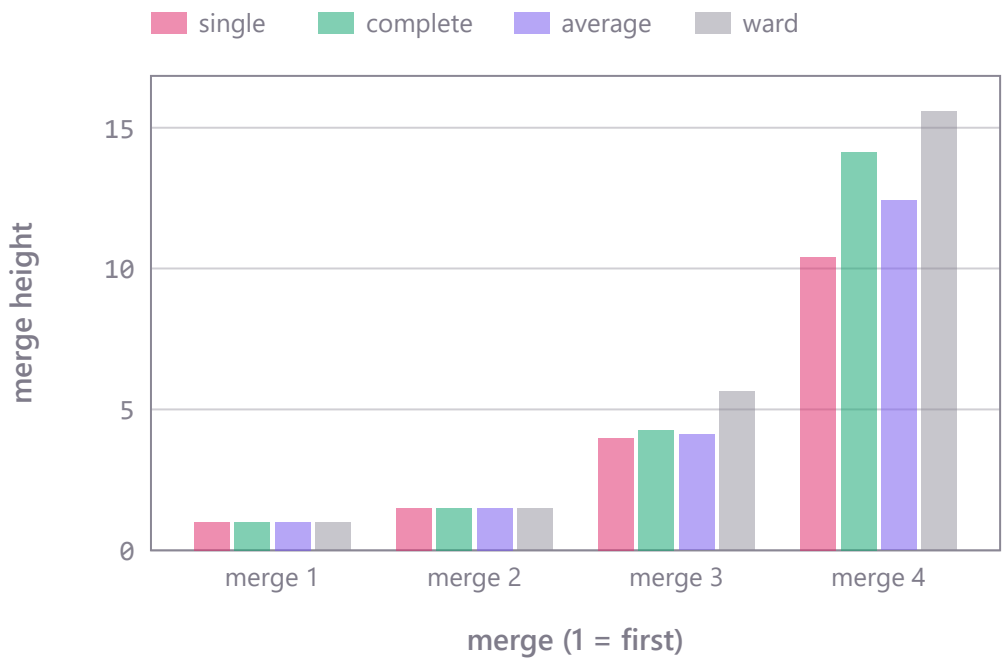
Agglomerative clustering of points with single, complete, average or Ward linkage (Euclidean distance): the distance at which each of the $n - 1$ merges happens, in order, and the final merge height.

$$d(k, i \cup j) = \sqrt{\frac{(n_i + n_k) d_{ki}^2 + (n_j + n_k) d_{kj}^2 - n_k d_{ij}^2}{n_i + n_j + n_k}} \quad \text{(Ward)}$$

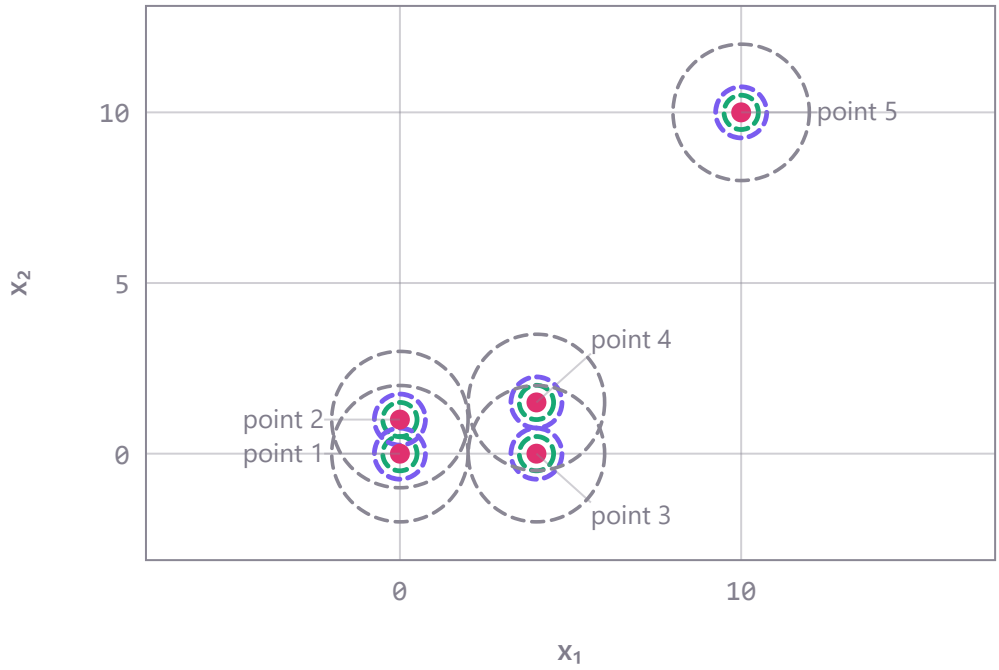
input	meaning	condition
`X`	points: one row per sample (2 to 40)	
`method`	linkage: single, complete, average or ward	

output	meaning
`heights`	merge distances in merge order (ascending)
`final_height`	height of the last merge

- **Method:** repeatedly merge the closest pair; Lance-Williams updates of the cluster distances
- **Assumes:** 2 to 40 points; no tied merge distances (complete, average, ward)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The merge heights the package returns for the 5 stated points X = [[0, 0], [0, 1], [4, 0], [4, 1.5], [10, 10]] under four linkages, one run each: single: 1, 1.5, 4, 10.4043; complete: 1, 1.5, 4.272, 14.1421; average: 1, 1.5, 4.10656, 12.4155; ward: 1, 1.5, 5.66789, 15.5893. The last merge joins the far point (10, 10).



The stated points with dashed circles of radius half of each of the first three returned single-linkage merge heights (1, 1.5, 4): two points merge at a height when their circles touch; the far point joins last at 10.4043.

```
data_science_ml_ai999x.solve('ml_hierarchical_clustering', X=[[0, 0], [0, 1], [4, 0], [4, 1.5], [10, 10]], method='single')
# -> heights = [1.0, 1.5, 4.0, 10.4043260233], final_height = 10.4043   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_hierarchical_clustering', X=[[0, 0], [0, 1], [4, 0], [4, 1.5], [10, 10]], method='ward')
# -> heights = [1.0, 1.5, 5.66789202438, 15.5892591229], final_height = 15.5893   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hierarchical clustering: X = [[0, 0], [0, 1], [4, 0], [4, 1.5], [10, 10]], method = single	SOLVED	heights = [1.0, 1.5, 4.0, 10.4043260233], final_height = 10.4043
Hierarchical clustering: X = [[0, 0], [0, 1], [4, 0], [4, 1.5], [10, 10]], method = ward	SOLVED	heights = [1.0, 1.5, 5.66789202438, 15.5892591229], final_height = 15.5893
Hierarchical clustering: method = single	NEEDS_INPUT	ml_hierarchical_clustering needs X (points: one row per sample (2 to 40)). Example: solve('ml_hierarchical_clustering',
Hierarchical clustering: X = [[0, 0], [0, 1], [4, 0], [4, 1.5], [10, 10]], method = single, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Principal component analysis (explained variance and first component), `ml_pca`

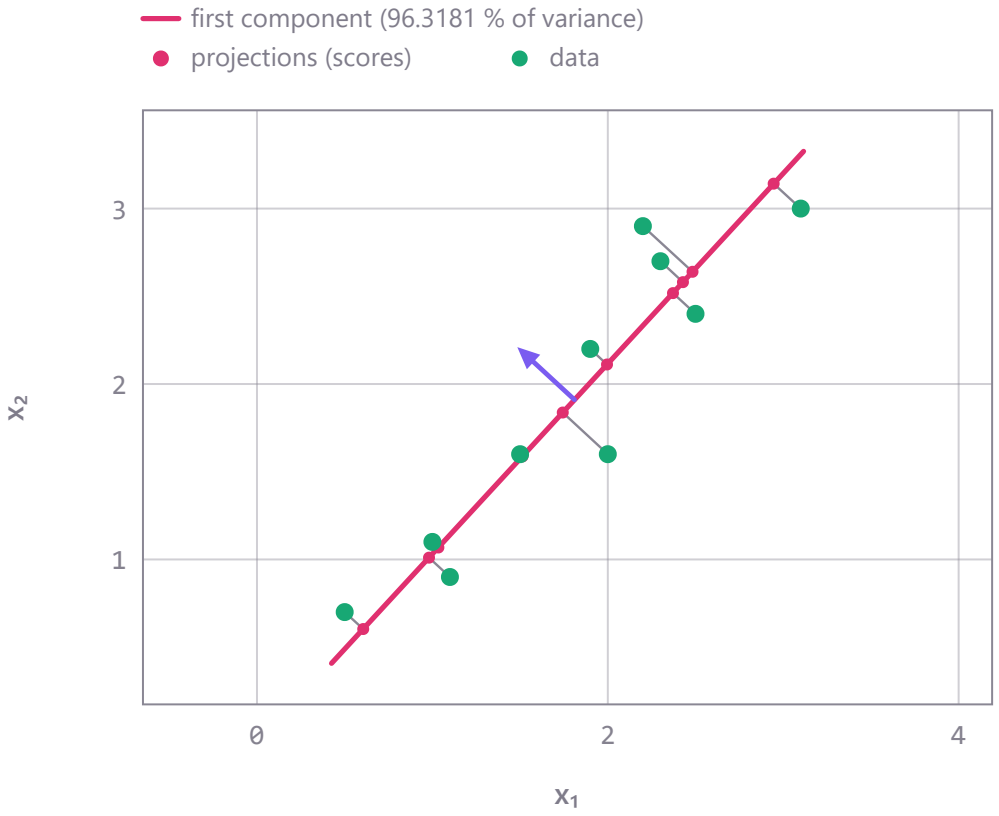
Principal component analysis of a data matrix: the variance explained by every component (sample covariance, n - 1), the explained variance ratios, the first principal direction and the first component score of every sample.

$$\Sigma = \frac{1}{n-1}(X - \bar{X})^\top (X - \bar{X}) = V \Lambda V^\top$$

input	meaning	condition
`X`	data: one row per sample, one column per feature (more samples than features, at most 8 features)	

output	meaning
`explained_variance`	eigenvalues of the sample covariance, descending
`explained_variance_ratio`	each eigenvalue / their sum
`component_1`	first principal direction (unit length, largest-magnitude entry positive)
`scores_1`	projection of each centred sample on component_1

- **Method:** exact centring and covariance, cyclic Jacobi eigen-decomposition
- **Assumes:** n > number of features; full rank; a unique first component
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Principal components of 10 points: the package's first component (0.677873, 0.735179) through the mean of the points explains 96.3181 % of the variance (variance 1.28403 against 0.0490834); each point is joined to its projection at the score the package returns; the short arrow is two standard deviations (0.443095) along the second component.



projection of each centred sample on component_1 (scores_1) returned by data-science-ml-ai999x for the worked example, one value for each position in the list (10 values, first 0.82797, last -1.22382, largest 1.6758). Inputs: X = [[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.....

```
data_science_ml_ai999x.solve('ml_pca', X=[[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0], [2.3, 2.7], [2, 1.6], [1, 1.1], [1.5, 1.6], [1.1, 0.9]])
# -> explained_variance = [1.28402771217, 0.0490833989383], explained_variance_ratio = [0.963181314349, 0.0368186856514], component_1 = [0.677873398528, 0.735178655544], scores_1 = [0.827970186201, -1.77758032528, 0.992197494415, 0.274210415975, 1.67580141864, 0.912949103159, -0.0991094374984, -1.1445721638, -0.438046136762, -1.22382055505]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_pca', X=[[1, 2, 3], [2, 1, 3], [4, 5, 7], [3, 3, 2], [5, 6, 4]])
# -> explained_variance = [8.2379768711, 2.0120231289, 0.25], explained_variance_ratio = [0.784569225819, 0.191621250371, 0.0238095238095], component_1 = [0.498282977681, 0.691615919337, 0.522858962124], scores_1 = [-2.38311541213, -2.57644835379, 3.27801712742, -1.21779249956, 2.89933913806]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PCA: X = [[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3], [2.3, 2.7], [2, 1.6], [1, 1.1], [1.5, 1.6], [1.1, 0.9]]	SOLVED	explained_variance = [1.28402771217, 0.0490833989383], explained_variance_ratio = [0.963181314349, 0.0368186856514], component_1 = [0.677873398528, 0.735178655544], scores_1 = [0.827970186201, -1.77758032528, 0.992197494415, 0.274210415975, 1.67580141864, 0.912949103159, -0.0991094374984, -1.1445721638, -0.438046136762, -1.22382055505]
PCA: X = [[1, 2, 3], [2, 1, 3], [4, 5, 7], [3, 3, 2], [5, 6, 4]]	SOLVED	explained_variance = [8.2379768711, 2.0120231289, 0.25], explained_variance_ratio = [0.784569225819, 0.191621250371, 0.0238095238095], component_1 = [0.498282977681, 0.691615919337, 0.522858962124], scores_1 = [-2.38311541213, -2.57644835379, 3.27801712742, -1.21779249956, 2.89933913806]
PCA:	NEEDS_INPUT	ml_pca needs X (data: one row per sample, one column per feature (more samples than features, at most 8 features)). Exam

question	outcome	result
PCA: X = [[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3], [2.3, 2.7], [2, 1.6], [1, 1.1], [1.5, 1.6], [1.1, 0.9]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
PCA: X = [[1, 2], [2, 4], [3, 6], [4, 8]]	INVALID_INPUT	Principal component analysis (explained variance and first component): the data are rank-deficient: a component's varian

Standardisation (z-scores of features), `ml_standard_scaler`

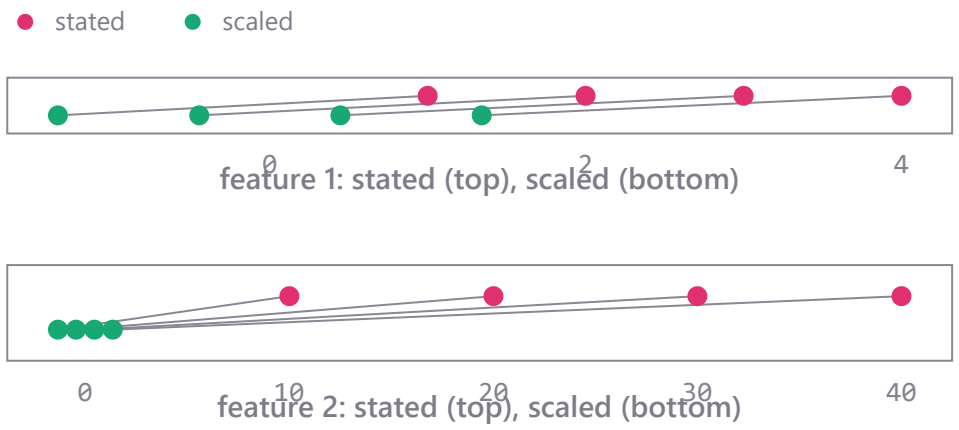
Standardise each feature column to mean 0 and unit variance: the column means, the population standard deviations and the scaled matrix (a constant column scales to 0).

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}, \quad s_j^2 = \frac{1}{n}\sum_i (x_{ij} - \bar{x}_j)^2$$

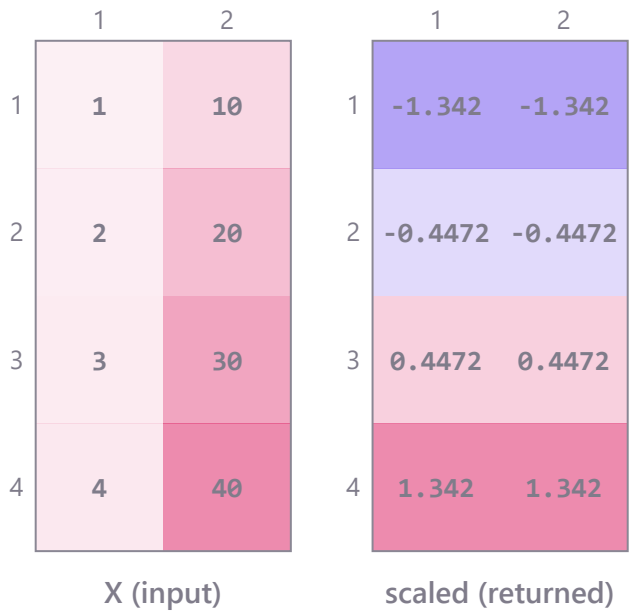
input	meaning	condition
`X`	data: one row per sample, one column per feature	

output	meaning
`means`	column means
`stds`	population standard deviations (divide by n)
`scaled`	(x - mean) / std

- **Method:** exact means and variances; one square root per column
- **Assumes:** a constant column scales to 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Each feature of X = [[1, 10], [2, 20], [3, 30], [4, 40]] before (top dots) and after (bottom dots) standardising to mean 0, standard deviation 1, the scaled values as returned; also returned: means ['2.5', '25'], stds ['1.11803', '11.1803'].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); scaled (returned).

```
data_science_ml_ai999x.solve('ml_standard_scaler', X=[[1, 10], [2, 20], [3, 30], [4, 40]])
# -> means = [2.5, 25.0], stds = [1.11803398875, 11.1803398875], scaled = [[-1.3416407865,
-1.3416407865], [-0.4472135955, -0.4472135955], [0.4472135955, 0.4472135955], [1.3416407865,
1.3416407865]]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_standard_scaler', X=[[0, 5], [0, 5], [1, 5], [1, 5]])
# -> means = [0.5, 5.0], stds = [0.5, 0.0], scaled = [[-1.0, 0.0], [-1.0, 0.0], [1.0, 0.0], [1.0,
0.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Standard scaler: X = [[1, 10], [2, 20], [3, 30], [4, 40]]	SOLVED	means = [2.5, 25.0], stds = [1.11803398875, 11.1803398875], scaled = [[-1.3416407865, -1.3416407865], [-0.4472135955, -0.4472135955], [0.4472135955, 0.4472135955], [1.3416407865, 1.3416407865]]
Standard scaler: X = [[0, 5], [0, 5], [1, 5], [1, 5]]	SOLVED	means = [0.5, 5.0], stds = [0.5, 0.0], scaled = [[-1.0, 0.0], [-1.0, 0.0], [1.0, 0.0], [1.0, 0.0]]
Standard scaler:	NEEDS_INPUT	ml_standard_scaler needs X (data: one row per sample, one column per feature). Example: solve('ml_standard_scaler', X=[[
Standard scaler: X = [[1, 10], [2, 20], [3, 30], [4, 40]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Min-max scaling to [0, 1], `ml_minmax_scaler`

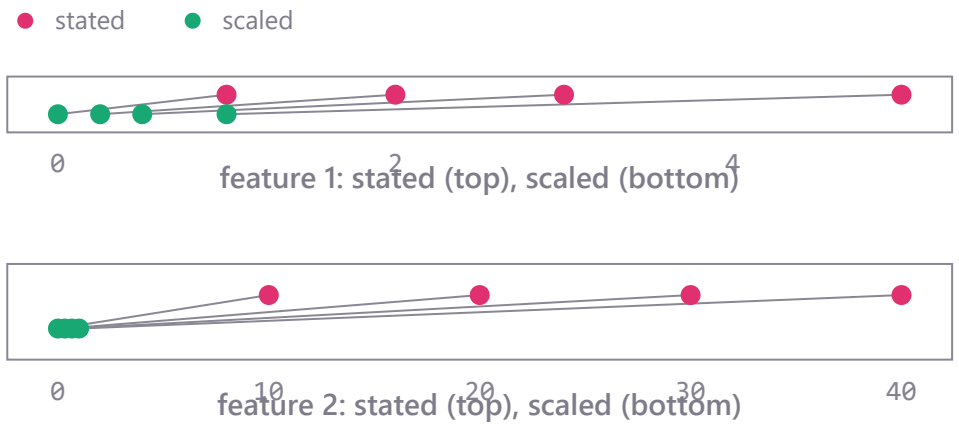
Scale each feature column linearly to the range [0, 1] using its minimum and maximum (a constant column scales to 0).

$$x' = \frac{x - \min_j}{\max_j - \min_j}$$

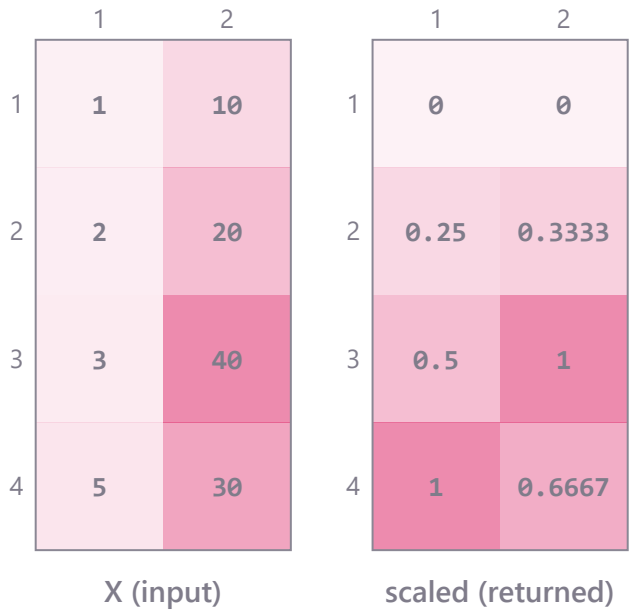
input	meaning	condition
`X`	data: one row per sample, one column per feature	

output	meaning
`mins`	column minima
`maxs`	column maxima
`scaled`	(x - min) / (max - min)

- **Method:** exact rational arithmetic
- **Assumes:** a constant column scales to 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Each feature of X = [[1, 10], [2, 20], [3, 40], [5, 30]] before (top dots) and after (bottom dots) min–max scaling to [0, 1], the scaled values as returned; also returned: maxs ['5', '40'], mins ['1', '10'].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); scaled (returned).

```
data_science_ml_ai999x.solve('ml_minmax_scaler', X=[[1, 10], [2, 20], [3, 40], [5, 30]])
# -> mins = [1.0, 10.0], maxs = [5.0, 40.0], scaled = [[0.0, 0.0], [0.25, 0.333333333333], [0.5,
1.0], [1.0, 0.666666666667]]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_minmax_scaler', X=[[-3, 7], [0, 7], [9, 7]])
# -> mins = [-3.0, 7.0], maxs = [9.0, 7.0], scaled = [[0.0, 0.0], [0.25, 0.0], [1.0, 0.0]]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Min-max scaler: X = [[1, 10], [2, 20], [3, 40], [5, 30]]	SOLVED	mins = [1.0, 10.0], maxs = [5.0, 40.0], scaled = [[0.0, 0.0], [0.25, 0.333333333333], [0.5, 1.0], [1.0, 0.666666666667]]
Min-max scaler: X = [[-3, 7], [0, 7], [9, 7]]	SOLVED	mins = [-3.0, 7.0], maxs = [9.0, 7.0], scaled = [[0.0, 0.0], [0.25, 0.0], [1.0, 0.0]]
Min-max scaler:	NEEDS_INPUT	ml_minmax_scaler needs X (data: one row per sample, one column per feature). Example: solve('ml_minmax_scaler', X=[[1, 1
Min-max scaler: X = [[1, 10], [2, 20], [3, 40], [5, 30]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Label encoding and one-hot encoding of categories, `ml_one_hot_encoding`

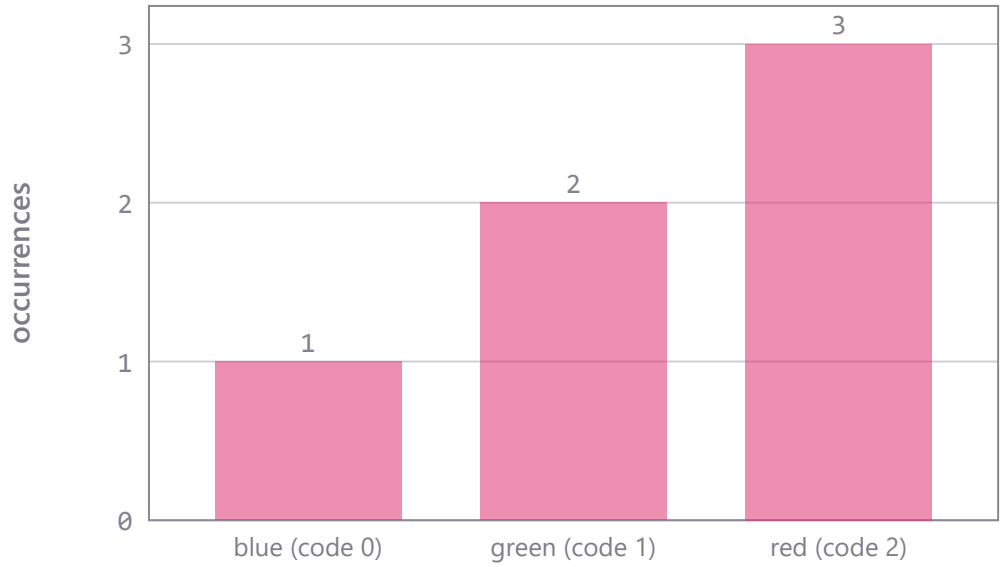
Encode a list of category values: the categories in sorted order, how often each occurs, the integer code (label encoding) of each value and its one-hot row.

$$\mathrm{onehot}(v)_j = \mathbb{1}(v = c_j)$$

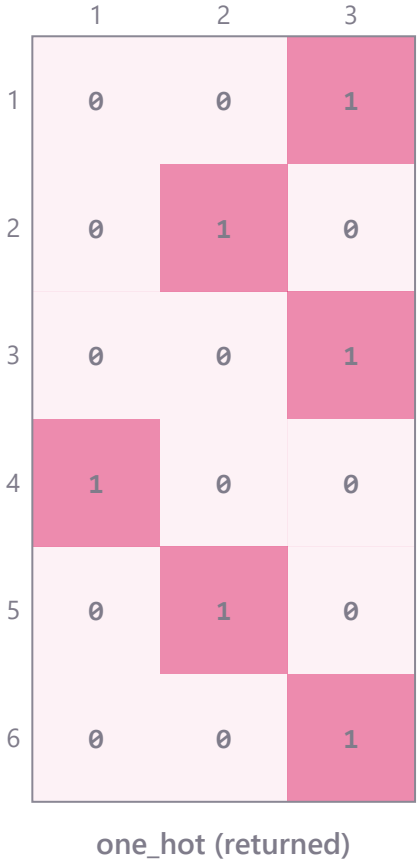
input	meaning	condition
`categories`	category values, written as a bracketed list of words: [red, green, red]	

output	meaning
`categories`	distinct categories, sorted
`counts`	occurrences of each category
`codes`	label code of each value (position of its category)
`one_hot`	one-hot row of each value

- **Method:** sort the distinct values; index each value
- **Assumes:** non-empty category words
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The categories of 'red, green, red, blue, green, red' in sorted order with their returned codes and counts (blue: 1, green: 2, red: 3); each becomes one column of the one-hot matrix.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): one_hot (returned). Other inputs: categories = red, green, red, blue, green, red.

```
data_science_ml_ai999x.solve('ml_one_hot_encoding', categories='red, green, red, blue, green, red')
# -> categories = ['blue', 'green', 'red'], counts = [1, 2, 3], codes = [2, 1, 2, 0, 1, 2], one_hot
= [[0, 0, 1], [0, 1, 0], [0, 0, 1], [1, 0, 0], [0, 1, 0], [0, 0, 1]]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_one_hot_encoding', categories='low, high, medium, low')
# -> categories = ['high', 'low', 'medium'], counts = [1, 2, 1], codes = [1, 0, 2, 1], one_hot =
[[0, 1, 0], [1, 0, 0], [0, 0, 1], [0, 1, 0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
One-hot encoding: categories = [red, green, red, blue, green, red]	SOLVED	categories = ['blue', 'green', 'red'], counts = [1, 2, 3], codes = [2, 1, 2, 0, 1, 2], one_hot = [[0, 0, 1], [0, 1, 0], [0, 0, 1], [1, 0, 0], [0, 1, 0], [0, 0, 1]]
One-hot encoding: categories = [low, high, medium, low]	SOLVED	categories = ['high', 'low', 'medium'], counts = [1, 2, 1], codes = [1, 0, 2, 1], one_hot = [[0, 1, 0], [1, 0, 0], [0, 0, 1], [0, 1, 0]]
One-hot encoding:	NEEDS_INPUT	ml_one_hot_encoding needs categories (category values, written as a bracketed list of words: [red, green, red]). Example
One-hot encoding: categories = [red, green, red, blue, green, red], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Train/test split sizes, `ml_train_test_split_sizes`

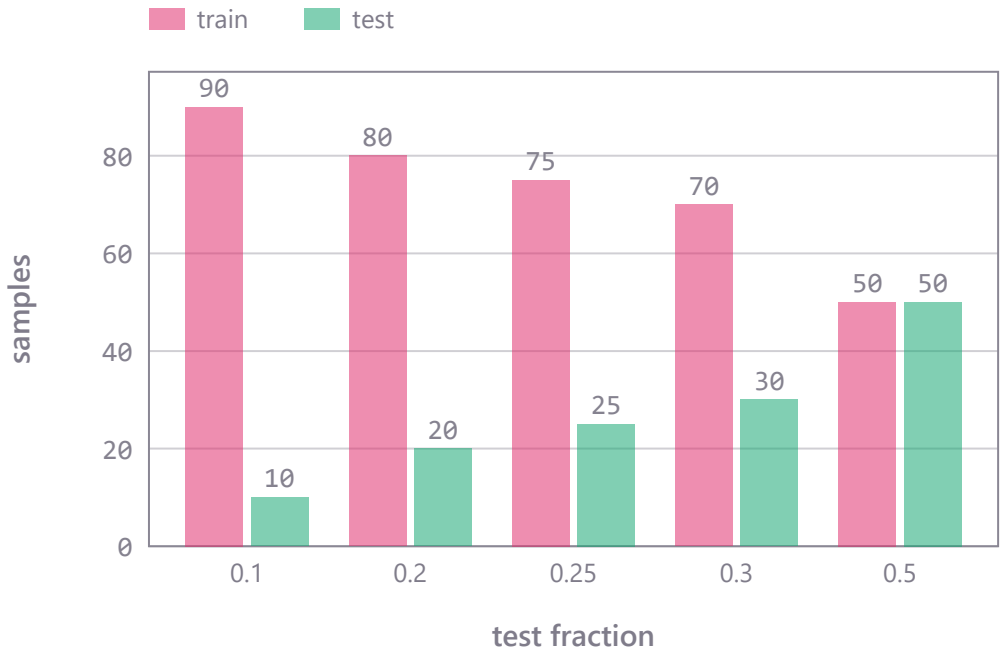
The number of test and training samples when n samples are split with a given test fraction (the test size is rounded up).

$$n_{\text{test}} = \lceil t\, n \rceil, \quad n_{\text{train}} = n - n_{\text{test}}$$

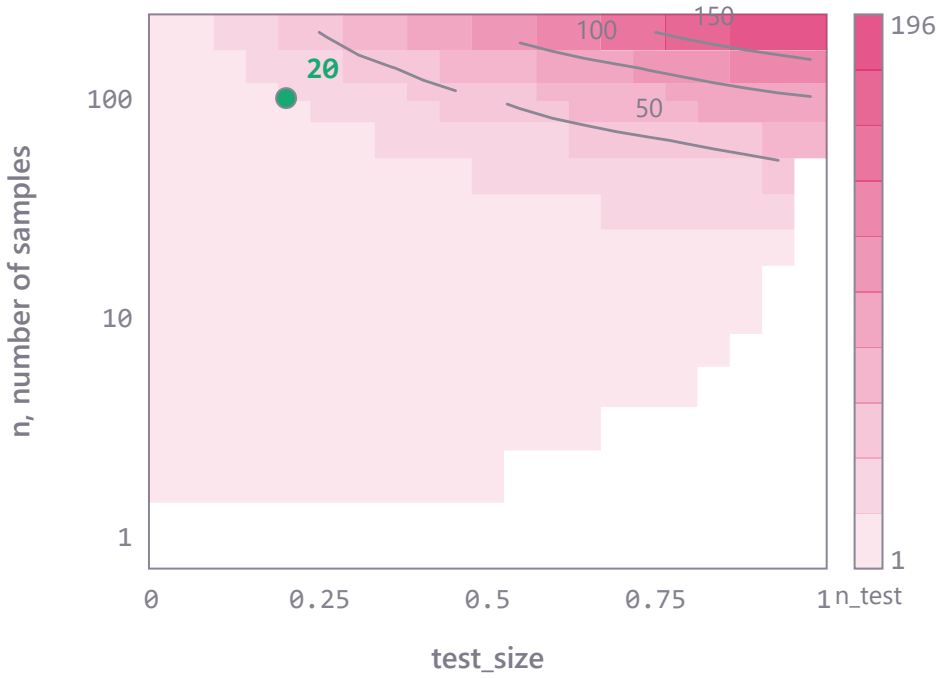
input	meaning	condition
`n`	number of samples	
`test_size`	fraction of the samples used for testing (0 to 1, or a percentage)	

output	meaning
`n_test`	ceil(test_size * n)
`n_train`	n - n_test

- **Method:** exact ceiling of the written fraction times n
- **Assumes:** 0 < test_size < 1; a non-empty training set
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Train and test sizes of 100 samples for test fractions 0.1, 0.2, 0.25, 0.3, 0.5, one run each: 0.1: 90 / 10; 0.2: 80 / 20; 0.25: 75 / 25; 0.3: 70 / 30; 0.5: 50 / 50.



n_test (ceil(test_size * n)) over test_size (fraction of the samples used for testing (0 to 1, or a percentage)) from 0.02 to 0.98 and n (number of samples) from 1 to 200 (logarithmic vertical axis): each of the 21 × 15 cells is one run of data-science-ml-ai999x, shaded by its value (light 1 to dark 196); iso-lines at n_test = 50, 100, 150; blank cells: inputs the capability refuses. Marked: the worked example, test_size = 0.2, n = 100, n_test = 20.

```
data_science_ml_ai999x.solve('ml_train_test_split_sizes', n=100, test_size=0.2)
# -> n_test = 20, n_train = 80   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_train_test_split_sizes', n=47, test_size=0.25)
# -> n_test = 12, n_train = 35   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Train test split sizes: n = 100, test_size = 0.2	SOLVED	n_test = 20, n_train = 80
Train test split sizes: n = 47, test_size = 0.25	SOLVED	n_test = 12, n_train = 35
Train test split sizes: test_size = 0.2	NEEDS_INPUT	ml_train_test_split_sizes needs n (number of samples). Example: solve('ml_train_test_split_sizes', n=100, test_size=0.2)
Train test split sizes: n = 100, test_size = 0.2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Train test split sizes: n = 100, test_size = 1.5	INVALID_INPUT	Train/test split sizes: test_size must be a fraction strictly between 0 and 1.

k-fold cross-validation fold sizes, `ml_kfold_sizes`

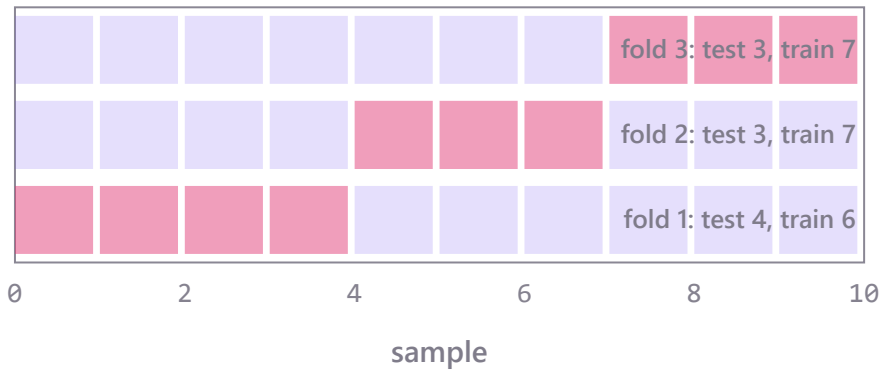
The test and training set sizes of each fold when n samples are split into k consecutive folds (the first n mod k folds get one extra sample), and the index range of the first test fold.

$$|F_i| = \lfloor n/k \rfloor + \mathbb{1}(i < n \bmod k)$$

input	meaning	condition
`n`	number of samples	
`k`	number of folds	

output	meaning
`test_sizes`	size of each test fold
`train_sizes`	size of each training set
`first_test_fold`	first and last index (0-based) of the first test fold

- **Method:** $n = k$ q + r: r folds of $q + 1$, k - r folds of q, in order
- **Assumes:** $2 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



10 samples split into 3 folds: each row is one fold, its test samples dark, training samples light, the test blocks consecutive with the returned sizes [4, 3, 3] (train [6, 7, 7]; first test fold covers samples 0 … 3).

```
data_science_ml_ai999x.solve('ml_kfold_sizes', n=10, k=3)
# -> test_sizes = [4, 3, 3], train_sizes = [6, 7, 7], first_test_fold = [0, 3]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_kfold_sizes', n=100, k=5)
# -> test_sizes = [20, 20, 20, 20, 20], train_sizes = [80, 80, 80, 80, 80], first_test_fold = [0,
19]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
K-fold sizes: n = 10, k = 3	SOLVED	test_sizes = [4, 3, 3], train_sizes = [6, 7, 7], first_test_fold = [0, 3]
K-fold sizes: n = 100, k = 5	SOLVED	test_sizes = [20, 20, 20, 20, 20], train_sizes = [80, 80, 80, 80, 80], first_test_fold = [0, 19]
K-fold sizes: k = 3	NEEDS_INPUT	ml_kfold_sizes needs n (number of samples). Example: solve('ml_kfold_sizes', n=10, k=3).
K-fold sizes: n = 10, k = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
K-fold sizes: n = 3, k = 5	INVALID_INPUT	k-fold cross-validation fold sizes: k must be at least 2 and at most n.

k-fold cross-validated MSE of linear regression, `ml_kfold_cv_linear_regression`

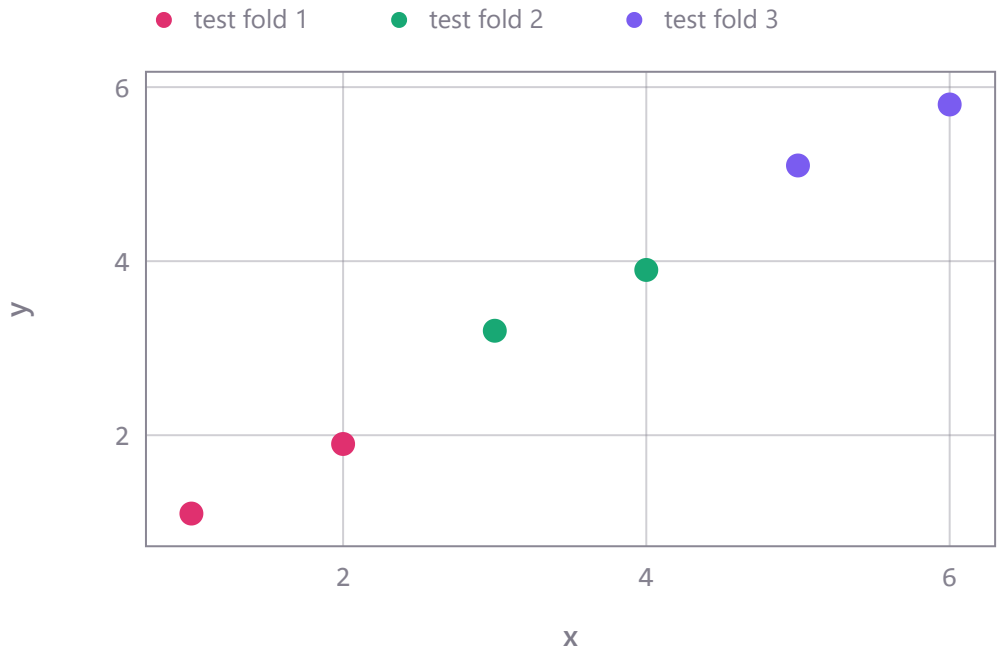
The mean squared error of ordinary least squares (with intercept) on each of k consecutive, unshuffled folds, trained on the other folds, and the mean over folds.

$$\mathrm{CV}_k = \frac{1}{k}\sum_{i=1}^{k}\mathrm{MSE}_i$$

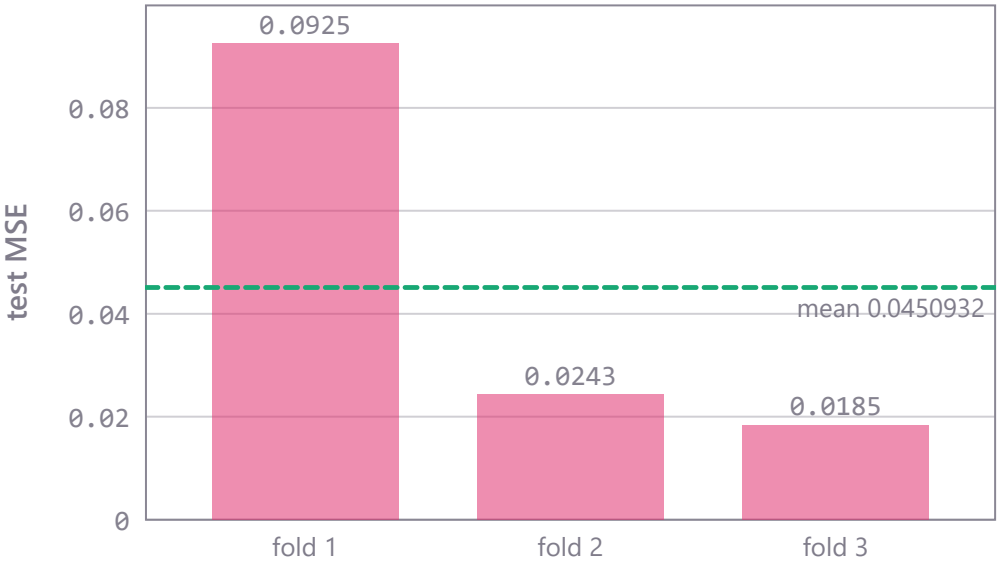
input	meaning	condition
`X`	features: one row per sample	
`y`	targets	
`k`	number of folds	

output	meaning
`fold_mse`	test MSE of each fold
`mean_mse`	mean of the fold MSEs

- **Method:** consecutive folds (first n mod k folds one larger); exact least squares by the normal equations
- **Assumes:** every training set has at least $d + 2$ samples and full rank
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated data x = [1, 2, 3, 4, 5, 6], y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8] coloured by test fold, the folds being consecutive blocks of the sizes the package's fold-size capability returns (2, 2, 2); each fold's points are predicted by a line fitted to the others.



3-fold cross-validation of a straight-line fit to x = [1, 2, 3, 4, 5, 6], y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8]: the returned test MSE of each fold (0.0925, 0.0243296, 0.01845) and their mean 0.0450932.

```
data_science_ml_ai999x.solve('ml_kfold_cv_linear_regression', X=[[1], [2], [3], [4], [5], [6]], y=
[1.1, 1.9, 3.2, 3.9, 5.1, 5.8], k=3)
# -> fold_mse = [0.0925, 0.0243295847751, 0.01845], mean_mse = 0.0450932   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_kfold_cv_linear_regression', X=[[1, 0], [2, 1], [3, 1], [4, 3], [5,
2], [6, 5], [7, 4], [8, 6]], y=[2, 4, 5, 9, 9, 14, 13, 17], k=4)
# -> fold_mse = [0.0537764003673, 0.0391984182789, 0.326697530864, 0.411248852158], mean_mse =
0.20773   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
K-fold cross-validation of linear regression: X = [[1], [2], [3], [4], [5], [6]], y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8], k = 3	SOLVED	fold_mse = [0.0925, 0.0243295847751, 0.01845], mean_mse = 0.0450932
K-fold cross-validation of linear regression: X = [[1, 0], [2, 1], [3, 1], [4, 3], [5, 2], [6, 5], [7, 4], [8, 6]], y = [2, 4, 5, 9, 9, 14, 13, 17], k = 4	SOLVED	fold_mse = [0.0537764003673, 0.0391984182789, 0.326697530864, 0.411248852158], mean_mse = 0.20773
K-fold cross-validation of linear regression: y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8], k = 3	NEEDS_INPUT	ml_kfold_cv_linear_regression needs X (features: one row per sample). Example: solve('ml_kfold_cv_linear_regression', X=
K-fold cross-validation of linear regression: X = [[1], [2], [3], [4], [5], [6]], y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8], k = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Leave-one-out cross-validation of linear regression (PRESS), `ml_loocv_linear_regression`

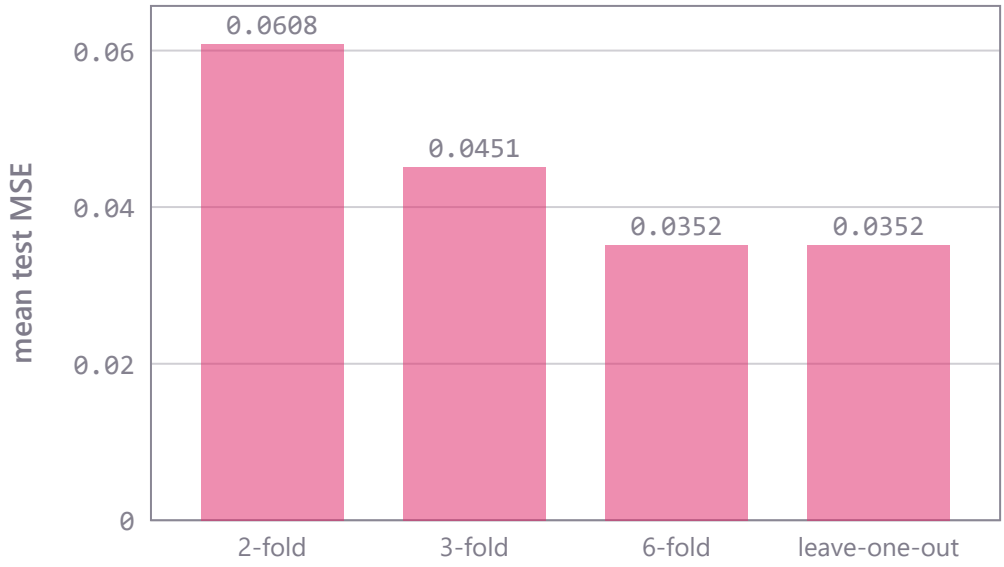
The leave-one-out cross-validated MSE and the PRESS statistic of ordinary least squares with intercept, from one fit through the leverage shortcut $e_i/(1-h_ii)$.

$$\mathrm{PRESS} = \sum_i \left(\frac{e_i}{1 - h_{ii}} \right)^2, \quad h_{ii} = x_i^\top (X^\top X)^{-1} x_i$$

input	meaning	condition
`X`	features: one row per sample	
`y`	targets	

output	meaning
`press`	sum of squared leave-one-out errors
`loocv_mse`	PRESS / n

- **Method:** hat-matrix diagonal from the exact inverse of the normal matrix; $e_i/(1-h_ii)$
- **Assumes:** at least $d+3$ samples; no leverage equal to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cross-validated error of a line fitted to x = [1, 2, 3, 4, 5, 6], y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8]: k-fold mean MSE for k = 2, 3, 6 (from the package's k-fold capability) and the returned leave-one-out MSE 0.0351518 (PRESS 0.210911); 6-fold is leave-one-out and agrees.

```
data_science_ml_ai999x.solve('ml_loocv_linear_regression', X=[[1], [2], [3], [4], [5], [6]], y=[1.1, 1.9, 3.2, 3.9, 5.1, 5.8])
# -> press = 0.210911, loocv_mse = 0.0351518   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_loocv_linear_regression', X=[[1, 0], [2, 1], [3, 1], [4, 3], [5, 2], [6, 5], [7, 4], [8, 6]], y=[2, 4, 5, 9, 9, 14, 13, 17])
# -> press = 1.47096, loocv_mse = 0.18387   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Leave-one-out cross-validation of linear regression: X = [[1], [2], [3], [4], [5], [6]], y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8]	SOLVED	press = 0.210911, loocv_mse = 0.0351518
Leave-one-out cross-validation of linear regression: X = [[1, 0], [2, 1], [3, 1], [4, 3], [5, 2], [6, 5], [7, 4], [8, 6]], y = [2, 4, 5, 9, 9, 14, 13, 17]	SOLVED	press = 1.47096, loocv_mse = 0.18387
Leave-one-out cross-validation of linear regression: y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8]	NEEDS_INPUT	ml_loocv_linear_regression needs X (features: one row per sample). Example: solve('ml_loocv_linear_regression', X=[[1],
Leave-one-out cross-validation of linear regression: X = [[1], [2], [3], [4], [5], [6]], y = [1.1, 1.9, 3.2, 3.9, 5.1, 5.8], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Bias-variance decomposition of predictions, `ml_bias_variance_decomposition`

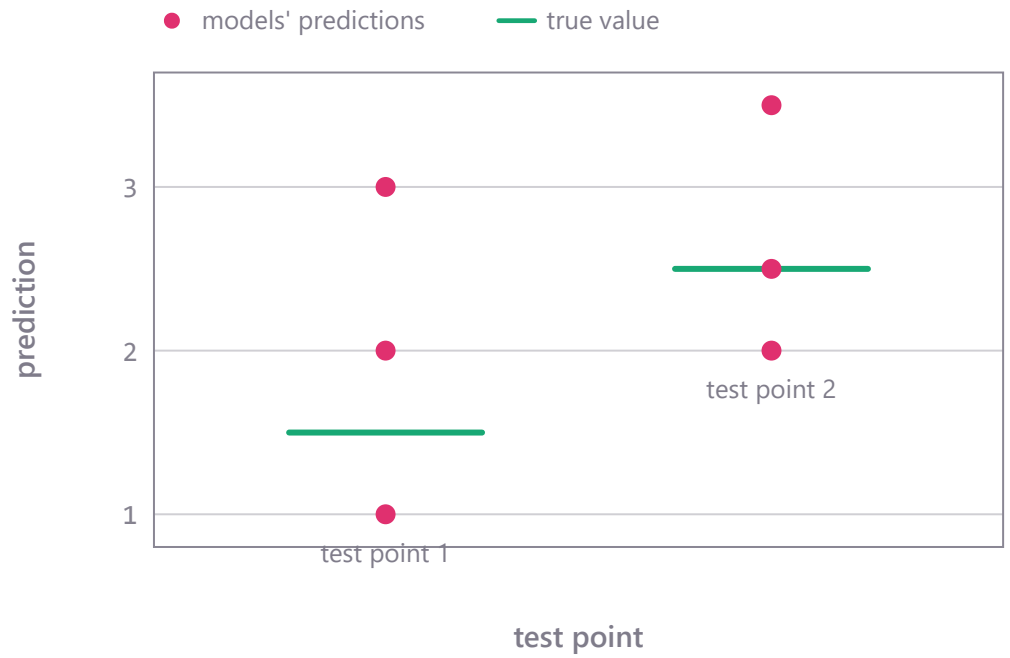
For predictions of several models (one row each, e.g. trained on different samples) at the same test points with known noise-free values: the squared bias, the variance and the expected squared error, which equals their sum.

$$E[(\hat{f} - f)^2] = (E[\hat{f}] - f)^2 + E[(\hat{f} - E[\hat{f}])^2]$$

input	meaning	condition
`P`	predictions: one row per trained model, one column per test point (at least two rows)	
`f`	true (noise-free) value at each test point	

output	meaning
`bias_squared`	mean over points of (mean prediction - f)^2
`variance`	mean over points of the variance of the predictions (divide by the number of models)
`expected_mse`	mean squared error over models and points = bias^2 + variance

- **Method:** exact rational sums
- **Assumes:** at least two models
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Predictions of 3 models (rows of P = [[1.0, 2.0], [3.0, 2.5], [2.0, 3.5]]) at 2 test points with true values f = [1.5, 2.5]: their spread is the variance, their mean's distance from the truth the bias. Returned: bias^2 0.138889, variance 0.527778, expected MSE 0.666667 = bias^2 + variance.

```
data_science_ml_ai999x.solve('ml_bias_variance_decomposition', P=[[1.0, 2.0], [3.0, 2.5], [2.0, 3.5]], f=[1.5, 2.5])
# -> bias_squared = 0.138889, variance = 0.527778, expected_mse = 0.666667   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_bias_variance_decomposition', P=[[0.9, 2.1, 2.9], [1.2, 1.8, 3.3]], f=[1, 2, 3])
# -> bias_squared = 0.005, variance = 0.0283333, expected_mse = 0.0333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bias-variance decomposition: P = [[1, 2], [3, 2.5], [2, 3.5]], f = [1.5, 2.5]	SOLVED	bias_squared = 0.138889, variance = 0.527778, expected_mse = 0.666667
Bias-variance decomposition: P = [[0.9, 2.1, 2.9], [1.2, 1.8, 3.3]], f = [1, 2, 3]	SOLVED	bias_squared = 0.005, variance = 0.0283333, expected_mse = 0.0333333
Bias-variance decomposition: f = [1.5, 2.5]	NEEDS_INPUT	ml_bias_variance_decomposition needs P (predictions: one row per trained model, one column per test point (at least two
Bias-variance decomposition: P = [[1, 2], [3, 2.5], [2, 3.5]], f = [1.5, 2.5], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Logistic regression fit (L2, stated C), `ml_logistic_regression_fit`

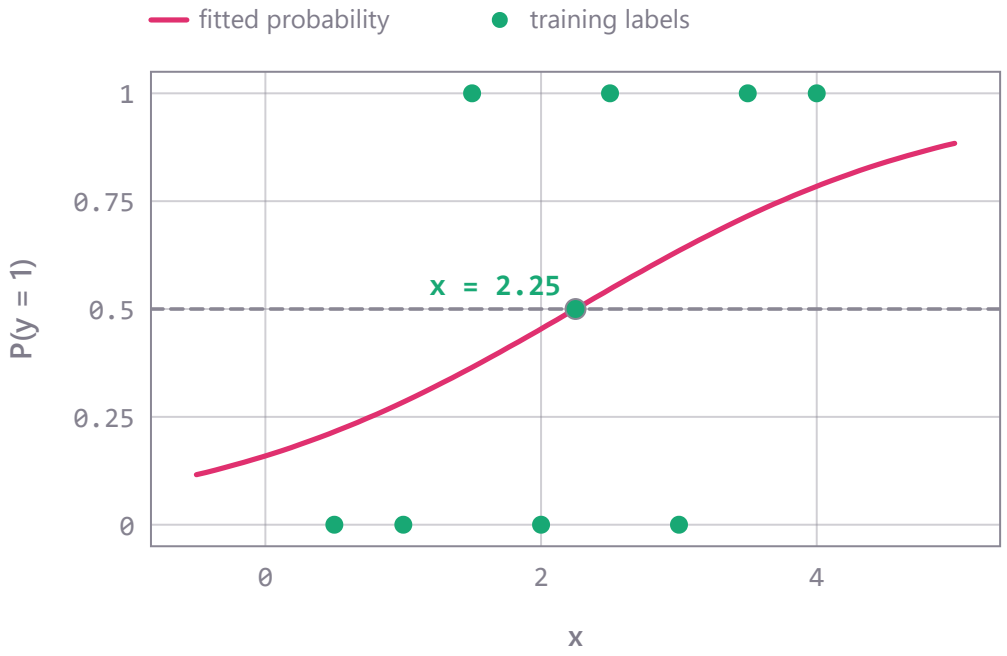
Fit binary logistic regression with an L2 penalty: minimise 0.5 ||w||^2 + C sum of log-losses (the intercept is not penalised); give the weights, the intercept, the objective and the training accuracy.

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \log\left(1 + e^{-\tilde{y}_i(w^\top x_i + b)}\right)$$

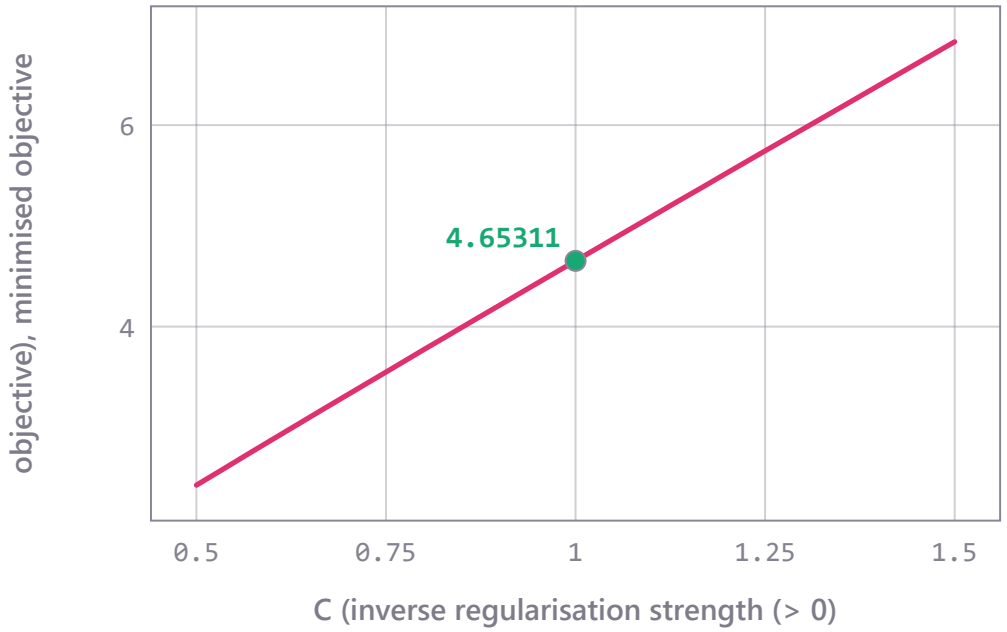
input	meaning	condition
`X`	features: one row per sample	
`y`	labels (0 and 1)	
`C`	inverse regularisation strength (> 0)	

output	meaning
`w`	fitted weights
`b`	fitted intercept
`objective`	minimised objective
`train_accuracy`	fraction of training samples classified correctly (z > 0 -> 1)

- **Method:** Newton's method on the strictly convex objective (exact gradient and Hessian) to machine precision
- **Assumes:** labels 0 and 1, both present; $C > 0$; Newton's method converges
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Logistic regression fitted by the package to x = 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4 with labels 0, 0, 1, 0, 1, 0, 1, 1 (C = 1): weight 0.738776, intercept -1.66225, training accuracy 0.75; the curve is the package's predicted probability at 81 points; marked: the decision boundary x = 2.25, P = 0.5.



objective (minimised objective) against C (inverse regularisation strength (> 0)) from 0.5 to 1.5. Other inputs as in the worked example: X = [[0.5], [1.0], [1.5], [2.0], [2.5], [3.0], [3..., y = [0, 0, 1, 0, 1, 0, 1, 1]. 31 points computed by data-science-ml-ai999x; marked: the worked example, C = 1, objective = 4.65311.

```
data_science_ml_ai999x.solve('ml_logistic_regression_fit', X=[[0.5], [1.0], [1.5], [2.0], [2.5],
[3.0], [3.5], [4.0]], y=[0, 0, 1, 0, 1, 0, 1, 1], C=1)
# -> w = [0.73877634783], b = -1.66225, objective = 4.65311, train_accuracy = 0.75   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_logistic_regression_fit', X=[[1, 2], [2, 1], [3, 3], [4, 5], [5,
4], [6, 6], [1, 4], [5, 1]], y=[0, 0, 1, 1, 1, 1, 0, 1], C=0.5)
# -> w = [0.893826419271, 0.274742800239], b = -2.98572, objective = 1.2827, train_accuracy = 1
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic regression fit: X = [[0.5], [1], [1.5], [2], [2.5], [3], [3.5], [4]], y = [0, 0, 1, 0, 1, 0, 1, 1], C = 1	SOLVED	w = [0.73877634783], b = -1.66225, objective = 4.65311, train_accuracy = 0.75
Logistic regression fit: X = [[1, 2], [2, 1], [3, 3], [4, 5], [5, 4], [6, 6], [1, 4], [5, 1]], y = [0, 0, 1, 1, 1, 1, 0, 1], C = 0.5	SOLVED	w = [0.893826419271, 0.274742800239], b = -2.98572, objective = 1.2827, train_accuracy = 1
Logistic regression fit: y = [0, 0, 1, 0, 1, 0, 1, 1], C = 1	NEEDS_INPUT	ml_logistic_regression_fit needs X (features: one row per sample). Example: solve('ml_logistic_regression_fit', X= [[0.5]
Logistic regression fit: X = [[0.5], [1], [1.5], [2], [2.5], [3], [3.5], [4]], y = [0, 0, 1, 0, 1, 0, 1, 1], C = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Logistic regression fit: X = [[1], [2]], y = [1, 1], C = 1	INVALID_INPUT	Logistic regression fit (L2, stated C): labels must be 0 and 1, both present.

Logistic regression prediction from given weights, `ml_logistic_regression_predict`

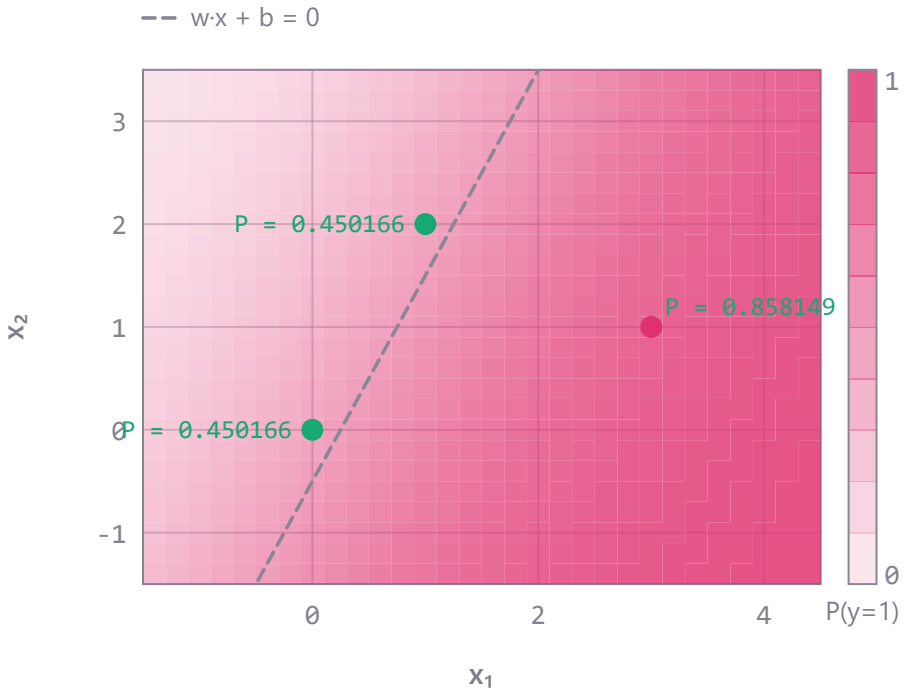
The logits, predicted probabilities of class 1 and predicted classes of a logistic regression model with given weights and intercept.

$$P(y = 1 \mid x) = \sigma(w^\top x + b)$$

input	meaning	condition
`w`	weights	
`b`	intercept	
`X`	samples to predict: one row each	

output	meaning
`probabilities`	sigma(w.x + b) for each sample
`classes`	1 when $w.x + b > 0$, else 0
`logits`	w.x + b

- **Method:** stable sigmoid of the linear score
- **Assumes:** w has one weight per feature
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Predicted probability of class 1 for w = [0.8, -0.4], b = -0.2 over the plane, 750 cells from one run of the package; dashed: the boundary w·x + b = 0 (from the stated w, b). Marked: the worked example's points [[1, 2], [3, 1], [0, 0]] with their returned probabilities and classes [0, 1, 0].

```
data_science_ml_ai999x.solve('ml_logistic_regression_predict', w=[0.8, -0.4], b=-0.2, X=[[1, 2], [3, 1], [0, 0]])
# -> probabilities = [0.450166002688, 0.8581489351, 0.450166002688], classes = [0, 1, 0], logits = [-0.2, 1.8, -0.2]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_logistic_regression_predict', w=[2], b=-3, X=[[1], [1.5], [2]])
# -> probabilities = [0.26894142137, 0.5, 0.73105857863], classes = [0, 0, 1], logits = [-1.0, 0.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logistic regression prediction: w = [0.8, -0.4], b = -0.2, X = [[1, 2], [3, 1], [0, 0]]	SOLVED	probabilities = [0.450166002688, 0.8581489351, 0.450166002688], classes = [0, 1, 0], logits = [-0.2, 1.8, -0.2]
Logistic regression prediction: w = [2], b = -3, X = [[1], [1.5], [2]]	SOLVED	probabilities = [0.26894142137, 0.5, 0.73105857863], classes = [0, 0, 1], logits = [-1.0, 0.0, 1.0]
Logistic regression prediction: b = -0.2, X = [[1, 2], [3, 1], [0, 0]]	NEEDS_INPUT	ml_logistic_regression_predict needs w (weights). Example: solve('ml_logistic_regression_predict', w=[0.8, -0.4], b=-0.2
Logistic regression prediction: w = [0.8, -0.4], b = -0.2, X = [[1, 2], [3, 1], [0, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Ridge regression coefficients (stated alpha), `ml_ridge_regression`

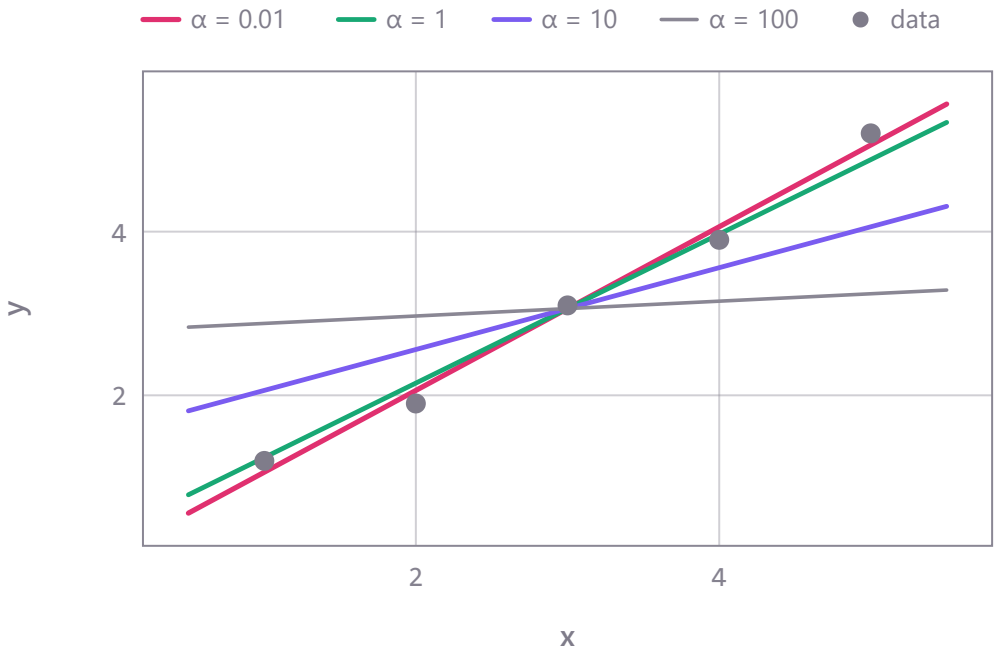
Fit ridge regression: minimise ||y - X w - b||^2 + α ||w||^2 (the intercept is not penalised); give the coefficients, the intercept and the training MSE.

$$\hat{w} = (X_c^\top X_c + \alpha I)^{-1} X_c^\top y_c$$

input	meaning	condition
`X`	features: one row per sample	
`y`	targets	
`alpha`	L2 penalty strength (>= 0; 0 is least squares)	

output	meaning
`w`	coefficients
`b`	intercept
`train_mse`	training mean squared error

- **Method:** centre X and y, solve (Xc^T Xc + alpha I) $w = Xc^T$ yc exactly, b = mean(y) - mean(X) w
- **Assumes:** $\alpha \geq 0$; a non-singular system (alpha > 0 always is)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ridge lines through x = [1, 2, 3, 4, 5], y = [1.2, 1.9, 3.1, 3.9, 5.2] for penalties α = 0.01, 1, 10, 100, one run each (α = 0.01: w = 0.999001, b = 0.062997; α = 1: w = 0.909091, b = 0.332727; α = 10: w = 0.5, b = 1.56; α = 100: w = 0.0909091, b = 2.78727): the slope shrinks toward zero as α grows.



train_mse (training mean squared error) against alpha (L2 penalty strength (>= 0; 0 is least squares)) from 0.1 to 10 on a logarithmic axis. Other inputs as in the worked example: X = [[1], [2], [3], [4], [5]], y = [1.2, 1.9, 3.1, 3.9, 5.2]. 41 points computed by data-science-ml-ai999x; marked: the worked example, alpha = 1, train_mse = 0.0349289.

```
data_science_ml_ai999x.solve('ml_ridge_regression', X=[[1], [2], [3], [4], [5]], y=[1.2, 1.9, 3.1,
3.9, 5.2], alpha=1)
# -> w = [0.909090909091], b = 0.332727, train_mse = 0.0349289   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_ridge_regression', X=[[1, 0], [2, 1], [3, 1], [4, 3], [5, 2], [6,
5]], y=[2, 4, 5, 9, 9, 14], alpha=0.5)
# -> w = [1.13541666667, 1.27083333333], b = 0.651042, train_mse = 0.0401024   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ridge regression: X = [[1], [2], [3], [4], [5]], y = [1.2, 1.9, 3.1, 3.9, 5.2], alpha = 1	SOLVED	w = [0.909090909091], b = 0.332727, train_mse = 0.0349289
Ridge regression: X = [[1, 0], [2, 1], [3, 1], [4, 3], [5, 2], [6, 5]], y = [2, 4, 5, 9, 9, 14], alpha = 0.5	SOLVED	w = [1.13541666667, 1.27083333333], b = 0.651042, train_mse = 0.0401024
Ridge regression: y = [1.2, 1.9, 3.1, 3.9, 5.2], alpha = 1	NEEDS_INPUT	ml_ridge_regression needs X (features: one row per sample). Example: solve('ml_ridge_regression', X=[[1], [2], [3], [4],
Ridge regression: X = [[1], [2], [3], [4], [5]], y = [1.2, 1.9, 3.1, 3.9, 5.2], alpha = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Ridge regression: X = [[1], [2]], y = [1, 2], alpha = -1	INVALID_INPUT	Ridge regression coefficients (stated alpha): alpha must be >= 0.

Lasso regression coefficients (stated alpha), `ml_lasso_regression`

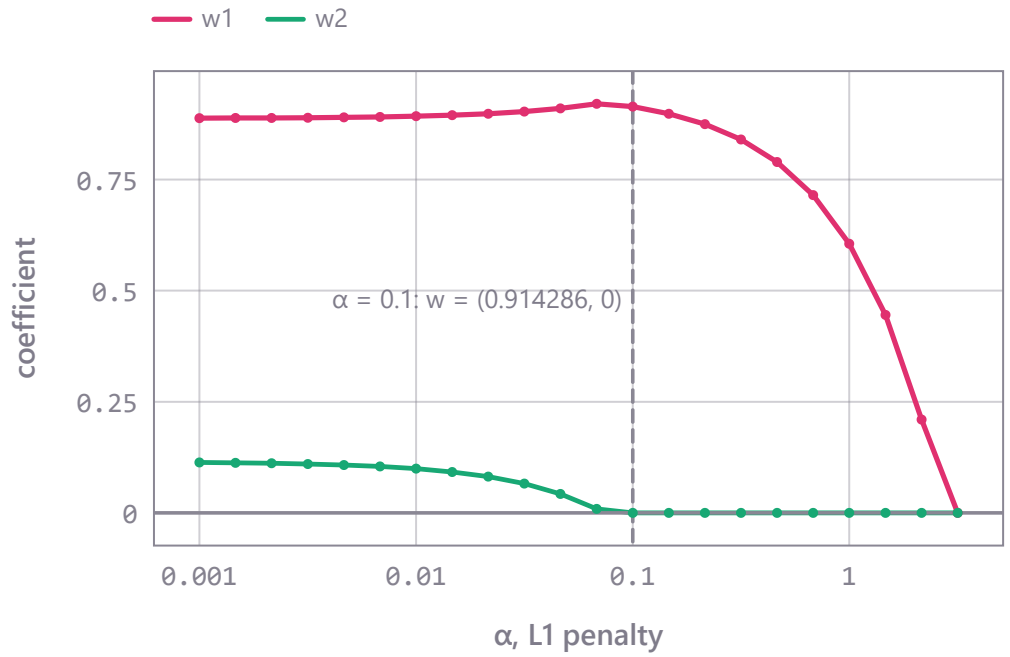
Fit lasso regression: minimise ||y - X w - b||^$2/(2n) + \alpha$ ||w||_1 (the intercept is not penalised); give the coefficients (some exactly zero), the intercept and the number of non-zero coefficients.

$$\min_{w,b} \frac{1}{2n}\|y - Xw - b\|^2 + \alpha\|w\|_1$$

input	meaning	condition
`X`	features: one row per sample	
`y`	targets	
`alpha`	L1 penalty strength (> 0)	

output	meaning
`w`	coefficients
`b`	intercept
`n_nonzero`	number of non-zero coefficients

- **Method:** coordinate descent to find the active set and signs, then the exact solution on that set; the optimality conditions are checked
- **Assumes:** $\alpha > 0$; non-constant features; a non-degenerate solution
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lasso coefficients for the stated X and y as the penalty α runs from 0.001 to about 4.6 (22 runs): the second feature's weight reaches exactly zero first, then the first's. Marked: the worked example α = 0.1, returned w = (0.914286, 0), b = 0.366667, 1 nonzero.

```
data_science_ml_ai999x.solve('ml_lasso_regression', X=[[1, 0.5], [2, 1.5], [3, 1.0], [4, 3.0], [5, 2.0], [6, 3.5]], y=[1.1, 2.3, 2.9, 4.2, 5.1, 5.8], alpha=0.1)
# -> w = [0.914285714286, 0.0], b = 0.366667, n_nonzero = 1   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_lasso_regression', X=[[1, 2, 0], [2, 1, 1], [3, 3, 0], [4, 1, 1], [5, 4, 0], [6, 2, 1], [7, 5, 1]], y=[3, 4, 6, 6, 9, 8, 12], alpha=0.5)
# -> w = [1.01785714286, 0.5, 0.0], b = 1.5, n_nonzero = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lasso regression: X = [[1, 0.5], [2, 1.5], [3, 1], [4, 3], [5, 2], [6, 3.5]], y = [1.1, 2.3, 2.9, 4.2, 5.1, 5.8], alpha = 0.1	SOLVED	w = [0.914285714286, 0.0], b = 0.366667, n_nonzero = 1
Lasso regression: X = [[1, 2, 0], [2, 1, 1], [3, 3, 0], [4, 1, 1], [5, 4, 0], [6, 2, 1], [7, 5, 1]], y = [3, 4, 6, 6, 9, 8, 12], alpha = 0.5	SOLVED	w = [1.01785714286, 0.5, 0.0], b = 1.5, n_nonzero = 2
Lasso regression: y = [1.1, 2.3, 2.9, 4.2, 5.1, 5.8], alpha = 0.1	NEEDS_INPUT	ml_lasso_regression needs X (features: one row per sample). Example: solve('ml_lasso_regression', X=[[1, 0.5], [2, 1.5],
Lasso regression: X = [[1, 0.5], [2, 1.5], [3, 1], [4, 3], [5, 2], [6, 3.5]], y = [1.1, 2.3, 2.9, 4.2, 5.1, 5.8], alpha = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Linear SVM decision values, classes and margin, `ml_linear_svm_decision`

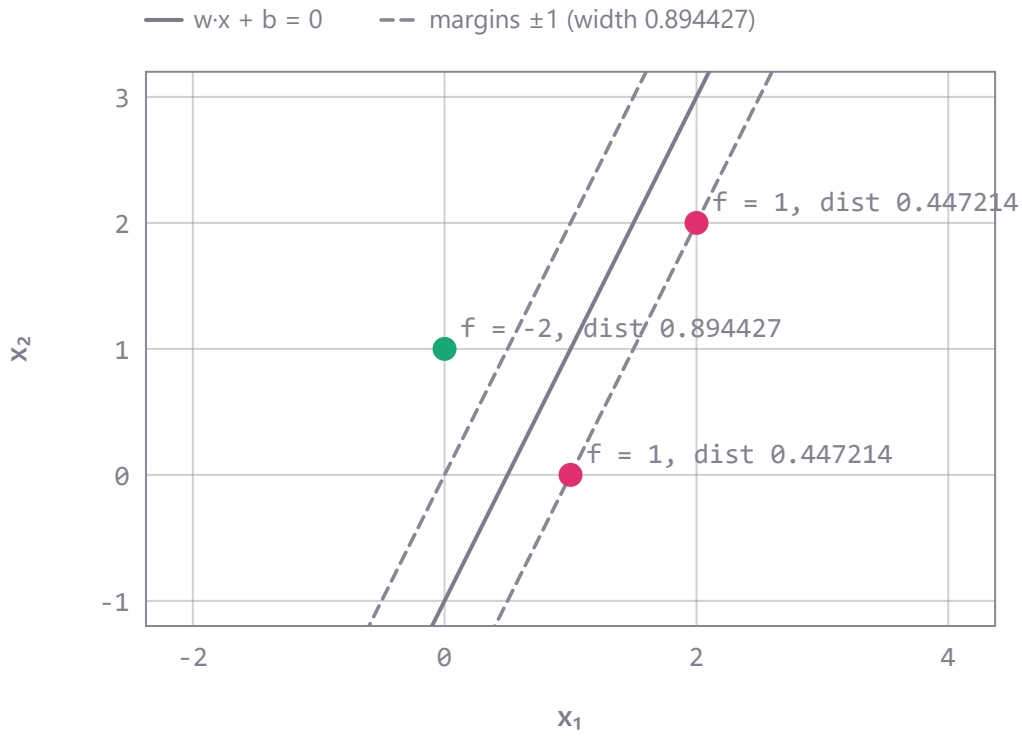
For a linear SVM with given weights and bias: the decision value w.x + b of each sample, its class (+1 when the value is positive, else -1), the margin width 2 / ||w|| and each sample's distance to the hyperplane.

$$f(x) = w^\top x + b, \quad \text{margin} = \frac{2}{\|w\|}$$

input	meaning	condition
`w`	weights	
`b`	bias	
`X`	samples: one row each	

output	meaning
`decision`	w.x + b
`classes`	+1 if $w.x + b > 0$, else -1
`margin_width`	2 / ||w||
`distances`	|w.x + b| / ||w||

- **Method:** exact decision values; one square root for ||w||
- **Assumes:** w is not zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The linear SVM w = [2, -1], b = -1.0: decision line and the margins w·x + b = ±1 (drawn from the stated w, b; returned margin width 0.894427). The stated points [[1, 0], [0, 1], [2, 2]] are coloured by the returned class [1, -1, 1] and labelled with the returned decision value and distance to the line.

```
data_science_ml_ai999x.solve('ml_linear_svm_decision', w=[2, -1], b=-1, X=[[1, 0], [0, 1], [2, 2]])
# -> decision = [1.0, -2.0, 1.0], classes = [1, -1, 1], margin_width = 0.894427, distances =
[0.4472135955, 0.894427191, 0.4472135955]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_linear_svm_decision', w=[0.5, 0.5, 1], b=0.25, X=[[1, 1, -1], [0,
0, 0]])
# -> decision = [0.25, 0.25], classes = [1, 1], margin_width = 1.63299, distances = [0.204124145232,
0.204124145232]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linear SVM decision: w = [2, -1], b = -1, X = [[1, 0], [0, 1], [2, 2]]	SOLVED	decision = [1.0, -2.0, 1.0], classes = [1, -1, 1], margin_width = 0.894427, distances = [0.4472135955, 0.894427191, 0.4472135955]
Linear SVM decision: w = [0.5, 0.5, 1], b = 0.25, X = [[1, 1, -1], [0, 0, 0]]	SOLVED	decision = [0.25, 0.25], classes = [1, 1], margin_width = 1.63299, distances = [0.204124145232, 0.204124145232]
Linear SVM decision: b = -1, X = [[1, 0], [0, 1], [2, 2]]	NEEDS_INPUT	ml_linear_svm_decision needs w (weights). Example: solve('ml_linear_svm_decision', w=[2, -1], b=-1, X=[[1, 0], [0, 1], [
Linear SVM decision: w = [2, -1], b = -1, X = [[1, 0], [0, 1], [2, 2]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Perceptron training epoch, `ml_perceptron_epoch`

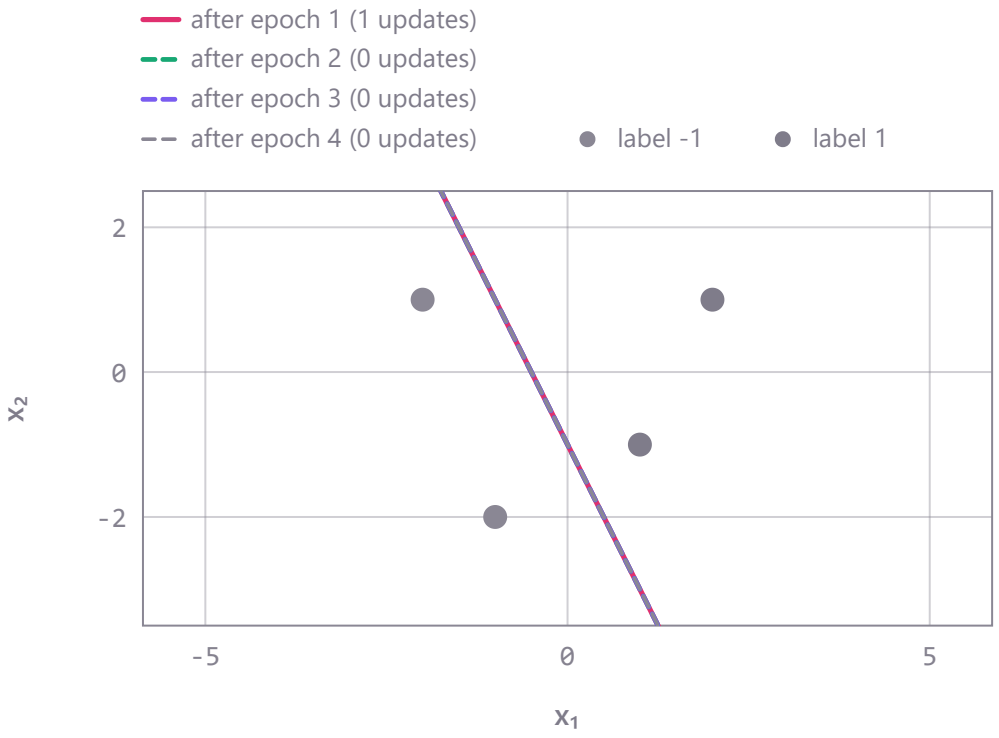
One pass of the perceptron learning rule through the samples in order (labels -1/+1): whenever y $(w . x + b) \leq 0$, update w += lr y x and b += lr y; the final weights, bias and number of updates.

$$\text{if } y_i(w^\top x_i + b) \leq 0: \; w \leftarrow w + \eta y_i x_i, \; b \leftarrow b + \eta y_i$$

input	meaning	condition
`X`	samples in presentation order	
`y`	labels (-1 or +1)	
`w`	initial weights	
`b`	initial bias	
`lr`	learning rate (> 0)	

output	meaning
`w_new`	weights after the epoch
`b_new`	bias after the epoch
`n_updates`	number of updates (mistakes, including y f = 0)

- **Method:** exact rational arithmetic, samples in the given order
- **Assumes:** labels -1 and +1; $lr > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Perceptron epochs on the 4 stated points (labels [1, -1, 1, -1], start w = [0, 0], b = 0, lr = 1): each epoch is one run of the package from the w, b the previous run returned; the line w·x + b = 0 after each: epoch 1: w = (2, 1), b = 1; epoch 2: w = (2, 1), b = 1; epoch 3: w = (2, 1), b = 1; epoch 4: w = (2, 1), b = 1.

```
data_science_ml_ai999x.solve('ml_perceptron_epoch', X=[[2, 1], [-1, -2], [1, -1], [-2, 1]], y=[1,
-1, 1, -1], w=[0, 0], b=0, lr=1)
# -> w_new = [2.0, 1.0], b_new = 1, n_updates = 1   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_perceptron_epoch', X=[[1], [3], [-1], [-2]], y=[1, 1, -1, -1], w=
[-0.5], b=0.5, lr=0.5)
# -> w_new = [0.5], b_new = 0.5, n_updates = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perceptron epoch: X = [[2, 1], [-1, -2], [1, -1], [-2, 1]], y = [1, -1, 1, -1], w = [0, 0], b = 0, lr = 1	SOLVED	w_new = [2.0, 1.0], b_new = 1, n_updates = 1
Perceptron epoch: X = [[1], [3], [-1], [-2]], y = [1, 1, -1, -1], w = [-0.5], b = 0.5, lr = 0.5	SOLVED	w_new = [0.5], b_new = 0.5, n_updates = 2
Perceptron epoch: y = [1, -1, 1, -1], w = [0, 0], b = 0, lr = 1	NEEDS_INPUT	ml_perceptron_epoch needs X (samples in presentation order). Example: solve('ml_perceptron_epoch', X=[[2, 1], [-1, -2],
Perceptron epoch: X = [[2, 1], [-1, -2], [1, -1], [-2, 1]], y = [1, -1, 1, -1], w = [0, 0], b = 0, lr = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Forward pass of a one-hidden-layer dense network, `ml_dense_forward_pass`

The forward pass of a dense network x -> activation(W1 x + b1) -> W2 $h + b_2$ with an identity, sigmoid or softmax output: the hidden activations and the outputs.

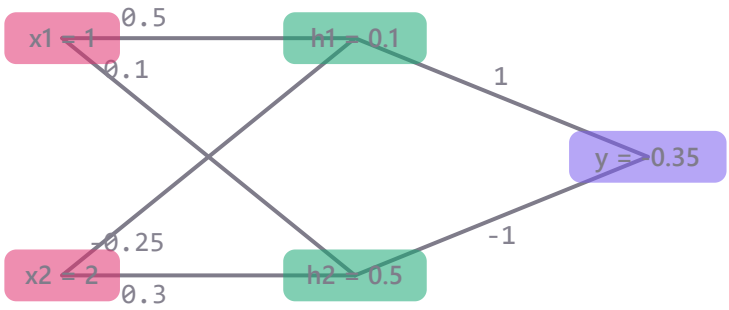
$$h = \phi(W_1 x + b_1), \quad \hat{y} = g(W_2 h + b_2)$$

input	meaning	condition
`x`	input vector	
`W1`	hidden-layer weights (rows = hidden units, columns = inputs)	
`b1`	hidden-layer biases	
`activation`	hidden activation: relu, tanh or sigmoid	
`W2`	output-layer weights (rows = outputs, columns = hidden units)	
`b2`	output biases	
`output`	output activation: identity, sigmoid or softmax	

output	meaning
`hidden`	hidden activations h
`output`	network outputs
`max_output`	largest output

- **Method:** matrix-vector products with fsum, then the activations
- **Assumes:** shapes agree (W1: hidden x inputs, W2: outputs x hidden)

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2–2–1 network of the worked example (x = [1, 2], W1 = [[0.5, -0.25], [0.1, 0.3]], b1 = [0.1, -0.2], relu hidden units, W2 = [[1, -1]], b2 = [0.05], identity output): every edge carries its stated weight; the boxes hold the returned hidden activations [0.1, 0.5] and output -0.35.

```
data_science_ml_ai999x.solve('ml_dense_forward_pass', x=[1, 2], W1=[[0.5, -0.25], [0.1, 0.3]], b1=
[0.1, -0.2], activation='relu', W2=[[1, -1]], b2=[0.05], output='identity')
# -> hidden = [0.1, 0.5], output = [-0.35], max_output = -0.35   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_dense_forward_pass', x=[0.5, -1, 2], W1=[[0.2, 0.4, -0.1], [-0.3,
0.1, 0.5]], b1=[0, 0.1], activation='tanh', W2=[[0.7, -0.2], [0.1, 0.9], [-0.5, 0.3]], b2=[0, 0.1,
-0.1], output='softmax')
# -> hidden = [-0.46211715726, 0.691069469833], output = [0.157616744625, 0.491574315988,
0.350808939386], max_output = 0.491574   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dense network forward pass: x = [1, 2], W1 = [[0.5, -0.25], [0.1, 0.3]], b1 = [0.1, -0.2], activation = relu, W2 = [[1, -1]], b2 = [0.05], output = identity	SOLVED	hidden = [0.1, 0.5], output = [-0.35], max_output = -0.35
Dense network forward pass: x = [0.5, -1, 2], W1 = [[0.2, 0.4, -0.1], [-0.3, 0.1, 0.5]], b1 = [0, 0.1], activation = tanh, W2 = [[0.7, -0.2], [0.1, 0.9], [-0.5, 0.3]], b2 = [0, 0.1, -0.1], output = softmax	SOLVED	hidden = [-0.46211715726, 0.691069469833], output = [0.157616744625, 0.491574315988, 0.350808939386], max_output = 0.491574
Dense network forward pass: W1 = [[0.5, -0.25], [0.1, 0.3]], b1 = [0.1, -0.2], activation = relu, W2 = [[1, -1]], b2 = [0.05], output = identity	NEEDS_INPUT	ml_dense_forward_pass needs x (input vector). Example: solve('ml_dense_forward_pass', x=[1, 2], W1=[[0.5, -0.25], [0.1,
Dense network forward pass: x = [1, 2], W1 = [[0.5, -0.25], [0.1, 0.3]], b1 = [0.1,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
-0.2], activation = relu, W2 = [[1, -1]], b2 = [0.05], output = identity, zz = 1		data_science_ml_ai999x.templates(
Dense network forward pass: x = [1, 2], W1 = [[0.5, -0.25, 1]], b1 = [0.1], activation = relu, W2 = [[1]], b2 = [0], output = identity	INVALID_INPUT	Forward pass of a one-hidden-layer dense network: W1 needs one column per input (shape hidden x inputs).

Backpropagation gradients of a one-hidden-layer network, `ml_backpropagation`

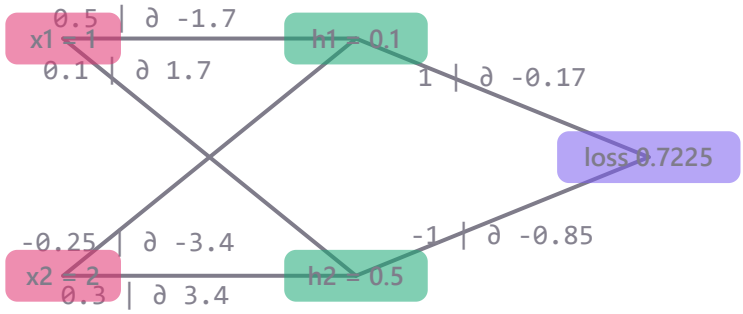
One backpropagation pass of a dense network with one hidden layer: the loss (mean squared error with identity outputs, or cross-entropy of softmax outputs against a target distribution) and its gradients with respect to every weight and bias.

$$\delta_2 = \frac{\partial L}{\partial z_2},\ \nabla_{W_2} = \delta_2 h^\top,\ \delta_1 = (W_2^\top \delta_2) \odot \phi'(z_1),\ \nabla_{W_1} = \delta_1 x^\top$$

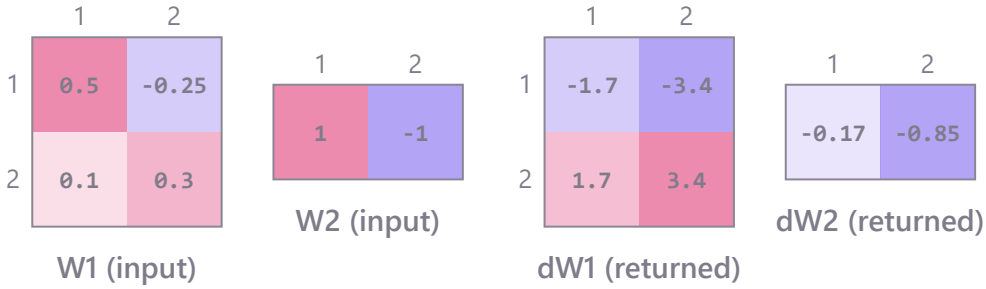
input	meaning	condition
`x`	input vector	
`t`	target: values for mse, a probability distribution for cross-entropy	
`W1`	hidden-layer weights (rows = hidden units, columns = inputs)	
`b1`	hidden-layer biases	
`activation`	hidden activation: relu, tanh or sigmoid	
`W2`	output-layer weights (rows = outputs, columns = hidden units)	
`b2`	output biases	
`loss`	loss: mse (identity outputs, mean over outputs) or cross_entropy (softmax outputs)	

output	meaning
`loss`	loss value
`dW1`	$dL/dW1$
`db1`	$dL/db1$
`dW2`	$dL/dW2$
`db2`	$dL/db2$

- **Method:** chain rule backwards through the output layer, the activation and the hidden layer
- **Assumes:** shapes agree; no ReLU pre-activation exactly 0; cross-entropy: target sums to 1, at least two outputs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2–2–1 network of the worked example (x = [1, 2], target [0.5], relu hidden units, mse loss). Each edge: its stated weight | the returned gradient of the loss with respect to it (dW1 = [[-1.7, -3.4], [1.7, 3.4]], dW2 = [[-0.17, -0.85]]); hidden activations [0.1, 0.5] from the package's forward pass; returned loss 0.7225, bias gradients db1 = [-1.7, 1.7], db2 = [-1.7].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): W1 (input); W2 (input); dW1 (returned); dW2 (returned). Returned with them by data-science-ml-ai999x: loss = 0.7225. Other inputs: x = [1, 2], t = [0.5], b1 = [0.1, -0.2], activation = relu, b2 = [0.05], loss = mse.

```
data_science_ml_ai999x.solve('ml_backpropagation', x=[1, 2], t=[0.5], W1=[[0.5, -0.25], [0.1, 0.3]], b1=[0.1, -0.2], activation='relu', W2=[[1, -1]], b2=[0.05], loss='mse')
# -> loss = 0.7225, dW1 = [[-1.7, -3.4], [1.7, 3.4]], db1 = [-1.7, 1.7], dW2 = [[-0.17, -0.85]], db2 = [-1.7]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_backpropagation', x=[0.5, -1], t=[0, 1, 0], W1=[[0.2, 0.4], [-0.3, 0.1]], b1=[0, 0.1], activation='sigmoid', W2=[[0.7, -0.2], [0.1, 0.9], [-0.5, 0.3]], b2=[0, 0.1, -0.1], loss='cross_entropy')
# -> loss = 0.780588, dW1 = [[0.00744901987142, -0.0148980397428], [-0.0604066962162, 0.120813392432]], db1 = [0.0148980397428, -0.120813392432], dW2 = [[0.136909362353, 0.148816992814], [-0.230594092978, -0.250649910918], [0.093684730625, 0.101832918104]], db2 = [0.321717670965, -0.541863560359, 0.220145889395]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Backpropagation gradients: x = [1, 2], t = [0.5], W1 = [[0.5, -0.25], [0.1, 0.3]], b1 = [0.1, -0.2], activation = relu, W2 = [[1, -1]], b2 = [0.05], loss = mse	SOLVED	loss = 0.7225, dW1 = [[-1.7, -3.4], [1.7, 3.4]], db1 = [-1.7, 1.7], dW2 = [[-0.17, -0.85]], db2 = [-1.7]
Backpropagation gradients: x = [0.5, -1], t = [0, 1, 0], W1 = [[0.2, 0.4], [-0.3, 0.1]], b1 = [0, 0.1], activation = sigmoid, W2 = [[0.7, -0.2], [0.1, 0.9], [-0.5, 0.3]], b2 = [0, 0.1, -0.1], loss = cross_entropy	SOLVED	loss = 0.780588, dW1 = [[0.00744901987142, -0.0148980397428], [-0.0604066962162, 0.120813392432]], db1 = [0.0148980397428, -0.120813392432], dW2 = [[0.136909362353, 0.148816992814], [-0.230594092978, -0.250649910918], [0.093684730625, 0.101832918104]], db2 = [0.321717670965, -0.541863560359, 0.220145889395]
Backpropagation gradients: t = [0.5], W1 = [[0.5, -0.25], [0.1, 0.3]], b1 = [0.1, -0.2], activation = relu, W2 = [[1, -1]], b2 = [0.05], loss = mse	NEEDS_INPUT	ml_backpropagation needs x (input vector). Example: solve('ml_backpropagation', x=[1, 2], t=[0.5], W1=[[0.5, -0.25], [0.
Backpropagation gradients: x = [1, 2], t = [0.5], W1 = [[0.5, -0.25], [0.1, 0.3]], b1 = [0.1, -0.2], activation = relu, W2 = [[1, -1]], b2 = [0.05], loss = mse, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Parameter count of a dense (fully connected) network, `ml_dense_parameter_count`

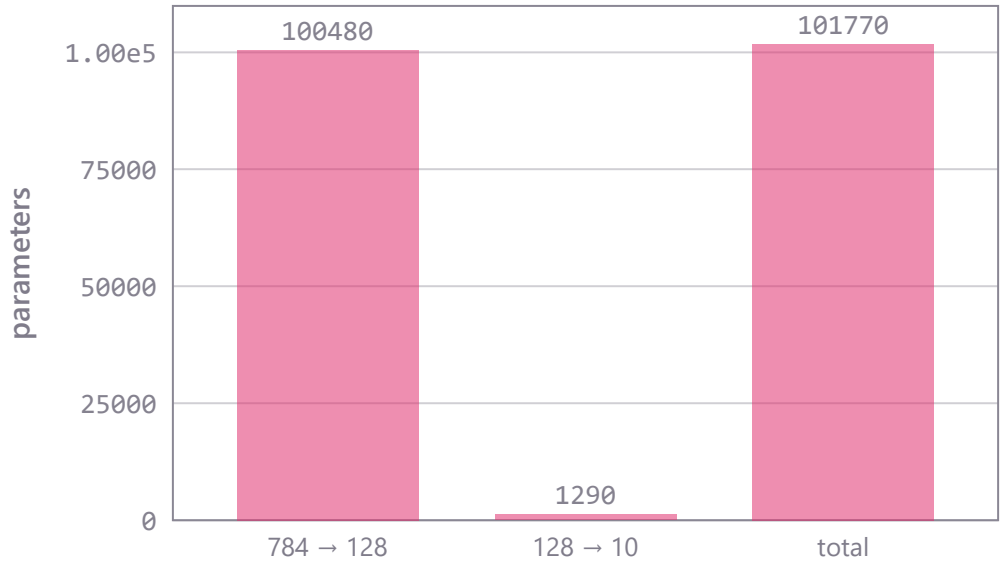
The number of trainable parameters of each dense layer and of the whole network, from the layer sizes (input size first), with or without biases.

$$P = \sum_{l} (n_{l-1} n_l + n_l)$$

input	meaning	condition
`sizes`	layer sizes from input to output, e.g. [784, 128, 10]	
`bias`	whether every layer has biases (yes or no)	

output	meaning
`per_layer`	parameters of each layer: n_in n_out (+ n_out biases)
`total`	total parameters

- **Method:** n_in * n_out + n_out per layer
- **Assumes:** positive whole-number sizes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Trainable parameters of the dense network with layer sizes [784, 128, 10] (biases: True): returned per layer 100480, 1290, total 101770; the first layer holds almost all.

```
data_science_ml_ai999x.solve('ml_dense_parameter_count', sizes=[784, 128, 10], bias=True)
# -> per_layer = [100480, 1290], total = 101770   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_dense_parameter_count', sizes=[3, 4, 4, 1], bias=False)
# -> per_layer = [12, 16, 4], total = 32   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dense network parameter count: sizes = [784, 128, 10], bias = yes	SOLVED	per_layer = [100480, 1290], total = 101770
Dense network parameter count: sizes = [3, 4, 4, 1], bias = no	SOLVED	per_layer = [12, 16, 4], total = 32
Dense network parameter count: bias = yes	NEEDS_INPUT	ml_dense_parameter_count needs sizes (layer sizes from input to output, e.g. [784, 128, 10]). Example: solve('ml_dense_p
Dense network parameter count: sizes = [784, 128, 10], bias = yes, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Convolution output size, `ml_conv_output_size`

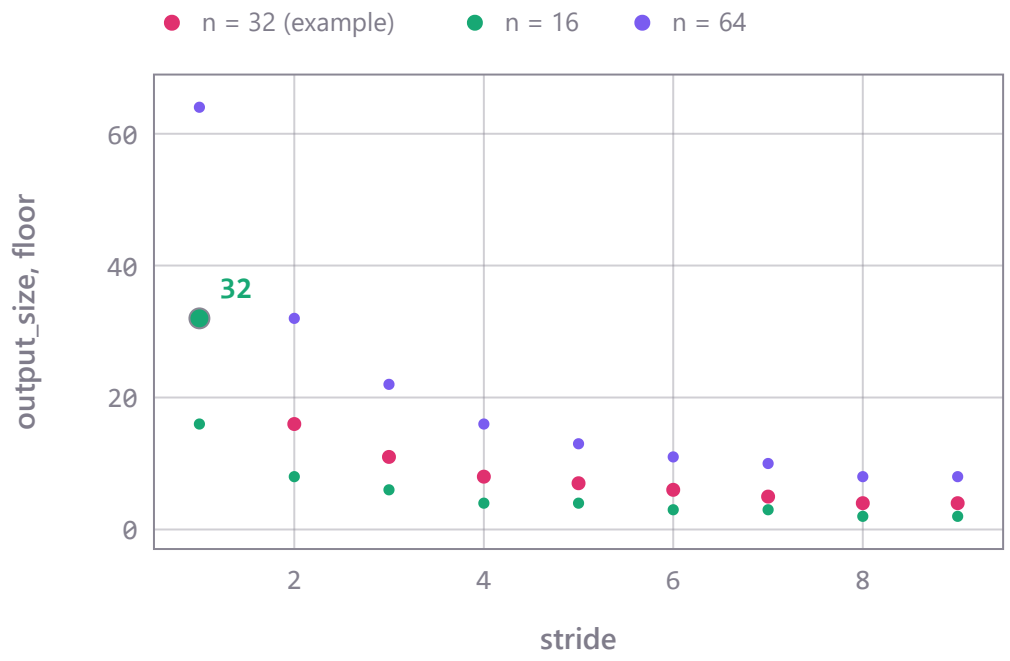
The output length (per spatial dimension) of a convolution or pooling window from the input size, kernel size, stride, zero padding and dilation.

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1$$

input	meaning	condition
`n`	input size	
`kernel`	kernel size	
`stride`	stride	
`padding`	zero padding on each side	
`dilation`	dilation (1 = ordinary)	

output	meaning
`output_size`	floor((n + 2p - d(k - 1) - 1) / s) + 1
`effective_kernel`	d(k - 1) + 1

- **Method:** integer floor division
- **Assumes:** the dilated kernel fits in the padded input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



output_size (floor((n + 2p - d(k - 1) - 1) / s) + 1) against stride (stride) from 1 to 9. One curve for each value of n (input size): 32, 16, 64. Other inputs as in the worked example: kernel = 3, padding = 1, dilation = 1. 9 points per curve computed by data-science-ml-ai999x; marked: the worked example, stride = 1, output_size = 32.

```
data_science_ml_ai999x.solve('ml_conv_output_size', n=32, kernel=3, stride=1, padding=1, dilation=1)
# -> output_size = 32, effective_kernel = 3   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_conv_output_size', n=224, kernel=7, stride=2, padding=3,
dilation=1)
# -> output_size = 112, effective_kernel = 7   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_conv_output_size', n=64, kernel=3, stride=1, padding=0, dilation=2)
# -> output_size = 60, effective_kernel = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convolution output size: n = 32, kernel = 3, stride = 1, padding = 1, dilation = 1	SOLVED	output_size = 32, effective_kernel = 3
Convolution output size: n = 224, kernel = 7, stride = 2, padding = 3, dilation = 1	SOLVED	output_size = 112, effective_kernel = 7
Convolution output size: n = 64, kernel = 3, stride = 1, padding = 0, dilation = 2	SOLVED	output_size = 60, effective_kernel = 5
Convolution output size: kernel = 3, stride = 1, padding = 1, dilation = 1	NEEDS_INPUT	ml_conv_output_size needs n (input size). Example: solve('ml_conv_output_size', n=32, kernel=3, stride=1, padding=1, dil
Convolution output size: n = 32, kernel = 3, stride = 1, padding = 1, dilation = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Convolution output size: n = 2, kernel = 5, stride = 1, padding = 0, dilation = 1	INVALID_INPUT	Convolution output size: the (dilated) kernel is larger than the padded input.

Parameter count of a 2-D convolution layer, `ml_conv_parameter_count`

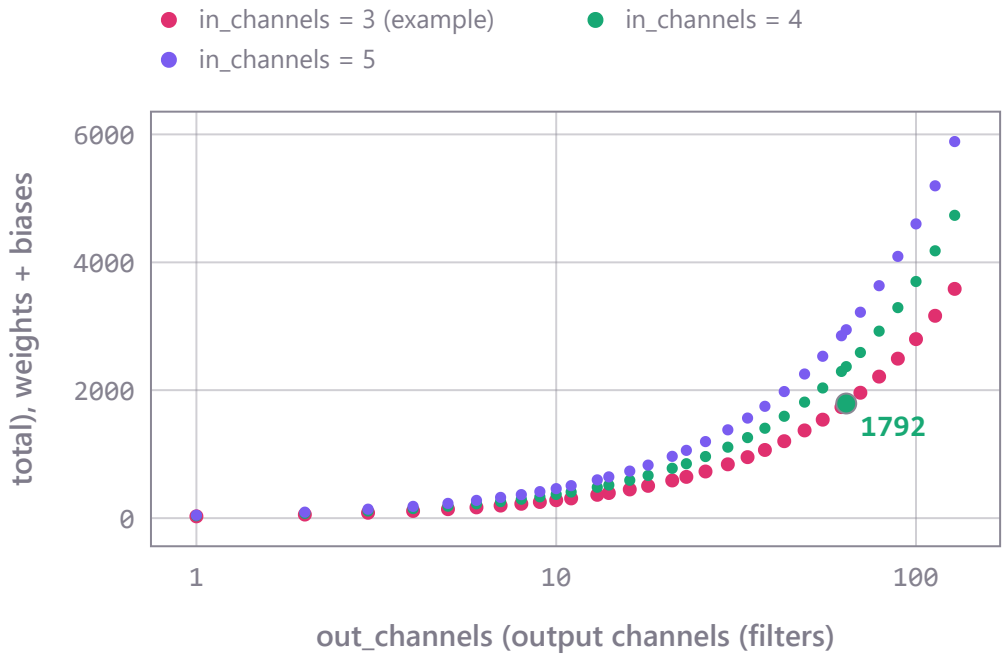
The number of weights, biases and total parameters of a $\mathbf{2}-D$ convolution layer, including grouped and depthwise convolutions.

$$P = C_{\text{out}} \frac{C_{\text{in}}}{g} k_h k_w + C_{\text{out}}$$

input	meaning	condition
`in_channels`	input channels	
`out_channels`	output channels (filters)	
`kernel_h`	kernel height	
`kernel_w`	kernel width	
`groups`	groups (1 = ordinary; in_channels = depthwise)	
`bias`	whether the layer has a bias per filter	

output	meaning
`weights`	C_out (C_in / g) k_h k_w
`biases`	C_out or 0
`total`	weights + biases

- **Method:** integer arithmetic
- **Assumes:** channels divisible by groups
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



total (weights + biases) against out_channels (output channels (filters)) from 1 to 128 on a logarithmic axis. One curve for each value of in_channels (input channels): 3, 4, 5. Other inputs as in the worked example: kernel_h = 3, kernel_w = 3, groups = 1, bias = True. 32 points per curve computed by data-science-ml-ai999x; marked: the worked example, out_channels = 64, total = 1792.

```
data_science_ml_ai999x.solve('ml_conv_parameter_count', in_channels=3, out_channels=64, kernel_h=3, kernel_w=3, groups=1, bias=True)
# -> weights = 1728, biases = 64, total = 1792   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_conv_parameter_count', in_channels=32, out_channels=32, kernel_h=3, kernel_w=3, groups=32, bias=False)
# -> weights = 288, biases = 0, total = 288   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convolution parameter count: in_channels = 3, out_channels = 64, kernel_h = 3, kernel_w = 3, groups = 1, bias = yes	SOLVED	weights = 1728, biases = 64, total = 1792
Convolution parameter count: in_channels = 32, out_channels = 32, kernel_h = 3, kernel_w = 3, groups = 32, bias = no	SOLVED	weights = 288, biases = 0, total = 288
Convolution parameter count: out_channels = 64, kernel_h = 3, kernel_w = 3, groups = 1, bias = yes	NEEDS_INPUT	ml_conv_parameter_count needs in_channels (input channels). Example: solve('ml_conv_parameter_count', in_channels=3, out
Convolution parameter count: in_channels = 3, out_channels = 64, kernel_h = 3, kernel_w = 3, groups = 1, bias = yes, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Convolution parameter count: in_channels = 3, out_channels = 64, kernel_h = 3, kernel_w = 3, groups = 2, bias = yes	INVALID_INPUT	Parameter count of a 2-D convolution layer: in_channels and out_channels must both be divisible by groups.

2-D convolution of an image with a kernel (deep-learning convention), `ml_conv2d_forward`

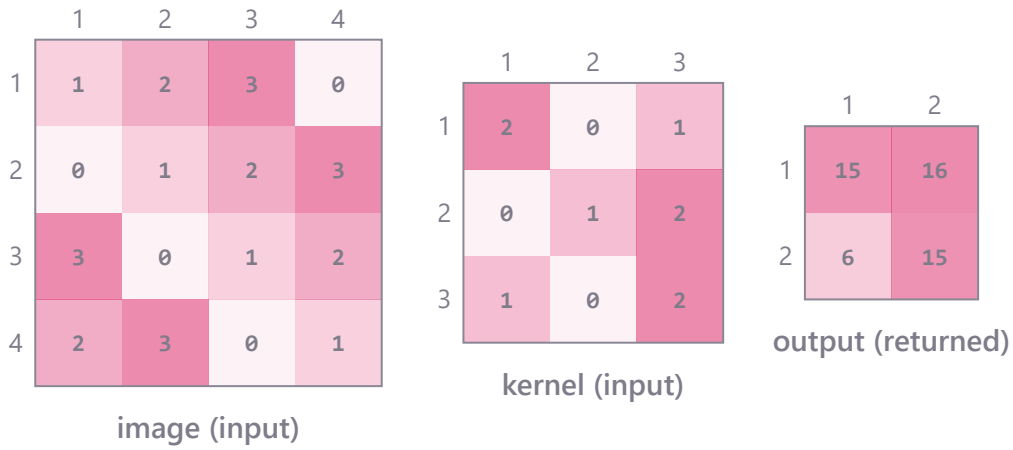
The output feature map of a single-channel $2-D$ convolution as computed in deep learning (cross-correlation: the kernel is not flipped), with a stride and zero padding.

$$Y_{ij} = \sum_{a,b} X_{is+a,\, js+b}\, K_{ab}$$

input	meaning	condition
`image`	input image (rows of pixel values)	
`kernel`	kernel (filter) weights	
`stride`	stride	
`padding`	zero padding on each side	

output	meaning
`output`	output feature map
`output_sum`	sum of all outputs

- **Method:** exact sums of products over each window
- **Assumes:** the kernel fits in the padded image
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): image (input); kernel (input); output (returned). Returned with them by data-science-ml-ai999x: output_sum = 52. Other inputs: stride = 1, padding = 0.



output_sum (sum of all outputs) against padding (zero padding on each side) from -10 to 10. One curve for each value of stride (stride): 1, 2, 3. Other inputs as in the worked example: image = [[1, 2, 3, 0], [0, 1, 2, 3], [3, 0, 1, 2], [2..., kernel = [[2, 0, 1], [0, 1, 2], [1, 0, 2]]. 11 points per curve computed by data-science-ml-ai999x; marked: the worked example, padding = 0, output_sum = 52.

```
data_science_ml_ai999x.solve('ml_conv2d_forward', image=[[1, 2, 3, 0], [0, 1, 2, 3], [3, 0, 1, 2],
[2, 3, 0, 1]], kernel=[[2, 0, 1], [0, 1, 2], [1, 0, 2]], stride=1, padding=0)
# -> output = [[15.0, 16.0], [6.0, 15.0]], output_sum = 52   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_conv2d_forward', image=[[1, 2, 0], [4, 5, 6], [7, 8, 9]], kernel=
[[1, -1], [0, 2]], stride=2, padding=1)
# -> output = [[2.0, 0.0], [10.0, 17.0]], output_sum = 29   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
2D convolution: image = [[1, 2, 3, 0], [0, 1, 2, 3], [3, 0, 1, 2], [2, 3, 0, 1]], kernel = [[2, 0, 1], [0, 1, 2], [1, 0, 2]], stride = 1, padding = 0	SOLVED	output = [[15.0, 16.0], [6.0, 15.0]], output_sum = 52
2D convolution: image = [[1, 2, 0], [4, 5, 6], [7, 8, 9]], kernel = [[1, -1], [0, 2]], stride = 2, padding = 1	SOLVED	output = [[2.0, 0.0], [10.0, 17.0]], output_sum = 29
2D convolution: kernel = [[2, 0, 1], [0, 1, 2], [1, 0, 2]], stride = 1, padding = 0	NEEDS_INPUT	ml_conv2d_forward needs image (input image (rows of pixel values)). Example: solve('ml_conv2d_forward', image=[[1, 2, 3,
2D convolution: image = [[1, 2, 3, 0], [0, 1, 2, 3], [3, 0, 1, 2], [2, 3, 0, 1]], kernel = [[2, 0, 1], [0, 1, 2], [1, 0, 2]], stride = 1, padding = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Max pooling and average pooling, `ml_pooling`

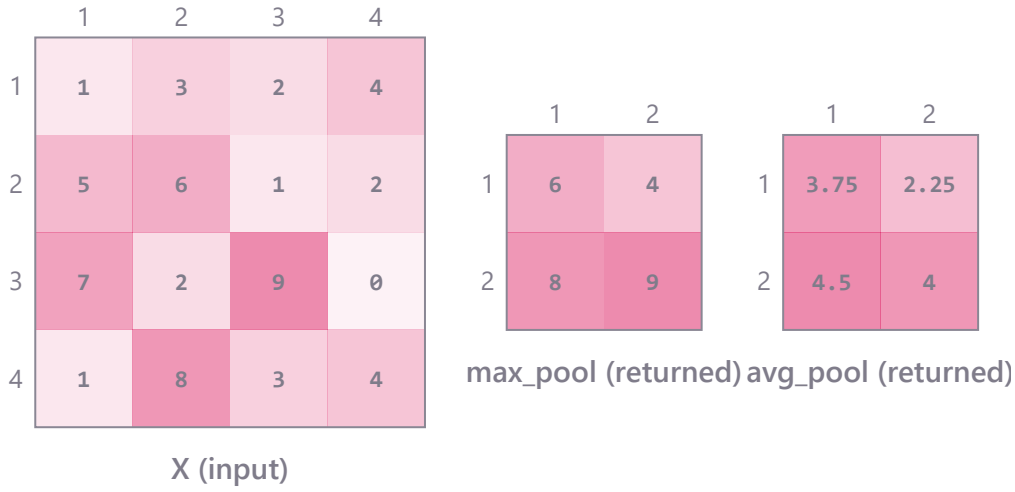
Max pooling and average pooling of a $\mathbf{2}-\boldsymbol{D}$ feature map with a square window, a stride and no padding.

$$Y_{ij} = \max_{a,b<k} X_{is+a,\, js+b}$$

input	meaning	condition
`X`	feature map	
`size`	window size	
`stride`	stride	

output	meaning
`max_pool`	maximum of each window
`avg_pool`	mean of each window

- **Method:** exact window maxima and means
- **Assumes:** the window fits in the input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); max_pool (returned); avg_pool (returned). Other inputs: size = 2, stride = 2.

```
data_science_ml_ai999x.solve('ml_pooling', X=[[1, 3, 2, 4], [5, 6, 1, 2], [7, 2, 9, 0], [1, 8, 3, 4]], size=2, stride=2)
# -> max_pool = [[6.0, 4.0], [8.0, 9.0]], avg_pool = [[3.75, 2.25], [4.5, 4.0]]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_pooling', X=[[1, 2, 3], [4, 5, 6], [7, 8, 9]], size=2, stride=1)
# -> max_pool = [[5.0, 6.0], [8.0, 9.0]], avg_pool = [[3.0, 4.0], [6.0, 7.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Max pooling: X = [[1, 3, 2, 4], [5, 6, 1, 2], [7, 2, 9, 0], [1, 8, 3, 4]], size = 2, stride = 2	SOLVED	max_pool = [[6.0, 4.0], [8.0, 9.0]], avg_pool = [[3.75, 2.25], [4.5, 4.0]]

question	outcome	result
Max pooling: X = [[1, 2, 3], [4, 5, 6], [7, 8, 9]], size = 2, stride = 1	SOLVED	max_pool = [[5.0, 6.0], [8.0, 9.0]], avg_pool = [[3.0, 4.0], [6.0, 7.0]]
Max pooling: size = 2, stride = 2	NEEDS_INPUT	ml_pooling needs X (feature map). Example: solve('ml_pooling', X=[[1, 3, 2, 4], [5, 6, 1, 2], [7, 2, 9, 0], [1, 8, 3, 4]
Max pooling: X = [[1, 3, 2, 4], [5, 6, 1, 2], [7, 2, 9, 0], [1, 8, 3, 4]], size = 2, stride = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Scaled dot-product attention, `ml_scaled_dot_product_attention`

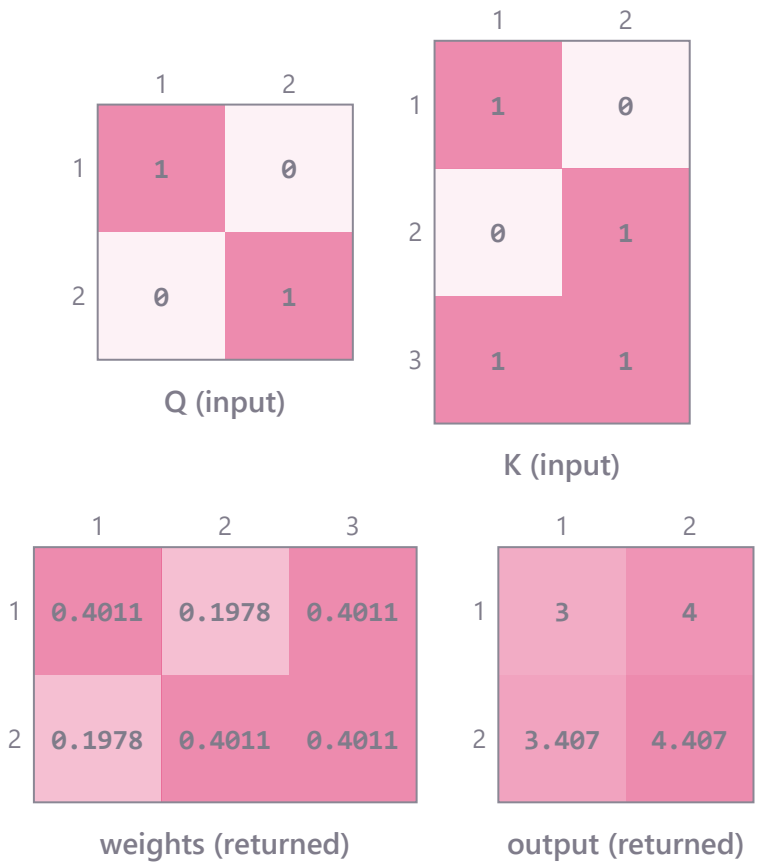
Scaled dot-product attention softmax(Q $K^T/sqrt$(d)) V for given query, key and value matrices, optionally with a causal mask (query i sees keys 0..i): the attention weights and the output.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

input	meaning	condition
`Q`	queries: one row per query	
`K`	keys: one row per key	
`V`	values: one row per key	
`causal`	apply a causal mask (yes or no)	

output	meaning
`weights`	attention weights (rows sum to 1)
`output`	weights V
`output_sum`	sum of all output entries

- **Method:** scores scaled by $1/sqrt$(d), stable softmax per row, weighted sum of values
- **Assumes:** Q and K have the same width; K and V have the same rows; causal: as many queries as keys
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): Q (input); K (input); weights (returned); output (returned). Returned with them by data-science-ml-ai999x: output_sum = 14.8133. Other inputs: causal = False.

```
data_science_ml_ai999x.solve('ml_scaled_dot_product_attention', Q=[[1, 0], [0, 1]], K=[[1, 0], [0, 1], [1, 1]], V=[[1, 2], [3, 4], [5, 6]], causal=False)
# -> weights = [[0.40111209268, 0.19777581464, 0.40111209268], [0.19777581464, 0.40111209268, 0.40111209268]], output = [[3.0, 4.0], [3.40667255608, 4.40667255608]], output_sum = 14.8133 (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_scaled_dot_product_attention', Q=[[1, 0], [0, 1], [1, 1]], K=[[1, 0], [0, 1], [1, 1]], V=[[1], [2], [3]], causal=True)
# -> weights = [[1.0, 0.0, 0.0], [0.330238450673, 0.669761549327, 0.0], [0.248255078258, 0.248255078258, 0.503489843485]], output = [[1.0], [1.66976154933], [2.25523476523]], output_sum = 4.925   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Scaled dot-product attention: Q = [[1, 0], [0, 1]], K = [[1, 0], [0, 1], [1, 1]], V = [[1, 2], [3, 4], [5, 6]], causal = no	SOLVED	weights = [[0.40111209268, 0.19777581464, 0.40111209268], [0.19777581464, 0.40111209268, 0.40111209268]], output = [[3.0, 4.0], [3.40667255608, 4.40667255608]], output_sum = 14.8133
Scaled dot-product attention: Q = [[1, 0], [0, 1], [1, 1]], K = [[1, 0], [0, 1], [1,	SOLVED	weights = [[1.0, 0.0, 0.0], [0.330238450673, 0.669761549327, 0.0], [0.248255078258, 0.248255078258, 0.503489843485]], output = [[1.0], [1.66976154933], [2.25523476523]], output_sum = 4.925

question	outcome	result
1]], V = [[1], [2], [3]], causal = yes		
Scaled dot-product attention: K = [[1, 0], [0, 1], [1, 1]], V = [[1, 2], [3, 4], [5, 6]], causal = no	NEEDS_INPUT	ml_scaled_dot_product_attention needs Q (queries: one row per query). Example: solve('ml_scaled_dot_product_attention',
Scaled dot-product attention: Q = [[1, 0], [0, 1]], K = [[1, 0], [0, 1], [1, 1]], V = [[1, 2], [3, 4], [5, 6]], causal = no, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Multi-head attention (identity projections), `ml_multihead_attention`

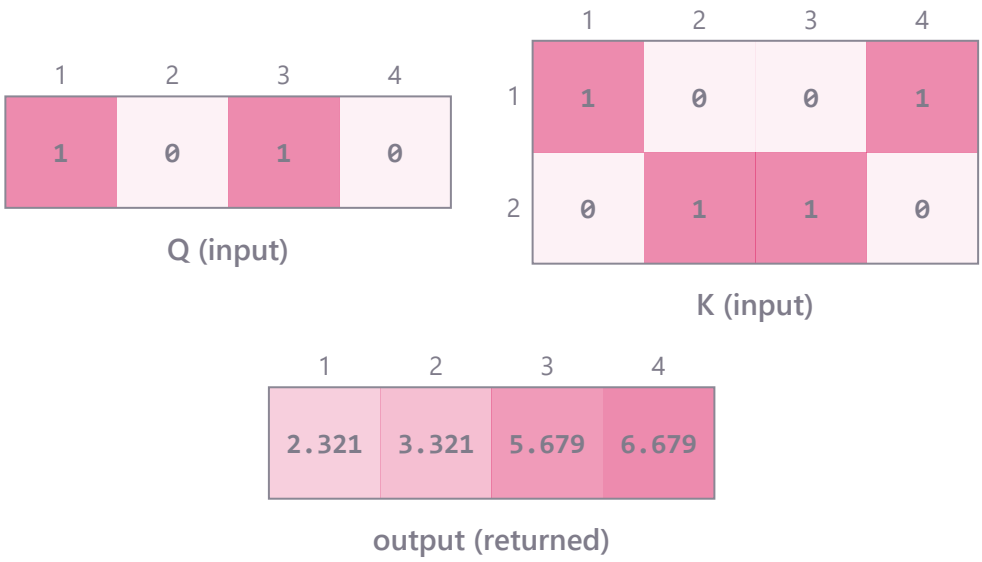
Multi-head attention with identity input and output projections: the model dimension is split into equal heads, scaled dot-product attention runs in each head (scale $1/sqrt$(head size)) and the head outputs are concatenated.

$$\mathrm{MHA} = [\mathrm{head}_1, \ldots, \mathrm{head}_h], \quad \mathrm{head}_i = \mathrm{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d/h}}\right) V_i$$

input	meaning	condition
`Q`	queries (rows), width = model dimension	
`K`	keys	
`V`	values	
`heads`	number of heads (divides the model dimension)	

output	meaning
`output`	concatenated head outputs
`output_sum`	sum of all output entries

- **Method:** per head: softmax(Q_h $K_h^T/sqrt$(d_h)) V_h; concatenate
- **Assumes:** equal widths; model dimension divisible by heads
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): Q (input); K (input); output (returned). Returned with them by data-science-ml-ai999x: output_sum = 18. Other inputs: heads = 2.

```
data_science_ml_ai999x.solve('ml_multihead_attention', Q=[[1, 0, 1, 0]], K=[[1, 0, 0, 1], [0, 1, 1, 0]], V=[[1, 2, 3, 4], [5, 6, 7, 8]], heads=2)
# -> output = [[2.32095380269, 3.32095380269, 5.67904619731, 6.67904619731]], output_sum = 18 (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_multihead_attention', Q=[[0.5, -1], [1, 1]], K=[[1, 0], [0, 1], [1, 1]], V=[[1, 0], [0, 1], [2, 2]], heads=2)
# -> output = [[1.15095519357, 0.635824672851], [1.26695639475, 1.26695639475]], output_sum = 4.32069   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multi-head attention: Q = [[1, 0, 1, 0]], K = [[1, 0, 0, 1], [0, 1, 1, 0]], V = [[1, 2, 3, 4], [5, 6, 7, 8]], heads = 2	SOLVED	output = [[2.32095380269, 3.32095380269, 5.67904619731, 6.67904619731]], output_sum = 18
Multi-head attention: Q = [[0.5, -1], [1, 1]], K = [[1, 0], [0, 1], [1, 1]], V = [[1, 0], [0, 1], [2, 2]], heads = 2	SOLVED	output = [[1.15095519357, 0.635824672851], [1.26695639475, 1.26695639475]], output_sum = 4.32069
Multi-head attention: K = [[1, 0, 0, 1], [0, 1, 1, 0]], V = [[1, 2, 3, 4], [5, 6, 7, 8]], heads = 2	NEEDS_INPUT	ml_multihead_attention needs Q (queries (rows), width = model dimension). Example: solve('ml_multihead_attention', Q=[[1
Multi-head attention: Q = [[1, 0, 1, 0]], K = [[1, 0, 0, 1], [0, 1, 1, 0]], V = [[1, 2, 3, 4], [5, 6, 7, 8]], heads = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Multi-head attention: Q = [[1, 0, 1]], K = [[1, 0, 0]], V = [[1, 2, 3]], heads = 2	INVALID_INPUT	Multi-head attention (identity projections): the model dimension must be divisible by the number of heads.

Batch normalisation forward pass (training mode), `ml_batch_norm_forward`

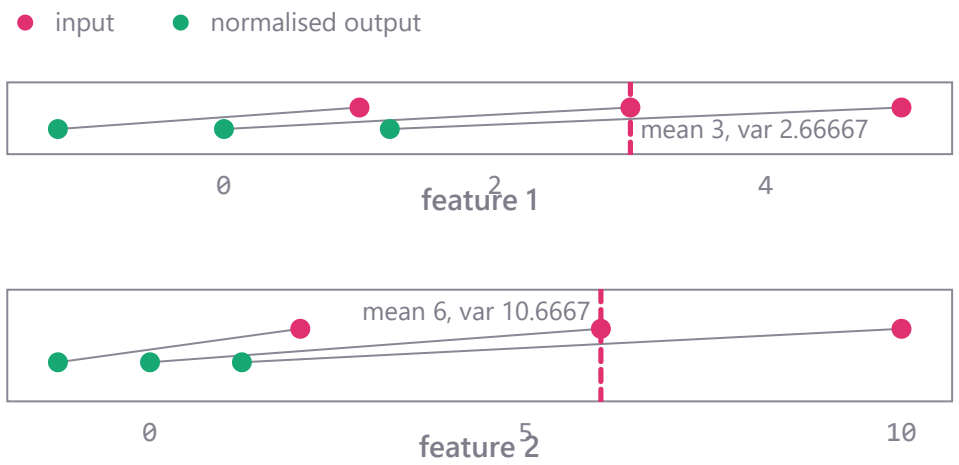
Batch normalisation of a mini-batch in training mode: each feature is normalised with the batch mean and the biased batch variance plus eps, then scaled by gamma and shifted by beta.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta$$

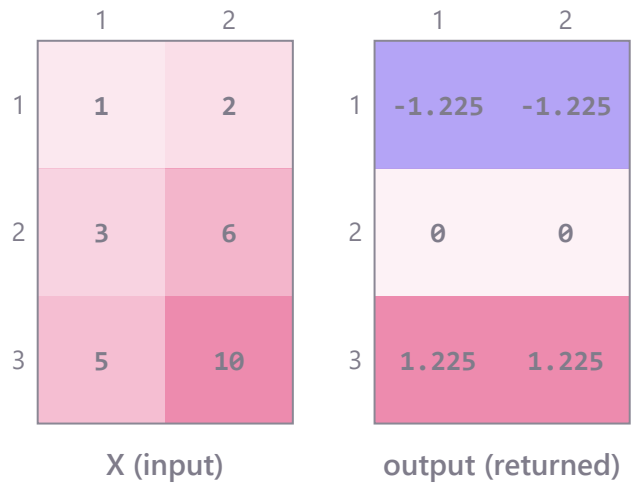
input	meaning	condition
`X`	mini-batch: one row per sample, one column per feature (at least two rows)	
`gamma`	scale per feature	
`beta`	shift per feature	
`eps`	small constant added to the variance (e.g. 1e-5)	

output	meaning
`output`	gamma (x - mean)/sqrt(var + eps) + beta
`batch_mean`	batch mean of each feature
`batch_var`	biased batch variance of each feature

- **Method:** exact batch mean and variance; one square root per feature
- **Assumes:** at least two samples; var + eps > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Each feature of the batch X = [[1, 2], [3, 6], [5, 10]] (top row of dots) and the returned normalised output (bottom row; γ = [1, 1], β = [0, 0], ε = 0): every feature is shifted by its returned batch mean [3.0, 6.0] and divided by the root of its batch variance ['2.66667', '10.6667'].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); output (returned). Other inputs: gamma = [1, 1], beta = [0, 0], eps = 0.

```
data_science_ml_ai999x.solve('ml_batch_norm_forward', X=[[1, 2], [3, 6], [5, 10]], gamma=[1, 1],
beta=[0, 0], eps=0)
# -> output = [[-1.22474487139, -1.22474487139], [0.0, 0.0], [1.22474487139, 1.22474487139]],
batch_mean = [3.0, 6.0], batch_var = [2.66666666667, 10.6666666667]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_batch_norm_forward', X=[[0.5, -1, 2], [1.5, 0, 2.5], [2.5, 1, 1]],
gamma=[1, 0.5, 2], beta=[0, 1, -1], eps=1e-05)
# -> output = [[-1.22473568591, 0.387632157046, -0.465484388475], [0.0, 1.0, 1.1380624461],
[1.22473568591, 1.61236784295, -3.67257805763]], batch_mean = [1.5, 0.0, 1.83333333333], batch_var =
[0.666666666667, 0.666666666667, 0.388888888889]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Batch normalization: X = [[1, 2], [3, 6], [5, 10]], gamma = [1, 1], beta = [0, 0], eps = 0	SOLVED	output = [[-1.22474487139, -1.22474487139], [0.0, 0.0], [1.22474487139, 1.22474487139]], batch_mean = [3.0, 6.0], batch_var = [2.66666666667, 10.6666666667]
Batch normalization: X = [[0.5, -1, 2], [1.5, 0, 2.5], [2.5, 1, 1]], gamma = [1, 0.5, 2], beta = [0, 1, -1], eps = 1e-05	SOLVED	output = [[-1.22473568591, 0.387632157046, -0.465484388475], [0.0, 1.0, 1.1380624461], [1.22473568591, 1.61236784295, -3.67257805763]], batch_mean = [1.5, 0.0, 1.83333333333], batch_var = [0.666666666667, 0.666666666667, 0.388888888889]
Batch normalization: gamma = [1, 1], beta = [0, 0], eps = 0	NEEDS_INPUT	ml_batch_norm_forward needs X (mini-batch: one row per sample, one column per feature (at least two rows)). Example: sol
Batch normalization: X = [[1, 2], [3, 6], [5, 10]], gamma = [1, 1], beta = [0, 0], eps = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

SGD with momentum: one update, `ml_sgd_momentum_step`

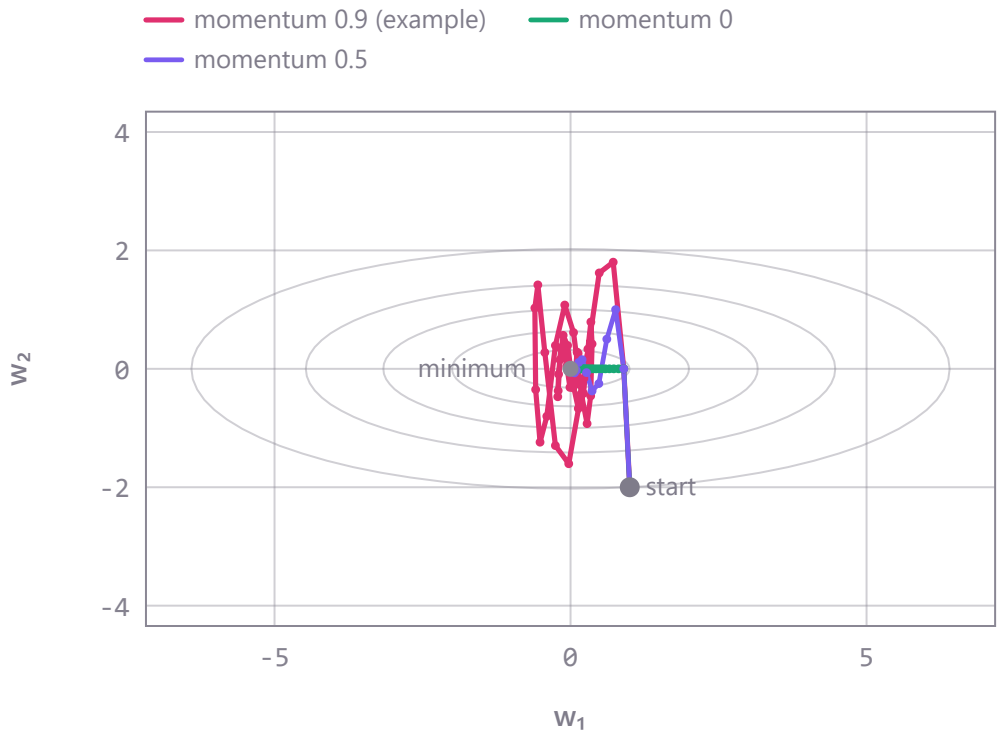
One step of stochastic gradient descent with momentum in PyTorch's form (optional Nesterov and weight decay): $d = g + wd$ w, buf = mu buf + d, $w = w - lr$ (buf, or d + mu buf with Nesterov).

$$b_t = \mu b_{t-1} + g_t, \quad w_t = w_{t-1} - \eta\, b_t$$

input	meaning	condition
`w`	parameters	
`grad`	gradient	
`buf`	momentum buffer before the step (zeros at the start)	
`lr`	learning rate	
`momentum`	momentum mu $(0 \leq mu < 1)$	
`nesterov`	use Nesterov momentum (yes or no)	
`weight_decay`	L2 weight decay (>= 0)	

output	meaning
`buf_new`	momentum buffer after the step
`w_new`	parameters after the step

- **Method:** exact rational arithmetic
- **Assumes:** $lr > 0$; 0 <= momentum < 1; weight_decay >= 0; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



40 SGD steps with momentum from the worked example's start w = (1, -2) on the loss L(w) = ½(w_1^2 + 10 w_2^2) chosen here (its gradient (w_1, 10 w_2) is passed to every step; grey ellipses are its level sets L = 0.5, 2, 5, 10, 20.5); lr = 0.1, momentum 0, 0.5, 0.9, Nesterov False, weight decay 0. Each dot is the w the package returned, its returned buffer fed to the next step. Ends after 40 steps: momentum 0.9 (example): (0.116015, -0.251647); momentum 0: (0.0147809, 0); momentum 0.5: (-3.0529e-07, -8.9382e-08). Momentum speeds the slow w_1 direction and overshoots across the steep w_2 one.

```
data_science_ml_ai999x.solve('ml_sgd_momentum_step', w=[1, -2], grad=[0.5, -1], buf=[0.1, 0.2],
lr=0.1, momentum=0.9, nesterov=False, weight_decay=0)
# -> buf_new = [0.59, -0.82], w_new = [0.941, -1.918]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_sgd_momentum_step', w=[0.3], grad=[0.2], buf=[0], lr=0.01,
momentum=0.9, nesterov=True, weight_decay=0.01)
# -> buf_new = [0.203], w_new = [0.296143]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
SGD momentum step: w = [1, -2], grad = [0.5, -1], buf = [0.1, 0.2], lr = 0.1, momentum = 0.9, nesterov = no, weight_decay = 0	SOLVED	buf_new = [0.59, -0.82], w_new = [0.941, -1.918]
SGD momentum step: w = [0.3], grad = [0.2], buf = [0], lr = 0.01, momentum = 0.9, nesterov = yes, weight_decay = 0.01	SOLVED	buf_new = [0.203], w_new = [0.296143]
SGD momentum step: grad = [0.5, -1], buf = [0.1, 0.2], lr = 0.1, momentum = 0.9, nesterov = no, weight_decay = 0	NEEDS_INPUT	ml_sgd_momentum_step needs w (parameters). Example: solve('ml_sgd_momentum_step', w=[1, -2], grad=[0.5, -1], buf=[0.1, 0
SGD momentum step: w = [1, -2], grad = [0.5, -1], buf = [0.1, 0.2], lr = 0.1, momentum = 0.9, nesterov = no, weight_decay = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Adam optimizer: one update, `ml_adam_step`

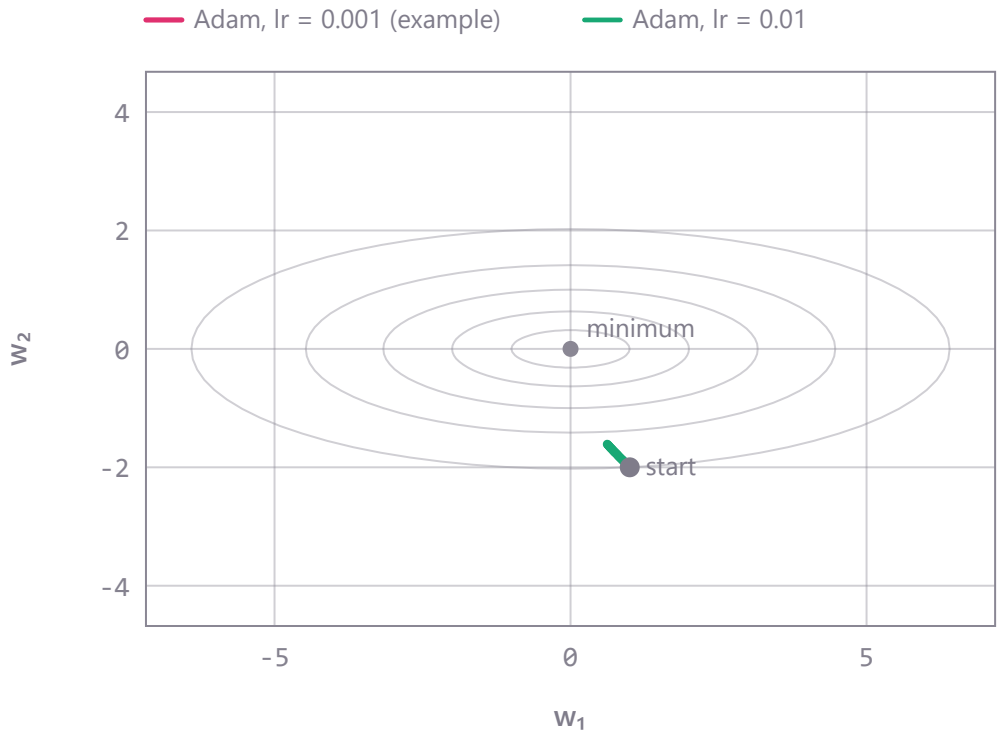
One Adam update: the new first and second moment estimates and the parameters after the bias-corrected step w - lr m_hat / (sqrt(v_hat) + eps).

$$\hat{m} = \frac{m_t}{1-\beta_1^t},\ \hat{v} = \frac{v_t}{1-\beta_2^t},\quad w_t = w_{t-1} - \eta\frac{\hat{m}}{\sqrt{\hat{v}}+\varepsilon}$$

input	meaning	condition
`w`	parameters	
`grad`	gradient	
`m`	first-moment estimate before the step (zeros at the start)	
`v`	second-moment estimate before the step (zeros at the start)	
`t`	step number after this update (1 for the first step)	
`lr`	learning rate	
`beta1`	first-moment decay (e.g. 0.9)	
`beta2`	second-moment decay (e.g. 0.999)	
`eps`	denominator constant (e.g. 1e-8)	

output	meaning
`m_new`	beta1 $m + (1 - beta1)$ g
`v_new`	beta2 $v + (1 - beta2)\ g^2$
`w_new`	parameters after the step

- **Method:** PyTorch's update rule (eps added after the square root of the bias-corrected v)
- **Assumes:** $t \geq 1$; $0 \leq beta1$, $beta2 < 1$; $v \geq 0$; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



40 Adam steps from the worked example's start w = (1, -2) on the loss L(w) = ½(w_1^2 + 10 w_2^2) chosen here (its gradient (w_1, 10 w_2) is passed to every step; grey ellipses are its level sets L = 0.5, 2, 5, 10, 20.5); β_1 = 0.9, β_2 = 0.999, ε = 1.0000e-08, learning rates 0.001 (the worked example's) and 0.01. Each dot is the w the package returned, its returned moments m and v fed to the next step. Marked: after 40 steps at lr = 0.001, w = (0.960168, -1.96008). Adam moves each coordinate by about lr per step whatever the gradient's scale.

```
data_science_ml_ai999x.solve('ml_adam_step', w=[1, -2], grad=[0.5, -1], m=[0, 0], v=[0, 0], t=1,
lr=0.001, beta1=0.9, beta2=0.999, eps=1e-08)
# -> m_new = [0.05, -0.1], v_new = [0.00025, 0.001], w_new = [0.99900000002, -1.99900000001]
(SOLVED)
```

```
data_science_ml_ai999x.solve('ml_adam_step', w=[0.3, 0.1], grad=[0.2, -0.05], m=[0.01, -0.02], v=
[0.001, 0.0004], t=10, lr=0.01, beta1=0.9, beta2=0.999, eps=1e-08)
# -> m_new = [0.029, -0.023], v_new = [0.001039, 0.0004021], w_new = [0.298621781734,
0.101757067997]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Adam step: w = [1, -2], grad = [0.5, -1], m = [0, 0], v = [0, 0], t = 1, lr = 0.001, beta1 = 0.9, beta2 = 0.999, eps = 1e-08	SOLVED	m_new = [0.05, -0.1], v_new = [0.00025, 0.001], w_new = [0.99900000002, -1.99900000001]
Adam step: w = [0.3, 0.1], grad = [0.2, -0.05], m = [0.01, -0.02], v = [0.001, 0.0004], t = 10, lr = 0.01, beta1 = 0.9, beta2 = 0.999, eps = 1e-08	SOLVED	m_new = [0.029, -0.023], v_new = [0.001039, 0.0004021], w_new = [0.298621781734, 0.101757067997]
Adam step: grad = [0.5, -1], m = [0, 0], v = [0, 0], t = 1, lr = 0.001, beta1 = 0.9, beta2 = 0.999, eps = 1e-08	NEEDS_INPUT	ml_adam_step needs w (parameters). Example: solve('ml_adam_step', w=[1, -2], grad=[0.5, -1], m=[0, 0], v=[0, 0], t=1, lr
Adam step: w = [1, -2], grad = [0.5, -1], m = [0, 0], v = [0, 0], t = 1, lr = 0.001,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

question	outcome	result
beta1 = 0.9, beta2 = 0.999, eps = 1e-08, zz = 1		

AdamW optimizer (decoupled weight decay): one update, `ml_adamw_step`

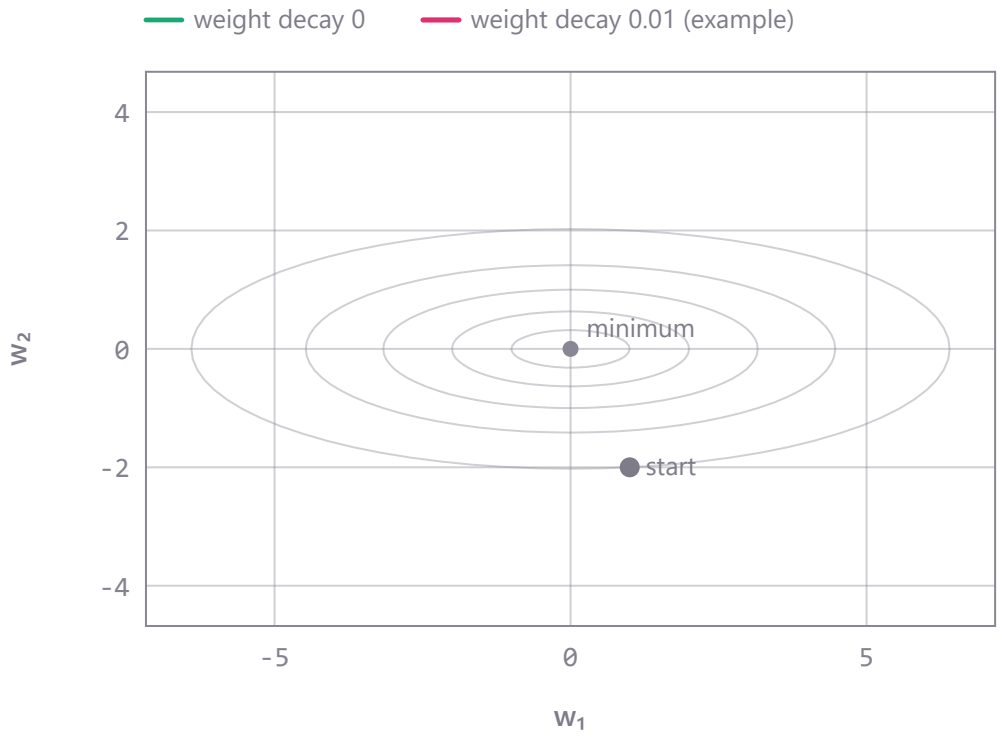
One AdamW update: the parameters first decay by the factor (1 - lr weight_decay), then take the bias-corrected Adam step.

$$w \leftarrow w(1-\eta\lambda), \quad w_t = w - \eta\frac{\hat{m}}{\sqrt{\hat{v}}+\varepsilon}$$

input	meaning	condition
`w`	parameters	
`grad`	gradient	
`m`	first-moment estimate before the step (zeros at the start)	
`v`	second-moment estimate before the step (zeros at the start)	
`t`	step number after this update (1 for the first step)	
`lr`	learning rate	
`beta1`	first-moment decay (e.g. 0.9)	
`beta2`	second-moment decay (e.g. 0.999)	
`eps`	denominator constant (e.g. 1e-8)	
`weight_decay`	decoupled weight decay (e.g. 0.01)	

output	meaning
`m_new`	beta1 $m + (1 - beta1)$ g
`v_new`	beta2 $v + (1 - beta2)\ g^2$
`w_new`	parameters after decay and step

- **Method:** PyTorch's AdamW rule
- **Assumes:** $t \geq 1$; $0 \leq beta1$, $beta2 < 1$; $v \geq 0$; weight_decay >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



40 AdamW steps from the worked example's start w = (1, -2) on the loss L(w) = ½(w_1^2 + 10 w_2^2) chosen here (its gradient (w_1, 10 w_2) is passed to every step; grey ellipses are its level sets L = 0.5, 2, 5, 10, 20.5); lr = 0.001, β_1 = 0.9, β_2 = 0.999, ε = 1.0000e-08, decoupled weight decay 0 and 0.01 (the worked example's 0.01 among them). Each dot is the w the package returned, its moments fed to the next step. Marked ends: (0.960168, -1.96008) and (0.959777, -1.95929).

```
data_science_ml_ai999x.solve('ml_adamw_step', w=[1, -2], grad=[0.5, -1], m=[0, 0], v=[0, 0], t=1,
lr=0.001, beta1=0.9, beta2=0.999, eps=1e-08, weight_decay=0.01)
# -> m_new = [0.05, -0.1], v_new = [0.00025, 0.001], w_new = [0.99899000002, -1.99898000001]
(SOLVED)
```

```
data_science_ml_ai999x.solve('ml_adamw_step', w=[0.3, 0.1], grad=[0.2, -0.05], m=[0.01, -0.02], v=
[0.001, 0.0004], t=10, lr=0.01, beta1=0.9, beta2=0.999, eps=1e-08, weight_decay=0.1)
# -> m_new = [0.029, -0.023], v_new = [0.001039, 0.0004021], w_new = [0.298321781734,
0.101657067997]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
AdamW step: w = [1, -2], grad = [0.5, -1], m = [0, 0], v = [0, 0], t = 1, lr = 0.001, beta1 = 0.9, beta2 = 0.999, eps = 1e-08, weight_decay = 0.01	SOLVED	m_new = [0.05, -0.1], v_new = [0.00025, 0.001], w_new = [0.99899000002, -1.99898000001]
AdamW step: w = [0.3, 0.1], grad = [0.2, -0.05], m = [0.01, -0.02], v = [0.001, 0.0004], t = 10, lr = 0.01, beta1 = 0.9, beta2 = 0.999, eps = 1e-08, weight_decay = 0.1	SOLVED	m_new = [0.029, -0.023], v_new = [0.001039, 0.0004021], w_new = [0.298321781734, 0.101657067997]
AdamW step: grad = [0.5, -1], m = [0, 0], v = [0, 0], t = 1, lr = 0.001, beta1 = 0.9, beta2 = 0.999, eps = 1e-08, weight_decay = 0.01	NEEDS_INPUT	ml_adamw_step needs w (parameters). Example: solve('ml_adamw_step', w=[1, -2], grad=[0.5, -1], m=[0, 0], v=[0, 0], t=1,
AdamW step: w = [1, -2], grad = [0.5, -1], m = [0, 0], v = [0, 0], t = 1, lr = 0.001, beta1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

question	outcome	result
= 0.9, beta2 = 0.999, eps = 1e-08, weight_decay = 0.01, zz = 1		

Step learning-rate schedule, `ml_step_lr_schedule`

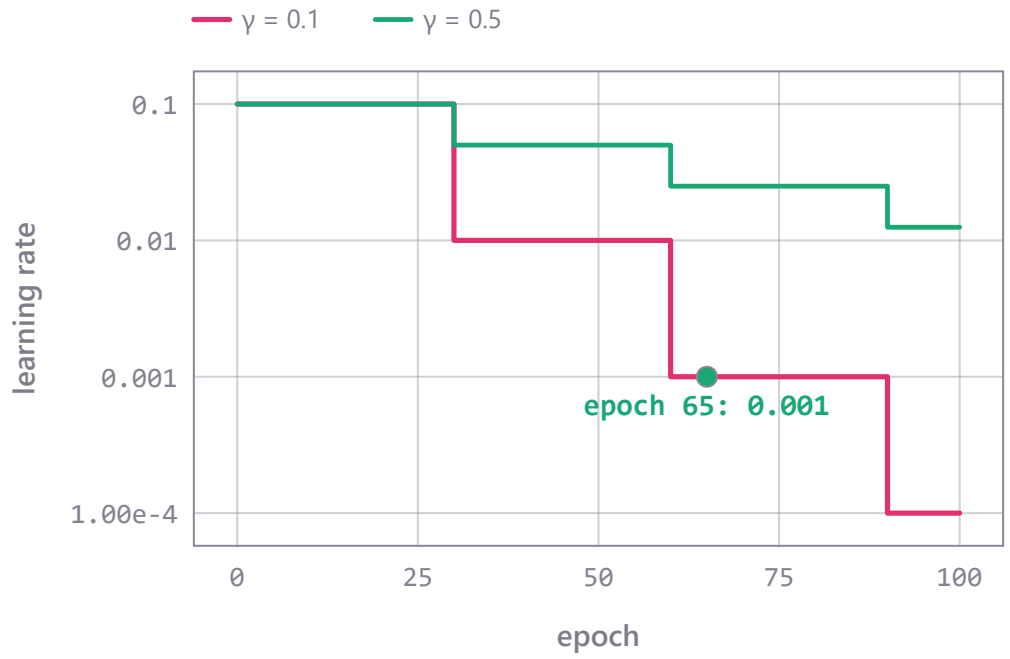
The learning rate at a given epoch under step decay: multiplied by gamma every step_size epochs.

$$\eta_t = \eta_0 \, \gamma^{\lfloor t/s \rfloor}$$

input	meaning	condition
`lr0`	initial learning rate	
`step_size`	epochs between decays	
`gamma`	decay factor $(0 < \gamma \leq 1)$	
`epoch`	epoch number (0 = start, at most 1000)	

output	meaning
`lr`	lr0 gamma^floor(epoch / step_size)
`decays`	number of decays so far

- **Method:** closed form
- **Assumes:** $lr0 > 0$; $0 < \gamma \leq 1$; 0 <= epoch <= 1000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Step decay from lr0 = 0.1, multiplied by γ every 30 epochs, for γ = 0.1 and 0.5 over 100 epochs (101 runs each; logarithmic axis). Marked: the worked example, epoch 65, lr = 0.001 after 2 decays.

```
data_science_ml_ai999x.solve('ml_step_lr_schedule', lr0=0.1, step_size=30, gamma=0.1, epoch=65)
# -> lr = 0.001, decays = 2   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_step_lr_schedule', lr0=0.05, step_size=10, gamma=0.5, epoch=9)
# -> lr = 0.05, decays = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Step learning rate schedule: lr0 = 0.1, step_size = 30, gamma = 0.1, epoch = 65	SOLVED	lr = 0.001, decays = 2
Step learning rate schedule: lr0 = 0.05, step_size = 10, gamma = 0.5, epoch = 9	SOLVED	lr = 0.05, decays = 0
Step learning rate schedule: step_size = 30, gamma = 0.1, epoch = 65	NEEDS_INPUT	ml_step_lr_schedule needs lr0 (initial learning rate). Example: solve('ml_step_lr_schedule', lr0=0.1, step_size=30, gamm
Step learning rate schedule: lr0 = 0.1, step_size = 30, gamma = 0.1, epoch = 65, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Cosine-annealing learning-rate schedule, `ml_cosine_lr_schedule`

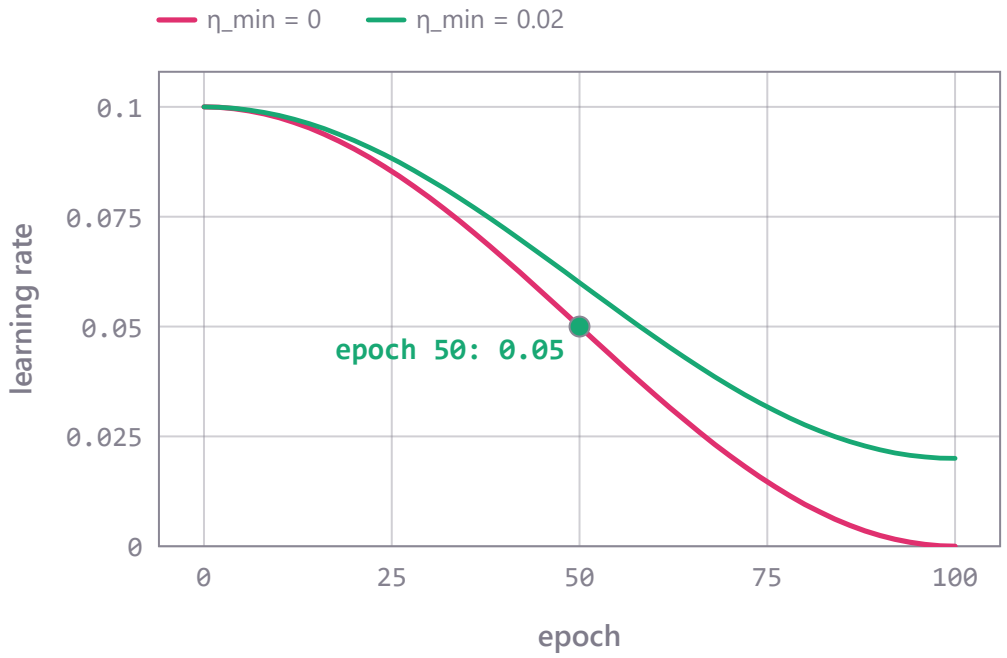
The learning rate at a given epoch under cosine annealing from lr_max down to eta_min over T_max epochs.

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{\pi t}{T}\right)$$

input	meaning	condition
`lr_max`	initial (maximum) learning rate	
`eta_min`	final (minimum) learning rate	
`T_max`	length of the schedule in epochs (at most 1000)	
`epoch`	epoch number (0 to T_max)	

output	meaning
`lr`	eta_min + (lr_max - eta_min)(1 + cos(pi t / T_max)) / 2

- **Method:** closed form
- **Assumes:** 0 <= eta_min < lr_max; 0 <= epoch <= T_max <= 1000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cosine annealing from lr_max = 0.1 over T_max = 100 epochs to η_min = 0 and 0.02 (51 runs each). Marked: the worked example, epoch 50, lr = 0.05, halfway down.

```
data_science_ml_ai999x.solve('ml_cosine_lr_schedule', lr_max=0.1, eta_min=0, T_max=100, epoch=50)
# -> lr = 0.05   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_cosine_lr_schedule', lr_max=0.01, eta_min=0.0001, T_max=200,
epoch=150)
# -> lr = 0.00154982   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cosine learning rate schedule: lr_max = 0.1, eta_min = 0, T_max = 100, epoch = 50	SOLVED	lr = 0.05
Cosine learning rate schedule: lr_max = 0.01, eta_min = 0.0001, T_max = 200, epoch = 150	SOLVED	lr = 0.00154982
Cosine learning rate schedule: eta_min = 0, T_max = 100, epoch = 50	NEEDS_INPUT	ml_cosine_lr_schedule needs lr_max (initial (maximum) learning rate). Example: solve('ml_cosine_lr_schedule', lr_max=0.1
Cosine learning rate schedule: lr_max = 0.1, eta_min = 0, T_max = 100, epoch = 50, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Cosine learning rate schedule: lr_max = 0.1, eta_min = 0, T_max = 100, epoch = 150	INVALID_INPUT	Cosine-annealing learning-rate schedule: 1 <= T_max <= 1000 and 0 <= epoch <= T_max (whole numbers).

Linear warm-up followed by cosine decay, `ml_warmup_cosine_lr_schedule`

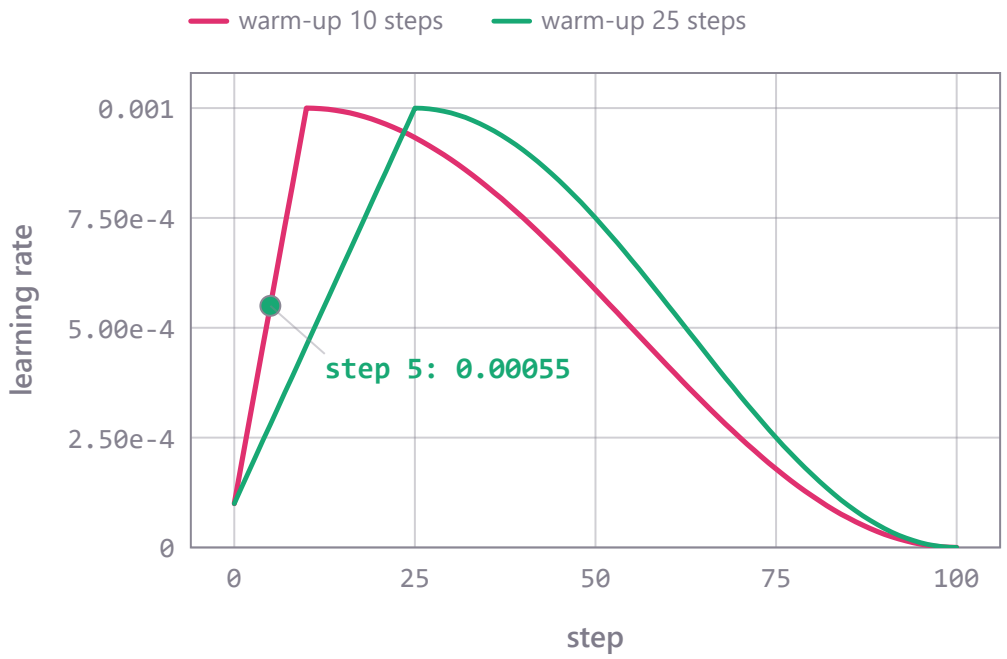
The learning rate at a given step of the common transformer schedule: linear warm-up from start_factor lr_max to lr_max over the warm-up steps, then cosine decay to eta_min at the last step.

$$\eta_t = \begin{cases} \eta_{\max}\left(f + (1-f)\frac{t}{W}\right) & t < W \\ \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{\pi(t-W)}{T-W}\right) & t \geq W \end{cases}$$

input	meaning	condition
`lr_max`	peak learning rate	
`start_factor`	fraction of lr_max at step 0 $(0 < f \leq 1)$	
`warmup`	warm-up steps	
`total`	total steps (at most 1000)	
`eta_min`	final learning rate	
`step`	step number (0 to total)	

output	meaning
`lr`	learning rate at the step

- **Method:** closed form of each phase
- **Assumes:** 1 <= warmup < total <= 1000; 0 <= step <= total
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Linear warm-up from 0.1 × lr_max to lr_max = 0.001, then cosine decay to 0 at step 100, for warm-ups of 10 and 25 steps (101 runs each). Marked: the worked example, step 5, lr = 0.00055.

```
data_science_ml_ai999x.solve('ml_warmup_cosine_lr_schedule', lr_max=0.001, start_factor=0.1,
warmup=10, total=100, eta_min=0, step=5)
# -> lr = 0.00055   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_warmup_cosine_lr_schedule', lr_max=0.001, start_factor=0.1,
warmup=10, total=100, eta_min=0, step=55)
# -> lr = 0.0005   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Warmup cosine schedule: lr_max = 0.001, start_factor = 0.1, warmup = 10, total = 100, eta_min = 0, step = 5	SOLVED	lr = 0.00055
Warmup cosine schedule: lr_max = 0.001, start_factor = 0.1, warmup = 10, total = 100, eta_min = 0, step = 55	SOLVED	lr = 0.0005
Warmup cosine schedule: start_factor = 0.1, warmup = 10, total = 100, eta_min = 0, step = 5	NEEDS_INPUT	ml_warmup_cosine_lr_schedule needs lr_max (peak learning rate). Example: solve('ml_warmup_cosine_lr_schedule', lr_max=0.
Warmup cosine schedule: lr_max = 0.001, start_factor = 0.1, warmup = 10, total = 100, eta_min = 0, step = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Parameter count of a transformer encoder, `ml_transformer_parameter_count`

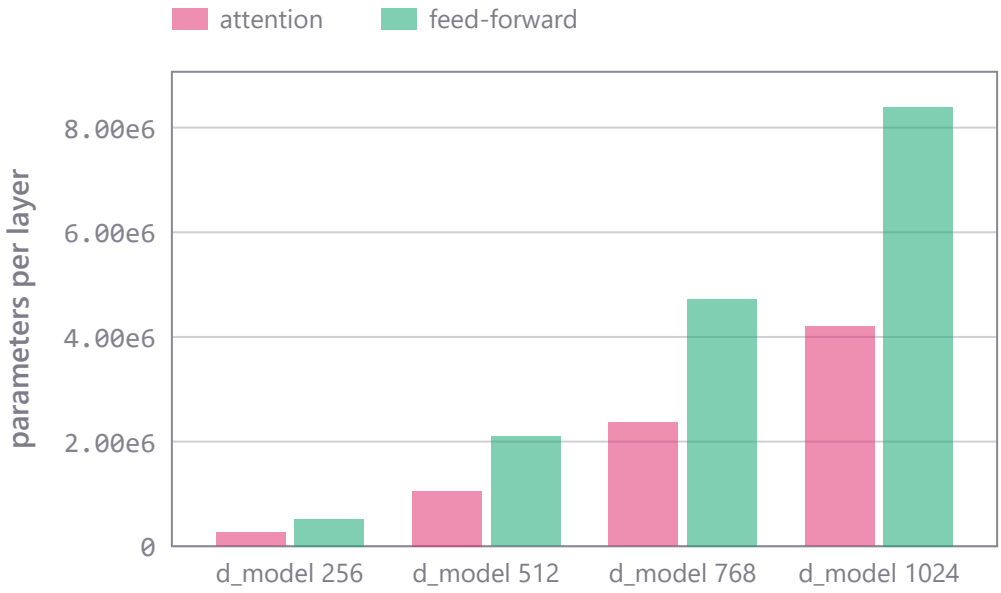
The parameters of a standard transformer encoder layer (multi-head self-attention with biases, a two-layer feed-forward block and two layer norms) and of a stack of such layers.

$$P_{\text{layer}} = 4d^2 + 4d + 2d\,d_{ff} + d_{ff} + d + 4d$$

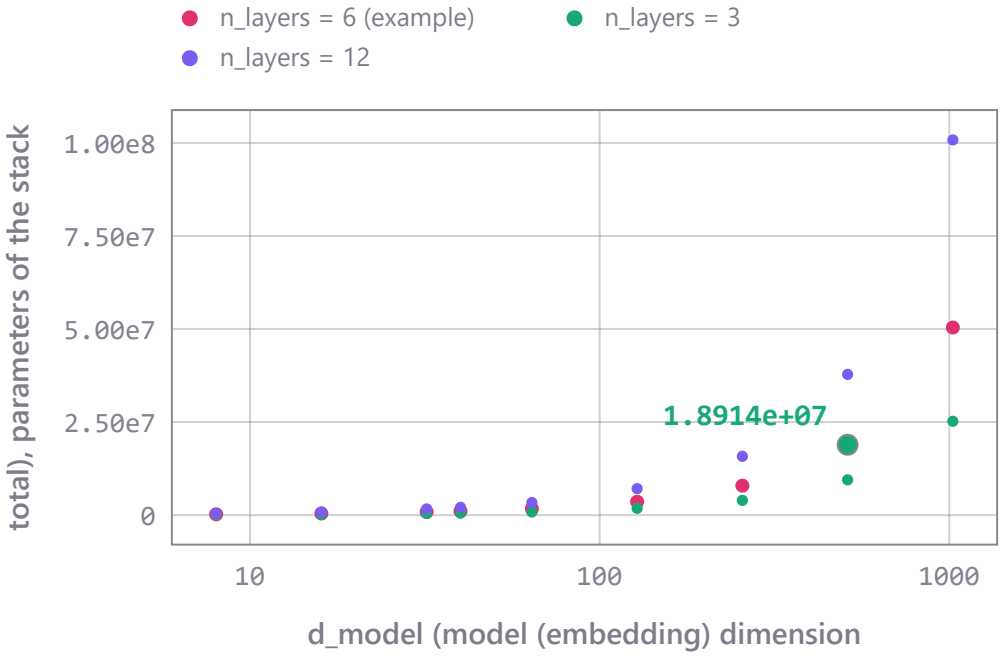
input	meaning	condition
`d_model`	model (embedding) dimension	
`n_heads`	attention heads (divide d_model)	
`d_ff`	feed-forward hidden size	
`n_layers`	number of layers	

output	meaning
`attention`	4 $d^2 + 4$ d (Q, K, V and output projections with biases)
`feed_forward`	2 d d_ff + d_ff + d
`layer_norms`	4 d
`per_layer`	parameters per layer
`total`	parameters of the stack

- **Method:** integer arithmetic
- **Assumes:** d_model divisible by n_heads
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Parameters of one transformer layer by part for d_model = 256, 512, 768, 1024 with d_ff = 4·d_model and 8 heads, one run each: 256: attention 263168, feed-forward 525568; 512: attention 1050624, feed-forward 2099712; 768: attention 2362368, feed-forward 4722432; 1024: attention 4198400, feed-forward 8393728. Worked example (d_model 512, 6 layers): total 18914304.



total (parameters of the stack) against d_model (model (embedding) dimension) from 1 to 1024 on a logarithmic axis. One curve for each value of n_layers (number of layers): 6, 3, 12. Other inputs as in the worked example: n_heads = 8, d_ff = 2048. 9 points per curve computed by data-science-ml-ai999x; marked: the worked example, d_model = 512, total = 1.8914e+07.

```
data_science_ml_ai999x.solve('ml_transformer_parameter_count', d_model=512, n_heads=8, d_ff=2048,
n_layers=6)
# -> attention = 1.05062e+06, feed_forward = 2.09971e+06, layer_norms = 2048, per_layer =
3.15238e+06, total = 1.89143e+07   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_transformer_parameter_count', d_model=768, n_heads=12, d_ff=3072,
n_layers=12)
# -> attention = 2.36237e+06, feed_forward = 4.72243e+06, layer_norms = 3072, per_layer =
7.08787e+06, total = 8.50545e+07   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transformer parameter count: d_model = 512, n_heads = 8, d_ff = 2048, n_layers = 6	SOLVED	attention = 1.05062e+06, feed_forward = 2.09971e+06, layer_norms = 2048, per_layer = 3.15238e+06, total = 1.89143e+07
Transformer parameter count: d_model = 768, n_heads = 12, d_ff = 3072, n_layers = 12	SOLVED	attention = 2.36237e+06, feed_forward = 4.72243e+06, layer_norms = 3072, per_layer = 7.08787e+06, total = 8.50545e+07
Transformer parameter count: n_heads = 8, d_ff = 2048, n_layers = 6	NEEDS_INPUT	ml_transformer_parameter_count needs d_model (model (embedding) dimension). Example: solve('ml_transformer_parameter_cou
Transformer parameter count: d_model = 512, n_heads = 8, d_ff = 2048, n_layers = 6, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Transformer parameter count: d_model = 100, n_heads = 8, d_ff = 400, n_layers = 2	INVALID_INPUT	Parameter count of a transformer encoder: d_model must be divisible by n_heads.

Parameter count of an LSTM, GRU or simple RNN, `ml_rnn_parameter_count`

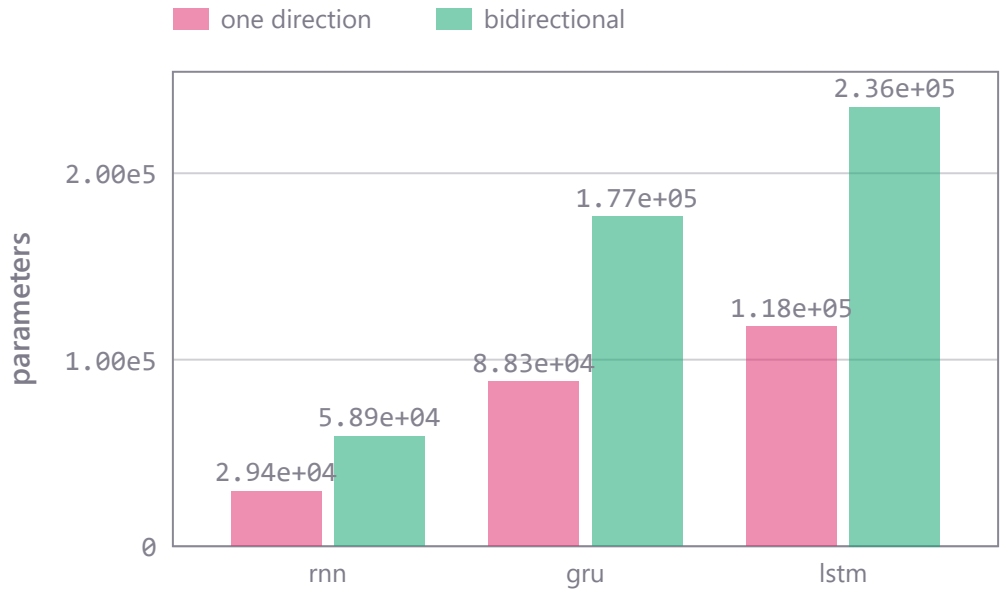
The trainable parameters of each layer and of a stacked recurrent network (LSTM: 4 gates, GRU: 3, simple RNN: 1), with two bias vectors per gate as in PyTorch, optionally bidirectional.

$$P = G\,h\,(n_{\text{in}} + h) + 2Gh$$

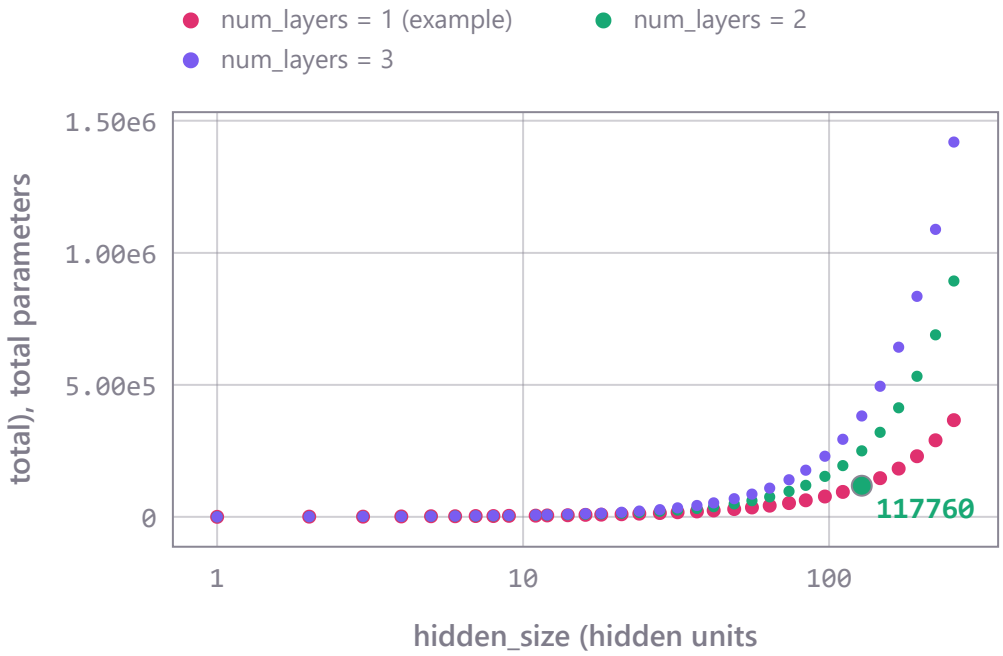
input	meaning	condition
`cell`	cell type: lstm, gru or rnn	
`input_size`	input features	
`hidden_size`	hidden units	
`num_layers`	stacked layers	
`bidirectional`	bidirectional (yes or no)	

output	meaning
`per_layer`	parameters of each layer (both directions)
`total`	total parameters

- **Method:** G h $(n_i n + h) + 2$ G h per direction; layers above the first take h (x 2 if bidirectional) inputs
- **Assumes:** positive whole-number sizes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Parameters of one recurrent layer with 100 inputs and 128 hidden units for each cell type, one run each: rnn 29440 (58880 bidirectional), gru 88320 (176640 bidirectional), lstm 117760 (235520 bidirectional), 1, 3 and 4 gate blocks.



total (total parameters) against hidden_size (hidden units) from 1 to 256 on a logarithmic axis. One curve for each value of num_layers (stacked layers): 1, 2, 3. Other inputs as in the worked example: cell = lstm, input_size = 100, bidirectional = False. 33 points per curve computed by data-science-ml-ai999x; marked: the worked example, hidden_size = 128, total = 117760.

```
data_science_ml_ai999x.solve('ml_rnn_parameter_count', cell='lstm', input_size=100, hidden_size=128,
num_layers=1, bidirectional=False)
# -> per_layer = [117760], total = 117760   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_rnn_parameter_count', cell='gru', input_size=32, hidden_size=64,
num_layers=2, bidirectional=True)
# -> per_layer = [37632, 74496], total = 112128   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
RNN parameter count: cell = lstm, input_size = 100, hidden_size = 128, num_layers = 1, bidirectional = no	SOLVED	per_layer = [117760], total = 117760
RNN parameter count: cell = gru, input_size = 32, hidden_size = 64, num_layers = 2, bidirectional = yes	SOLVED	per_layer = [37632, 74496], total = 112128
RNN parameter count: input_size = 100, hidden_size = 128, num_layers = 1, bidirectional = no	NEEDS_INPUT	ml_rnn_parameter_count needs cell (cell type: lstm, gru or rnn). Example: solve('ml_rnn_parameter_count', cell='lstm', i
RNN parameter count: cell = lstm, input_size = 100, hidden_size = 128, num_layers = 1, bidirectional = no, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Receptive field of stacked convolutions, `ml_receptive_field`

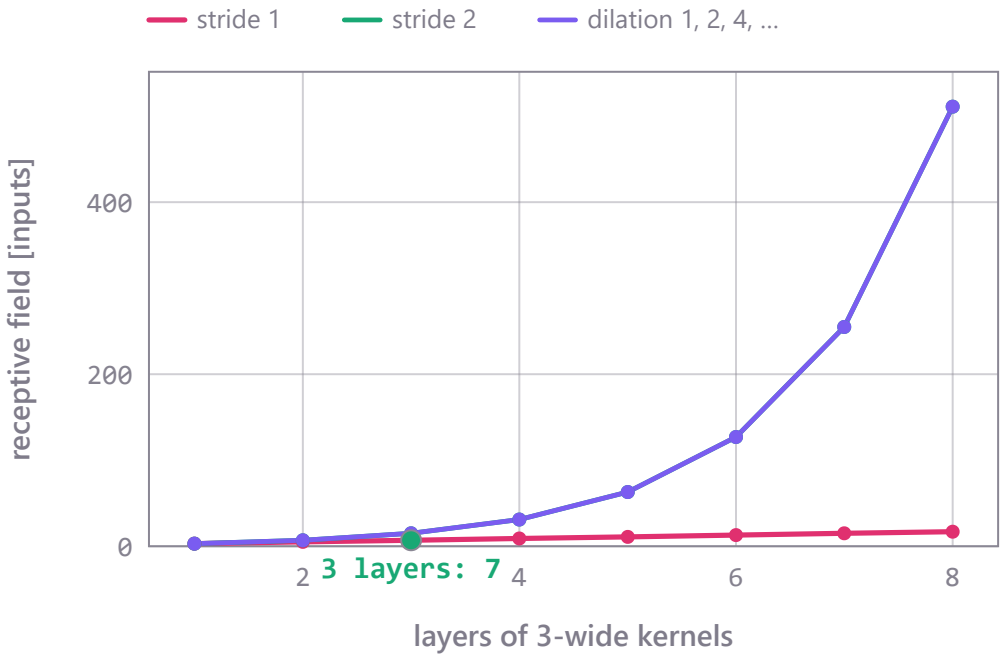
The receptive field (how many input positions one output depends on, end to end) and the jump (input distance between adjacent outputs) of a stack of convolution or pooling layers with given kernel sizes, strides and dilations.

$$r = 1 + \sum_{l} (k_l - 1)\, d_l \prod_{i<l} s_i$$

input	meaning	condition
`kernels`	kernel size of each layer (whole numbers)	
`strides`	stride of each layer	
`dilations`	dilation of each layer (1 = ordinary)	

output	meaning
`receptive_field`	1 + sum $(k_l - 1)$ d_l prod_{$i < l$} s_i
`jump`	product of the strides

- **Method:** layer-by-layer recursion r += $(k - 1)$ d j, j *= s
- **Assumes:** one kernel, stride and dilation per layer; at most 20 layers
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Receptive field of a stack of 1 to 8 convolutions with 3-wide kernels: stride 1 (grows by 2 per layer), stride 2 and dilations doubling per layer (both grow geometrically); one run per point. Marked: the worked example, three stride-1 layers, receptive field 7 (jump 1).

```
data_science_ml_ai999x.solve('ml_receptive_field', kernels=[3, 3, 3], strides=[1, 1, 1], dilations=
[1, 1, 1])
# -> receptive_field = 7, jump = 1   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_receptive_field', kernels=[7, 3, 3, 3], strides=[2, 2, 1, 1],
dilations=[1, 1, 1, 1])
# -> receptive_field = 27, jump = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Receptive field: kernels = [3, 3, 3], strides = [1, 1, 1], dilations = [1, 1, 1]	SOLVED	receptive_field = 7, jump = 1
Receptive field: kernels = [7, 3, 3, 3], strides = [2, 2, 1, 1], dilations = [1, 1, 1, 1]	SOLVED	receptive_field = 27, jump = 4
Receptive field: strides = [1, 1, 1], dilations = [1, 1, 1]	NEEDS_INPUT	ml_receptive_field needs kernels (kernel size of each layer (whole numbers)). Example: solve('ml_receptive_field', kerne
Receptive field: kernels = [3, 3, 3], strides = [1, 1, 1], dilations = [1, 1, 1], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Precision@k and recall@k of a ranking, `ml_precision_recall_at_k`

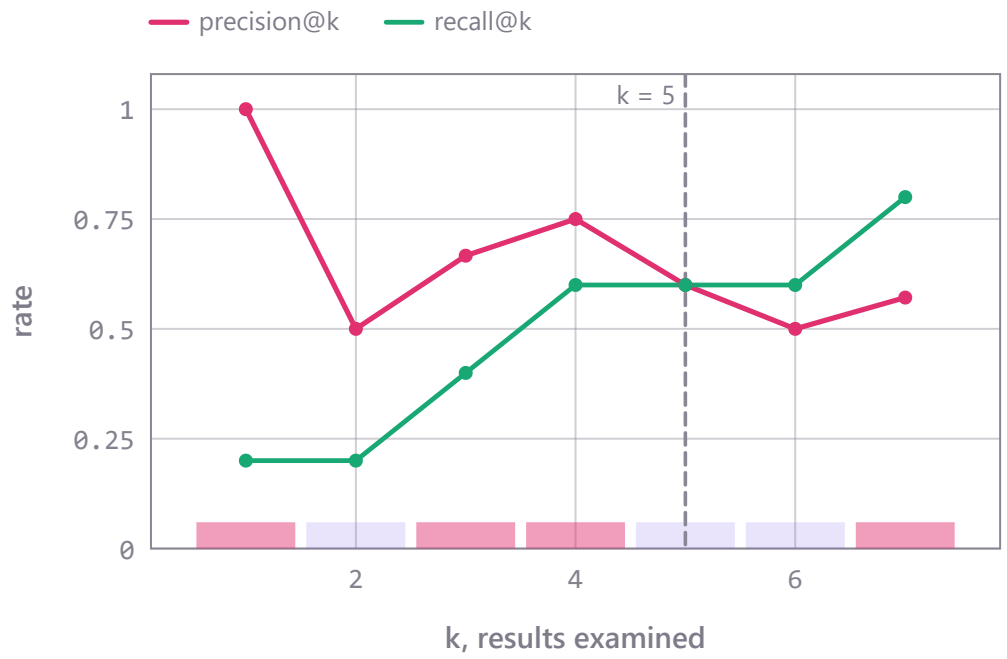
Precision at k and recall at k of a ranked list (relevance $0/1$ in rank order), with the number of relevant items in the top k and the hit indicator.

$$P@k = \frac{|\text{rel} \cap \text{top}_k|}{k}, \quad R@k = \frac{|\text{rel} \cap \text{top}_k|}{|\text{rel}|}$$

input	meaning	condition
`rel`	relevance of each ranked item, in rank order (0 or 1)	
`n_relevant`	total number of relevant items for the query	
`k`	cut-off rank	

output	meaning
`precision_at_k`	relevant in top k / k
`recall_at_k`	relevant in top k / n_relevant
`hits`	relevant items in the top k
`hit_rate`	1 if any relevant item is in the top k

- **Method:** exact counts
- **Assumes:** relevance 0 or 1; 1 <= k <= list length; n_relevant >= relevant items in the list
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Precision and recall among the top k of the ranked relevance list [1, 0, 1, 1, 0, 0, 1] (5 relevant items in all), one run per k; the strip at the bottom marks relevant results (dark). Worked example k = 5: precision 0.6, recall 0.6, 3 hits.

```
data_science_ml_ai999x.solve('ml_precision_recall_at_k', rel=[1, 0, 1, 1, 0, 0, 1], n_relevant=5,
k=5)
# -> precision_at_k = 0.6, recall_at_k = 0.6, hits = 3, hit_rate = 1   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_precision_recall_at_k', rel=[0, 0, 1, 0], n_relevant=2, k=2)
# -> precision_at_k = 0, recall_at_k = 0, hits = 0, hit_rate = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Precision at k: rel = [1, 0, 1, 1, 0, 0, 1], n_relevant = 5, k = 5	SOLVED	precision_at_k = 0.6, recall_at_k = 0.6, hits = 3, hit_rate = 1
Precision at k: rel = [0, 0, 1, 0], n_relevant = 2, k = 2	SOLVED	precision_at_k = 0, recall_at_k = 0, hits = 0, hit_rate = 0
Precision at k: n_relevant = 5, k = 5	NEEDS_INPUT	ml_precision_recall_at_k needs rel (relevance of each ranked item, in rank order (0 or 1)). Example: solve('ml_precision
Precision at k: rel = [1, 0, 1, 1, 0, 0, 1], n_relevant = 5, k = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Precision at k: rel = [1, 0, 1], n_relevant = 1, k = 2	INVALID_INPUT	Precision@k and recall@k of a ranking: n_relevant must be at least the number of relevant items in the list and at least

Mean average precision (MAP) of rankings, `ml_mean_average_precision`

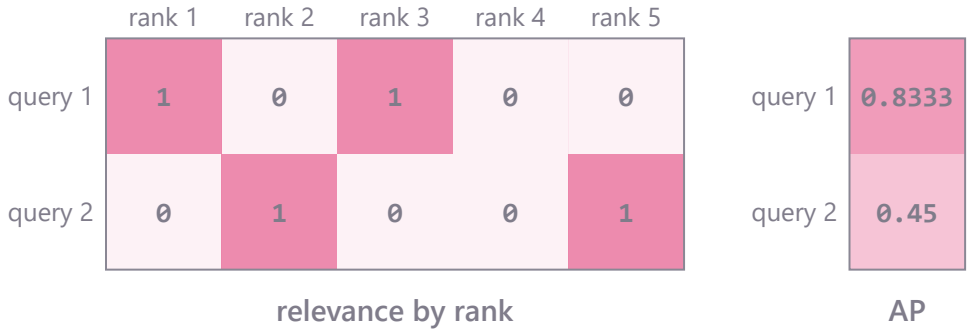
The average precision of each query's ranked list (mean of precision at the rank of each relevant item, over the relevant items in the list) and their mean, MAP.

$$\mathrm{AP} = \frac{1}{|\mathrm{rel}|} \sum_{k:\ \mathrm{rel}_k=1} P@k, \quad \mathrm{MAP} = \frac{1}{Q} \sum_{q} \mathrm{AP}_q$$

input	meaning	condition
`R`	relevance (0 or 1) in rank order: one row per query, rows of equal length	

output	meaning
`average_precision`	AP of each query
`map`	mean of the APs

- **Method:** exact fractions
- **Assumes:** relevance 0 or 1; every query has a relevant item
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated relevance of each query's ranked results R = [[1, 0, 1, 0, 0], [0, 1, 0, 0, 1]] and the returned average precision of each query (0.833333, 0.45); mean average precision 0.641667.

```
data_science_ml_ai999x.solve('ml_mean_average_precision', R=[[1, 0, 1, 0, 0], [0, 1, 0, 0, 1]])
# -> average_precision = [0.833333333333, 0.45], map = 0.641667   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_mean_average_precision', R=[[0, 0, 1, 1, 0, 1]])
# -> average_precision = [0.444444444444], map = 0.444444   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mean average precision: R = [[1, 0, 1, 0, 0], [0, 1, 0, 0, 1]]	SOLVED	average_precision = [0.833333333333, 0.45], map = 0.641667
Mean average precision: R = [[0, 0, 1, 1, 0, 1]]	SOLVED	average_precision = [0.444444444444], map = 0.444444
Mean average precision:	NEEDS_INPUT	ml_mean_average_precision needs R (relevance (0 or 1) in rank order: one row per query, rows of equal length). Example:
Mean average precision: R = [[1, 0, 1, 0, 0], [0, 1, 0, 0, 1]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Mean average precision: R = [[0, 0, 0], [1, 0, 1]]	INVALID_INPUT	Mean average precision (MAP) of rankings: a query has no relevant item (its average precision is undefined).

NDCG@k (normalised discounted cumulative gain), `ml_ndcg_at_k`

DCG@k of a ranked list with graded relevance (linear gain rel or exponential gain 2^rel - 1, discount $1/log2$ (rank + 1)), the ideal DCG@k of the same items sorted by relevance, and NDCG@k = DCG/IDCG.

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g_i}{\log_2(i+1)}, \quad \mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}$$

input	meaning	condition
`rel`	relevance grade of each item in rank order (>= 0)	
`k`	cut-off rank	
`gain`	gain: linear (rel) or exponential (2^rel - 1)	

output	meaning
`dcg`	DCG@k
`idcg`	ideal DCG@k
`ndcg`	DCG@k / IDCG@k

- **Method:** discounted sums with fsum
- **Assumes:** grades >= 0; 1 <= k <= list length; at least one relevant item
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



NDCG at each cut-off k for the ranked relevance grades [3, 2, 3, 0, 1, 2] (labelled along the bottom), linear and exponential gain, one run per point. Worked example (k = 6, linear gain): DCG 6.86113, ideal 7.141, NDCG 0.960808; the 3 at rank 3 placed below a 2 costs a little.

```
data_science_ml_ai999x.solve('ml_ndcg_at_k', rel=[3, 2, 3, 0, 1, 2], k=6, gain='linear')
# -> dcg = 6.86113, idcg = 7.141, ndcg = 0.960808   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_ndcg_at_k', rel=[3, 2, 3, 0, 1, 2], k=3, gain='exponential')
# -> dcg = 12.3928, idcg = 12.9165, ndcg = 0.959454   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
NDCG at k: rel = [3, 2, 3, 0, 1, 2], k = 6, gain = linear	SOLVED	dcg = 6.86113, idcg = 7.141, ndcg = 0.960808
NDCG at k: rel = [3, 2, 3, 0, 1, 2], k = 3, gain = exponential	SOLVED	dcg = 12.3928, idcg = 12.9165, ndcg = 0.959454
NDCG at k: k = 6, gain = linear	NEEDS_INPUT	ml_ndcg_at_k needs rel (relevance grade of each item in rank order (>= 0)). Example: solve('ml_ndcg_at_k', rel=[3, 2, 3,
NDCG at k: rel = [3, 2, 3, 0, 1, 2], k = 6, gain = linear, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
NDCG at k: rel = [0, 0, 0], k = 2, gain = linear	INVALID_INPUT	NDCG@k (normalised discounted cumulative gain): NDCG is undefined: no relevant item.

Mean reciprocal rank (MRR), `ml_mean_reciprocal_rank`

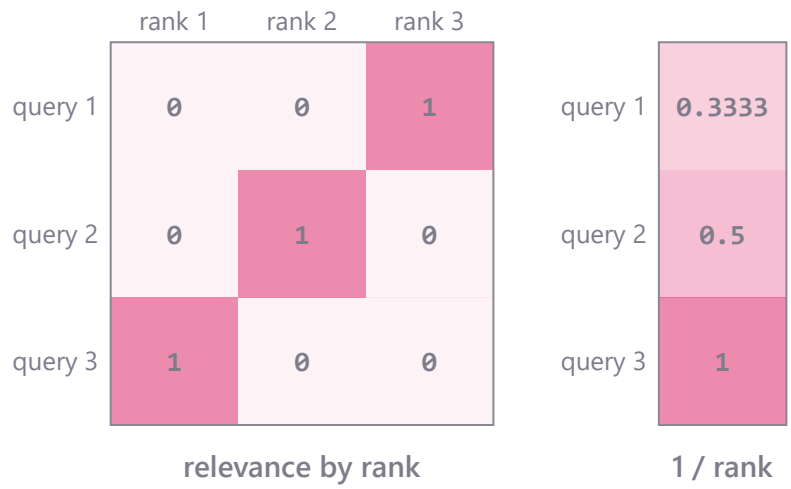
The reciprocal rank of the first relevant item of each query's ranked list (0 when none is relevant) and their mean, the MRR.

$$\mathrm{MRR} = \frac{1}{Q}\sum_{q}\frac{1}{\mathrm{rank}_q}$$

input	meaning	condition
`R`	relevance (0 or 1) in rank order: one row per query	

output	meaning
`reciprocal_ranks`	1 / rank of the first relevant item of each query
`mrr`	mean reciprocal rank

- **Method:** exact fractions
- **Assumes:** relevance 0 or 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated relevance of each query's ranked results R = [[0, 0, 1], [0, 1, 0], [1, 0, 0]] (1 = relevant) and the returned reciprocal rank of the first relevant result per query (0.333333, 0.5, 1); mean reciprocal rank 0.611111.

```
data_science_ml_ai999x.solve('ml_mean_reciprocal_rank', R=[[0, 0, 1], [0, 1, 0], [1, 0, 0]])
# -> reciprocal_ranks = [0.333333333333, 0.5, 1.0], mrr = 0.611111   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_mean_reciprocal_rank', R=[[0, 0, 0, 1], [0, 0, 0, 0]])
# -> reciprocal_ranks = [0.25, 0.0], mrr = 0.125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mean reciprocal rank: R = [[0, 0, 1], [0, 1, 0], [1, 0, 0]]	SOLVED	reciprocal_ranks = [0.333333333333, 0.5, 1.0], mrr = 0.611111
Mean reciprocal rank: R = [[0, 0, 0, 1], [0, 0, 0, 0]]	SOLVED	reciprocal_ranks = [0.25, 0.0], mrr = 0.125
Mean reciprocal rank:	NEEDS_INPUT	ml_mean_reciprocal_rank needs R (relevance (0 or 1) in rank order: one row per query). Example: solve('ml_mean_reciproca
Mean reciprocal rank: R = [[0, 0, 1], [0, 1, 0], [1, 0, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Confusion counts, precision and recall at a score threshold, `ml_threshold_metrics`

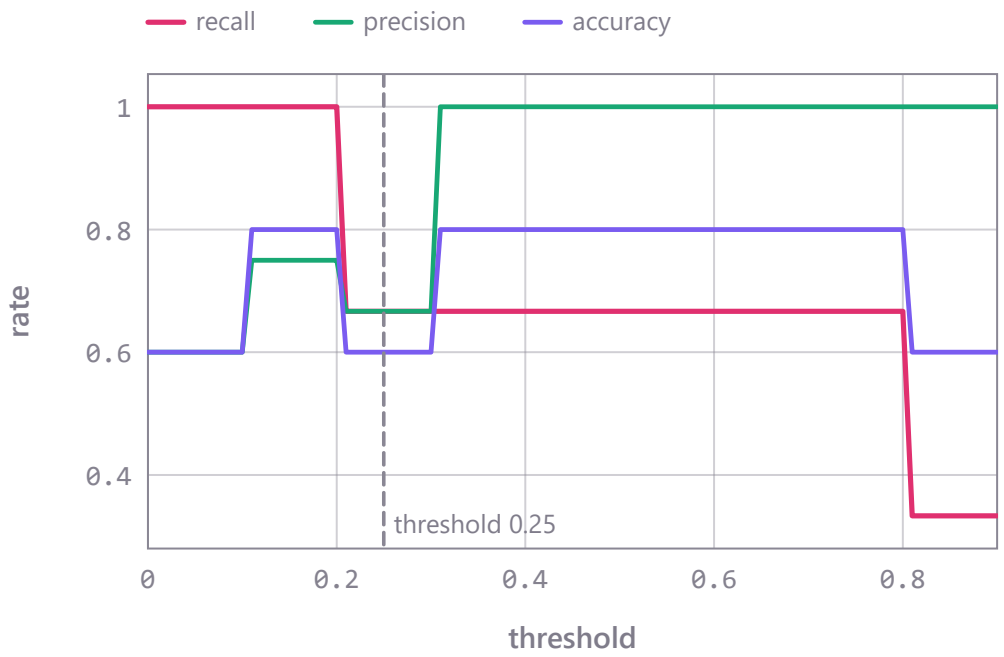
Classify by a score threshold (score >= threshold is positive) and give the confusion counts, precision, recall and accuracy.

$$\hat{y}_i = \mathbb{1}(s_i \geq t)$$

input	meaning	condition
`y_true`	true labels (0 or 1)	
`scores`	classifier scores	
`threshold`	decision threshold (a score at or above it is positive)	

output	meaning
`TP`	true positives
`FP`	false positives
`FN`	false negatives
`TN`	true negatives
`precision`	TP / (TP + FP)
`recall`	TP / (TP + FN)
`accuracy`	(TP + TN) / n

- **Method:** threshold the scores, count, exact fractions
- **Assumes:** labels 0 or 1; at least one score reaches the threshold; at least one positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Recall, precision and accuracy of labels [0, 1, 1, 0, 1] with scores [0.1, 0.9, 0.8, 0.3, 0.2] as the threshold runs from 0 to 1 (101 runs; a threshold where nothing is called positive leaves precision undrawn). Marked: threshold 0.25: TP 2, FP 1, FN 1, TN 1, precision 0.666667, recall 0.666667, accuracy 0.6.

```
data_science_ml_ai999x.solve('ml_threshold_metrics', y_true=[0, 1, 1, 0, 1], scores=[0.1, 0.9, 0.8, 0.3, 0.2], threshold=0.25)
# -> TP = 2, FP = 1, FN = 1, TN = 1, precision = 0.666667, recall = 0.666667, accuracy = 0.6   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_threshold_metrics', y_true=[1, 0, 1, 0, 0, 1], scores=[3, 1, 2, 2, 0, 4], threshold=2)
# -> TP = 3, FP = 1, FN = 0, TN = 2, precision = 0.75, recall = 1, accuracy = 0.833333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Metrics at a threshold: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2], threshold = 0.25	SOLVED	TP = 2, FP = 1, FN = 1, TN = 1, precision = 0.666667, recall = 0.666667, accuracy = 0.6
Metrics at a threshold: y_true = [1, 0, 1, 0, 0, 1], scores = [3, 1, 2, 2, 0, 4], threshold = 2	SOLVED	TP = 3, FP = 1, FN = 0, TN = 2, precision = 0.75, recall = 1, accuracy = 0.833333
Metrics at a threshold: scores = [0.1, 0.9, 0.8, 0.3, 0.2], threshold = 0.25	NEEDS_INPUT	ml_threshold_metrics needs y_true (true labels (0 or 1)). Example: solve('ml_threshold_metrics', y_true=[0, 1, 1, 0, 1],
Metrics at a threshold: y_true = [0, 1, 1, 0, 1], scores = [0.1, 0.9, 0.8, 0.3, 0.2], threshold = 0.25, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Softmax with temperature, `ml_softmax_temperature`

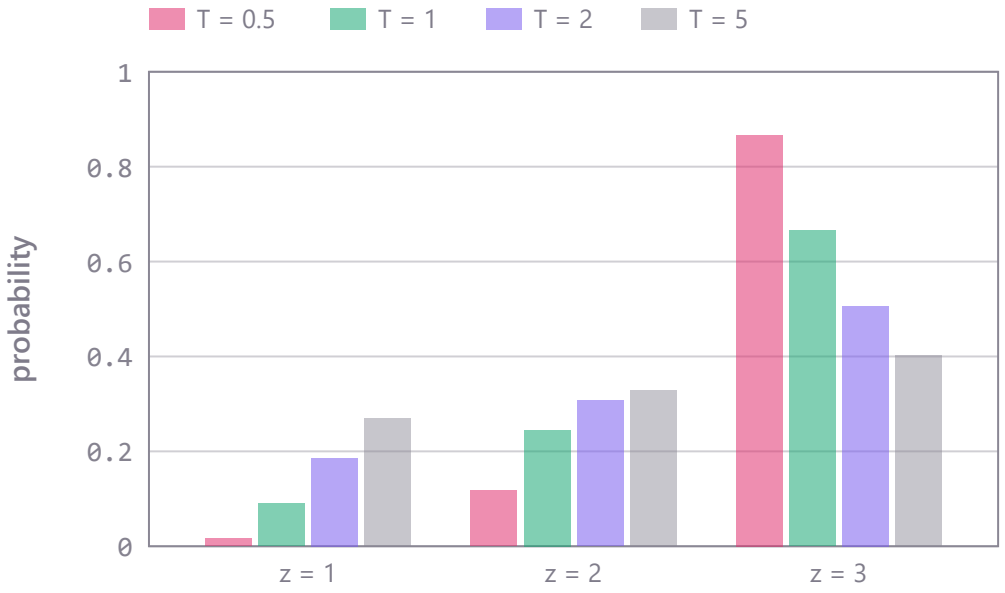
The softmax of logits divided by a temperature T (T > 1 flattens, $T < 1$ sharpens the distribution) and the entropy of the result in nats.

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

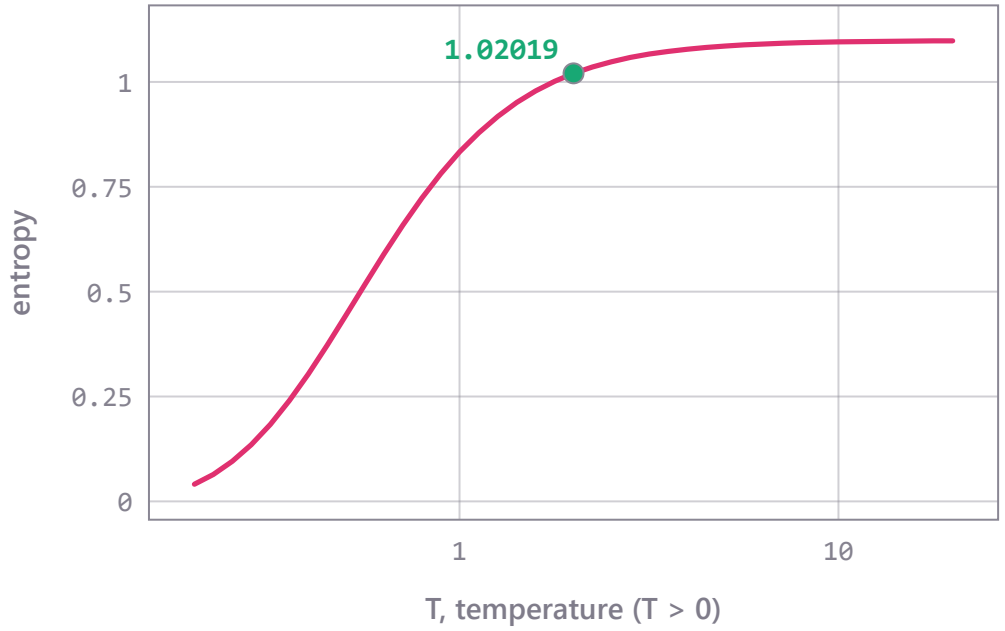
input	meaning	condition
`z`	logits	
`T`	temperature $(T > 0)$	

output	meaning
`probabilities`	softmax(z / T)
`entropy`	entropy of the distribution in nats

- **Method:** stable softmax of z / T
- **Assumes:** $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Softmax of z = [1, 2, 3] at temperatures T = 0.5, 1, 2, 5, one run each; returned entropies T = 0.5: 0.441057, T = 1: 0.832396, T = 2: 1.02019, T = 5: 1.08541. Low temperature sharpens the distribution, high temperature flattens it.



entropy (entropy of the distribution in nats) against T (temperature (T > 0)) from 0.2 to 20 on a logarithmic axis. Other inputs as in the worked example: z = [1, 2, 3]. 41 points computed by data-science-ml-ai999x; marked: the worked example, T = 2, entropy = 1.02019.

```
data_science_ml_ai999x.solve('ml_softmax_temperature', z=[1, 2, 3], T=2)
# -> probabilities = [0.186323723226, 0.307195885718, 0.506480391056], entropy = 1.02019   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_softmax_temperature', z=[2, 0, -1, 0.5], T=0.5)
# -> probabilities = [0.934071845901, 0.0171081226257, 0.00231533262119, 0.0465046988524], entropy =
0.290041   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Softmax with temperature: z = [1, 2, 3], T = 2	SOLVED	probabilities = [0.186323723226, 0.307195885718, 0.506480391056], entropy = 1.02019
Softmax with temperature: z = [2, 0, -1, 0.5], T = 0.5	SOLVED	probabilities = [0.934071845901, 0.0171081226257, 0.00231533262119, 0.0465046988524], entropy = 0.290041
Softmax with temperature: T = 2	NEEDS_INPUT	ml_softmax_temperature needs z (logits). Example: solve('ml_softmax_temperature', z=[1, 2, 3], T=2).
Softmax with temperature: z = [1, 2, 3], T = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Softmax with temperature: z = [1, 2], T = 0	INVALID_INPUT	Softmax with temperature: the temperature T must be positive.

Polynomial kernel, `ml_polynomial_kernel`

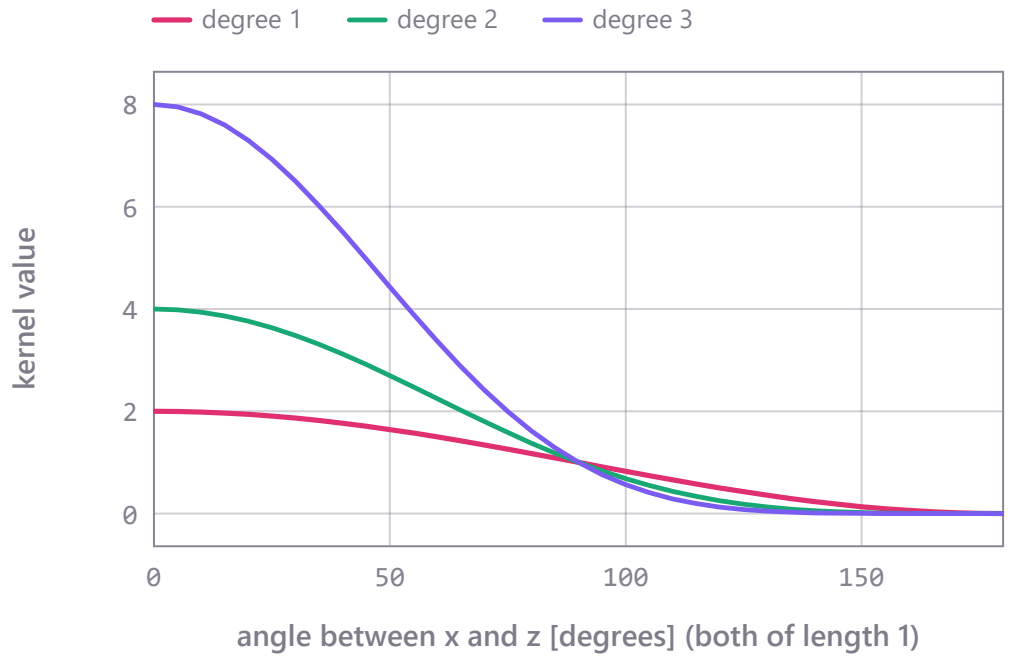
The polynomial kernel (gamma x.z + coef0)^degree between two vectors, as used by kernel SVMs.

$$k(x, z) = (\gamma\, x^\top z + c_0)^d$$

input	meaning	condition
`x`	first vector	
`z`	second vector	
`gamma`	scale of the dot product	
`coef0`	constant term	
`degree`	degree (1 to 10)	

output	meaning
`kernel`	(gamma x.z + coef0)^degree

- **Method:** exact rational arithmetic
- **Assumes:** degree 1 to 10; equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Polynomial kernel (γ·x·z + c_0)^d with γ = 1, c_0 = 1 between two unit vectors at an angle from 0° to 180° (chosen here; 37 runs per degree, d = 1, 2, 3): higher degree magnifies similarity. Worked example x = [1, 2], z = [3, 4], degree 2: kernel 144.

```
data_science_ml_ai999x.solve('ml_polynomial_kernel', x=[1, 2], z=[3, 4], gamma=1, coef0=1, degree=2)
# -> kernel = 144   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_polynomial_kernel', x=[1, -1, 0.5], z=[2, 0, 1], gamma=0.5,
coef0=2, degree=3)
# -> kernel = 34.3281   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Polynomial kernel: x = [1, 2], z = [3, 4], gamma = 1, coef0 = 1, degree = 2	SOLVED	kernel = 144
Polynomial kernel: x = [1, -1, 0.5], z = [2, 0, 1], gamma = 0.5, coef0 = 2, degree = 3	SOLVED	kernel = 34.3281
Polynomial kernel: z = [3, 4], gamma = 1, coef0 = 1, degree = 2	NEEDS_INPUT	ml_polynomial_kernel needs x (first vector). Example: solve('ml_polynomial_kernel', x=[1, 2], z=[3, 4], gamma=1, coef0=1
Polynomial kernel: x = [1, 2], z = [3, 4], gamma = 1, coef0 = 1, degree = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Triplet margin loss, `ml_triplet_loss`

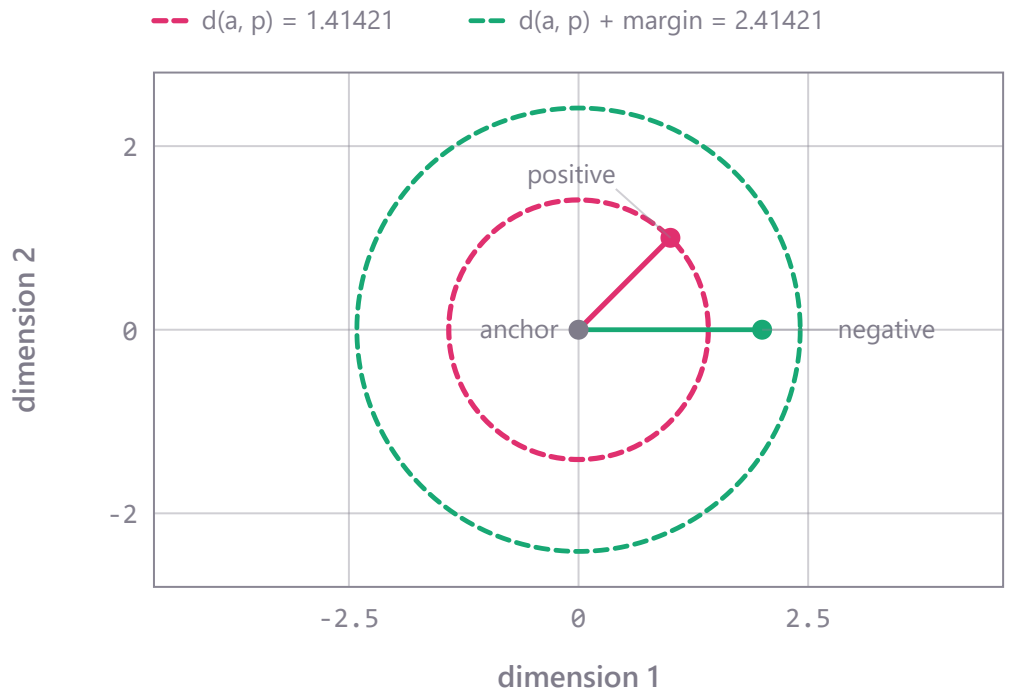
The triplet margin loss max(0, d(a, p) - d(a, n) + margin) of an anchor, a positive and a negative embedding, with both Euclidean distances.

$$L = \max\left(0,\ \|a - p\|_2 - \|a - n\|_2 + m\right)$$

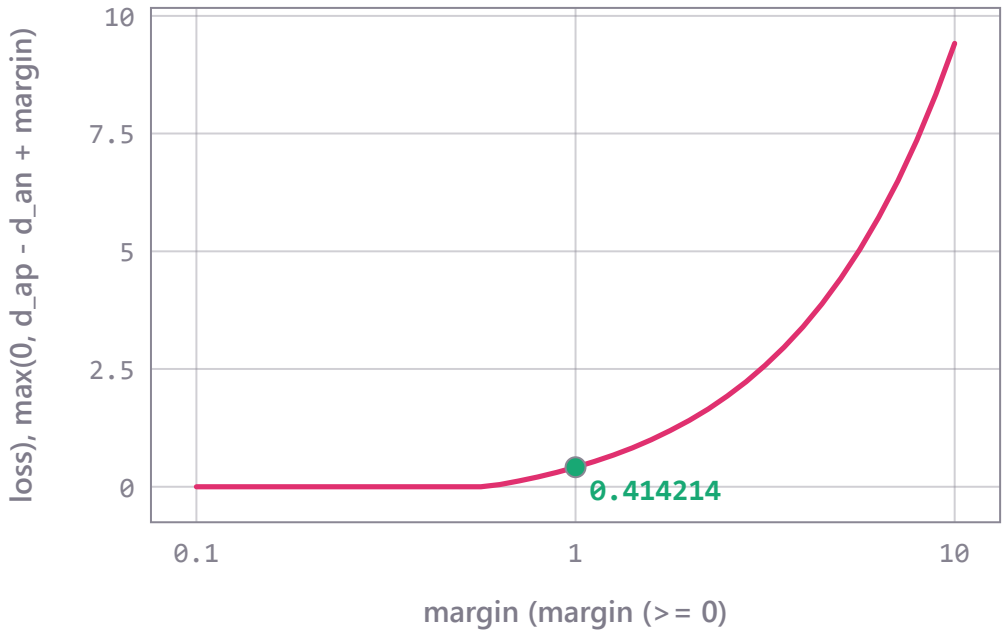
input	meaning	condition
`anchor`	anchor embedding	
`positive`	embedding of the same identity	
`negative`	embedding of a different identity	
`margin`	margin (>= 0)	

output	meaning
`loss`	max(0, d_ap - d_an + margin)
`d_ap`	anchor-positive distance
`d_an`	anchor-negative distance

- **Method:** Euclidean distances with fsum
- **Assumes:** equal lengths; margin >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Anchor [0, 0], positive [1, 1] and negative [2, 0] with margin 1: the returned distances d(a, p) = 1.41421 and d(a, n) = 2. The loss is zero only if the negative lies outside the outer circle (radius d(a, p) + margin, sum shown here); here it lies inside, and the returned loss is 0.414214.



loss (max(0, d_ap - d_an + margin)) against margin (margin (>= 0)) from 0.1 to 10 on a logarithmic axis. Other inputs as in the worked example: anchor = [0, 0], positive = [1, 1], negative = [2, 0]. 41 points computed by data-science-ml-ai999x; marked: the worked example, margin = 1, loss = 0.414214.

```
data_science_ml_ai999x.solve('ml_triplet_loss', anchor=[0, 0], positive=[1, 1], negative=[2, 0],
margin=1)
# -> loss = 0.414214, d_ap = 1.41421, d_an = 2   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_triplet_loss', anchor=[1, 2, 3], positive=[1.5, 2, 2], negative=[0,
3, 3.5], margin=0.5)
# -> loss = 0.118034, d_ap = 1.11803, d_an = 1.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Triplet loss: anchor = [0, 0], positive = [1, 1], negative = [2, 0], margin = 1	SOLVED	loss = 0.414214, d_ap = 1.41421, d_an = 2
Triplet loss: anchor = [1, 2, 3], positive = [1.5, 2, 2], negative = [0, 3, 3.5], margin = 0.5	SOLVED	loss = 0.118034, d_ap = 1.11803, d_an = 1.5
Triplet loss: positive = [1, 1], negative = [2, 0], margin = 1	NEEDS_INPUT	ml_triplet_loss needs anchor (anchor embedding). Example: solve('ml_triplet_loss', anchor=[0, 0], positive=[1, 1], negat
Triplet loss: anchor = [0, 0], positive = [1, 1], negative = [2, 0], margin = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Node impurity from class counts (entropy, Gini, misclassification), `ml_impurity_from_counts`

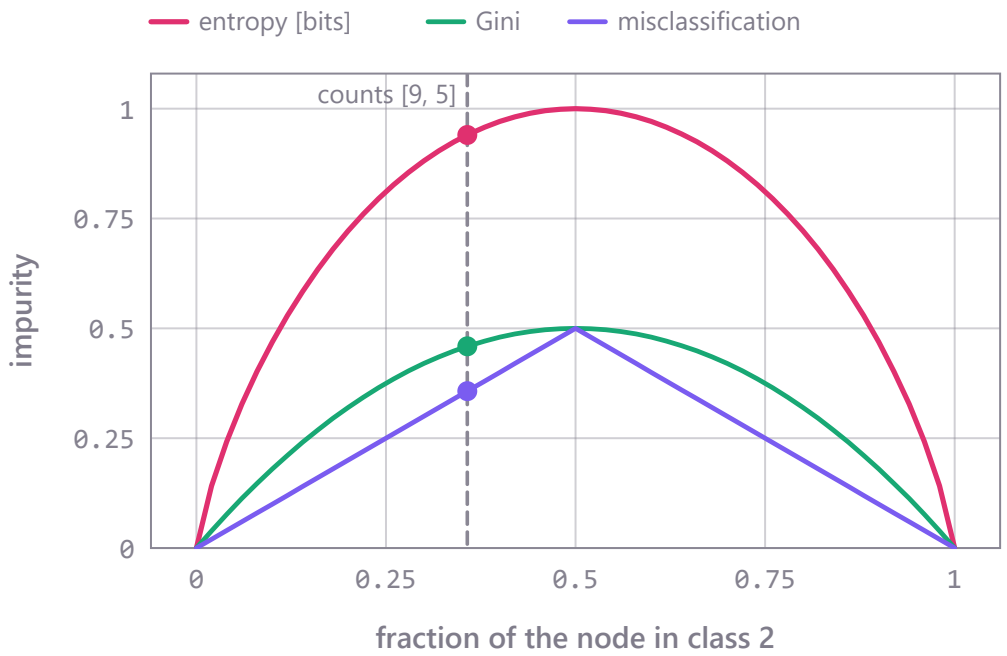
The impurity of a decision-tree node from its class counts: entropy in bits, Gini impurity and misclassification error.

$$G = 1 - \sum_k p_k^2, \quad H = -\sum_k p_k \log_2 p_k$$

input	meaning	condition
`counts`	number of samples of each class in the node (whole numbers)	

output	meaning
`entropy_bits`	entropy in bits
`gini`	1 - sum p_k^2
`misclassification`	1 - max p_k

- **Method:** class proportions; Gini and misclassification exactly as fractions
- **Assumes:** counts >= 0 with a positive total
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The three impurities of a two-class node with counts (100 − k, k), k = 0 ... 100 (51 runs each). Marked: the worked example counts [9, 5] (fraction 0.357143): entropy 0.940286 bits, Gini 0.459184, misclassification 0.357143.

```
data_science_ml_ai999x.solve('ml_impurity_from_counts', counts=[9, 5])
# -> entropy_bits = 0.940286, gini = 0.459184, misclassification = 0.357143   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_impurity_from_counts', counts=[10, 10, 0])
# -> entropy_bits = 1, gini = 0.5, misclassification = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Impurity from class counts: counts = [9, 5]	SOLVED	entropy_bits = 0.940286, gini = 0.459184, misclassification = 0.357143
Impurity from class counts: counts = [10, 10,	SOLVED	entropy_bits = 1, gini = 0.5, misclassification = 0.5

question	outcome	result
0]		
Impurity from class counts:	NEEDS_INPUT	ml_impurity_from_counts needs counts (number of samples of each class in the node (whole numbers)). Example: solve('ml_i
Impurity from class counts: counts = [9, 5], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(
Impurity from class counts: counts = [0, 0]	INVALID_INPUT	Node impurity from class counts (entropy, Gini, misclassification): counts must be whole numbers >= 0 with a positive to

Robust scaling (median and interquartile range), `ml_robust_scaler`

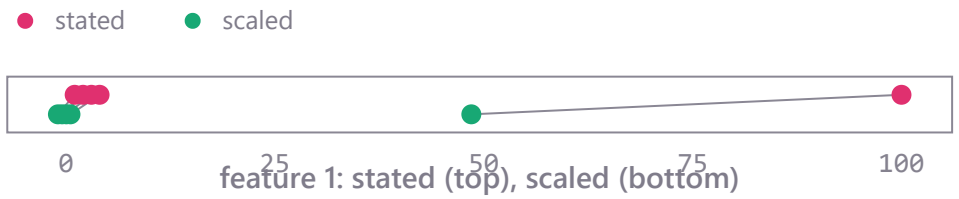
Scale each feature by subtracting its median and dividing by its interquartile range (quartiles by linear interpolation), which resists outliers; a column with zero IQR is only centred.

$$x' = \frac{x - \text{median}}{Q_3 - Q_1}$$

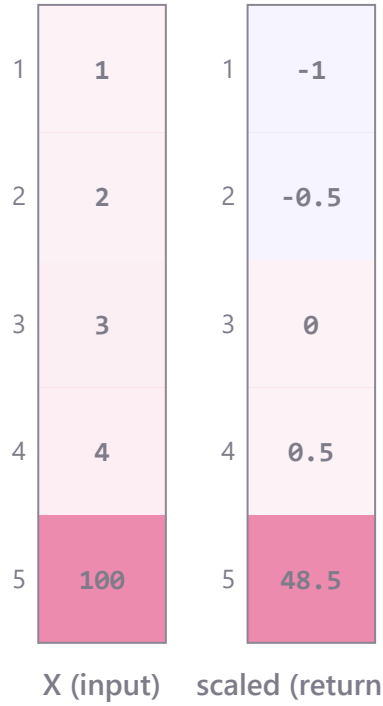
input	meaning	condition
`X`	data: one row per sample, one column per feature	

output	meaning
`medians`	column medians
`iqr`	column interquartile ranges $Q_3 - Q_1$
`scaled`	(x - median) / IQR

- **Method:** quartiles at position $(n-1)$ p with linear interpolation; exact fractions
- **Assumes:** zero IQR: centred only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Each feature of X = [[1], [2], [3], [4], [100]] before (top dots) and after (bottom dots) subtracting the median and dividing by the interquartile range, the scaled values as returned; also returned: iqr ['2'], medians ['3'].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); scaled (returned).

```
data_science_ml_ai999x.solve('ml_robust_scaler', X=[[1], [2], [3], [4], [100]])
# -> medians = [3.0], iqr = [2.0], scaled = [[-1.0], [-0.5], [0.0], [0.5], [48.5]]   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_robust_scaler', X=[[1, 5], [4, 2], [2, 8], [8, 1], [3, 3], [5, 9]])
# -> medians = [3.5, 4.0], iqr = [2.5, 5.0], scaled = [[-1.0, 0.2], [0.2, -0.4], [-0.6, 0.8], [1.8,
-0.6], [-0.2, -0.2], [0.6, 1.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Robust scaler: X = [[1], [2], [3], [4], [100]]	SOLVED	medians = [3.0], iqr = [2.0], scaled = [[-1.0], [-0.5], [0.0], [0.5], [48.5]]
Robust scaler: X = [[1, 5], [4, 2], [2, 8], [8, 1], [3, 3], [5, 9]]	SOLVED	medians = [3.5, 4.0], iqr = [2.5, 5.0], scaled = [[-1.0, 0.2], [0.2, -0.4], [-0.6, 0.8], [1.8, -0.6], [-0.2, -0.2], [0.6, 1.0]]
Robust scaler:	NEEDS_INPUT	ml_robust_scaler needs X (data: one row per sample, one column per feature). Example: solve('ml_robust_scaler', X=[[1],
Robust scaler: X = [[1], [2], [3], [4], [100]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(

Best regression-tree split (squared error), `ml_decision_stump_regression`

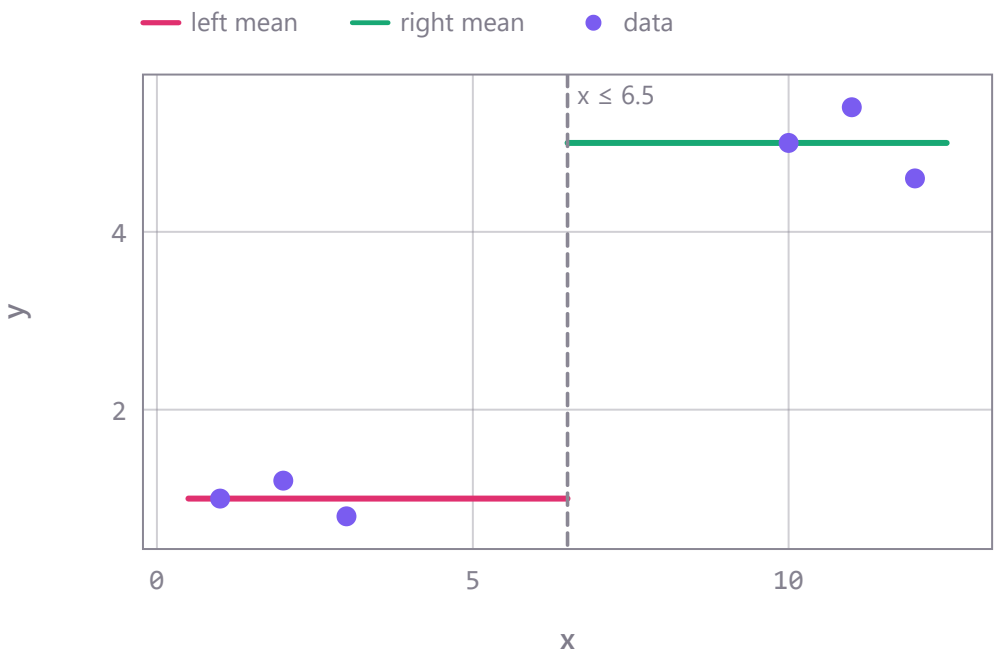
The best single split $x_j \le t$ of a regression tree's root by squared error: the feature, the threshold, the decrease of the mean squared error and the mean target on each side.

$$\Delta = \frac{1}{n}[SSE(S) - SSE(S_L) - SSE(S_R)]$$

input	meaning	condition
`X`	features (whole numbers)	
`y`	targets	

output	meaning
`feature`	index of the best feature
`threshold`	threshold t (left when x_j <= t)
`mse_decrease`	decrease of the node's mean squared error
`left_mean`	mean target on the left
`right_mean`	mean target on the right

- **Method:** scan every feature and midpoint; exact sums of squares
- **Assumes:** whole-number features; targets not all equal; a unique best split
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The best single split of x = [1, 2, 3, 10, 11, 12] for y = [1.0, 1.2, 0.8, 5.0, 5.4, 4.6]: the package returns threshold 6.5, left mean 1, right mean 5 and a decrease of the squared error by 4; the stump predicts the mean of each side.

```
data_science_ml_ai999x.solve('ml_decision_stump_regression', X=[[1], [2], [3], [10], [11], [12]], y=[1.0, 1.2, 0.8, 5.0, 5.4, 4.6])
# -> feature = 0, threshold = 6.5, mse_decrease = 4, left_mean = 1, right_mean = 5   (SOLVED)
```

```
data_science_ml_ai999x.solve('ml_decision_stump_regression', X=[[1, 5], [2, 3], [3, 8], [4, 1], [5, 7]], y=[2.0, 1.0, 4.0, 0.5, 3.5])
# -> feature = 1, threshold = 6, mse_decrease = 1.60167, left_mean = 1.16667, right_mean = 3.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Regression tree best split: X = [[1], [2], [3], [10], [11], [12]], y = [1, 1.2, 0.8, 5, 5.4, 4.6]	SOLVED	feature = 0, threshold = 6.5, mse_decrease = 4, left_mean = 1, right_mean = 5
Regression tree best split: X = [[1, 5], [2, 3], [3, 8], [4, 1], [5, 7]], y = [2, 1, 4, 0.5, 3.5]	SOLVED	feature = 1, threshold = 6, mse_decrease = 1.60167, left_mean = 1.16667, right_mean = 3.75
Regression tree best split: y = [1, 1.2, 0.8, 5, 5.4, 4.6]	NEEDS_INPUT	ml_decision_stump_regression needs X (features (whole numbers)). Example: solve('ml_decision_stump_regression', X=[[1],
Regression tree best split: X = [[1], [2], [3], [10], [11], [12]], y = [1, 1.2, 0.8, 5, 5.4, 4.6], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: data_science_ml_ai999x.templates(